

Combined Lecture Notes

Source: audited lecture notes for Lectures 02-23.

Lecture 02 Notes

Course Motivation: Geometry Behind Algebra

The lecture begins with the main goal of the course: become comfortable looking at algebraic expressions and asking for the geometric picture behind them. An expression might define a quadratic equation, another expression might define a hyperplane, and the real object of interest might be the intersection of those objects. The course will repeatedly translate algebraic formulas, constraints, objective functions, and data into geometric language.

Historically, algebra and geometry were treated mostly separately. Geometry involved figures and proofs based on those figures. Algebra involved numbers and symbolic expressions. The instructor gives Descartes as the main historical figure who linked the two through coordinates, while adding the caveat that the whole credit should not necessarily go to him alone. By assigning numbers to geometric locations, a point in the plane can be represented by an ordered pair, and a geometric object can be represented by an equation.

This link is analytical geometry: equations can be visualized as circles, lines, planes, or other geometric sets, and geometric objects can be described with algebraic equations. Linear algebra and vector space methods continue this idea. They let us take algebraic problems or data sets and bring a geometric view to them.

The course will use high-dimensional spaces to visualize or reason about objects such as objective functions and constraints in optimization problems. Beyond two or three dimensions we cannot draw the figures directly, but we can still use the same terminology and reasoning. The instructor warns that high-dimensional spaces sometimes have surprises that do not match ordinary two- or three-dimensional intuition.

Planned topics include:

- Vectors and matrices.
- Vector space definitions.
- Linear independence.
- Span.
- Basis.
- Norms.
- Inner products.
- Applications and significance of these definitions.

Vectors as Positions, Arrows, and Ordered Boxes of Numbers

The usual first exposure to vectors is often in physics, especially position vectors. In Cartesian coordinates, a position in the plane can be represented by two numbers organized as a column vector. The same object can be viewed either as a point or as an arrow from the origin to that point. The arrow view gives both location and direction.

Informally, from the mathematical point of view, a vector is a collection of numbers placed into a box for some purpose. For a two-dimensional vector, the entries are ordered. The first entry refers to the horizontal coordinate and the second entry refers to the vertical coordinate.

Exam note: the order of the entries is important. Swapping entries changes which coordinate is horizontal and which is vertical.

The instructor tells a Descartes anecdote from a children's book. Descartes was sick in bed and noticed a fly landing on the ceiling. He wondered whether it was landing in the same geometric location, so he drew horizontal and vertical lines, numbered them, and recorded the fly's positions as pairs of numbers. The point of the story is that position information was converted into numerical pairs. The instructor jokingly notes that this line of thinking eventually supports modern machine learning.

Velocity vectors are another geometric example. A velocity can be decomposed into horizontal and vertical components depending on the frame of reference, and these components can again be placed in a vector. The velocity vector tells us in which direction and how fast something is moving.

Non-Geometric Data as Vectors: Student Grades

The lecture then moves from physical vectors to data that is not naturally geometric. A student can be represented, for the purpose of a class, by two numbers: midterm grade and final grade.

For example,

```
x = [midterm grade, final grade]^T
```

This is a two-dimensional vector. The first element is the midterm grade, and the second element is the final grade. The instructor uses a special vector notation, often an arrow or bold symbol, to distinguish vectors from scalars.

Although grades are not physical positions, we can make an analogy with position vectors:

- Treat the midterm grade as the horizontal coordinate.
- Treat the final grade as the vertical coordinate.
- Represent the student as a point in a plane.

This is a simple observation, but conceptually important: non-geometric information has been converted into a geometric form. The instructor remarks that this helps humans because visual processing is powerful. If multiple students are plotted as points, we can use geometric language:

- Two students may have close performance.
- One student may be an outlier.
- The class distribution can be visually inspected.

This same idea extends to machine intelligence: numerical data is represented as vectors, and geometric relationships between those vectors become meaningful.

Overall Grades, Hyperplanes, and Half-Spaces in Two Dimensions

Suppose the class has two grading components: midterm and final. Each student is a two-dimensional vector. If the overall grade is computed as a weighted combination, for example

```
overall = 0.4 * midterm + 0.6 * final
```

and the passing condition is

```
overall > 40,
```

then the equality case

```
0.4 * midterm + 0.6 * final = 40
```

defines a line in the midterm-final plane. That line is the set of all potential grade pairs that produce exactly the passing threshold. The instructor calls this line a hyperplane.

In two dimensions, a hyperplane is a line. The line divides the plane into two regions:

- One half-space consists of grade pairs with overall grade greater than 40. The instructor calls this the half-space of “happy students” because these students pass.
- The other half-space consists of grade pairs with overall grade less than 40. The instructor calls this the half-space of “sad students” because these students fail.

The instructor uses notation such as H^+ for the passing half-space. These sets contain all potential student grade vectors satisfying the property, not necessarily only students who are actually in the class.

Instructor clarification: a hyperplane does not need to pass through the origin. In the grade example, the passing threshold line is generally away from the origin. The coordinate axes are arbitrary reference choices in an infinite plane. Since both sides of the line extend infinitely, one should not think of one side as inherently “larger”; they are just two half-spaces determined by the line.

Extension to Three and Four Grading Components

With three grading components, such as midterm, final, and homework, each student becomes a point in three-dimensional space:

$$\mathbf{x} = [\text{midterm}, \text{final}, \text{homework}]^T$$

The instructor gives an overall grade rule of the form

$$0.3 * \text{midterm} + 0.4 * \text{final} + 0.3 * \text{homework}$$

The weights sum to one, and the instructor notes that this will later be called a convex combination. The equality condition

$$0.3 * \text{midterm} + 0.4 * \text{final} + 0.3 * \text{homework} = 40$$

defines a plane in three-dimensional space. That plane divides the space into two half-spaces: passing grade triples and failing grade triples.

With four grading components, such as midterm, final, homework, and project, each student becomes a four-dimensional vector:

$$\mathbf{x} = [\text{midterm}, \text{final}, \text{homework}, \text{project}]^T$$

The instructor gives a rule like

$$0.3 * \text{midterm} + 0.4 * \text{final} + 0.2 * \text{homework} + 0.1 * \text{project} = 40$$

The set of grade vectors satisfying this equality is the set of barely passing combinations:

$$H = \{ (m, f, h, p) : 0.3m + 0.4f + 0.2h + 0.1p = 40 \}$$

The passing half-space is

$$H^+ = \{ (m, f, h, p) : 0.3m + 0.4f + 0.2h + 0.1p > 40 \}$$

The instructor emphasizes that we cannot draw this four-dimensional object, but we can still use the same terminology. In two dimensions the hyperplane is a line. In three dimensions it is a plane. In four and higher dimensions, it is called a hyperplane. The key idea is to extend familiar geometric terminology to higher-dimensional vector spaces.

Machine Learning Interpretation: Credit Approval and Neurons

The instructor connects the half-space idea to machine learning and artificial intelligence using a bank credit approval example. A customer applies for credit and declares information such as salary and real-estate value. These numbers form a feature vector:

$$\mathbf{x} = [\text{salary}, \text{real_estate_value}]^T$$

The bank wants to decide whether to approve or reject the credit application. Past data may show approved customers in one region of the feature plane and rejected customers in another. The instructor remarks that financial institutions may be required by government or the financial system to declare a decision rule: what criterion is used for giving or denying credit, and whether the rule discriminates against anyone. This is one reason machine-learning credit rules raise explainability issues. A simple explainable decision rule can be represented by a hyperplane:

$$w_1 * \text{salary} + w_2 * \text{real_estate_value} = \text{threshold}$$

One half-space corresponds to approved applications, and the other corresponds to rejected applications. Once we know that such a hyperplane can separate the data, the remaining problem is to determine the weights and threshold. The instructor says this is essentially the machine learning problem.

A simple artificial neuron implements this kind of half-space test. It computes something like

$$w_1 * \text{salary} + w_2 * \text{real_estate_value} - \text{threshold}$$

Then a nonlinear rule maps the result to an output:

- If the value is positive, output 1.
- If the value is negative, output 0.

So the neuron checks whether the input vector lies in a particular half-space. The instructor's geometric interpretation is that a neuron with this kind of threshold nonlinearity is a half-space identifier.

The instructor also briefly describes a biological neuron. A real neuron has dendrites that receive inputs, a cell body, and an axon that produces outputs. When a neuron fires, an action potential propagates along the axon, chemicals are released, and those chemicals bind to dendrites of another neuron. This opens gates, allowing charges such as sodium ions to enter, increasing the membrane potential. If the potential exceeds a threshold, the next neuron fires. The instructor notes that artificial models simplify this substantially, and real neurons have complications such as different types of dendrites and dendrite positions. On average, a biological neuron has about 10,000 connections.

When One Hyperplane Is Not Enough

The instructor then considers a data pattern where approved and rejected points cannot be separated by a single half-space. In such a case, one neuron is not enough. However, the same half-space concept can be extended by using intersections of half-spaces.

For example, a desired region may be described as the intersection of:

- A left-right half-space determined by one hyperplane.
- An up-down half-space determined by another hyperplane.

Each half-space can be checked by a separate neuron:

```
neuron 1: w_11 * salary + w_12 * real_estate - threshold_1
neuron 2: w_21 * salary + w_22 * real_estate - threshold_2
```

Then another neuron can check whether both outputs are 1. The instructor mentions McCulloch-Pitts logic elements in this context but says the lecture is not meant to go through neural network theory. The point is geometric: combinations of neurons can test membership in intersections of half-spaces.

Question/clarification: a student asks why the weights changed. The instructor explains that each neuron corresponds to a different hyperplane, so the weights are different. One neuron checks one half-space, another checks another half-space, and a later neuron can combine their outputs.

The instructor also notes that real neural networks may have thousands of inputs per neuron. We cannot draw those spaces, but the same geometric intuition about hyperplanes and half-spaces extends to high-dimensional cases.

Matrices as Collections of Vectors

The lecture then shifts from vectors to matrices. A matrix can be viewed as a rectangular organization of numbers, like an Excel sheet. But it can also be viewed as a collection of column vectors or a collection of row vectors.

In the student grade example:

- Each column might represent one student vector.
- Each row might represent one measurement or grading component.

This simple distinction, viewing a matrix by columns or by rows, is fundamental in linear algebra. The instructor says it lies at the heart of vector space concepts and the analysis of systems of linear equations, which will be studied later.

Scalar Multiplication of a Vector

The first formal vector operation introduced is scaling a vector by a scalar. A scalar is typically a real number or complex number from a field. The operation takes:

- One scalar α .
- One vector x .

If

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T,$$

then

$$\alpha \mathbf{x} = [\alpha x_1, \alpha x_2, \dots, \alpha x_n]^T.$$

The output is a vector of the same dimension as the input vector. Geometrically:

- If $\alpha > 1$, the vector is magnified in the same direction.
- If $0 < \alpha < 1$, the vector is shrunk in the same direction.
- If $\alpha < 0$, the vector points in the opposite direction.

The instructor stresses that this geometric interpretation is easy in two or three dimensions and then extended abstractly to higher-dimensional spaces.

Vector Addition

The second operation is vector addition. Addition is defined for two vectors of the same dimension. If

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T$$

$$\mathbf{y} = [y_1, y_2, \dots, y_n]^T,$$

then

$$\mathbf{x} + \mathbf{y} = [x_1 + y_1, x_2 + y_2, \dots, x_n + y_n]^T.$$

Geometrically, one can translate the tail of the second vector to the tip of the first vector. The sum is the vector from the original starting point to the final tip. The two vectors and their sum form a triangle.

The instructor gives a displacement example. If someone first moves a certain amount north and east, and then moves another amount west and north, the final location is obtained by summing the displacement vectors. In geometric terms, the second motion vector can be moved so that it starts at the end of the first motion vector, and the resulting final location is the vector sum.

Exam note: the final location after sequential vector displacements is given by vector addition.

Linear Combination

The lecture combines scalar multiplication and vector addition into the central operation of a linear combination.

For two vectors $x_1, x_2 \in \mathbb{R}^n$ and scalars $\alpha_1, \alpha_2 \in \mathbb{R}$,

$$\mathbf{y} = \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2$$

is a linear combination. The scalars are degrees of freedom. Each vector is scaled first, and the scaled vectors are then added.

The definition extends to any number of vectors:

$$\mathbf{y} = \sum_{i=1}^m \alpha_i \mathbf{x}_i.$$

Here n is the dimension of each vector, while m is the number of vectors in the combination. The instructor explicitly clarifies that m does not have to equal n . You may take any number of vectors in a linear

combination, even more than the dimension. Later, the number of vectors relative to dimension will matter for linear independence, but not for the basic definition of linear combination.

There are no restrictions on the scalars in a linear combination. They can be arbitrary real numbers in the current discussion, with complex extensions to come later in the course.

Span

The span of a set of vectors is the set of all possible linear combinations of those vectors.

If we are given vectors $x_1, x_2, \dots, x_m$, then

$$\text{span}\{x_1, \dots, x_m\} \\ = \{ \sum_{i=1}^m \alpha_i x_i : \alpha_i \text{ are scalars} \}.$$

For two vectors in the plane:

- If x_1 and x_2 are not collinear, their span is the whole plane $\mathbb{R}^2$.
- If x_1 and x_2 point in the same or opposite direction, their span is a line through the origin.

The instructor calls this line a hyperplane in the two-dimensional setting. It goes through the origin because linear combinations allow all scalars, including zero, and the zero vector is always included.

In three dimensions, if two non-collinear vectors are considered as position vectors from the origin, their linear combinations form a plane through the origin. The instructor warns that a picture can be misleading if one forgets the role of the origin. Linear combinations of x_1 and x_2 form the plane through the origin and those vectors, not merely the line through the endpoints x_1 and x_2 .

Relationship: span is built directly from linear combinations. The geometric object produced by a span depends on the directions and dependencies among the vectors.

Affine Combination

An affine combination is a restricted linear combination. The formula still looks like a weighted sum:

$$y = \sum_{i=1}^m \alpha_i x_i,$$

but the weights must satisfy

$$\sum_{i=1}^m \alpha_i = 1.$$

There is no nonnegativity requirement for affine combinations. The coefficients may be negative, as long as they sum to one.

The instructor gives the overall grade of a student as an example. If the grade weights sum to one, then the overall grade is an affine combination of the grading components. This makes the grading method meaningful as a weighted average.

Two-Point Affine Combinations

For two points x_1 and x_2 , an affine combination has the form

$$\alpha_1 x_1 + \alpha_2 x_2,$$

with

$$\alpha_1 + \alpha_2 = 1.$$

Since $\alpha_2 = 1 - \alpha_1$, we can write

$$\alpha_1 x_1 + (1 - \alpha_1)x_2.$$

This can be rearranged as

$$x_2 + \alpha_1(x_1 - x_2).$$

Proof idea from the instructor: $x_1 - x_2$ is the direction vector from x_2 to x_1 . Starting at x_2 and moving by an arbitrary scalar multiple of $x_1 - x_2$ traces the whole line through the two points. Therefore all affine combinations of two points lie on the line through them, and every point on that line can be represented this way.

Special cases:

- If $\alpha_1 = 0$, the point is x_2 .
- If $\alpha_1 = 1$, the point is x_1 .
- If $0 < \alpha_1 < 1$, the point lies between x_1 and x_2 .
- If $\alpha_1 > 1$, the point lies beyond x_1 on the same line.
- If $\alpha_1 < 0$, the point lies beyond x_2 on the other side.

This remains true in three dimensions: all affine combinations of two points form the line through the two points.

Relationship to linear combinations: linear combinations of two non-collinear position vectors in three dimensions can form a plane through the origin. Affine combinations of two points form only the line through those two points. The restriction that the weights sum to one changes the geometry.

Affine Hull

The affine hull of a set is the set of all possible affine combinations of points in that set.

For three points x_1, x_2, x_3 , affine combinations have the form

$$\alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3$$

with

$$\alpha_1 + \alpha_2 + \alpha_3 = 1.$$

If the three points are not collinear, their affine hull is the plane containing the three points. If the points are collinear, their affine hull is the line containing them. The instructor says these dimension issues will be discussed later and asks students to verify the plane case at home.

Relationship: affine hull is to affine combinations what span is to linear combinations.

Convex Combination

A convex combination is an even more restricted affine combination. It has the same weighted-sum form:

$$y = \sum_{i=1}^m \alpha_i x_i,$$

but now the weights must satisfy both:

$$\sum_{i=1}^m \alpha_i = 1$$

and

$$\alpha_i \geq 0 \text{ for all } i.$$

Thus:

- Linear combination: no restriction on coefficients.
- Affine combination: coefficients sum to one.
- Convex combination: coefficients sum to one and are nonnegative.

The instructor again uses grades as an example. A normal overall grade calculation is a convex combination if the weights sum to one and are nonnegative. It is also an affine combination and a linear combination, but convex combination is the most specific description.

For two points, all convex combinations form the line segment joining them. The instructor says this line segment is sometimes called a chord. Unlike affine combinations, convex combinations cannot go beyond the endpoints because nonnegative weights summing to one keep the point between the endpoints.

Convex Hull

Given a set S , the convex hull of S , often written $\text{conv}(S)$, is the set of all possible convex combinations of elements of S .

In symbolic form:

```
conv(S) =
{ \sum_{i=1}^m \alpha_i x_i :
  x_i in S,
  \alpha_i >= 0,
  \sum_{i=1}^m \alpha_i = 1,
  m is any positive integer }.
```

For two points, the convex hull is the line segment between them.

For three non-collinear points, the convex hull is the triangle whose vertices are those three points. This contrasts with the affine hull of the same three points, which is the entire plane through them.

The instructor also describes the triangle as an intersection of half-spaces within the plane of the three points. Each line through a pair of vertices defines a half-space inside that plane. The triangular convex hull is the intersection of the appropriate three half-spaces, restricted to the affine hull of the three points.

Convex Set

A set is convex if, whenever we pick any two points in the set, the entire line segment joining those points also stays inside the set.

Using convex-combination notation: a set S is convex if for any $x_1, x_2 \in S$ and any $\alpha \in [0, 1]$,

$\alpha x_1 + (1 - \alpha)x_2 \in S$.

The instructor describes a round or bowl-like set as convex because every chord between two points of the set remains inside the set. A heart-shaped set is not convex because one can choose two points inside it such that part of the line segment between them goes outside the set.

Exam note: convexity is an important definition.

Convex Optimization and Convex Functions

The instructor relates convex sets to optimization. In optimization theory, convex optimization is the simpler case, while non-convex optimization is hard and remains an active difficult area.

The intuitive reason is that searching inside a convex set is simpler. If line segments between feasible points stay feasible, the geometry is better behaved. The instructor also notes that convex optimization is not only about the search domain; the function being optimized is potentially even more important.

For a scalar function of a scalar variable, a convex function looks like a bowl or parabola. A parabola is a special case. Such a function does not have multiple separate bumps or local minima in the way a non-convex function might. This supports the existence of a simple global optimum structure.

The instructor introduces the epigraph of a function: the set of points lying above the graph of the function. A function is convex when its epigraph is convex, with the domain also convex. An optimization problem involving such a function and domain is called a convex optimization problem.

Relationship:

- Convex combination leads to convex hull.
- Convex hull and line-segment containment lead to convex sets.
- Convex sets and convex epigraphs are central to convex optimization.

Applications of Convex Combinations

The instructor returns to the student-grade setting with three grading components: midterm, final, and homework.

There are two natural ways to organize the data:

- Student vectors: each vector contains one student's midterm, final, and homework grades.
- Component vectors: one vector contains all students' midterm grades, one contains all students' final grades, and one contains all students' homework grades.

The instructor notes that, in the small example, there are three student vectors and three grading-component vectors, so six related vectors appear. He also warns that the equality between the number of students and the number of grading components in this example is accidental; he should have chosen different counts to avoid suggesting they must coincide.

If the overall grade vector for the whole class is computed from the midterm vector, final vector, and homework vector using nonnegative weights that sum to one, then the overall grade vector is a convex combination of the component vectors. For example:

```
overall_vector =
0.3 * midterm_vector +
0.4 * final_vector +
0.3 * homework_vector
```

This produces each student's overall grade as the corresponding entry of the resulting vector.

The instructor then connects this to probability. For a discrete random vector, expectation is computed by multiplying each possible realization by its probability and summing:

$$E[X] = \sum_i p_i x_i.$$

The probabilities satisfy:

$$p_i \geq 0 \\ \sum_i p_i = 1.$$

Therefore, expectation is a convex combination of the possible realizations of the random vector.

Another example is the average student vector. If all students are averaged with equal nonnegative weights that sum to one, the result is a vector containing the midterm average, final average, and homework average. This average vector is a convex combination of the student vectors.

End-of-Lecture Summary and Next Lecture

The instructor summarizes the progression of ideas:

1. Scale a vector by a scalar.
2. Add vectors.
3. Combine scaling and addition to get linear combinations.
4. Restrict the weights to sum to one to get affine combinations.
5. Add nonnegativity of weights to get convex combinations.

These ideas support span, affine hull, convex hull, convex sets, hyperplanes, half-spaces, and geometric reasoning in high-dimensional vector spaces.

The next lecture will introduce another operation, but the transcript is garbled at this point. The instructor then says the next lecture will introduce norms, define distance for vectors, extend distance concepts from two-

and three-dimensional spaces to high-dimensional and arbitrary vector spaces, and later extend these ideas to matrices. After this introductory phase, the course will move toward the formal vector space definition.

Exam-Relevant Topics Marked in the Source

The corrected transcript explicitly marks these as likely exam topics:

- Hyperplanes and half-spaces.
- Affine and convex combinations.
- Convex sets and convex hulls.

The transcript also marks as exam-relevant the ordered nature of vector entries, the student-grade vector example, grade-threshold hyperplanes and half-spaces in two, three, and four dimensions, the final location interpretation of vector addition, the importance of linear combination, the two-point convex-combination line segment, convexity as an important definition, convex functions/epigraphs in optimization, the three-point convex hull triangle versus affine hull plane, and the grade/probability/average examples of convex combinations.

Source and Coverage Note

Source used: `corrected/lecture2_corrected.md` only.

Coverage: These notes preserve the lecture's chronological structure, examples, exam-marked comments, instructor warnings, definitions, relationships between concepts, proof ideas, and end-of-lecture preview.

Lecture 03 Notes

1. Opening Recap: Vector Operations And Combinations

The lecture opens by connecting back to the previous lecture's geometric intuition for vectors and vector spaces.

Previously covered operations:

- Scaling a vector.
- Adding vectors.
- Forming a linear combination of vectors.

A linear combination is a weighted sum of multiple vectors. The instructor emphasizes that when the weights are restricted, special kinds of linear combinations arise.

Special combinations:

- Affine combination: a linear combination where the weights sum to 1.
- Convex combination: a more restricted affine combination where the weights sum to 1 and each weight is nonnegative.

Geometric interpretation of convex combinations:

- For two vectors or points, all convex combinations form the line segment connecting the two points.
- For three vectors or points, all convex combinations form the triangle whose vertices are those three vectors.
- For multiple vectors, all possible convex combinations define the higher-dimensional analogue of polygons in two dimensions. This is the idea behind the convex hull of a set of points.

Exam relevance:

- The transcript explicitly marks this discussion as a likely exam topic.
- The important relationship is that algebraic restrictions on linear combination weights produce geometric objects such as line segments, triangles, polygons, and their higher-dimensional analogues.

2. Euclidean Norm

The lecture then moves to an additional operation on vectors: the Euclidean norm.

The norm is introduced as a way to answer the question: how big is a vector? A vector can have many components, but the norm maps the vector to a single number measuring its size.

The instructor stresses that there are different possible definitions of “how big” a vector is. The Euclidean norm is only one choice, although it is the main reference norm at this stage of the course.

2.1 Euclidean Norm In Two Dimensions

For a two-dimensional vector

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

the Euclidean norm is the distance from the point x to the origin. Geometrically, it is the length of the arrow from the origin to the point (x_1, x_2) .

Using the right triangle with side lengths x_1 and x_2 , the norm is

$$\|x\| = \sqrt{x_1^2 + x_2^2}.$$

The instructor notes that later the notation $\|x\|_2$ may be used to distinguish the Euclidean norm from other norms. For now, when no special subscript is written, $\|x\|$ means the Euclidean norm.

Properties visible from the definition:

- The norm maps a vector to a scalar.
- The scalar is nonnegative.
- Example: if $x = (3, 4)$, then $\|x\| = 5$, the familiar 3-4-5 triangle.

2.2 Euclidean Norm In Three Dimensions

For a three-dimensional vector

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix},$$

the Euclidean norm is again interpreted as distance from the point to the origin.

Exam note from the transcript:

- The instructor explicitly discusses projecting the vector onto the x_1x_2 -plane.

Derivation idea:

1. Project the vector onto the x_1x_2 -plane.
2. In that plane, the first right triangle has side lengths x_1 and x_2 .
3. The hypotenuse in the x_1x_2 -plane has length

$$\sqrt{x_1^2 + x_2^2}.$$

4. A second right triangle uses that hypotenuse as one side and x_3 as the other side.
5. Applying the Pythagorean theorem again gives

$$\|x\| = \sqrt{x_1^2 + x_2^2 + x_3^2}.$$

The three-dimensional formula extends the two-dimensional formula by squaring the additional coordinate, adding it to the sum, and taking the square root.

2.3 Distance Between Two Vectors

Once the norm is defined, geometric language becomes available.

The size of one vector is $\|x\|$.

The distance between two vectors x and y is defined as the norm of their difference:

$$d(x, y) = \|x - y\|.$$

The difference vector $x - y$ is itself a vector in the same space, so its norm measures how far apart x and y are.

Important relationship:

- The norm gives a notion of vector size.
- The norm of a difference vector gives a notion of distance between vectors.

The instructor warns that different norm definitions give different distance values. Therefore, optimization problems that minimize distances can produce different solutions depending on the chosen norm.

Instructor remark:

- A significant amount of signal processing and machine learning research has studied choices of norms, how they affect solution behavior, and what kinds of solutions they promote.

2.4 Why Euclidean Norm Is The Main Starting Point

The instructor describes the Euclidean norm as the “typical” norm in much of 20th-century signal processing, especially in engineering contexts.

Reasons for its popularity:

- It is convenient.
- It is easy to handle analytically.
- It often allows closed-form solutions.
- It can avoid iterative computations in many settings.

The instructor also notes that other norms are useful for different purposes and became heavily used in later parts of the 20th century, especially in different applications of signal processing and machine learning.

Course trajectory:

- The Euclidean norm is the main reference for now.
- Later the course will generalize the norm concept and study alternatives.

2.5 Euclidean Norm In Higher Dimensions

The instructor identifies the extension beyond three dimensions as a major conceptual leap.

In four-dimensional real space, or in a 2000-dimensional real space, a vector cannot be drawn and measured with a ruler. However, the formula can still be extended as a definition.

For

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n,$$

the Euclidean norm is

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\sum_{i=1}^n x_i^2}.$$

Important instructor remark:

- In dimensions higher than three, this is a definition, not a geometric derivation from a drawing.
- We are assigning geometric properties such as size and distance to objects we cannot draw.

The resulting setting is called n -dimensional Euclidean space equipped with the Euclidean norm.

Even though high-dimensional vectors cannot be drawn, the instructor encourages thinking by analogy with two- and three-dimensional pictures.

2.6 Fundamental Properties Of The Euclidean Norm

The Euclidean norm satisfies several key properties.

Nonnegativity:

$$\|x\| \geq 0.$$

This follows because the norm squares the components, sums nonnegative quantities, and then takes a square root.

Definiteness:

$$\|x\| = 0 \quad \text{if and only if} \quad x = 0.$$

Any vector other than the origin has at least one nonzero component. Squaring that component gives a positive term in the sum, so the norm cannot be zero.

Homogeneity under scalar multiplication:

$$\|\alpha x\| = |\alpha| \|x\|.$$

The absolute value is important. Multiplying by a negative scalar reverses orientation, but length is scaled by the magnitude of the scalar.

Triangle inequality:

$$\|x + y\| \leq \|x\| + \|y\|.$$

Geometric interpretation:

- If x and y are placed tip-to-tail, then $x + y$ is the third side of the triangle.
- The length of one side of a triangle is no larger than the sum of the other two side lengths.

Equality case mentioned in lecture:

- Equality occurs when x and y are collinear and point in the same direction.

Exam relevance:

- The transcript explicitly marks the generalization of norm properties as a likely exam topic.
- The instructor emphasizes that the Euclidean norm is only one norm satisfying these properties. Later, other norms satisfying analogous norm properties will be introduced.

3. Inner Product, Angles, And Orthogonality

After size and distance, the next geometric concept introduced is angle between vectors.

In two dimensions, the angle between vectors can be drawn directly. The instructor asks how this concept can be extended to arbitrary-dimensional real spaces, such as spaces of thousand-dimensional vectors.

The key tool is the Euclidean inner product.

Exam note:

- The transcript explicitly marks the connection between angles and orthogonality as exam-relevant.

3.1 Orthogonality Motivation

Once angle is available, orthogonality becomes available.

Definition from geometry:

- Two vectors are orthogonal if the angle between them is 90° .

Goal:

- Extend the notion of orthogonality to higher-dimensional vector spaces.

The instructor emphasizes that inner products are choices. The Euclidean inner product is the standard initial choice, but later other inner products will be introduced.

Important relationship:

- Different inner products can define different geometries.
- Two vectors may be orthogonal with respect to one inner product but not orthogonal with respect to another.
- Orthogonality is therefore relative to the chosen inner product.

3.2 Euclidean Inner Product In Two Dimensions

For

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix},$$

the Euclidean inner product is

$$\langle x, y \rangle = x_1 y_1 + x_2 y_2.$$

The operation:

- Takes two vector inputs.
- Produces one scalar output in the real case.
- Is formed by multiplying matching-index components and summing the products.

Extension to n -dimensional real vectors:

For

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix},$$

the Euclidean inner product is

$$\langle x, y \rangle = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n = \sum_{i=1}^n x_i y_i.$$

This is the same extension pattern as the Euclidean norm: start from the two- and three-dimensional geometric picture, then use the component formula as the definition in dimensions that cannot be drawn.

For complex vectors, the instructor notes that the output can be complex and the definition requires modifications later.

Important warning:

- The inner product output is not necessarily nonnegative.
- In the real case, $\langle x, y \rangle$ can be positive, zero, or negative.

3.3 Norm Induced By Inner Product

Exam note:

- The transcript explicitly marks the relationship between inner product and norm as exam-relevant.

If a vector is inner-producted with itself, then

$$\langle x, x \rangle = x_1x_1 + x_2x_2 = x_1^2 + x_2^2.$$

Since the Euclidean norm satisfies

$$\|x\| = \sqrt{x_1^2 + x_2^2},$$

we have

$$\langle x, x \rangle = \|x\|^2.$$

Therefore,

$$\|x\| = \sqrt{\langle x, x \rangle}.$$

Terminology:

- The Euclidean norm is induced by the Euclidean inner product.
- This means the norm can be derived from the inner product by taking the inner product of a vector with itself and then taking the square root.

Important relationship:

- Defining an inner product automatically gives a way to define a norm, provided the positivity and definiteness requirements hold.

3.4 Deriving The Angle Formula From Inner Product

The instructor derives the connection between the inner product and the angle between two vectors.

Let θ be the angle between x and y .

Geometric setup:

- Consider the triangle formed by x , y , and the difference vector $x - y$.
- The side corresponding to the difference vector has length $\|x - y\|$.
- Drop an orthogonal projection so that right triangles appear in the picture.

Using the right-triangle decomposition:

- One side has length $\|x\| \sin \theta$.
- Another side has length $\|y\| - \|x\| \cos \theta$.

Applying the Pythagorean theorem gives

$$\|x - y\|^2 = \|x\|^2 \sin^2 \theta + (\|y\| - \|x\| \cos \theta)^2.$$

Expanding:

$$\|x - y\|^2 = \|x\|^2 \sin^2 \theta + \|y\|^2 - 2\|x\|\|y\| \cos \theta + \|x\|^2 \cos^2 \theta.$$

Using

$$\sin^2 \theta + \cos^2 \theta = 1,$$

this becomes

$$\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2\|x\|\|y\| \cos \theta.$$

This is the law of cosines in vector notation.

Now write the left side using the inner product:

$$\|x - y\|^2 = \langle x - y, x - y \rangle.$$

Using bilinearity of the real inner product:

$$\langle x - y, x - y \rangle = \langle x, x \rangle - \langle x, y \rangle - \langle y, x \rangle + \langle y, y \rangle.$$

For the real Euclidean inner product, $\langle x, y \rangle = \langle y, x \rangle$, so

$$\|x - y\|^2 = \|x\|^2 - 2\langle x, y \rangle + \|y\|^2.$$

Matching this with the law-of-cosines expression gives

$$\langle x, y \rangle = \|x\|\|y\| \cos \theta.$$

Therefore,

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\|\|y\|}.$$

Important relationship:

- The inner product does not directly give the angle.
- The inner product, together with the induced norms, gives the cosine of the angle.

3.5 Orthogonality From Inner Product

If the angle between two vectors is 90° , then

$$\cos 90^\circ = 0.$$

Using the angle formula,

$$\langle x, y \rangle = \|x\| \|y\| \cos \theta,$$

orthogonality implies

$$\langle x, y \rangle = 0.$$

Definition extended from geometry:

$$x \perp y \quad \text{if and only if} \quad \langle x, y \rangle = 0.$$

Example from the lecture:

Let

$$x = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad y = \begin{bmatrix} -4 \\ 3 \end{bmatrix}.$$

Then

$$\langle x, y \rangle = 3(-4) + 4(3) = -12 + 12 = 0.$$

Therefore the vectors are orthogonal, and the angle between them is 90° .

3.6 Inner Product Signs And Angle Types

In two and three dimensions, and then by extension in n -dimensional space:

- If $\langle x, y \rangle > 0$, then $\cos \theta > 0$, so the angle is acute.
- If $\langle x, y \rangle < 0$, then $\cos \theta < 0$, so the angle is obtuse.
- If $\langle x, y \rangle = 0$, then $\cos \theta = 0$, so the vectors are orthogonal.

In three dimensions, two non-collinear vectors span a plane, so the same two-dimensional picture applies inside that plane. The formula

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\| \|y\|}$$

again gives the cosine of the angle between the vectors.

In n -dimensional space, the same formula is adopted as the definition of the cosine of the angle between two nonzero vectors.

Warning:

- The angle formula requires x and y to be nonzero, because $\|x\| \|y\|$ appears in the denominator.
- The orthogonality definition $\langle x, y \rangle = 0$ is the algebraic condition that extends cleanly to high-dimensional spaces; the angle interpretation comes from the cosine formula when both vectors are nonzero.

3.7 Need For The Cauchy-Schwarz Inequality

When defining

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\| \|y\|}$$

in high-dimensional space, the expression must always lie between -1 and 1 to be a valid cosine value.

The required guarantee is the Cauchy-Schwarz inequality:

$$|\langle x, y \rangle| \leq \|x\| \|y\|.$$

This inequality ensures that

$$-1 \leq \frac{\langle x, y \rangle}{\|x\| \|y\|} \leq 1$$

whenever x and y are nonzero.

Instructor remark:

- Cauchy-Schwarz is used in many areas, including estimation theory.
- The instructor specifically mentions that the Cramer-Rao lower bound in estimation theory is obtained from the Cauchy-Schwarz inequality.
- The instructor also mentions a book containing multiple proofs of the Cauchy-Schwarz inequality.

3.8 Proof Idea For Cauchy-Schwarz

The proof begins with the fact that a squared norm is always nonnegative:

$$\|u - v\|^2 \geq 0.$$

Using the inner product induced norm:

$$\|u - v\|^2 = \langle u - v, u - v \rangle.$$

For real inner products, expand:

$$\langle u - v, u - v \rangle = \|u\|^2 - 2\langle u, v \rangle + \|v\|^2.$$

Therefore,

$$\|u\|^2 - 2\langle u, v \rangle + \|v\|^2 \geq 0.$$

Rearranging:

$$2\langle u, v \rangle \leq \|u\|^2 + \|v\|^2.$$

This is not yet Cauchy-Schwarz. It is an additive bound, because it bounds the inner product by a sum of squared norms.

Equivalently,

$$\langle u, v \rangle \leq \frac{\|u\|^2 + \|v\|^2}{2}.$$

The transcript includes a suggested alternative of completing the square. The instructor notes that algebraic manipulation may be possible, but then uses a simpler normalization trick.

The instructor's trick is to convert the additive bound into a multiplicative bound by normalizing the vectors.

Assume x and y are nonzero and choose

$$u = \frac{x}{\|x\|}, \quad v = \frac{y}{\|y\|}.$$

Then

$$\|u\| = 1, \quad \|v\| = 1.$$

Substituting into the additive bound gives

$$2 \left\langle \frac{x}{\|x\|}, \frac{y}{\|y\|} \right\rangle \leq 1^2 + 1^2 = 2.$$

So

$$\left\langle \frac{x}{\|x\|}, \frac{y}{\|y\|} \right\rangle \leq 1.$$

Because $\|x\|$ and $\|y\|$ are positive, multiply both sides by $\|x\|\|y\|$:

$$\langle x, y \rangle \leq \|x\|\|y\|.$$

To get the lower bound, replace one normalized vector by its negative, for example

$$u = -\frac{x}{\|x\|}, \quad v = \frac{y}{\|y\|}.$$

This gives

$$-\langle x, y \rangle \leq \|x\|\|y\|.$$

Combining both inequalities yields

$$|\langle x, y \rangle| \leq \|x\|\|y\|.$$

Thus Cauchy-Schwarz justifies using the quotient

$$\frac{\langle x, y \rangle}{\|x\|\|y\|}$$

as a cosine value in n -dimensional Euclidean space.

3.9 Properties Of The Inner Product

The instructor lists the main properties of the Euclidean inner product for real n -dimensional vectors.

Scaling in the first argument:

$$\langle \alpha x, y \rangle = \alpha \langle x, y \rangle.$$

Scaling in the second argument:

$$\langle x, \beta y \rangle = \beta \langle x, y \rangle.$$

Scaling both arguments:

$$\langle \alpha x, \beta y \rangle = \alpha \beta \langle x, y \rangle.$$

There is no absolute value in these scaling rules. This differs from the norm homogeneity rule, where the scalar appears as $|\alpha|$.

Additivity in the first argument:

$$\langle x + z, y \rangle = \langle x, y \rangle + \langle z, y \rangle.$$

Additivity in the second argument:

$$\langle x, y + z \rangle = \langle x, y \rangle + \langle x, z \rangle.$$

Linearity in each argument:

- If the second argument is fixed, the inner product is linear in the first argument.
- If the first argument is fixed, the inner product is linear in the second argument.

This is why, in the real case, the Euclidean inner product is bilinear.

Symmetry in the real Euclidean case:

$$\langle x, y \rangle = \langle y, x \rangle.$$

This symmetry is what makes the two cross terms in expansions such as

$$\langle u - v, u - v \rangle$$

combine into $-2\langle u, v \rangle$. The instructor warns during the proof discussion that the complex case has deviations, so the real-vector formulas are the setting for this lecture.

Positivity:

$$\langle x, x \rangle \geq 0.$$

Definiteness:

$$\langle x, x \rangle = 0 \quad \text{if and only if} \quad x = 0.$$

These properties are required for the positive definite inner product setting used to induce a true norm.

3.10 Instructor Remarks On Inner Product Generalizations

A student asks whether all these properties are required for old and new inner products.

Instructor response, preserved in substance:

- These properties are required for the kind of inner product being used in the course.
- Some extensions exist, such as semidefinite families and indefinite spaces.
- The course will not go deeply into those extensions.
- Such extensions are nevertheless used in practice.

Important clarification:

- The condition $\langle x, x \rangle = 0$ only when $x = 0$ is the definiteness property.
- It is needed for the induced norm to be positive definite.
- If this condition is removed, one obtains indefinite spaces rather than the positive definite normed spaces being used here.

Instructor example of where indefinite spaces appear:

- Relativity and related theories use indefinite spaces.

Exam relevance:

- The transcript explicitly marks this distinction as likely exam-relevant.

4. Hyperplanes And Half-Spaces From Inner Products

The lecture next uses inner products and orthogonality to define hyperplanes and half-spaces.

4.1 Hyperplane Through The Origin

The instructor begins with an incomplete hyperplane definition that applies to hyperplanes passing through the origin.

Let a be a vector. The vector a is the normal vector of the hyperplane.

The hyperplane defined by a is the collection of all vectors x orthogonal to a :

$$H_a = \{x : \langle x, a \rangle = 0\}.$$

In two dimensions:

$$H_a = \left\{ \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} : a_1 x_1 + a_2 x_2 = 0 \right\}.$$

Geometric interpretation in two dimensions:

- H_a is a line through the origin.
- The vector a is perpendicular to that line.
- All vectors lying on the line are orthogonal to a .

Geometric interpretation in three dimensions:

- H_a is a plane through the origin.
- The vector a is normal to that plane.

The instructor emphasizes that this is not yet the complete hyperplane definition, because it only covers hyperplanes through the origin.

4.2 Positive And Negative Half-Spaces

Using the same normal vector a , define a positive half-space:

$$H_a^+ = \{x : \langle x, a \rangle > 0\}.$$

These are vectors making an acute angle with a .

Define a negative half-space:

$$H_a^- = \{x : \langle x, a \rangle < 0\}.$$

These are vectors making an obtuse angle with a .

In two dimensions:

- The hyperplane H_a is a line.
- H_a^+ is one side of the line.
- H_a^- is the other side of the line.

In three dimensions:

- The hyperplane H_a is a plane.
- The positive and negative half-spaces are the two sides of that plane.

Important relationship:

- The sign of the inner product with the normal vector identifies which side of the hyperplane a vector lies on.

4.3 General Hyperplanes With A Threshold

Exam note:

- The transcript explicitly marks the generalized hyperplane definition as exam-relevant.

The full hyperplane definition compares the inner product not necessarily to zero, but to a scalar threshold b :

$$H_{a,b} = \{x : \langle x, a \rangle = b\}.$$

Here:

- a is the normal vector.
- b is a scalar threshold.
- If $b = 0$, the hyperplane passes through the origin.
- If $b \neq 0$, the hyperplane is shifted away from the origin.

The instructor connects this to an earlier grade-threshold example:

- A passing/failing threshold can be represented by comparing an inner product to a nonzero value b .
- The passing students and failing students lie on different sides of a threshold hyperplane.

The corresponding shifted half-spaces are

$$H_{a,b}^+ = \{x : \langle x, a \rangle > b\},$$

and

$$H_{a,b}^- = \{x : \langle x, a \rangle < b\}.$$

The sign of $\langle x, a \rangle - b$ determines which side of the shifted hyperplane x lies on. When $b = 0$, these reduce to the origin-centered half-space definitions H_a^+ and H_a^- .

4.4 Deriving The Shifted Hyperplane Form

Start from the origin hyperplane condition, but shift it by a vector x_0 :

$$\langle x - x_0, a \rangle = 0.$$

Using linearity:

$$\langle x, a \rangle - \langle x_0, a \rangle = 0.$$

Therefore

$$\langle x, a \rangle = \langle x_0, a \rangle.$$

Let

$$b = \langle x_0, a \rangle.$$

Then the shifted hyperplane is

$$H_{a,b} = \{x : \langle x, a \rangle = b\}.$$

Important interpretation:

- x_0 is a shift vector.
- a remains the normal vector.
- b is the threshold quantity produced by the shift.

4.5 Projection Interpretation Of The General Hyperplane

The instructor says projection will be covered later, but gives a preview because of a student question.

For a vector x projected onto the direction of a vector a , the projection vector is

$$x_a = \frac{\langle x, a \rangle}{\langle a, a \rangle} a = \frac{\langle x, a \rangle}{\|a\|^2} a.$$

Equivalently,

$$x_a = \frac{\langle x, a \rangle}{\|a\|} \frac{a}{\|a\|}.$$

Here:

- $\frac{a}{\|a\|}$ is the unit vector in the direction of a .
- The signed projection length along a is $\frac{\langle x, a \rangle}{\|a\|}$.
- The magnitude of the projection length is $\frac{|\langle x, a \rangle|}{\|a\|}$.

Warning from the instructor:

- $\langle x, a \rangle$ is not itself the projection length unless a is a unit vector.

- To obtain the projection length, divide by $\|a\|$.
- The threshold b in $\langle x, a \rangle = b$ is not automatically a projection length.

For all vectors x in $H_{a,b}$:

$$\langle x, a \rangle = b.$$

Since a is fixed, all such vectors have the same projection component along a . This is why $H_{a,b}$ is a shifted line, plane, or higher-dimensional hyperplane.

Important clarification:

- In $H_{a,b}$, the angle between x and a is not fixed.
- What is fixed is the orthogonal projection component of x along a .
- A set of constant-angle vectors would not be the same object as this hyperplane.

4.6 Neuron As A Half-Space Test

Exam note:

- The transcript explicitly marks the neuron connection as exam-relevant.

The instructor connects hyperplanes and half-spaces to a neuron model with a threshold/nonlinearity.

For an input vector x and weight vector w , the neuron computes an inner product:

$$\langle x, w \rangle = w_1x_1 + w_2x_2 + \cdots + w_nx_n.$$

Then it subtracts or compares to a threshold b :

$$\langle x, w \rangle - b > 0.$$

Interpretation:

- The weights are synaptic weights.
- The weight vector w defines a direction in high-dimensional space.
- The threshold b shifts the separating hyperplane.
- The neuron checks whether the input vector lies in a particular half-space determined by w and b .

Important relationship:

- A threshold neuron is geometrically a half-space classifier.
- The decision boundary is a hyperplane.
- The normal direction of the boundary is the weight vector.

The instructor says this example may be revisited later.

4.7 High-Dimensional Geometric Thinking

The instructor summarizes that inner products allow the course to extend geometric language into high-dimensional spaces:

- Cosine of angle.
- Acute angle.
- Obtuse angle.
- Orthogonality.
- Hyperplanes.
- Half-spaces.

Even when the space cannot be drawn, the signs and values of inner products allow us to reason as if the two- and three-dimensional geometric pictures still guide the intuition.

5. Matrix Multiplication As A Box Of Inner Products

The lecture then moves to matrix-vector and matrix-matrix operations.

Matrix multiplication is described as a “box of inner products.”

For matrices

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times p},$$

the product

$$C = AB$$

has entries

$$c_{ij} = \text{inner product of row } i \text{ of } A \text{ with column } j \text{ of } B.$$

Explicitly:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

Dimension requirement:

- A row of A has length n .
- A column of B has length n .
- These dimensions must match so the inner product is defined.

This is the usual inner-dimension matching condition for matrix multiplication:

$$(m \times n)(n \times p) = m \times p.$$

Instructor example:

- The element c_{25} is the inner product of the second row of the first matrix with the fifth column of the second matrix.

Important relationship:

- Matrix multiplication is not an unrelated operation. It is built from many inner products arranged in matrix form.

6. Systems Of Linear Equations In Matrix Form

The instructor next explains why matrix multiplication is useful by writing systems of linear equations compactly.

Example system:

$$2x_1 + 3x_2 = 10,$$

$$x_1 - x_2 = 10.$$

The left-hand sides and right-hand sides can be boxed into column vectors:

$$\begin{bmatrix} 2x_1 + 3x_2 \\ x_1 - x_2 \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}.$$

Each left-hand side is an inner product:

$$2x_1 + 3x_2 = \begin{bmatrix} 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

$$x_1 - x_2 = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Therefore the system can be written as

$$\begin{bmatrix} 2 & 3 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}.$$

In compact notation:

$$Ax = b.$$

Interpretation:

- A is the known coefficient matrix.
- x is the vector of unknown variables.
- b is the known right-hand-side vector.

Instructor remark:

- Matrix notation separates known quantities from unknown variables.
- This compact notation becomes the starting point for studying linear algebra and matrix analysis.

7. Existence And Uniqueness Questions For $Ax = b$

Once systems are written as

$$Ax = b,$$

the course will ask fundamental questions.

Existence:

- Is there an x such that $Ax = b$?
- Can we find a vector x satisfying the given matrix A and right-hand side b ?

Uniqueness:

- If a solution exists, is it unique?
- Are there alternative solutions?
- How large is the solution set?

Instructor question:

- Can a system of linear equations have exactly five solutions only?

The later vector-space analysis will show the typical classification:

- No solution.
- A unique solution.
- Infinitely many solutions.

The instructor says the course will study which types of matrices guarantee unique solutions and what properties of A and b characterize each case.

Important relationship:

- Some questions depend only on A .
- Other questions depend on A together with b .
- Vector space concepts will be used to answer these existence and uniqueness questions.

8. Geometric View: Equations As Hyperplanes

The lecture closes by connecting linear systems back to hyperplanes.

Each row of a linear system defines one scalar equation. For example,

$$2x_1 + 3x_2 = 10$$

defines a hyperplane. In two dimensions, that hyperplane is a line.

Likewise,

$$x_1 - x_2 = 10$$

defines another line.

The solution set of the full system is the intersection of the solution sets of the individual equations.

General relationship:

- One equation $a_i^T x = b_i$ defines a hyperplane.
- A system $Ax = b$ asks for the intersection of all those hyperplanes.

8.1 Two Equations And Two Unknowns

For two equations in two unknowns, the geometric picture consists of two lines.

General form:

$$a_{11}x_1 + a_{12}x_2 = b_1,$$

$$a_{21}x_1 + a_{22}x_2 = b_2.$$

Each equation defines a line in the x_1x_2 -plane.

The possible cases:

1. Unique solution.

The two lines intersect at exactly one point. That intersection point satisfies both equations.

Instructor remark:

- For estimation purposes, this is the ideal case when the goal is to “nail down” x_1 and x_2 .

2. No solution.

The two lines are parallel and distinct. Each individual equation has infinitely many solutions, but there is no point satisfying both equations simultaneously.

3. Infinitely many solutions.

The two equations define the same line. The second equation is redundant or artificial, so the system effectively contains only one independent equation.

The instructor emphasizes that vector-space methods will show the same trichotomy for arbitrary numbers of equations and arbitrary numbers of unknowns:

- Unique solution.
- No solution.
- Infinitely many solutions.

The course will then characterize these cases using properties of A and b .

9. Source And Coverage Note

Source transcript used:

- `C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture3_corrected.md`

Coverage:

- These notes follow only Lecture 3 and preserve the chronological development of the transcript.
- Included topics: recap of linear, affine, and convex combinations; Euclidean norm; distance; norm properties; two-dimensional and n -dimensional Euclidean inner products; induced norm; angle formula; orthogonality; Cauchy-Schwarz inequality and proof idea; inner product properties; positive definiteness and indefinite-space remarks; origin-centered and shifted hyperplanes and half-spaces; projection preview; neuron half-space interpretation; matrix multiplication; matrix form of linear systems; and geometric solution-set cases.
- Explicit transcript exam markers were incorporated in the relevant sections.
- No other lecture transcript was processed for these notes.

Lecture 04 Notes

Recap: Angles, Cosine Distance, and Orthogonality

The lecture begins by continuing the discussion of angles between vectors. Even in high-dimensional spaces, the angle between two vectors can be defined through the inner product:

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\| \|y\|}$$

The instructor emphasized that this quantity is used in machine learning to judge how close or similar vectors are. This use is connected to what is often called cosine distance or cosine similarity, depending on whether one uses the cosine value directly or converts it into a distance-like measure.

The expression above is guaranteed to lie between -1 and 1 . The reason is the Cauchy-Schwarz inequality, also referred to in the transcript as the Schwarz inequality:

$$|\langle x, y \rangle| \leq \|x\| \|y\|$$

Because of this inequality, the normalized inner product can legitimately be interpreted as a cosine value.

Exam-related remark: the instructor said a proof of this inequality was already given, and an alternative proof would appear in the homework. The purpose of the proof is to justify why the normalized inner product is always in the valid range for a cosine.

This recap also connects to orthogonality. In ordinary two- or three-dimensional geometry, perpendicular vectors have angle 90° . The generalized definition used in vector spaces is simpler:

$$x \perp y \quad \text{if and only if} \quad \langle x, y \rangle = 0$$

So two vectors are called orthogonal when their inner product is zero. This definition extends the familiar geometric notion of perpendicularity into high-dimensional spaces.

The instructor also reminded the class that hyperplanes and half-spaces can be defined using inner products. These were part of the previous lecture's conclusion and will remain important for understanding systems of linear equations geometrically.

Matrix Multiplication as a Collection of Inner Products

The lecture then returns to matrix multiplication.

The instructor described the product of two matrices as a “bag of inner products.” Each entry of a matrix product is computed by taking the inner product of:

- one row of the first matrix
- one column of the second matrix

Thus matrix multiplication can be understood as computing all possible row-column inner products.

This viewpoint is important because it connects matrix multiplication directly to geometry. Matrix products are not just formal algebraic rules; they organize many inner products into one object.

Systems of Linear Equations and the Form $Ax = b$

Matrix multiplication is used to write systems of linear equations compactly. A collection of equations can be represented as:

$$Ax = b$$

Here:

- A is the coefficient matrix.
- x is the vector of unknown variables.
- b is the right-hand side vector.

The instructor described this as isolating the unknown variables into a vector or matrix and putting the known coefficients into another matrix. This compact form is convenient for both analysis and computation.

The system $Ax = b$ will be used as an excuse or motivation to introduce several important concepts in linear algebra. These concepts are useful not only for solving systems of equations, but also for many other purposes in engineering, optimization, signal processing, and machine learning.

The main questions are:

- When does a solution exist?
- When is the solution unique?
- Can there be no solution?
- Can there be a finite number of solutions?
- Can there be infinitely many solutions?
- Can the answer be characterized from A alone, or does it depend on the pair (A, b) ?

The instructor emphasized that solving systems of linear equations appears either explicitly or implicitly in many applications.

Geometric Picture for Two Equations in Two Unknowns

For a two-dimensional example, each equation defines a hyperplane. In two dimensions, a hyperplane is a line.

For example, an equation can be viewed as an inner product between a coefficient vector and the unknown vector (x_1, x_2) , equal to a constant. The solution set of one equation is the set of all x -vectors satisfying that equality. Geometrically, this is a line.

For two equations in two unknowns, the solution set of the full system is the intersection of two lines, because both equations must be satisfied at the same time.

The instructor described three possible cases:

1. The two lines intersect at exactly one point.
 - There is a unique solution.
2. The two equations look different but define the same line.
 - The lines overlap.
 - There are infinitely many solutions.
3. The two lines are parallel and distinct.
 - They cannot be satisfied at the same time.
 - This means there is no solution.
 - Note: the transcript's wording at this point appears to say "infinitely many solutions" after saying the lines cannot be satisfied simultaneously. The surrounding context indicates the intended conclusion is no solution.

Likely exam topic: the instructor said that these same types of conclusions for the two-variable, two-equation case will be extended to arbitrary numbers of equations and unknowns. The main tool for that extension will be vector spaces.

Why Vector Spaces Are Introduced

Vector spaces will be the major tool for analyzing existence and uniqueness of solutions to systems such as:

$$Ax = b$$

They are also useful beyond this specific problem. The instructor repeatedly emphasized that the system of linear equations is a motivation for introducing concepts that have much wider use.

Vector spaces allow geometric ideas from two- and three-dimensional vectors to be generalized to many other kinds of objects.

The examples mentioned include:

- ordinary position vectors in $\mathbb{R}^2$, $\mathbb{R}^3$, and higher dimensions
- student grade vectors
- matrices
- polynomials
- functions
- random variables
- random processes
- discrete-time sequences

The instructor gave a grade-vector example:

- If a student has two grade components, such as midterm and homework, this can be viewed as a point in a two-dimensional space.
- If a student has three components, this can be viewed as a point in three-dimensional space.
- If a project component is added, the vector becomes four-dimensional.

In four dimensions, we cannot draw the object directly, but we can still talk about its projections or shadows in lower-dimensional spaces. We can also still discuss hyperplanes, half-spaces, inner products, norms, distances, and angles.

The major conceptual point is that geometry is not limited to objects we can draw. Once a vector space structure exists, geometric language can be transported to much more abstract objects.

General Systems $Ax = b$: Role of m , n , A , and b

The instructor stated the general setup as follows:

- A has m rows.
- Each row corresponds to one equation.
- A has n columns.
- The number n is the number of unknown variables.

The basic questions are:

- Given A and b , is there a solution?
- If there is a solution, is it unique?
- Are there many solutions?
- If only A is given, can we say whether every b has a solution?
- For a fixed A , can some choices of b have solutions while others do not?
- For a fixed A , if a solution exists, is it always unique?

The instructor said the course will characterize existence and uniqueness in terms of:

- the pair (A, b)
- and, when possible, properties of A alone

The relevant properties will involve the dimensions of A , and relationships among its rows and columns.

Special forms of A , including symmetric matrices, will also be studied. Symmetric matrices are important because they help develop high-dimensional versions of quadratic functions, which are key functions in optimization.

Instructor clarification: when a student asked about a random matrix, the instructor clarified that the immediate analysis is not mainly about probabilistic methods. The course first treats the general matrix A , characterizing existence and uniqueness through dimensions and row/column relationships, then studies special matrix forms and how those forms affect solutions.

Exam-related remark: the instructor said a significant amount of time will be spent on symmetric matrices because of their role in quadratic functions and optimization.

Definition: Vector Space

A vector space consists of four ingredients:

1. A set of vectors.
2. A set of scalars, usually a field such as $\mathbb{R}$ or $\mathbb{C}$.
3. A vector addition operation.
4. A scalar multiplication operation.

The set of vectors is the main object of interest. The instructor called it the “star of the show.”

Vector addition takes two vectors and returns another vector:

$$+ : V \times V \rightarrow V$$

Scalar multiplication takes one scalar and one vector and returns another vector:

$$\cdot : F \times V \rightarrow V$$

These operations must satisfy certain properties. Some of the properties are closure conditions: performing the operations should not take us outside the original vector set. Other properties are compatibility conditions with the scalar field’s own addition and multiplication.

Vector Space Axioms Discussed in Lecture

Let x, y, z be vectors from the vector set V , and let a, b be scalars from the scalar field.

The lecture discussed the following requirements.

Closure Under Vector Addition

For any $x, y \in V$,

$$x + y \in V$$

Adding two vectors should produce another vector in the same vector set.

Additive Identity

There must be a zero vector $0 \in V$ such that:

$$x + 0 = x$$

and, conceptually, also:

$$0 + x = x$$

The zero vector is the additive identity.

Commutativity of Vector Addition

Vector addition should not depend on the order:

$$x + y = y + x$$

The instructor described this as symmetry of the addition operation.

Additive Inverses

For every vector x , there must be a vector $-x$ such that:

$$x + (-x) = 0$$

The vector $-x$ is the negative of x .

Associativity of Vector Addition

When adding three vectors, it should not matter which pair is added first:

$$(x + y) + z = x + (y + z)$$

The instructor noted that vector addition is a binary operation: it takes two inputs and produces one output. Associativity makes repeated addition well-defined.

Instructor remark: these addition properties are essentially the group-like part of the vector space definition for the vector set together with vector addition.

Closure Under Scalar Multiplication

For any scalar a and vector x ,

$$ax \in V$$

Scalar multiplication should produce a vector in the same vector set.

Compatibility Between Scalar Multiplication and Field Multiplication

The lecture emphasized the property:

$$a(bx) = (ab)x$$

The left-hand side applies scalar multiplication twice. The right-hand side first multiplies a and b inside the scalar field, then performs scalar multiplication once. These must agree.

This is a compatibility requirement between multiplication in the scalar field and scalar multiplication in the vector space.

Multiplicative Identity of the Scalar Field

The scalar field has a multiplicative identity, 1. It must satisfy:

$$1x = x$$

for every vector x .

Zero Scalar Produces the Zero Vector

If the scalar zero is used in scalar multiplication, the result should be the additive identity of the vector space:

$$0x = 0$$

The zero on the left is the scalar zero. The zero on the right is the zero vector.

Linear Combinations and Closure

The instructor stressed that these properties imply closure under linear combinations.

If scaled versions of vectors stay inside V , and sums of vectors stay inside V , then any expression of the form:

$$a_1x_1 + a_2x_2 + \cdots + a_kx_k$$

also stays inside V , as long as the x_i are in V and the a_i are scalars.

This is one of the most important consequences of the vector space structure:

A vector space is closed under linear combinations.

This idea later becomes central to span, subspaces, basis, and the analysis of $Ax = b$.

Example: $\mathbb{R}^3$

The first example is ordinary three-dimensional real space:

$$\mathbb{R}^3$$

A vector has three components:

$$x = (x_1, x_2, x_3)$$

Vector addition is defined elementwise:

$$x + y = (x_1 + y_1, x_2 + y_2, x_3 + y_3)$$

Scalar multiplication is also elementwise:

$$ax = (ax_1, ax_2, ax_3)$$

The zero vector is:

$$0 = (0, 0, 0)$$

All vector space properties are satisfied in the usual way.

Example: $\mathbb{R}^n$

The same construction extends from $\mathbb{R}^3$ to $\mathbb{R}^n$.

Vectors have n real components. Addition and scalar multiplication are defined component by component. The zero vector has all entries equal to zero.

This is the natural finite-dimensional generalization of two- and three-dimensional Euclidean space.

Example: Complex Vector Spaces

Complex vectors are common in electrical engineering.

A vector in $\mathbb{C}^n$ has complex entries:

$$x = (x_1, \dots, x_n)$$

where each x_i is a complex number.

The scalar field is $\mathbb{C}$, the complex numbers. The instructor explicitly noted that the notation for complex numbers is different from $\mathbb{R}$, the real numbers.

Vector addition is still elementwise:

$$x + y = (x_1 + y_1, \dots, x_n + y_n)$$

Scalar multiplication uses complex multiplication:

$$ax = (ax_1, \dots, ax_n)$$

This is a direct extension of real vector spaces, but over the complex scalar field.

Example: Matrix Spaces

Another vector space is the set of $m \times n$ matrices.

In this setting, each matrix is treated as one vector in an abstract sense. The instructor emphasized that this may feel different because the object does not look like a column vector, but the abstract vector-space definition allows it.

An $m \times n$ matrix can be regarded as one point in a high-dimensional space.

The operations are:

- usual matrix addition
- multiplication of a matrix by a scalar

If two $m \times n$ matrices are added, the result is still an $m \times n$ matrix. If an $m \times n$ matrix is scaled by a scalar, the result is still an $m \times n$ matrix.

Thus closure is satisfied, and the remaining vector space properties follow from ordinary arithmetic on matrix entries.

This example is important because it allows us to discuss:

- distances between matrices
- angles between matrices
- orthogonality of matrices
- subspaces of matrix spaces

These ideas appear later in the lecture through symmetric, trace-zero, and orthogonal matrices.

Example: Polynomial Spaces

The instructor next discussed polynomial spaces, which can be confusing at first because polynomials may be nonlinear functions of their argument.

The vector space is the set of polynomials of degree at most n . A typical polynomial is:

$$p(x) = a_0 + a_1x + \cdots + a_nx^n$$

The coefficients $a_0, \dots, a_n$ define the polynomial. Therefore, a polynomial of degree at most n has $n + 1$ coefficients.

Polynomial addition is defined by adding coefficients of matching powers. If:

$$p(x) = a_0 + a_1x + \cdots + a_nx^n$$

and:

$$q(x) = b_0 + b_1x + \cdots + b_nx^n$$

then:

$$(p + q)(x) = (a_0 + b_0) + (a_1 + b_1)x + \cdots + (a_n + b_n)x^n$$

Scalar multiplication scales each coefficient:

$$(cp)(x) = ca_0 + ca_1x + \cdots + ca_nx^n$$

The instructor pointed out that there is a one-to-one connection between such polynomials and vectors of coefficients in $\mathbb{R}^{n+1}$. Under this connection, polynomial addition corresponds to vector addition of coefficient vectors.

Likely exam topic: once polynomial spaces are treated as vector spaces, we can define inner products, norms, and orthogonality for polynomials. We can speak of one polynomial being orthogonal to another polynomial.

The key warning is that the polynomial may be nonlinear in the variable x , but the vector space operations act on the polynomial as an object, or equivalently on its coefficient vector.

Example: Discrete-Time Sequences

The lecture also introduced the set of discrete-time sequences as a vector space.

A discrete-time sequence is a collection of values indexed by integers, as in signals and systems notation. The instructor referenced Oppenheim's notation for such sequences.

A real-valued sequence may be written conceptually as:

$$\{x[n]\}_{n \in \mathbb{Z}}$$

Complex-valued sequences can also be used.

The instructor said one may imagine such a sequence as an infinite-dimensional vector containing infinitely many samples. However, the axiomatic vector space definition is useful because we do not need to explicitly force every object into a finite column-vector representation.

With suitable addition and scalar multiplication:

- adding two sequences means adding corresponding samples
- scaling a sequence means scaling every sample

Then each sequence can be viewed as a point in a high-dimensional or infinite-dimensional space.

This allows geometric language to be used for sequences. For example, we can later define when two discrete-time sequences are orthogonal.

Example: Function Spaces

Function spaces were also mentioned.

For example, one can consider functions defined on an interval $[a, b]$ that are square-integrable on that interval.

Function addition is pointwise:

$$(f + g)(t) = f(t) + g(t)$$

Scalar multiplication is also pointwise:

$$(cf)(t) = cf(t)$$

The instructor's point was that vector space language is broad enough to include functions, not only finite-dimensional coordinate vectors.

Example: Zero-Mean Random Variables

Another important example is the set of zero-mean random variables.

If X and Y are zero-mean random variables, then:

$$E[X] = 0, \quad E[Y] = 0$$

Their sum is also zero-mean:

$$E[X + Y] = E[X] + E[Y] = 0$$

Scaling also preserves zero mean:

$$E[aX] = aE[X] = 0$$

Therefore, zero-mean random variables form a vector space under the usual addition and scalar multiplication of random variables.

The instructor said random variables, random vectors, and random processes can be viewed through vector space ideas.

Inner Products for Random Variables: Correlation and Orthogonality

The instructor then highlighted an important leap: once random variables are viewed as vectors, one can define inner products on them.

For two random variables X and Y , correlation can be interpreted as an inner product. In this setting, the condition that the inner product is zero corresponds to the random variables being uncorrelated.

For zero-mean real random variables, the correlation inner product can be thought of as:

$$\langle X, Y \rangle = E[XY]$$

So the orthogonality condition becomes:

$$E[XY] = 0$$

The instructor explicitly distinguished this from independence:

- Zero correlation means uncorrelated.
- It does not necessarily mean independent.

Under this interpretation, uncorrelated random variables can be described geometrically as orthogonal.

Exam-related remark: this point was marked as important. The instructor said this viewpoint is very useful in estimation theory and signal processing.

Applications mentioned:

- Kalman filters
- Wiener filters
- estimation theory
- signal processing

These are built on the basic terminology of orthogonality in spaces of random variables.

Subspaces: Definition and Motivation

The next major concept is subspace.

Exam-related remark: the instructor said subspaces will be a major tool for addressing existence and uniqueness problems for systems of linear equations. Subspaces are also used in algorithmic constructions and analysis.

Let V be a vector space. A subset $S \subseteq V$ is called a subspace of V if S itself forms a vector space using the same vector addition and scalar multiplication as V .

The key requirements are:

- S is a subset of V .
- S contains the zero vector.
- S is closed under vector addition.

- S is closed under scalar multiplication.

The instructor phrased the formal definition as: S is a subset of V , and S itself can be used to construct a vector space with the same operations.

Subspace Examples and Non-Examples in $\mathbb{R}^2$

The instructor used several subsets of $\mathbb{R}^2$ to clarify what is and is not a subspace.

A Subset That Does Not Contain Zero

The first pictured subset was some region in the plane. It was a subset of $\mathbb{R}^2$, but it was not a subspace.

Reasons:

- It did not contain the zero vector.
- It was not closed under vector addition.
- Adding two vectors in the subset could produce a vector outside the subset.

The instructor emphasized that being a subset is not enough. A subspace must satisfy the vector space requirements.

First and Third Quadrants Together

Another example was the union of the first and third quadrants.

This example is misleading because it contains the origin. It also seems partly compatible with scalar multiplication:

- Positive scaling keeps a vector in the same quadrant.
- Negative scaling can move a vector from the first quadrant to the third, or from the third to the first.

Also:

- the sum of two vectors in the first quadrant remains in the first quadrant
- the sum of two vectors in the third quadrant remains in the third quadrant

However, if one vector is taken from the first quadrant and another from the third quadrant, their sum can land outside the union of the first and third quadrants.

Therefore this set is not closed under vector addition, so it is not a subspace.

A Line Not Passing Through the Origin

A line in $\mathbb{R}^2$ that does not pass through the origin is not a subspace.

Reasons:

- It does not contain the zero vector.
- Adding two vectors on the line can produce a vector outside the line.

The instructor warned that this object looks like a linear object geometrically, but it is not a linear subspace unless it passes through the origin.

Lines Through the Origin

Lines through the origin are subspaces of $\mathbb{R}^2$.

Reasons:

- They contain the zero vector.
- The sum of any two vectors on the same line through the origin stays on that line.
- Any scalar multiple of a vector on the line stays on that line.

The instructor described the vectors as collinear.

Trivial Subspaces of $\mathbb{R}^2$

Two trivial subspaces were also mentioned:

- the entire ambient space $\mathbb{R}^2$
- the set containing only the origin, $\{0\}$

Both satisfy the subspace requirements.

Subspaces of $\mathbb{R}^3$

For $\mathbb{R}^3$, the instructor asked what the subspaces are.

The examples given were:

- planes through the origin
- lines through the origin
- the origin alone
- the entire space $\mathbb{R}^3$

A student suggested something like $\mathbb{R}^2$, and the instructor clarified that inside $\mathbb{R}^3$, the relevant geometric object is a plane through the origin.

The instructor said planes through the origin satisfy the main condition: all possible linear combinations of vectors in the plane remain inside the plane.

Matrix Subspaces: Symmetric Matrices

The lecture then moved to more interesting examples inside matrix spaces.

The ambient vector space is the set of square real matrices, for example $m \times m$ real matrices. The instructor noted that, strictly speaking, one should specify the vector set, scalar set, and operations, but in common mathematical speech the operations are often implied.

Define:

$$S_1 = \{X : X^T = X\}$$

This is the set of real symmetric matrices.

The transpose operation swaps rows and columns. A matrix is symmetric when it is equal to its transpose.

The instructor checked whether this set is a subspace.

Zero Matrix

The zero matrix is symmetric:

$$0^T = 0$$

So the origin of the ambient matrix space is included.

Closure Under Scalar Multiplication

If X is symmetric, then scaling every entry of X by a scalar preserves symmetry:

$$(\alpha X)^T = \alpha X^T = \alpha X$$

Thus scalar multiplication does not break symmetry.

Closure Under Addition

If X and Y are symmetric, then:

$$(X + Y)^T = X^T + Y^T = X + Y$$

Therefore the sum is symmetric.

Conclusion:

$$S_1$$

is a subspace of the square matrix space.

The instructor said one may imagine this subspace as a plane or slice in a high-dimensional geometric region, although such geometric imagination can sometimes mislead.

Matrix Operation: Trace

The instructor then introduced the trace of a square matrix and called it a very important, though simple, operation.

For a square matrix X , the trace is the sum of its diagonal elements:

$$\text{tr}(X) = \sum_{i=1}^m x_{ii}$$

For example, if:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots \\ x_{21} & x_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

then:

$$\text{tr}(X) = x_{11} + x_{22} + \cdots + x_{mm}$$

The trace ignores off-diagonal elements.

Exam-related remark: trace was explicitly flagged as important.

Matrix Subspaces: Zero-Trace Matrices

Define:

$$S_2 = \{X : \text{tr}(X) = 0\}$$

This is the set of square matrices whose diagonal elements sum to zero.

The instructor checked whether this is a subspace.

Zero Matrix

The zero matrix has trace zero:

$$\text{tr}(0) = 0$$

So the origin is included.

Closure Under Scalar Multiplication

If $\text{tr}(X) = 0$, then:

$$\text{tr}(\alpha X) = \alpha \text{tr}(X) = \alpha \cdot 0 = 0$$

So scaling preserves the zero-trace condition.

Closure Under Addition

If $\text{tr}(X) = 0$ and $\text{tr}(Y) = 0$, then:

$$\text{tr}(X + Y) = \text{tr}(X) + \text{tr}(Y) = 0 + 0 = 0$$

So addition preserves the zero-trace condition.

The instructor emphasized that trace is a linear operator:

$$\text{tr}(X + Y) = \text{tr}(X) + \text{tr}(Y)$$

and:

$$\text{tr}(\alpha X) = \alpha \text{tr}(X)$$

Conclusion:

$$S_2$$

is a subspace of the square matrix space.

The instructor again described it geometrically as a plane through the origin in a high-dimensional space.

Degrees of Freedom and Dimension Preview

The lecture used matrix examples to preview the idea of dimension.

An arbitrary $m \times m$ matrix has:

$$m^2$$

free parameters.

For symmetric $m \times m$ matrices:

- all m diagonal entries can be chosen freely
- the off-diagonal entries are mirrored across the diagonal
- there are $m^2 - m$ off-diagonal entries total
- half of those, $(m^2 - m)/2$, can be chosen freely

So the number of free parameters is:

$$\frac{m^2 - m}{2} + m = \frac{m^2 + m}{2}$$

For zero-trace $m \times m$ matrices:

- the full matrix would have m^2 parameters
- the trace condition imposes one linear constraint on the diagonal entries
- $m - 1$ diagonal entries can be chosen freely, and the final diagonal entry must make the trace equal zero

Therefore the number of degrees of freedom is:

$$m^2 - 1$$

The instructor said this means the zero-trace subspace is one dimension less than the full square-matrix space. It is a hyperplane.

Zero-Trace Matrices as Orthogonal to the Identity

The instructor previewed a matrix inner product that will be defined later:

$$\langle A, B \rangle = \text{tr}(B^T A)$$

Using this inner product, the zero-trace condition can be viewed as an orthogonality condition involving the identity matrix:

$$\text{tr}(X) = \langle X, I \rangle = 0$$

Thus zero-trace matrices can be thought of as all matrices orthogonal to the identity matrix, according to the matrix inner product.

The instructor was careful to note that this matrix inner product had not yet been formally defined in the lecture.

Exam-related remark: this was flagged as important because it connects trace, inner products, orthogonality, and hyperplanes.

Hyperplanes Through the Origin Versus Shifted Hyperplanes

The instructor asked whether every set defined by a condition equal to zero is a subspace. In the context being discussed, the idea was that a linear condition or inner-product condition equal to zero, such as trace zero, gives a subspace because it passes through the origin and is closed under linear combinations.

If the condition is changed from zero to a nonzero number, the result is generally not a subspace.

For example:

$$\{X : \text{tr}(X) = 1\}$$

is not a subspace.

Reasons:

- The zero matrix is not included because its trace is zero, not one.
- If two matrices each have trace one, their sum has trace two.
- Scalar multiplication also does not preserve trace one.

However, this set can still be viewed geometrically as a hyperplane. It is just a hyperplane that does not pass through the origin.

The instructor also described a shifted version of the zero-trace subspace. If S_2 is the trace-zero subspace and Z has nonzero trace, then:

$$S_3 = Z + S_2$$

is a shifted subset of the square matrix space. It is not a subspace, because it is moved away from the origin and its elements have a nonzero trace determined by the shift.

More explicitly, every element of S_3 has trace $\text{tr}(Z)$, since adding an element of S_2 contributes zero trace. Thus S_3 is a translated hyperplane, not a subspace through the origin.

Matrix Subspaces and Non-Subspaces: Real Orthogonal Matrices

The instructor next introduced real orthogonal matrices.

Define:

$$S_4 = \{X : X^T X = I\}$$

Such matrices are called real orthogonal matrices.

The ambient space is again the set of $m \times m$ real matrices. The instructor mentioned that complex extensions will be considered later.

The condition:

$$X^T X = I$$

has a geometric interpretation using matrix multiplication as inner products.

Recall that matrix multiplication forms entries by taking inner products of rows and columns. In $X^T X$, the rows of X^T are the columns of X . Therefore, the entries of $X^T X$ are inner products between columns of X .

The identity matrix condition means:

- the inner product of a column with itself is 1

- the inner product of two different columns is 0

Therefore, the columns of X are mutually orthogonal and each has norm 1. For square real orthogonal matrices, this also implies corresponding orthogonality properties for rows.

Exam-related remark: the instructor explicitly connected this condition back to matrix multiplication as inner products.

Orthogonal Matrices Are Not a Subspace

The set of real orthogonal matrices is not a subspace.

The quickest reason is that the zero matrix is not included:

$$0^T 0 = 0 \neq I$$

Equivalently, the zero matrix's columns do not have norm one.

So the origin of the ambient matrix space is not in the set, and the set cannot be a subspace.

The instructor emphasized that orthogonal matrices are still important even though they are not a subspace.

Orthogonal Matrices, Circles, Manifolds, and Closest Points

The instructor used the orthogonal-matrix condition to motivate future distance and projection problems.

If the object were a single vector $x = (x_1, x_2)$ satisfying:

$$x^T x = 1$$

then the set would be the unit circle.

Given a point outside or inside the circle, the closest point on the circle in Euclidean distance is found by drawing the line through the point and the origin. The closest point is where this line intersects the circle.

The instructor then generalized this idea:

- The set of orthogonal matrices is not a linear subspace.
- It behaves more like a curved set, such as a sphere or manifold.
- The transcript uses wording like “hyper spherical thing” and “manifold.”

Even though it is not a subspace, we can still ask optimization questions about it:

- Given a matrix Y not in the set, what is the closest matrix in the set?
- What does “closest” mean?
- Which distance measure are we using?

The instructor said that there are infinitely many ways to define distance between matrices. Some distance measures are algebraically attractive because they lead to closed-form solutions.

The practical problem is:

Given an arbitrary matrix, find a matrix with orthogonal columns that is as close as possible to it.

The instructor stressed that the goal is not to find just any orthogonal-column matrix, but one close to the original matrix.

Applications mentioned include:

- orthogonalization-like procedures (the transcript wording appears as “auto organization”)
- independent component analysis
- machine learning gradient-search-related algorithms

This topic will be revisited after defining appropriate notions of matrix distance and closeness.

Subspace Example: Causal Discrete-Time Sequences

The lecture returned to sequence spaces for another subspace example.

Consider two-sided discrete-time sequences. These have values indexed over both positive and negative integers.

A causal sequence is one whose values are zero at all negative indices:

$$x[n] = 0 \quad \text{for } n < 0$$

The instructor said such sequences correspond to causal impulse responses of causal systems.

The set of causal sequences is a subset of the set of all two-sided sequences. It is a subspace.

Reason:

- If two causal sequences are added, their negative-index entries remain zero, because $0 + 0 = 0$.
- If a causal sequence is multiplied by a scalar, its negative-index entries remain zero, because $\alpha \cdot 0 = 0$.

Therefore causal sequences are closed under addition and scalar multiplication.

Instructor remark: students missing signals and systems background should review causal systems, causal impulse responses, impulse response, and frequency response, because these ideas will be needed later.

Dimension, Span, Basis, and Linear Independence: Motivation

After discussing degrees of freedom in matrix subspaces, the instructor introduced the next formal topic: dimension.

Dimension is meant to answer questions such as:

- How large is a vector space?
- How large is a subspace?
- How many degrees of freedom are needed to describe its elements?

The instructor said the formal discussion of dimension will require:

- span
- basis
- linear independence

Span is introduced first. Basis is the reverse direction of span, and basis requires linear independence.

Definition: Span

Given a set $A \subseteq V$, the span of A is the set of all possible linear combinations of elements of A .

The set A may be:

- finite
- large
- infinite

To form the span, one takes arbitrary numbers of elements from A , multiplies them by arbitrary scalars, and sums them.

For example, if $A = \{x, y\}$, then:

$$\text{span}(A) = \{ax + by : a, b \text{ are scalars}\}$$

If x and y are suitable non-collinear vectors in three-dimensional space, all their linear combinations form the plane containing them.

The instructor described span as a kind of machine:

1. You give the machine a set of vectors.
2. It generates all possible linear combinations.
3. The output is a subspace containing the original set.

This description reinforces the idea that span produces a vector space or subspace from a given collection of vectors.

Span Example: Vectors in a Plane

The instructor described a set of vectors lying in a plane such as the $z = 0$ plane.

The transcript's example appears as vectors like 100 and 110, which can be read as coordinate vectors such as:

$$(1, 0, 0), \quad (1, 1, 0)$$

Their span is the plane containing all linear combinations of them, namely a plane with zero third coordinate, assuming the vectors are independent within that plane.

The instructor then considered adding another vector that already lies in the same span.

If the new vector can be generated as a linear combination of the previous vectors, adding it does not change the span. It does not bring a new dimension or new information.

The instructor connected this to random variables as well: if a new vector or random variable is already derivable from existing ones, it does not add a new independent form of information.

Alternative Definition of Span

The instructor gave another definition:

The span of a set A is the smallest subspace containing A .

“Smallest” means that if a subspace contains A , then the span is contained in that subspace. There is no smaller subspace that still contains all elements of A .

This definition is equivalent to the “all possible linear combinations” definition.

Basis as the Reverse Problem of Span

The instructor described basis as the reverse of the span problem.

For span:

- You are given a set of vectors.
- You generate the subspace they produce.

For basis:

- You are given a vector space or subspace.
- You try to find a set of vectors that spans it.

The goal is to find a minimal set of vectors that can represent or generate the whole space.

The instructor used the plane example:

- If a plane is given, one can communicate the plane by giving two suitable spanning vectors p_1 and p_2 .
- One could also give p_1, p_2, p_3 , but if p_3 is already generated by p_1 and p_2 , then this includes redundant information.

A basis is a spanning set with no redundant vector. No element of a basis can be written as a linear combination of the others.

This connects directly to linear independence.

Dictionaries Versus Bases

The instructor introduced the term dictionary for a spanning set that may include redundant vectors.

If p_1, p_2, p_3 span the same space as p_1, p_2 , and p_3 can be derived from the others, then:

- $\{p_1, p_2\}$ may be a basis
- $\{p_1, p_2, p_3\}$ is not a basis because it is not minimal
- the larger redundant set can be called a dictionary

The instructor noted that dictionaries are useful in machine learning. Sometimes it is useful to represent a space with more than the minimal number of elements.

However, the strict definition of a basis involves the minimum number of elements needed to represent the space, with no element derivable from the others.

This topic will continue in the next lecture.

Instructor Study Advice

At the end, the instructor gave two study recommendations.

First, students who are missing signals and systems background should review:

- causal systems
- causal impulse responses
- impulse response
- frequency response

The instructor said these will be needed in the future.

Second, students who are missing linear algebra background should begin reading a linear algebra book in some detail. The instructor mentioned Strang's book and advised reading the basic explanatory text, not only the equations.

Exam-Relevant Items Mentioned in the Transcript

The transcript explicitly marks or strongly signals the following as exam-relevant:

- Cauchy-Schwarz inequality guarantees the normalized inner product lies in $[-1, 1]$, justifying the cosine-angle definition.
- Orthogonality is generalized by the condition $\langle x, y \rangle = 0$.
- Systems of linear equations can be analyzed geometrically through hyperplanes and their intersections.
- The two-equation, two-unknown cases generalize to arbitrary numbers of equations and unknowns.
- Vector spaces and subspaces are likely exam topics.
- Span, basis, and linear independence are likely exam topics.
- Symmetric matrices are important for high-dimensional quadratic functions and optimization.
- Random variables can be treated as vectors, with correlation acting like an inner product.

- Uncorrelated random variables correspond to orthogonality in that vector-space interpretation.
- Subspaces are a major tool for existence and uniqueness of solutions to $Ax = b$.
- Trace is an important matrix operation.
- Zero-trace matrices form a subspace and can be viewed as matrices orthogonal to the identity under a matrix inner product.
- Orthogonal matrices satisfy $X^T X = I$, meaning their columns are mutually orthogonal and unit norm.
- Orthogonal matrices are important but do not form a subspace.

Source and Coverage Note

Source: `corrected/lecture4_corrected.md`.

Coverage: These notes use only the specified Lecture 4 corrected transcript. They preserve the lecture's chronological flow, examples, instructor remarks, exam flags, proof ideas, warnings, and relationships among concepts. No other lecture transcript was processed.

Lecture 05 Notes

Main Theme: Existence, Uniqueness, and Vector Spaces

Likely exam topic The lecture frames **existence and uniqueness of solutions** for linear systems as the main problem. The instructor uses that problem as the reason to introduce and use vector space ideas.

The guiding question is:

Given a linear system $Ax = b$:

- Does a solution x exist?
- If a solution exists, is it unique?
- If not unique, how many solutions or what kind of solution set should we expect?

The instructor emphasizes that the answer will come from vector spaces and from two different ways of looking at matrix-vector multiplication.

Review: Vector Spaces

[Exam note] A **vector space** was reviewed as a mathematical object with four components:

- A set of vectors.
- A set of scalars.
- Vector addition.
- Scalar multiplication.

These objects must satisfy a collection of vector space properties. The instructor did not relist all axioms here, but emphasized the most important conclusion for the current lecture:

Linear combinations of vectors in a vector space stay inside the vector set.

In other words, if vectors belong to the vector space, then any expression of the form

$$\alpha_1 v_1 + \alpha_2 v_2 + \cdots + \alpha_k v_k$$

also belongs to the same vector space.

This closure under linear combinations is the property that connects vector spaces to span, basis, column space, and the existence of solutions to $Ax = b$.

Review: Subspaces

[Exam note] A **subspace** is a subset of a vector space that is itself a vector space.

The lecture recalled geometric examples:

- In $\mathbb{R}^2$, subspaces include the origin, lines through the origin, and $\mathbb{R}^2$ itself.
- In $\mathbb{R}^3$, the variety of subspaces increases: examples include lines through the origin, planes through the origin, the origin, and the full space.

Important warning:

A set may be a subset of a vector space without being a subspace. To be a subspace, it must satisfy the vector space requirements, especially closure and inclusion of the zero vector.

Matrix Subspace Examples

[Exam note] The lecture reviewed examples in spaces of square matrices.

Symmetric Matrices

The set of symmetric matrices forms a subspace of the square matrix space.

The relevant idea is that linear combinations of symmetric matrices remain symmetric.

Zero-Trace Matrices

The set of square matrices with trace zero forms a subspace.

The trace condition is compatible with linear combinations:

$$\text{tr}(\alpha X + \beta Y) = \alpha \text{tr}(X) + \beta \text{tr}(Y).$$

So if both X and Y have zero trace, then any linear combination also has zero trace.

Matrices Orthogonal to a Fixed Matrix

The instructor mentioned matrices X that make a zero trace expression when multiplied by some A^T . This is pointing toward a matrix inner product:

$$\langle A, X \rangle = \text{tr}(A^T X).$$

The course had not yet formally defined matrix inner products at this point, but the instructor explained the interpretation:

If

$$\text{tr}(A^T X) = 0,$$

then X is orthogonal to A under that matrix inner product.

This gives a nontrivial example of a subspace: all matrices orthogonal to a fixed matrix A .

Orthogonal Matrices Are Not a Subspace

The set of real orthogonal matrices satisfies

$$XX^T = I$$

or, depending on convention, $X^T X = I$.

This set is a subset of the square matrix space, but it is **not** a subspace.

Main reason given:

The zero matrix is not included, because $0 \cdot 0^T = 0$, not I .

So the set fails a basic requirement for being a vector space/subspace.

Span

The instructor reviewed the **span** of a set.

Mechanistic definition:

Given a set of vectors A , take all possible linear combinations of those vectors. The set of all those combinations is the span of A .

$$\text{span}(A) = \{\alpha_1 a_1 + \cdots + \alpha_k a_k : a_i \in A, \alpha_i \text{ scalars}\}.$$

Conceptual definition:

$$\text{span}(A)$$

is the smallest vector space that contains A .

Equivalently, no proper smaller vector space can contain all of A while being strictly smaller than $\text{span}(A)$.

The instructor also phrased this as span taking a smaller set of vectors and generating a bigger set that is itself a vector space, because all possible linear combinations are included.

This idea is important because the column space of a matrix will later be defined as the span of its columns.

From Span to Basis and Dictionary

[Exam note] The instructor shifted from the span question to the opposite question.

Span starts with a set of vectors and asks:

What vector space is generated by this set?

The opposite question is:

Given a vector space, what set of vectors can span it?

This motivates **basis** and **dictionary**.

The instructor's motivation was representation: even if a vector space contains infinitely many vectors, one may want to represent or parameterize the whole space using only a small generating set.

The instructor used the example of a plane through the origin:

- The plane can be represented as all linear combinations of two non-collinear vectors p_1 and p_2 .
- The same plane can also be represented as all linear combinations of three vectors p_1, p_2, p_3 , if all three lie in the plane and span it.

The set $\{p_1, p_2\}$ is minimal in the sense that two non-collinear vectors are enough to generate the plane.

The set $\{p_1, p_2, p_3\}$ is not minimal if one vector can be written as a linear combination of the others. For example,

$$p_3 = \alpha_1 p_1 + \alpha_2 p_2.$$

Then p_3 is redundant.

Because of that redundancy, one could remove p_3 and keep p_1, p_2 , or possibly remove p_2 and keep p_1, p_3 , as long as the remaining vectors still span the same plane. The important point is that a redundant spanning set contains at least one vector that is not essential.

Instructor remark:

At this stage, the instructor called $\{p_1, p_2\}$ a basis only intuitively, before giving the official definition later.

Dictionary Versus Basis

A **basis** has no redundancy.

A **dictionary** may have redundancy. The instructor used the spanning set $\{p_1, p_2, p_3\}$ for a two-dimensional plane as a dictionary example.

The instructor emphasized that dictionaries are not bad. In some applications, dictionaries are preferred because they allow **sparse representations**.

Non-Uniqueness in a Dictionary

If a vector x lies in the plane spanned by $\{p_1, p_2, p_3\}$, then it can be represented as

$$x = \alpha_1 p_1 + \alpha_2 p_2 + \alpha_3 p_3.$$

Because the dictionary is redundant, the coefficients $\alpha_1, \alpha_2, \alpha_3$ are not unique. There may be infinitely many coefficient choices for the same vector.

This non-uniqueness gives a degree of freedom. One can choose coefficients so that many of them are zero.

This is called **sparsity**.

Sparse Representation Example

The instructor said the sparsity idea is not very clear in the simple case of three vectors in a two-dimensional plane, but it becomes useful in high-dimensional settings.

Example:

- Suppose the space is 100-dimensional.
- Choose a dictionary with 150 vectors.
- Because the dictionary has more vectors than a basis would need, representations are redundant.
- The dictionary can be chosen based on the type of data of interest.
- Then vectors from the data may be represented with coefficient vectors that are mostly zero.

Relationship:

Redundancy creates non-uniqueness, and non-uniqueness can be used to impose sparsity constraints.

The instructor said this topic would be discussed later and presented it here mainly to give the flavor of dictionary versus basis.

Linear Independence

The restriction that a basis has no redundancy leads to **linear independence**.

English definition:

A set A is linearly independent if no element of A can be written as a linear combination of the other elements of A .

Interpretation:

Each element has value because it cannot be produced from the others.

Equivalently, no vector in the set is redundant.

The set $\{p_1, p_2, p_3\}$ from the plane example is not linearly independent if any of the vectors can be written using the other two. For example:

- p_3 may be a linear combination of p_1 and p_2 .
- Or p_2 may be a linear combination of p_1 and p_3 .

Algebraic Definition of Linear Independence

The more standard definition is:

A set of vectors is linearly independent if the only way a finite linear combination can equal zero is for all coefficients to be zero.

For vectors $v_1, \dots, v_k$, this means:

$$\alpha_1 v_1 + \dots + \alpha_k v_k = 0 \iff \alpha_1 = \dots = \alpha_k = 0.$$

Equivalently, there should be no nonzero coefficient choice that produces the zero vector.

For an infinite set, the same idea is checked through finite selections of vectors from the set: any finite nontrivial zero combination reveals redundancy.

Proof Idea for Equivalence

If a nontrivial linear combination equals zero, then at least one coefficient is nonzero. One can move that vector term to the other side and solve for the corresponding vector as a linear combination of the others.

Example from the lecture:

Suppose

$$2p_1 + p_2 = 0.$$

Then

$$p_2 = -2p_1.$$

So p_2 can be produced from p_1 , meaning the set is redundant and not linearly independent.

This explains why the zero-combination definition captures the “no redundancy” idea.

Basis

After defining linear independence, the instructor gave the official basis definition.

A set B is a **basis** of a vector space V if:

1. B spans V :

$$\text{span}(B) = V.$$

2. B is linearly independent.

So a basis is a spanning set with no redundancy.

The basis concept restricts redundancy. A dictionary may span the same space but fail to be a basis if it contains redundant vectors.

Dimension

The **dimension** of V is the cardinality of a basis of V .

For finite-dimensional spaces, cardinality means the number of basis elements.

For infinite-dimensional spaces, cardinality involves a notion of infinity.

The lecture focused on finite-dimensional examples, but the instructor noted the broader cardinality language.

Standard Basis for $\mathbb{C}^n$

For n -dimensional complex vectors, the standard basis consists of vectors

$$e_1, e_2, \dots, e_n.$$

Each e_i is zero everywhere except at the i -th location, where it equals one.

Example:

$$e_1 = (1, 0, \dots, 0)^T, \quad e_2 = (0, 1, 0, \dots, 0)^T.$$

These basis vectors align with the coordinate axes and have unit length.

In $\mathbb{R}^3$, these are the usual e_1, e_2, e_3 vectors along the three coordinate axes.

Standard Basis and Kronecker Delta

The instructor connected standard basis vectors to delta functions used in electrical engineering.

The instructor first mentioned Dirac delta language, then corrected to **Kronecker delta functions** for the discrete finite-dimensional setting.

If the vector index is interpreted as a discrete time axis:

- One standard basis vector corresponds to a delta at sample time 0.
- Another corresponds to a delta shifted by one sample.
- Others correspond to further time-shifted Kronecker deltas.

This matters because standard basis vectors can be interpreted as time-localized impulses in signal processing.

Coordinates

Coordinates are basis-dependent.

If a vector x is written as a linear combination of basis vectors,

$$x = x_1 e_1 + x_2 e_2 + \dots + x_n e_n,$$

then $x_1, \dots, x_n$ are the coordinates of x with respect to that basis.

When the basis is the standard basis, these are called **standard coordinates**.

For another basis $b_1, \dots, b_n$, the same vector can be written as

$$x = \alpha_1 b_1 + \dots + \alpha_n b_n.$$

Then $\alpha_1, \dots, \alpha_n$ are the coordinates of x with respect to that chosen basis.

Important relationship:

The same vector can have different coordinate representations depending on the basis.

Complex Exponential Basis for $\mathbb{C}^n$

The instructor introduced another important basis for $\mathbb{C}^n$: the complex exponential vectors.

There are n such vectors, denoted f_k .

The entries are of the form

$$(f_k)_i = e^{j2\pi(k-1)(i-1)/n}.$$

The transcript discussion involved some index clarification:

- Vector indexing in linear algebra often starts at 1.
- Time indexing in signal processing often starts at 0.
- The expression uses $i - 1$ so that the first vector entry corresponds to time/sample index 0.

The instructor noted this as a source of confusion.

Real and Imaginary Parts of Complex Exponentials

By Euler's identity,

$$e^{j\theta} = \cos(\theta) + j \sin(\theta).$$

Therefore:

- The real part of a complex exponential vector behaves like a sampled cosine.
- The imaginary part behaves like a sampled sine.
- The parameter k determines the frequency.

The instructor noted that one cannot directly draw $e^{j\theta}$ as a real-valued curve because it is complex-valued, but one can draw its real and imaginary components.

For each k , the cosine and sine have a different frequency as functions of the vector index.

Complex Exponentials Form a Basis

[Exam note / homework emphasis] The instructor said that for different k values, the vectors f_k are independent and span the n -dimensional space.

Therefore, the complex exponential vectors form a basis of $\mathbb{C}^n$.

This means any vector in n -dimensional complex space can be written as a linear combination of those complex exponential vectors.

The instructor said this would be examined in more detail in homework.

Bases Are Not Unique

The class discussed that every finite-dimensional vector space has infinitely many possible bases.

The complex exponential basis is just one choice.

Important relationship:

A basis is not unique, but each basis gives a coordinate system. Choosing a basis is choosing how to represent vectors.

The instructor then asked why one would focus on the complex exponential basis in particular.

Why Complex Exponentials Matter: LTI Systems

The key reason is that complex exponential vectors are eigenvectors of linear time-invariant systems.

The instructor said this is especially familiar to electrical engineers, and suggested that students from other backgrounds may need to connect it to discrete-time signals and systems.

Linearity

A system is linear if it satisfies scaling and superposition.

Scaling:

If input x produces output y , then input αx produces output αy .

Superposition:

If x_1 produces y_1 and x_2 produces y_2 , then

$$\alpha_1 x_1 + \alpha_2 x_2$$

produces

$$\alpha_1 y_1 + \alpha_2 y_2.$$

Shift Invariance / Time Invariance

In the finite-dimensional discrete setting, the instructor described a shift as a circular shift or rotation:

- One entry moves to the top.
- The other entries move down.

If the input is shifted, the output should be shifted in the same way.

That property is time invariance in this finite-dimensional discrete-time context.

Complex Exponentials as Eigenvectors

For a linear time-invariant system, if a complex exponential is given as input, the output is the same complex exponential multiplied by a scalar.

The scalar depends on the frequency.

Geometrically:

- The direction of the vector does not change.
- The vector is scaled by a complex number.

This is exactly eigenvector behavior.

[Exam note] This property is important because it makes the output easy to compute. For each complex exponential component, one only needs to know the scaling factor.

Fourier Transform as Basis Change

If an input can be represented as a linear combination of complex exponential vectors,

$$x = x'_1 f_1 + x'_2 f_2 + \cdots + x'_n f_n,$$

then for an LTI system whose response to f_k is $H_k f_k$, linearity gives

$$\text{output} = x'_1 H_1 f_1 + x'_2 H_2 f_2 + \cdots + x'_n H_n f_n.$$

So the output is easy to compute once the input is represented in the complex exponential basis.

The instructor summarized this as the reason electrical engineers spend significant time learning how to write arbitrary vectors/signals as sums of complex exponentials.

This representation is the **Fourier transform**.

Important interpretation:

The Fourier transform is a basis change.

- Time-domain samples are coordinates with respect to the standard basis.
- Frequency-domain coefficients are coordinates with respect to the complex exponential basis.

The complex exponential basis is useful because LTI systems are easier to analyze in that basis.

The instructor said the course would return to this topic later.

Return to the Main Problem: $Ax = b$

After reviewing span, linear independence, bases, dictionaries, and examples of bases, the instructor returned to the main problem:

$$Ax = b.$$

The goal is to analyze existence and uniqueness of solutions.

[Exam note] Existence and uniqueness mean:

- Given A and b , is there a solution x ?
- If there is a solution, is it unique?
- Or are there multiple solutions?

The instructor emphasized that the analysis uses vector space methods and depends on two different ways of perceiving the matrix-vector product Ax .

Two Views of Matrix-Vector Multiplication

The instructor called these two views simple but fundamental.

They are based on whether A is treated as:

- A collection of column vectors.
- A collection of row vectors.

These two views lead to different analyses:

- Column view: important for existence.
- Row view: important for uniqueness.

First View: A as a Collection of Columns

Let A be an $m \times n$ matrix. Write it as columns:

$$A = [c_1 \ c_2 \ \cdots \ c_n].$$

Each column c_i is an m -dimensional vector.

For

$$x = (x_1, \dots, x_n)^T,$$

the product is

$$Ax = x_1c_1 + x_2c_2 + \dots + x_nc_n.$$

[Exam note] Multiplying a matrix on the right by a vector is taking a linear combination of the columns of the matrix.

The instructor justified this through partitioned multiplication: view the matrix as column blocks and the vector entries as scalar blocks. Matrix multiplication then becomes the first column block times x_1 , plus the second column block times x_2 , and so on.

This is the first key perception:

$$Ax = \text{linear combination of columns of } A.$$

The instructor emphasized that this is simple but important.

This view will drive the existence analysis.

Second View: A as a Collection of Rows

The same matrix A can be written in terms of its rows.

The instructor used notation where r_i is conceptually a column vector, so the actual row is r_i^T . Thus rows are written as

$$r_1^T, r_2^T, \dots, r_m^T.$$

This notation can be misleading because r_i is a column vector even though it represents row data after transposition.

Using the row view,

$$Ax = \begin{bmatrix} r_1^T x \\ r_2^T x \\ \vdots \\ r_m^T x \end{bmatrix}.$$

[Exam note] Matrix multiplication can be viewed as taking inner products of the rows of A with the vector x .

Using the Euclidean inner product,

$$r_i^T x = \langle r_i, x \rangle.$$

So the row view says:

$$Ax = \text{the vector of inner products of } x \text{ with the rows of } A.$$

This view will be important for uniqueness.

Left Multiplication by a Vector

The instructor also discussed the dual situation: multiplying A from the left by a row vector y^T :

$$y^T A.$$

For dimensions to match, if A is $m \times n$, then y has length m and $y^T A$ is a row vector of length n .

If A is viewed as a collection of row vectors, then $y^T A$ is a linear combination of the rows of A .

So:

- Right multiplication Ax gives linear combinations of columns.
- Left multiplication $y^T A$ gives linear combinations of rows.

If A is viewed as a collection of columns, then $y^T A$ gives inner products of y with the columns of A :

$$y^T A = [y^T c_1 \quad y^T c_2 \quad \cdots \quad y^T c_n].$$

The instructor briefly became tangled in notation but returned to the main summary:

- Right multiply: linear combination of columns, or inner products with rows.
- Left multiply: linear combination of rows, or inner products with columns.

Why the Two Views Matter

[Exam note] The instructor gave the punchline:

The column and row perceptions are important because the course will define subspaces based on them.

These subspaces will then be used to analyze existence and uniqueness.

Specifically:

- Existence will be addressed using the column/range space.
- Uniqueness will later be addressed using row-related or nullspace-related ideas.

When a student asked how these views explain existence and uniqueness, the instructor said this section was preparation: the course first builds the relevant subspaces, then uses them for the two questions.

The current lecture begins the existence side.

Geometric Picture of $Ax = b$

The instructor described Ax as a function or mapping.

For an $m \times n$ matrix A :

$$A : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

The input x is n -dimensional.

The output Ax is m -dimensional.

The right-hand side b also lies in the m -dimensional output space.

Instructor warning:

The symbols m and n are easy to confuse, but the convention here is:

- n : input dimension and number of columns.
- m : output dimension and number of rows.

The instructor noted that this m, n convention is standard in many references.

Instructor correction:

The instructor initially almost swapped the roles of m and n , then corrected the picture: x is n -dimensional and Ax , like b , is m -dimensional. The drawings are only toy geometric pictures, not necessarily actual three-dimensional-to-three-dimensional maps.

$Ax = b$ as an Inverse Problem

Solving

$$Ax = b$$

is an inverse problem.

The output b is known. The goal is to find an input x that generated it.

Question:

Which x , if any, maps to the given b ?

This interpretation is central to the existence question.

If no input maps to b , then the equation has no solution.

So an $Ax = b$ picture should be read as a search for a preimage of b , not as a guarantee that the displayed target is actually attainable.

Where Rows and Columns Live

For an $m \times n$ matrix A :

- Each row has length n , so rows live in the input space $\mathbb{R}^n$.
- Each column has length m , so columns live in the output space $\mathbb{R}^m$.
- The vector x lives in the input space $\mathbb{R}^n$.
- The vector b lives in the output space $\mathbb{R}^m$.

[Exam note] This matches the two product views:

- Rows must be in the same space as x , because Ax computes inner products with rows.
- Columns must be in the same space as b , because Ax is a linear combination of columns and therefore produces an output-space vector.

This is the fundamental geometric picture for analyzing $Ax = b$.

Column Space / Range Space

The first vector space introduced for existence analysis is the **column space**.

Definition:

The column space of A is the span of the columns of A .

$$\text{Col}(A) = \text{span}\{c_1, \dots, c_n\}.$$

Because the columns are m -dimensional, the column space is a subspace of the output space:

$$\text{Col}(A) \subseteq \mathbb{R}^m.$$

The instructor also called this the **range space**.

Geometric picture:

The instructor drew the column space as a subspace, such as a plane inside the output space. He cautioned that the drawing was only a toy picture; if the drawn columns look linearly dependent, that visual dependence should not be over-interpreted.

Warning about notation:

The transcript says the textbook/instructor may use $R(A)$ for range space. This can be confused with row space, but here $R(A)$ means range/column space.

Column Space as All Possible Outputs

Using the column view,

$$Ax = x_1c_1 + \cdots + x_nc_n.$$

As x varies over all possible inputs, Ax varies over all possible linear combinations of the columns.

Therefore:

$$\text{Col}(A) = \{Ax : x \in \mathbb{R}^n\}.$$

This is the image of the input space inside the output space.

Interpretation:

The column/range space is the set of all outputs that the linear mapping $x \mapsto Ax$ can generate.

Existence Criterion for $Ax = b$

The first major conclusion:

$$Ax = b \text{ has at least one solution} \iff b \in \text{Col}(A).$$

The instructor stated it in two parts:

- If b lies in the column/range space of A , then there is at least one x such that $Ax = b$.
- If b does not lie in the column/range space, then there is no solution.

Reason:

The only vectors A can produce are linear combinations of its columns. If b is not one of those vectors, no choice of x can produce it.

Procedure for checking existence conceptually:

1. Take the columns of A .
2. Find or understand their span.
3. Check whether b belongs to that span.

The instructor emphasized that this analysis comes directly from viewing A as a collection of columns.

When Every b Has a Solution

The next question:

For which matrices A does $Ax = b$ have a solution for every possible $b \in \mathbb{R}^m$?

This is equivalent to asking when the column/range space fills the entire output space:

$$\text{Col}(A) = \mathbb{R}^m.$$

If this holds, then every $b \in \mathbb{R}^m$ lies in the range space, so every right-hand side has at least one solution.

Important limitation:

This only says something about existence. It does not imply uniqueness.

Instructor remark:

The lecture connects this condition to full rank. The instructor noted that although this looks like a column-space property, it will be related to independence of rows, specifically full row rank.

Likely exam topic This is the bridge to the later full-row-rank condition: for an $m \times n$ matrix to have $\mathcal{R}(A) = \mathbb{R}^m$, the rows will have to supply m independent constraints. The instructor explicitly warned that “independent columns” is not the right automatic answer for this existence-for-every- b property.

Matrix Shapes and Relative Dimensions

[Exam note] The relative dimensions of the matrix are important.

For an $m \times n$ matrix:

- $m > n$: tall matrix, more rows than columns.
- $m = n$: square matrix.
- $n > m$: fat/wide matrix, more columns than rows.

The instructor used the terms tall and fat. The transcript also mentions that some references may use “skinny” for tall matrices.

Necessary Condition for Full Range

To have

$$\text{Col}(A) = \mathbb{R}^m,$$

the columns of A must span an m -dimensional output space.

Since A has only n columns, a necessary condition is:

$$n \geq m.$$

Interpretation:

There must be at least as many columns as the dimension of the output space.

The instructor gave the example:

If $m = 100$ and $n = 10$, then we have ten vectors in a 100-dimensional space. Ten vectors cannot span all of $\mathbb{R}^{100}$.

Therefore, a tall matrix $m > n$ cannot have column space equal to all of $\mathbb{R}^m$. There will be some b 's outside the range space, so one cannot guarantee a solution for arbitrary b .

Important warning:

$$\text{Col}(A) = \mathbb{R}^m \Rightarrow n \geq m.$$

This is a one-way implication. It is necessary but not sufficient.

Not every square or fat matrix has full range. Having enough columns does not guarantee that the columns actually span the entire output space.

Input and Output Dimension Interpretation

The condition $n \geq m$ means:

The input space must have dimension at least as large as the output space if we want the possibility of generating every output.

Again, this is only necessary. Additional rank/independence conditions are needed.

Alternative Definitions of Range Space

The instructor summarized three equivalent descriptions of the range space of A .

1. Image Definition

$$\mathcal{R}(A) = \{Ax : x \in \mathbb{R}^n\}.$$

This is the set of all outputs generated by all possible inputs.

2. Column Space Definition

$$\mathcal{R}(A) = \text{span}\{c_1, \dots, c_n\}.$$

This is the span of the columns of A .

3. Solvability Definition

$$\mathcal{R}(A) = \{b \in \mathbb{R}^m : Ax = b \text{ has at least one solution}\}.$$

This is the collection of right-hand sides for which the linear system is solvable.

These definitions are the same because Ax is exactly a linear combination of the columns of A .

Full Range and Right Inverses

Likely exam topic The instructor then introduced a two-way implication involving full range and right inverses.

The condition

$$\mathcal{R}(A) = \mathbb{R}^m$$

is equivalent to the existence of a matrix $D \in \mathbb{R}^{n \times m}$ such that

$$AD = I_m.$$

The transcript uses both Z and D language, but the constructed matrix is effectively D .

The dimensions are the transpose shape of A : if A is $m \times n$, then a right inverse D must be $n \times m$. Thus for a fat A , the right inverse D is tall.

Such a matrix D is called a **right inverse** of A , because it multiplies A on the right and produces the identity:

$$AD = I_m.$$

Important warning:

This does not claim that

$$DA = I_n.$$

For non-square matrices, right and left inverse properties are different.

Proof Idea: Full Range Implies a Right Inverse

Assume

$$\mathcal{R}(A) = \mathbb{R}^m.$$

Then every $b \in \mathbb{R}^m$ has at least one solution to $Ax = b$.

Now choose the standard basis vectors of the output space:

$$e_1, e_2, \dots, e_m.$$

Since each $e_i \in \mathbb{R}^m$, and the range is all of $\mathbb{R}^m$, each system

$$Ax = e_i$$

has at least one solution.

Pick one solution for each standard basis vector:

$$Ad_1 = e_1, \quad Ad_2 = e_2, \quad \dots, \quad Ad_m = e_m.$$

If there are multiple choices, choose any one; existence is what matters here.

Now assemble these solution vectors as columns:

$$D = [d_1 \ d_2 \ \cdots \ d_m].$$

Then

$$AD = A[d_1 \ d_2 \ \cdots \ d_m] = [Ad_1 \ Ad_2 \ \cdots \ Ad_m] = [e_1 \ e_2 \ \cdots \ e_m] = I_m.$$

So D is a right inverse of A .

This proof uses the column-partition view of matrix multiplication again: multiplying A by a matrix D column by column gives the columns Ad_i .

Converse Proof Idea: Right Inverse Implies Full Range

The instructor announced the implication as two-way. The immediate converse is:

If there exists D such that

$$AD = I_m,$$

then for any $b \in \mathbb{R}^m$, choose

$$x = Db.$$

Then

$$Ax = A(Db) = (AD)b = I_m b = b.$$

So every b has a solution, which means

$$\mathcal{R}(A) = \mathbb{R}^m.$$

This completes the equivalence between full range and existence of a right inverse.

Right Inverse, Square Matrices, and Non-Square Matrices

The instructor made several important remarks about right inverses.

Square Case

If A is square and a right inverse D exists, then:

- The right inverse is unique.
- It will also turn out to be a left inverse.
- So $DA = I$ as well.
- This unique matrix is the usual inverse A^{-1} .

The instructor said this would be shown later.

Fat Matrix Case

If A is fat/wide and has full range, then right inverses can exist.

In that case, the right inverse is not unique. There can be infinitely many such matrices D satisfying

$$AD = I_m.$$

This aligns with the idea that fat systems often have more input degrees of freedom than output constraints.

Tall Matrix Case

A tall matrix cannot have full range when $m > n$, because it does not have enough columns to span $\mathbb{R}^m$.

Therefore, tall matrices cannot have a right inverse of this type.

General Non-Square Case

For non-square matrices, one talks about right inverses and left inverses separately.

The usual unique inverse concept belongs to the square case.

Relationship Between Concepts

The lecture repeatedly ties advanced-looking matrix ideas back to simple vector space ideas.

Key relationships:

- Span is the set of all linear combinations.
- A basis is a spanning set without redundancy.
- Linear independence formalizes “no redundancy.”
- A dictionary is a spanning set that may contain redundancy.
- Redundancy produces non-unique coefficients.
- Non-unique coefficients can be used for sparsity.
- Coordinates are coefficients relative to a chosen basis.
- Changing basis changes coordinates but not the vector itself.
- Fourier transform is a basis change from standard/time coordinates to complex exponential/frequency coordinates.
- Ax as a column combination connects matrix multiplication to span.
- The column space/range space is the set of all possible outputs of $x \mapsto Ax$.
- $Ax = b$ is solvable exactly when b lies in that range.
- Every b is solvable exactly when the range is all of $\mathbb{R}^m$.
- Full range requires $n \geq m$, but this is not sufficient.
- Full range is equivalent to existence of a right inverse D with $AD = I_m$.

Instructor Warnings and Hints

Likely exam topic Existence and uniqueness questions are important, and they motivate the vector space machinery.

[Exam note] The column-space/range-space view is central to existence.

[Exam note] The row-space/inner-product view will be central to uniqueness later.

[Exam note] Remember both matrix-vector product perceptions:

- Ax as a linear combination of columns.
- Ax as inner products with rows.

[Exam note] Relative dimensions m and n matter. Be careful:

- m : number of rows and output dimension.
- n : number of columns and input dimension.

Warning:

The geometric sketches are intuition only. Always check the actual dimensions: rows live with x in the input space, columns live with b in the output space.

Warning:

The condition $n \geq m$ is necessary for full range, but it is not sufficient.

Warning:

Full range is not obtained merely by saying “the columns are independent.” The lecture flags full row rank as the relevant rank language for guaranteeing every $b \in \mathbb{R}^m$ is reachable.

Warning:

Do not confuse range-space notation $R(A)$ with row space. In this lecture, $R(A)$ refers to range/column space.

Warning:

A right inverse satisfies $AD = I_m$, not necessarily $DA = I_n$.

Warning:

Orthogonal matrices form a subset of square matrices but not a subspace, because the zero matrix is not included.

Instructor remark:

The complex exponential basis will be examined in homework and revisited later. Its importance comes from LTI systems and Fourier analysis.

Instructor closing remark:

The concepts of right inverse and solvability are built from the basic concepts of linear combination, span, and vector spaces. The instructor asked students to review this before the next lecture.

Source / Coverage Note

These notes were created only from `corrected/lecture5_corrected.md`. They cover the chronological lecture content, including definitions, examples, proof ideas, instructor remarks, warnings, and the transcript's exam-relevant markers through the closing review instruction. No other lecture was processed.

Lecture 6 Notes: Vector Space View of Linear Systems

1. Existence and Uniqueness for Linear Systems

Likely exam topic The lecture begins by separating two questions about a linear system $Ax = b$:

- **Existence:** Is there at least one solution?
- **Uniqueness:** If a solution exists, is it the only one, or are there many?

The instructor recalls the toy case of a 2×2 system. Each equation gives a line, and the possible geometric cases are:

- The lines intersect at one point: one unique solution.
- The lines are parallel and distinct: no solution.
- The lines lie on top of each other: infinitely many solutions.

The goal is to extend this picture from two equations and two unknowns to an arbitrary number of equations and unknowns.

2. Two Ways to View Matrix-Vector Multiplication

The instructor emphasizes that there are two key perceptions of Ax . This is described as a simple but important observation and a key to the later vector-space analysis.

2.1 Column View: Ax as a Linear Combination of Columns

If A is partitioned into columns, then multiplying A by x from the right forms a linear combination of the columns of A :

$$Ax = x_1c_1 + x_2c_2 + \cdots + x_nc_n.$$

From this point of view, the problem $Ax = b$ asks:

Can b be expressed as a linear combination of the columns of A ?

This column view is directly connected to the **existence** of solutions.

2.2 Row View: Ax as Inner Products with Rows

If A is partitioned into rows, then multiplying A by x forms a vector whose entries are inner products of x with the rows of A :

$$Ax = \begin{bmatrix} r_1^T x \\ r_2^T x \\ \vdots \\ r_m^T x \end{bmatrix}.$$

[Exam note] The instructor reminds students about his visual notation: a row vector such as r_1^T may be represented using a column-vector notation and then transposed.

This row view will be used later to understand **uniqueness**.

2.3 Transpose Mapping

Multiplying A^T from the right by a vector is equivalent, after transposing, to multiplying A from the left by a row vector. The instructor notes:

- $A^T x$ takes linear combinations of the columns of A^T .

- The columns of A^T are the rows of A .
- Therefore $A^T x$ can be interpreted as taking linear combinations of the rows of A .
- $A^T x$ can also be interpreted as taking inner products with the columns of A .

This A^T mapping will be used in the vector-space method for analyzing $Ax = b$.

3. A as a Linear Mapping

For an $m \times n$ matrix A , the product Ax maps

$$x \in \mathbb{R}^n \quad \longmapsto \quad Ax \in \mathbb{R}^m.$$

The instructor describes:

- The **input space** is $\mathbb{R}^n$.
- The **output space** is $\mathbb{R}^m$.
- b lies in the output space.
- The rows of A lie in the input space, because they take inner products with x .
- The columns of A lie in the output space.

He gives an informal brute-force interpretation: if one tried to solve $Ax = b$ by trial, one would vary x in the input space and watch the output Ax in $\mathbb{R}^m$, hoping it lands on the given b . The geometric question is therefore: which output vectors can this mapping actually generate?

All possible outputs generated by Ax are linear combinations of the columns of A . Therefore, all possible outputs form the span of the columns.

4. Column Space / Range Space of A

The first fundamental subspace introduced is the space spanned by the columns of A .

4.1 Definition

The **column space** of A is

$$\text{Col}(A) = \text{span}\{c_1, c_2, \dots, c_n\}.$$

The instructor also calls this the **range space** of A , because it is the set of all possible outputs:

$$\mathcal{R}(A) = \{Ax : x \in \mathbb{R}^n\}.$$

The transcript sometimes uses wording like “rank space” or “rate space”; in context, this is the range/column space.

4.2 Alternative Descriptions

The range space of A can be described in three equivalent ways:

1. The set of all possible outputs Ax generated by all possible $x \in \mathbb{R}^n$.
2. The span of the columns of A .
3. The set of all right-hand sides b for which $Ax = b$ has at least one solution.

5. Existence of Solutions

Likely exam topic The geometric conclusion about existence is:

$$Ax = b \text{ has a solution} \iff b \in \mathcal{R}(A) = \text{Col}(A).$$

If b lies outside the range/column space, then b cannot be generated by any Ax , so there is no solution.

If b lies inside the range/column space, then at least one x maps to b , so there is at least one solution.

5.1 Finite-Dimensional Setting

In response to a question, the instructor clarifies that the lecture is working with finite-dimensional spaces. Extensions to general operators are beyond the current level of discussion.

6. Guaranteeing Existence for Every b

The instructor then considers a stronger question:

For a given matrix A , can we guarantee a solution for every possible $b \in \mathbb{R}^m$?

This is not asking about one chosen b . It is classifying matrices A for which every right-hand side is solvable.

The condition is:

$$\mathcal{R}(A) = \mathbb{R}^m.$$

That means the column space covers the entire target/output space, so b has nowhere outside the column space to “hide.”

6.1 Necessary Shape Condition

Since the target space is m -dimensional and the range is spanned by the columns of A , there must be at least m columns available to span all of $\mathbb{R}^m$. Thus:

$$n \geq m.$$

So A must be either:

- square, or
- fat/wide.

This condition is necessary but not sufficient. Not every square or fat matrix has columns spanning $\mathbb{R}^m$.

For tall matrices, existence for every b cannot be guaranteed because there are not enough columns to cover the whole output space.

7. Right Inverses from Full Range Space

[Exam note] The instructor reviews how the right inverse conclusion is obtained.

Assume

$$\mathcal{R}(A) = \mathbb{R}^m.$$

Then $Ax = b$ has a solution for every $b \in \mathbb{R}^m$. In particular, it has solutions for every standard basis vector:

$$e_1, e_2, \dots, e_m.$$

Choose one solution d_i for each equation

$$Ad_i = e_i.$$

Place these solution vectors as columns of a matrix D :

$$D = [d_1 \ d_2 \ \dots \ d_m].$$

Then

$$AD = [Ad_1 \ Ad_2 \ \dots \ Ad_m] = [e_1 \ e_2 \ \dots \ e_m] = I_m.$$

So if the range space of A is all of $\mathbb{R}^m$, then A has a right inverse D satisfying

$$AD = I_m.$$

The instructor remarks that this property looks sophisticated, but it is a direct consequence of being able to solve $Ax = b$ for every b . If A is fat and the condition holds, there are infinitely many right inverses.

8. Full Range Space and Linear Independence Properties

[Exam note] The instructor mentions additional equivalent properties and leaves some proof directions to students.

If $\mathcal{R}(A) = \mathbb{R}^m$, then the columns of A contain a basis for $\mathbb{R}^m$. That means:

- We can find m columns of A that are linearly independent.
- These m columns form a basis for the output space.

One proof idea mentioned by the instructor is a pruning argument: if the columns span the whole m -dimensional target space and there are more than m columns, dependent columns can be removed without changing the span until a basis of m independent columns remains.

The instructor stresses a distinction:

- This does **not** say all columns of A are linearly independent.
- It says that some subset of m columns is linearly independent.
- If $n > m$, the full set of columns is necessarily linearly dependent because there are more vectors than the dimension of the output space.

Another implication is about rows:

If the range space covers the whole target space, then the rows of A are linearly independent.

This is interesting because a property stated in terms of the column space has a consequence for the rows.

9. Proof Idea: Row Independence and Full Column Space

The instructor develops the relationship between row independence and columns spanning the output space.

Let $\alpha \in \mathbb{R}^m$, where m is the number of rows and the dimension of the output space.

Left multiplication by α^T gives

$$\alpha^T A.$$

There are two ways to interpret this product:

1. As a linear combination of the rows of A .
2. As a row vector whose entries are inner products of α with the columns of A .

The rows of A are linearly independent exactly when

$$\alpha^T A = 0 \implies \alpha = 0.$$

Equivalently, the only vector orthogonal to all columns of A is the zero vector.

The instructor gives the geometric idea:

- If there were a nonzero vector α orthogonal to every column of A , then α could not lie in the range space of A .
- To see this, suppose $\alpha \in \mathcal{R}(A)$. Then

$$\alpha = \beta_1 c_1 + \beta_2 c_2 + \cdots + \beta_n c_n.$$

- Take inner product with α on both sides:

$$\alpha^T \alpha = \beta_1 \alpha^T c_1 + \beta_2 \alpha^T c_2 + \cdots + \beta_n \alpha^T c_n.$$

- Since α is orthogonal to every c_i , the right side is zero.
- Therefore $\alpha^T \alpha = 0$, which can happen only if $\alpha = 0$.

Thus a nonzero vector orthogonal to all columns cannot be in the range space. If the only such vector is zero, the columns span the whole output space.

This is the proof-by-contradiction/contrapositive logic the instructor wants students to notice:

- If the rows were linearly dependent, there would be a nonzero α with $\alpha^T A = 0$.
- That same condition says α is orthogonal to every column of A .
- If the range space were all of $\mathbb{R}^m$, then this nonzero α would have to lie in the range space.
- But a nonzero vector orthogonal to all columns cannot lie in their span, because taking the inner product with itself would force $\alpha^T \alpha = 0$.

So full range space forces row independence. Conversely, the instructor says the reverse implication can also be argued by contradiction, though the lecture does not spell out every detail.

10. Moving from Existence to Uniqueness

Up to this point, the lecture has focused on existence of solutions. The instructor then shifts to uniqueness.

The main duality is:

- Column space is related to existence.
- Row space is related to uniqueness.
- Existence for every b is guaranteed when the column space is the whole output space.
- Uniqueness, when a solution exists, is guaranteed when the row space is the whole input space.

11. Row Space of A

11.1 Definition

The **row space** of A is the span of the rows of A :

$$\text{Row}(A) = \text{span}\{r_1, r_2, \dots, r_m\}.$$

It is a subspace of the input space $\mathbb{R}^n$.

11.2 Row Space as Range Space of A^T

The rows of A are the columns of A^T . Therefore:

$$\text{Row}(A) = \text{Col}(A^T) = \mathcal{R}(A^T).$$

[Warning] Here $\mathcal{R}$ means **range**, not row. Thus the row space of A is the range space of A^T . The instructor calls this notation potentially confusing because the row space is being described with the same range-space symbol used earlier for column spaces.

The instructor warns that A^T is not necessarily the inverse of A . The map

$$A^T y$$

goes from the output space back into the input-space dimension, but in general it does not undo Ax . In other words, $A^T(Ax)$ is not generally equal to x .

12. Null Space of A

The instructor defines a set of input vectors that map to the origin:

$$\mathcal{N}(A) = \{x \in \mathbb{R}^n : Ax = 0\}.$$

This is the **null space** of A .

12.1 Null Space Is a Vector Space

The instructor first calls it a null set and then notes it is actually a vector space.

If $x \in \mathcal{N}(A)$, then $Ax = 0$. For any scalar α ,

$$A(\alpha x) = \alpha Ax = \alpha 0 = 0.$$

So scaled versions stay in the null space.

If $x, y \in \mathcal{N}(A)$, then

$$A(x + y) = Ax + Ay = 0 + 0 = 0.$$

So sums stay in the null space.

Therefore $\mathcal{N}(A)$ is a subspace of the input space.

12.2 Null Space as Orthogonal to the Row Space

Using the row interpretation of Ax , the equation $Ax = 0$ means every inner product between x and a row of A is zero.

Thus:

$$x \in \mathcal{N}(A) \iff x \text{ is orthogonal to every row of } A.$$

Therefore the null space is the space orthogonal to the row space:

$$\mathcal{N}(A) = \text{Row}(A)^\perp.$$

The instructor illustrates this with a three-dimensional picture:

- The row space can be represented as a plane.
- The null space can be represented as the perpendicular line.
- Both are subspaces of $\mathbb{R}^n$.

13. Why Null Space Controls Uniqueness

Suppose x is a solution of $Ax = b$:

$$Ax = b.$$

Let $z \in \mathcal{N}(A)$. Then $Az = 0$. Define

$$x' = x + z.$$

Then

$$Ax' = A(x + z) = Ax + Az = b + 0 = b.$$

So x' is also a solution.

If $z \neq 0$, then $x' \neq x$, giving at least two different solutions. Therefore, for uniqueness, there cannot be any nonzero vector in the null space.

The uniqueness condition is:

$$\mathcal{N}(A) = \{0\}.$$

This is called the **trivial null space**. The instructor notes that the null space can never be empty because 0 always maps to 0. The trivial null space is the smallest possible null space.

The instructor also frames this through the special right-hand side $b = 0$: if multiple input vectors map to the origin, then even for output 0 one cannot identify a unique generating input. The condition $\mathcal{N}(A) = \{0\}$ is therefore not only necessary but also sufficient for uniqueness whenever a solution exists.

14. Row Space Covering the Input Space

Since the null space is orthogonal to the row space, having only the zero vector in the null space means there is no nonzero direction orthogonal to all rows.

This implies the rows span the whole input space:

$$\text{Row}(A) = \mathbb{R}^n.$$

Thus the uniqueness condition can be stated equivalently as:

$$Ax = b \text{ has at most one solution for each } b \iff \mathcal{N}(A) = \{0\} \iff \text{Row}(A) = \mathbb{R}^n.$$

The instructor frames this as the dual of the existence condition:

- Existence for every b : $\text{Col}(A) = \mathbb{R}^m$.
- Uniqueness: $\text{Row}(A) = \mathbb{R}^n$.

15. Direct Sum, Disjoint Subspaces, and a Warning About Union

The instructor warns not to write

$$\mathbb{R}^n = \text{Row}(A) \cup \mathcal{N}(A).$$

The union of the row space and null space is not the whole ambient space in general. In the plane-line picture, the union would only include those two subspaces, not every vector in $\mathbb{R}^n$.

Instead, one uses a **direct sum** expression:

$$\mathbb{R}^n = \text{Row}(A) \oplus \mathcal{N}(A).$$

This means every vector in $\mathbb{R}^n$ can be written as

$$x = x_1 + x_2,$$

where

$$x_1 \in \text{Row}(A), \quad x_2 \in \mathcal{N}(A).$$

15.1 Disjoint Subspaces

Two subspaces are called **disjoint** in this lecture if their intersection is only the origin:

$$S_1 \cap S_2 = \{0\}.$$

Every subspace contains the origin, so disjoint subspaces still share 0.

The row space $\mathcal{R}(A^T)$ and the null space $\mathcal{N}(A)$ are disjoint in this sense. In this case they are also orthogonal complements.

The instructor notes that the definition of direct sum does not itself require orthogonality, but in the row-space/null-space case the two subspaces are orthogonal.

16. The First Three Fundamental Subspaces

At this stage, the lecture has defined three subspaces:

1. Column space / range space of A :

$$\mathcal{R}(A) = \text{Col}(A) \subseteq \mathbb{R}^m.$$

This is the span of the columns and determines existence.

2. Row space of A :

$$\text{Row}(A) = \mathcal{R}(A^T) \subseteq \mathbb{R}^n.$$

This is the span of the rows and is connected to uniqueness.

3. Null space of A :

$$\mathcal{N}(A) \subseteq \mathbb{R}^n.$$

This is the set of input vectors mapped to the origin and is orthogonal to the row space.

The instructor says the picture is still incomplete because we have described two subspaces in the input space but only one in the output space.

17. Null Space of A^T / Left Null Space

The missing output-space companion is the space of vectors orthogonal to all columns of A . This is the null space of A^T :

$$\mathcal{N}(A^T) = \{y \in \mathbb{R}^m : A^T y = 0\}.$$

Since $A^T y$ takes inner products of y with the columns of A , this is the set of all output-space vectors orthogonal to every column of A .

Thus:

$$\mathcal{N}(A^T) = \text{Col}(A)^\perp.$$

The row space of A^T is the column/range space of A , and the space orthogonal to it is $\mathcal{N}(A^T)$.

Therefore the output space can be decomposed as

$$\mathbb{R}^m = \mathcal{R}(A) \oplus \mathcal{N}(A^T).$$

In the instructor's picture, the extra "orange" output-space direction represents $\mathcal{N}(A^T)$, completing the four-subspace view: two subspaces in the input space and two in the output space.

The instructor explains the dimension intuition:

- If $\mathcal{R}(A)$ has dimension r , then its orthogonal complement $\mathcal{N}(A^T)$ has dimension $m - r$.
- Their dimensions add to the dimension m of the ambient output space.
- A basis for one together with a basis for the other spans the output space.

He notes that he has not given a complete proof here, but this is the intuition based on orthogonal complements.

17.1 Name: Left Null Space

$\mathcal{N}(A^T)$ is also called the **left null space** of A .

The reason is:

If $z \in \mathcal{N}(A^T)$, then

$$A^T z = 0.$$

Taking transposes gives

$$z^T A = 0^T.$$

So these are the vectors that annihilate A by left multiplication.

The instructor notes the naming convention:

- $\mathcal{N}(A)$ is usually just called the null space, not the right null space.
- $\mathcal{N}(A^T)$ is called the left null space of A .
- Kernel and null space refer to the same idea, but the phrase “left null space” specifically means the null space of A^T .

18. Example 1: A 3×3 Matrix with xy -Plane Column and Row Spaces

The instructor gives an easy-to-draw 3×3 example whose columns are:

$$c_1 = (1, 0, 0)^T, \quad c_2 = (0, 1, 0)^T, \quad c_3 = (0, 0, 0)^T.$$

The column space is the span of the first two coordinate vectors:

$$\text{Col}(A) = \text{span}\{(1, 0, 0)^T, (0, 1, 0)^T\},$$

which is the xy -plane.

In this particular example, the rows are also

$$(1, 0, 0), \quad (0, 1, 0), \quad (0, 0, 0),$$

so the row space is also the xy -plane. The instructor warns that this equality of row and column spaces is special to this example and not true in general.

18.1 Existence and Uniqueness in Example 1

Existence for every b is not guaranteed because the column space does not cover all of $\mathbb{R}^3$. Only b 's in the xy -plane are attainable.

Even when b lies in the column space, the solution is not unique because the null space is not trivial. In this example the null space is the whole z -axis direction.

19. Example 2: A 3×2 Tall Matrix

The second example is a 3×2 matrix mapping

$$\mathbb{R}^2 \rightarrow \mathbb{R}^3.$$

This is a tall matrix.

The instructor emphasizes:

- For tall matrices, existence for arbitrary $b \in \mathbb{R}^3$ cannot be guaranteed.
- The input dimension is lower than the output dimension, so the mapping cannot cover all of the output space.

In the example:

- The columns again span the xy -plane in $\mathbb{R}^3$.
- The row vectors include the two coordinate directions in the two-dimensional input space, and their span is the whole input space.
- Therefore the null space is just the origin.

19.1 Existence and Uniqueness in Example 2

For this tall matrix:

- A solution is not guaranteed for arbitrary b .
- If b lies in the range/column space, then there is a unique solution.

So such a matrix can have uniqueness for all solvable right-hand sides, while still failing to have existence for all right-hand sides.

20. Comparing Existence and Uniqueness Conditions

The lecture summarizes the dual conditions:

20.1 Existence for Every b

$$\mathcal{R}(A) = \mathbb{R}^m.$$

This requires the column space to cover the entire target/output space. A necessary shape condition is:

$$m \leq n.$$

So A must be square or fat. This is necessary, not sufficient.

When this condition holds, A has at least one right inverse:

$$AD = I_m.$$

20.2 Uniqueness

$$\mathcal{N}(A) = \{0\}.$$

Equivalently:

$$\text{Row}(A) = \mathbb{R}^n.$$

This requires enough rows to span the input space. A necessary shape condition is:

$$m \geq n.$$

So A must be square or tall. This is necessary, not sufficient.

When this condition holds, A has at least one left inverse.

The instructor obtains this by transpose duality: if $\mathcal{N}(A) = \{0\}$, then A^T satisfies the corresponding full-range/right-inverse condition; transposing that right-inverse relation gives a left inverse for A .

20.3 Both Existence for All b and Uniqueness

Both conditions require

$$m \leq n \quad \text{and} \quad m \geq n,$$

so both can only hold for square matrices.

For square matrices where both corresponding null spaces are trivial and both conditions hold, the right inverse and left inverse coincide. This common matrix is called the inverse of A , denoted A^{-1} .

The instructor also states:

- In the existence-for-all- b case, the rows are linearly independent.
- In the uniqueness case, the columns are linearly independent.
- These statements are related by transpose duality.

[Exam note] The instructor highlights the shape distinction: existence for every b needs square or fat matrices; uniqueness needs square or tall matrices; both together force square matrices.

21. Fundamental Result: Row Rank Equals Column Rank

[Exam note] The instructor introduces an important result before continuing the system-of-equations analysis.

Let

$$r_C = \dim(\text{Col}(A))$$

be the dimension of the column space in the output space, and let

$$r_R = \dim(\text{Row}(A))$$

be the dimension of the row space in the input space.

Although these subspaces live in different ambient spaces and are defined differently, their dimensions are equal:

$$r_C = r_R.$$

This common dimension is called the **rank** of A :

$$\text{rank}(A) = r_C = r_R.$$

The instructor notes that people may refer to column rank and row rank, but they are equal.

22. Proof Idea: Row Rank $\leq$ Column Rank

The instructor outlines one direction of the proof.

Let the row space have dimension r_R . Choose a basis for the row space:

$$v_1, v_2, \dots, v_{r_R}.$$

These vectors are linearly independent and span the row space.

Now map these basis vectors into the output space by multiplying by A :

$$Av_1, Av_2, \dots, Av_{r_R}.$$

Each Av_i lies in the column space $\text{Col}(A)$, because every output of A lies in the column space.

The key question is whether the set

$$\{Av_1, \dots, Av_{r_R}\}$$

is linearly independent.

Suppose a linear combination is zero:

$$\alpha_1 Av_1 + \alpha_2 Av_2 + \dots + \alpha_{r_R} Av_{r_R} = 0.$$

Factor out A :

$$A(\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_{r_R} v_{r_R}) = 0.$$

The vector

$$w = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_{r_R} v_{r_R}$$

lies in the row space because it is a linear combination of row-space basis vectors.

But $Aw = 0$, so w also lies in the null space.

The row space and null space intersect only at 0. Therefore:

$$w = 0.$$

Because the v_i 's are linearly independent, this implies

$$\alpha_1 = \alpha_2 = \dots = \alpha_{r_R} = 0.$$

Thus the vectors

$$Av_1, \dots, Av_{r_R}$$

are linearly independent. Since they lie in the column space, the column space must have dimension at least r_R :

$$r_R \leq r_C.$$

23. Reverse Direction and Rank

The instructor says the reverse argument is analogous:

- Choose a basis for the column space.
- Map it back using A^T .
- Show the corresponding independent set lies in the row space.

This gives

$$r_C \leq r_R.$$

Together:

$$r_R \leq r_C \quad \text{and} \quad r_C \leq r_R$$

so

$$r_R = r_C.$$

This shared dimension is the rank of A .

Likely exam topic Rank is the common dimension of the row space and the column space.

24. Instructor Remarks and Study Guidance

The instructor says this material is the core of the upcoming discussion of systems of linear equations. It is based on simple principles, but the class will build on these results.

He recommends that students who are not fully comfortable with the vector-space picture draw it on paper:

- Draw the range/column space.
- Draw the row space.
- Draw the null space.
- Draw the left null space.
- Write the main properties beside the picture.

[Exam note] The instructor says this is an important basis for the next lecture's conclusions about systems of linear equations.

The lecture ends with the note that the class will continue on Tuesday.

Source and Coverage Note

Source used: `corrected/lecture6_corrected.md` only.

Coverage: These notes preserve the lecture's chronological development: existence and uniqueness, column and row interpretations of Ax , range/column space, conditions for existence for all b , right inverses, row independence, row space, null space, direct sums, left null space, the two concrete examples, the duality between existence and uniqueness, and the proof idea that row rank equals column rank. Exam-marked comments and instructor warnings were folded into the relevant sections. No other lecture transcript was processed.

Lecture 7 Notes

Opening Administrative Remarks

The lecture begins with a brief administrative aside about calendars and an event or symposium schedule. The instructor mentions that sessions start around 9:00 and run until around 5:00, and that students can check the website or posters with QR codes for more information.

The instructor also notes that they are wearing a mask because of a health issue around the throat or voice area, but says their voice should still be clear.

Return to the Main Linear Algebra Problem

[Likely exam topic in transcript]

The lecture returns to the previous discussion: the central problem is the analysis of

$$\mathbf{Ax} = \mathbf{b}.$$

The instructor calls this the “mother of all problems” for the course. The main goals of the analysis are:

- Existence: does a solution exist?
- Uniqueness: if a solution exists, is it the only solution?
- Multiplicity: if it is not unique, how many solutions are there?

The course analyzes these questions using vector space methods.

Two Views of Matrix-Vector Multiplication

Before using vector spaces to analyze $\mathbf{Ax} = \mathbf{b}$, the instructor reviews two interpretations of multiplying a matrix by a vector.

For a matrix $\mathbf{A}$ multiplied on the right by a vector $\mathbf{x}$:

$$\mathbf{Ax}$$

there are two equivalent views:

1. Column-combination view:

$\mathbf{Ax}$ is a linear combination of the columns of $\mathbf{A}$, with the entries of $\mathbf{x}$ as coefficients.

2. Row-inner-product view:

Each entry of $\mathbf{Ax}$ is the inner product of $\mathbf{x}$ with one row of $\mathbf{A}$.

The same kind of interpretation can be made for the transpose matrix $\mathbf{A}^T$, with rows and columns interchanged.

These two perceptions connect matrix multiplication directly to the row space, column space, and the fundamental subspaces.

The Fundamental Spaces Picture

The lecture uses the “fundamental picture” of a matrix as a linear mapping.

For an $m \times n$ matrix $\mathbf{A}$,

$$\mathbf{A} : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

The input space is $\mathbb{R}^n$; the output or target space is $\mathbb{R}^m$. Four fundamental spaces are associated with this mapping:

- Column space / range space of $\mathbf{A}$, in the output space $\mathbb{R}^m$.
- Row space of $\mathbf{A}$, in the input space $\mathbb{R}^n$.

- Null space of A , in the input space $\mathbb{R}^n$.
- Left null space of A , in the output space $\mathbb{R}^m$.

The transcript repeatedly refers to the “green space”; from context this is the column space or range space, the set of all possible outputs of the map.

Column Space and Existence

The column space of A is:

$$\text{Col}(A) = \{Ax : x \text{ in } \mathbb{R}^n\}.$$

Equivalently, it is the span of the columns of A .

The column space is the set of all outputs that can be generated by the mapping $x \rightarrow Ax$. Therefore it answers the existence question for $Ax=b$:

- If $b \text{ in } \text{Col}(A)$, then at least one solution exists.
- If $b \text{ not in } \text{Col}(A)$, then no solution exists.

Important special case:

- If $\text{Col}(A) = \mathbb{R}^m$, then every possible right-hand side $b \text{ in } \mathbb{R}^m$ is reachable.
- In that case, a solution exists for every b .

This is marked in the transcript as important.

Null Space and Uniqueness

[Exam note in transcript]

The null space of A is:

$$N(A) = \{x \text{ in } \mathbb{R}^n : Ax = 0\}.$$

It is a subspace of the input space. The instructor emphasizes that the null space is orthogonal to the row space:

$$N(A) = \text{Row}(A)^\perp.$$

The null space controls uniqueness:

- If the null space is trivial, $N(A) = \{0\}$, then any solution is unique.
- If the null space contains any nonzero vector, then solutions cannot be unique.

Reason:

If x_p is one solution of $Ax=b$ and $z \text{ in } N(A)$, then

$$A(x_p + z) = Ax_p + Az = b + 0 = b.$$

So $x_p + z$ is also a solution. If there is any nonzero z in the null space, then there are infinitely many scalar multiples of z , and therefore infinitely many solutions whenever one solution exists.

The instructor also connects this to the orthogonal decomposition idea: a vector in the input space can be represented using components from the row space and its orthogonal complement, the null space.

Row Rank Equals Column Rank

[Exam note in transcript]

The instructor reviews the theorem:

$$\dim(\text{Row}(A)) = \dim(\text{Col}(A)).$$

The dimension of the column space is called the column rank:

$$R_c = \dim(\text{Col}(A)).$$

The dimension of the row space is called the row rank:

$$R_r = \dim(\text{Row}(A)).$$

The theorem says:

$$R_c = R_r.$$

This common dimension is called the rank of A:

$$r = \text{rank}(A).$$

Proof Idea Reviewed in Lecture

The instructor recalls the proof idea rather than fully redoing it.

One direction:

- Take a basis for the row space.
- Use the matrix mapping to show that the corresponding images are linearly independent in the column-space side.
- This gives an inequality of the form $R_c \geq R_r$.

The other direction:

- Apply the same argument to the transpose matrix A^T .
- Equivalently, start with a basis for the column space.
- This gives $R_r \geq R_c$.

Together:

$$R_c \geq R_r$$

and

$$R_r \geq R_c$$

so:

$$R_c = R_r.$$

This shared dimension is the matrix rank.

Rank Bounds

For an $m \times n$ matrix:

- $\text{Col}(A)$ is a subspace of $\mathbb{R}^m$, so $\text{rank}(A) \leq m$.
- $\text{Row}(A)$ is a subspace of $\mathbb{R}^n$, so $\text{rank}(A) \leq n$.

Therefore:

$$\text{rank}(A) \leq \min(m, n).$$

This upper bound leads to the distinction between full rank and rank-deficient matrices.

Full Rank and Rank Deficiency

The instructor defines the full rank case:

$$\text{rank}(A) = \min(m, n).$$

If this equality holds, A is full rank.

If not:

$$\text{rank}(A) < \min(m, n),$$

then the matrix is rank deficient. The transcript also uses the term “degenerate rank case” or “rank degenerate”; the instructor notes that “rank deficient” is the more common term.

Full rank has different implications depending on the shape of the matrix:

- For a tall matrix, full rank means the row space fills the input space.
- For a fat matrix, full rank means the column space fills the output space.
- For a square matrix, full rank means both happen at once.

Relationship Between Full Rank and Existence/Uniqueness

[Exam note in transcript]

The instructor uses the fundamental spaces picture to connect rank to existence and uniqueness.

Row Space Equals Input Space

If:

$$\text{Row}(A) = \mathbb{R}^n,$$

then:

$$N(A) = \{0\}.$$

This gives uniqueness whenever a solution exists.

This condition can be satisfied only for square or tall matrices, because the row space lies in $\mathbb{R}^n$ and needs enough independent rows to span it.

Column Space Equals Output Space

If:

$$\text{Col}(A) = \mathbb{R}^m,$$

then every $\mathbf{b}$ in $\mathbb{R}^m$ is reachable. This guarantees existence for every right-hand side $\mathbf{b}$.

This condition can be satisfied only for square or fat matrices, because the column space lies in $\mathbb{R}^m$ and needs enough independent columns to span it.

Non-Square Full Rank Matrices

For non-square matrices, full rank can satisfy only one of the two main conditions:

- A full rank tall matrix gives uniqueness, but not existence for every $\mathbf{b}$.
- A full rank fat matrix gives existence for every $\mathbf{b}$, but not uniqueness.

For square matrices, full rank satisfies both conditions.

Tall Matrices

Assume A is tall:

$$m > n.$$

There are more equations than unknowns.

The instructor suggests thinking of a tall matrix as having many rows. Because there are many row vectors, it is possible for the row space to span the whole input space $\mathbb{R}^n$, but this is not automatic.

Full Rank Tall Matrix

If A is tall and full rank, then:

$$\text{rank}(A) = n.$$

Consequences:

- $\text{Row}(A) = \mathbb{R}^n$.
- $N(A) = \{0\}$.
- If a solution exists, it is unique.

However, since the column space is a subspace of $\mathbb{R}^m$ with dimension at most n , and $n < m$, the column space cannot cover all of $\mathbb{R}^m$. Therefore:

- Existence is not guaranteed for arbitrary b .
- Some right-hand sides b will not lie in $\text{Col}(A)$.

So:

- If $b \in \text{Col}(A)$, there is a unique solution.
- If $b \notin \text{Col}(A)$, there is no solution.

Main point: for tall matrices, full rank is an indicator of uniqueness, not a guarantee of existence for every b .

Rank-Deficient Tall Matrix

If a tall matrix is rank deficient:

$$\text{rank}(A) < n.$$

Then:

- The row space does not fill $\mathbb{R}^n$.
- The null space is nontrivial.
- There are infinitely many vectors in the null space.

Therefore:

- If $b \in \text{Col}(A)$, then there are infinitely many solutions.
- If $b \notin \text{Col}(A)$, then there is no solution.

The instructor emphasizes that even though a tall matrix has many rows, linear dependence among the rows can prevent the row space from filling the input space.

Fat Matrices

Assume A is fat:

$$m < n.$$

There are more unknowns than equations.

The instructor says this is the opposite of the tall case: now there are many columns, so it may be possible for the column space to span the whole target space $\mathbb{R}^m$.

Full Rank Fat Matrix

If A is fat and full rank, then:

$$\text{rank}(A) = m.$$

Consequences:

- $\text{Col}(A) = \mathbb{R}^m$.
- The left null space is trivial.
- A solution exists for every b in $\mathbb{R}^m$.

But uniqueness is impossible for a fat matrix, because there are more unknowns than equations. The null space is nontrivial no matter what the rank is.

Therefore:

- For every b , there is at least one solution.
- For every b , there are actually infinitely many solutions.

Main point: for full rank fat matrices, existence for every b is guaranteed, but uniqueness is impossible.

Rank-Deficient Fat Matrix

If a fat matrix is rank deficient:

$$\text{rank}(A) < m.$$

Then:

- The column space does not span all of $\mathbb{R}^m$.
- The left null space is nontrivial.
- There are right-hand sides b outside the column space.

Therefore:

- If $b \notin \text{Col}(A)$, there is no solution.
- If $b \in \text{Col}(A)$, there are infinitely many solutions.

The instructor again notes terminology: “rank deficient” is the standard phrase, while “degenerate” means the matrix does not achieve the full possible rank $\min(m, n)$.

Square Matrices

Assume A is square:

$$m = n.$$

This is the only matrix shape where it is possible to satisfy both existence for every b and uniqueness at the same time.

Full Rank Square Matrix

If A is square and full rank:

$$\text{rank}(A) = m = n.$$

Consequences:

- $\text{Col}(A) = \mathbb{R}^m$.
- $\text{Row}(A) = \mathbb{R}^n$.
- The null space is trivial.
- The left null space is trivial.
- For every b , a solution exists.

- Since the null space is trivial, that solution is unique.

Such a matrix is invertible.

The unique solution is:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}.$$

The instructor also says that in this case the matrix has a unique left inverse and a unique right inverse, and they are the same object, the inverse $\mathbf{A}^{-1}$.

Rank-Deficient Square Matrix

If a square matrix is rank deficient:

$$\text{rank}(\mathbf{A}) < n,$$

then:

- The row space does not fill the input space.
- The column space does not fill the output space.
- The null space is nontrivial.
- The left null space is nontrivial.

Therefore:

- For $\mathbf{b}$ in $\text{Col}(\mathbf{A})$, there are infinitely many solutions.
- For $\mathbf{b}$ not in $\text{Col}(\mathbf{A})$, there is no solution.

Example: Identical Rows or Columns

The instructor describes a rank-deficient square example where rows or columns are identical. In a 2×2 example, both the row space and column space may be one-dimensional lines inside a two-dimensional ambient space.

If $\mathbf{b}$ is not on the column-space line, there is no solution.

If $\mathbf{b}$ lies on the column-space line, solutions exist, but because the null space is nontrivial, there are infinitely many solutions.

Left and Right Inverses for Non-Square Matrices

The lecture briefly discusses one-sided inverses for non-square matrices.

For a full rank tall matrix $\mathbf{A}$ with shape $m \times n$, $m > n$, the natural one-sided inverse is a left inverse:

$$\mathbf{L} \mathbf{A} = \mathbf{I}_n.$$

For a full rank fat matrix $\mathbf{A}$ with shape $m \times n$, $m < n$, the natural one-sided inverse is a right inverse:

$$\mathbf{A} \mathbf{R} = \mathbf{I}_m.$$

The transcript is noisy in this part, but the conceptual point is that non-square full rank matrices can have one-sided inverses, while a full rank square matrix has a unique two-sided inverse.

For full rank square matrices:

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

and

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

The same unique inverse is both a left and right inverse.

For non-square matrices, the side matters because the identity matrix has a different size on each side. A full rank tall matrix can satisfy $\mathbf{LA} = \mathbf{I}_n$, but it cannot generally satisfy $\mathbf{AR} = \mathbf{I}_m$; the latter would require spanning the larger output space. A full rank fat matrix can satisfy $\mathbf{AR} = \mathbf{I}_m$, but it cannot generally satisfy $\mathbf{LA} = \mathbf{I}_n$; the latter would require uniqueness in the larger input space.

The instructor also indicates that these one-sided inverses are not unique in the non-square full-rank cases:

- A full rank tall matrix generally has infinitely many left inverses.
- A full rank fat matrix generally has infinitely many right inverses.
- A full rank square matrix is the special case where the left and right inverse coincide and are unique.

When Uniqueness Is Useful and When Non-Uniqueness Is Useful

The instructor emphasizes that uniqueness is not always “good” and non-uniqueness is not always “bad.” It depends on the problem.

Estimation Example: Multi-User Signal Recovery

In an estimation problem, uniqueness is usually desired.

The instructor gives a communications example:

- Multiple users transmit signals.
- They transmit at the same time.
- They use the same frequency band.
- They interfere with one another.
- Such users are called co-channel users.
- A base station with multiple antennas receives mixtures of their signals.

Each antenna receives a linear combination of user signals. Conceptually:

$$y_i = h_{i1}x_1 + h_{i2}x_2 + \dots + h_{in}x_n.$$

Collecting all antenna observations gives a system:

$$\mathbf{y} = \mathbf{A}\mathbf{x}.$$

Here:

- $\mathbf{x}$ contains the individual transmitted user symbols or signals.
- $\mathbf{y}$ contains antenna observations.
- $\mathbf{A}$ contains channel coefficients, delays, gains, or related mixing parameters.

Goal:

Recover the actual transmitted user signals $\mathbf{x}$ without ambiguity.

In this estimation setting, infinitely many possible solutions are not useful. The receiver wants the actual transmitted symbols, not a set of equally valid candidates.

Without any prior assumptions on $\mathbf{x}$, the previous linear algebra analysis says:

- The number of equations must be at least the number of unknowns.
- In this communication example, the number of antenna observations should be at least the number of users.

In rough terms:

$$\text{number of antennas / observations} \geq \text{number of users}.$$

Prior Information Can Reduce Required Measurements

The instructor adds an important qualification:

The current analysis assumes no prior information about the source vector $\mathbf{x}$. It allows $\mathbf{x}$ to be any point in the full n -dimensional input space.

If additional structure is known, then fewer equations may be enough.

Examples of useful prior information:

- $\mathbf{x}$ lies in some restricted subset of $\mathbb{R}^n$.
- $\mathbf{x}$ is sparse: most entries are zero, but the zero locations are not known.
- The signals have other known structure, possibly such as coding or correlation properties.

For sparse vectors, the instructor says later techniques can recover the vector with high probability under assumptions on $\mathbf{A}$, even with fewer equations than unknowns.

The instructor says this will be discussed later and connects it to future homework or later course material. The current rank-based counting discussion assumes no prior information; the sparse or structured-signal case is a qualification, not the model being analyzed in this part of the lecture.

Control Example: Non-Uniqueness as Flexibility

The instructor contrasts estimation with control.

In a simple memoryless linear control system:

$$\mathbf{y} = \mathbf{A}\mathbf{x}.$$

Here:

- $\mathbf{x}$ is the input or control signal.
- $\mathbf{y}$ is the output.
- The goal is to choose $\mathbf{x}$ so that the system generates a desired target output $\mathbf{y}$.

If $\mathbf{A}$ is fat and full rank, then:

- Every target output $\mathbf{y}$ can be generated.
- For each target output, there are infinitely many inputs $\mathbf{x}$ that produce it.

In this setting, non-uniqueness is useful. It gives design freedom.

For example, among infinitely many inputs that generate the same target output, one can choose the input with smaller energy.

So:

- Estimation problems usually want uniqueness.
- Control problems can benefit from non-uniqueness.

The instructor frames control as a dual problem to estimation in this sense.

New Topic: Solving Linear Systems Through Matrix Factorization

[Exam note in transcript]

After finishing the analysis of existence and uniqueness, the lecture starts a new journey.

The stated goal is not mainly to study numerical algorithms in detail. Instead, solving $\mathbf{A}\mathbf{x}=\mathbf{b}$ is used as a motivation or excuse to introduce important matrix factorizations.

The central strategy:

Convert a hard system of linear equations into a sequence of simple problems.

The instructor emphasizes that this picture will lead to many important results in linear algebra.

Hard Problems and Simple Problems

For an arbitrary matrix A , solving:

$$Ax = b$$

can be hard because the equations are coupled. Each equation may involve many or all unknowns.

The instructor warns that being square, or even having some special-looking structure such as symmetry, is not by itself the notion of “simple” being used here. The difficulty is coupling: if each equation mixes many unknowns, then equal numbers of equations and unknowns do not automatically make the system easy.

The idea is to rewrite or transform the problem so that it becomes a sequence of problems involving special simple matrices.

This is exactly what matrix factorization does:

$$A = A_1 A_2 \dots A_k,$$

where each factor A_i is a simple matrix.

Then:

$$Ax = b$$

becomes:

$$A_1 A_2 \dots A_k x = b,$$

which can be solved step by step.

The instructor says that famous factorizations are essentially different ways of writing a matrix as a product of simple matrices.

Simple Matrix Type 1: Diagonal Matrices

The simplest case is a diagonal matrix.

For a diagonal matrix:

```
D =
\begin{bmatrix}
d_1 & 0 & \cdots & 0 \\
0 & d_2 & \cdots & 0 \\
\vdots & \vdots & \ddots & \vdots \\
0 & 0 & \cdots & d_n
\end{bmatrix},
```

the system:

$$Dx = b$$

is uncoupled:

$$d_i x_i = b_i.$$

Each unknown can be solved independently:

$$x_i = b_i / d_i.$$

For a square diagonal matrix to be full rank, every diagonal entry must be nonzero:

$$d_i \neq 0 \quad \text{for all } i.$$

The instructor notes that this is simple because there is no coupling among the variables.

Student Remark: Decoupled Does Not Have to Mean Diagonal

A student points out that the nonzero entries need not literally be on the main diagonal for the system to be easy. If each equation involves only one variable and each variable appears separately, the system is still decoupled, even if the nonzero entries are permuted away from the diagonal.

The instructor agrees. Such a matrix can be represented using a diagonal matrix times a permutation matrix.

This leads naturally to permutation matrices, discussed later as a special case of orthogonal matrices.

Simple Matrix Type 2: Triangular Matrices

The second simple case is a triangular matrix.

Gaussian elimination aims to transform a general system into triangular form, often an upper triangular or row echelon form.

Upper Triangular Case

For an upper triangular matrix U , all entries below the diagonal are zero.

In a system:

$$Ux = b,$$

the last row involves only the last unknown x_n . So:

1. Solve the last equation for x_n .
2. Move one row up.
3. That row involves x_{n-1} and x_n , but x_n is already known.
4. Substitute the known value of x_n and solve for x_{n-1} .
5. Continue upward.

This procedure is called back substitution.

The instructor says Gaussian elimination tries to reach triangular form and then uses back substitution to solve the system.

Solving a triangular system is equivalent to applying the inverse triangular matrix, but the instructor stresses it as an iterative substitution procedure rather than explicitly forming the inverse.

Lower Triangular Case

For a lower triangular matrix L , all entries above the diagonal are zero.

The solution order is the reverse of the upper triangular case:

1. Start from the first row.
2. Solve for x_1 .
3. Move to the second row and solve for x_2 , using the known value of x_1 .
4. Continue downward.

This is the dual of the upper triangular case. The standard name is forward substitution, although the transcript mostly describes it rather than naming it.

Simple Matrix Type 3: Orthogonal Matrices

[Exam note in transcript]

The third simple case is an orthogonal matrix. The transcript sounds like “alternate” in places, but the mathematical condition described is the orthogonal condition.

For a real orthogonal matrix Q :

$$Q^{-1} = Q^T.$$

Equivalently:

$$Q^T Q = I.$$

This makes solving:

$$Qx = b$$

easy:

$$x = Q^T b.$$

The inverse is cheap to compute because it is just the transpose.

The instructor notes that for complex matrices the definition must be modified; the corresponding operation would use the conjugate transpose rather than the ordinary transpose.

Permutation Matrices

Permutation matrices are examples of orthogonal matrices.

Example:

```
P =
\begin{bmatrix}
0 & 1 \\
1 & 0
\end{bmatrix}.
```

This matrix swaps two components. It is symmetric in this example:

$$P^T = P.$$

It is also orthogonal:

$$P^T P = I.$$

Permutation matrices can reorder rows or columns. The instructor connects them to the earlier student remark: a system that is decoupled but not diagonal can be viewed as a diagonal system combined with a permutation.

Famous Factorizations as Products of Simple Matrices

The instructor says that famous matrix factorizations are built from combinations of these simple matrix types:

- Diagonal matrices.
- Triangular matrices.
- Orthogonal matrices.
- Permutation matrices.

The general theme is:

hard matrix = product of simple matrices.

Then:

hard problem = sequence of simple problems.

Singular Value Decomposition

[Exam note in transcript]

The lecture mentions singular value decomposition as one example of this philosophy:

$$A = U \Sigma V^T.$$

Here:

- U is orthogonal.
- V is orthogonal.
- Σ is diagonal, or diagonal-like in the rectangular case.

So SVD writes a general matrix as a product of simple matrices.

The instructor says this is part of the “story of all factorizations.”

Symmetric Matrix Decomposition

For symmetric matrices, the lecture mentions a decomposition of the form:

$$A = Q \Lambda Q^T.$$

Here:

- Q is orthogonal.
- Λ is diagonal.

Again, the matrix is converted into simple pieces.

This foreshadows diagonalization and eigenvalue-related topics.

QR Factorization

The lecture then focuses on QR factorization as an example of turning one hard problem into two simple ones.

In QR factorization:

$$A = QR.$$

where:

- Q is orthogonal.
- R is upper triangular.

The instructor states that such a factorization can be obtained for matrices under the conditions to be discussed later.

Solving $Ax=b$ Using QR

Start with:

$$Ax = b.$$

If:

$$A = QR,$$

then:

$$QRx = b.$$

Define:

$$y = Rx.$$

Then the original system becomes two systems:

$$Qy = b$$

and

$$Rx = y.$$

Because Q is orthogonal:

$$y = Q^T b.$$

Then solve:

$$Rx = y$$

by back substitution, because R is upper triangular.

So the hard problem $Ax=b$ becomes:

1. Multiply by Q^T to solve the orthogonal part.
2. Use back substitution to solve the triangular part.

This is the core factorization perspective: factorization converts one hard system into a sequence of simple systems.

Software Remark: MATLAB Backslash Operator

The instructor makes an extended remark about software.

In MATLAB, writing:

$$x = A \backslash b$$

solves a system of linear equations. The instructor says it looks like division, but that appearance is misleading. It is actually a powerful operator that chooses appropriate linear algebra methods, often based on factorizations.

The instructor notes that the backslash operator is especially powerful because it also handles non-square systems and can return special solutions. The transcript says something like “minimum norm”; in underdetermined systems, this means a solution with minimum norm, while in overdetermined systems such tools often relate to least-squares solutions.

The instructor contrasts MATLAB with Python, where one typically has to call a linear algebra function more explicitly, such as from NumPy or SciPy. The exact Python function is not clearly stated in the transcript.

There is also a joking remark that one could ask Codex to solve the system in MATLAB or Python, but the instructor does not want to spend months debugging code.

LU Factorization and Gaussian Elimination

The next example is LU factorization:

$$A = LU.$$

Here:

- L is lower triangular.
- U is upper triangular.

The instructor notes a qualification:

Not every matrix can be written directly as LU without additional operations. Sometimes a permutation matrix is needed, for row exchanges or pivoting. This leads to forms such as a permutation times A being factored into L and U .

The core idea remains the same: A is represented as a product of simple matrices.

The instructor identifies Gaussian elimination as the process of finding lower and upper triangular factors.

Solving $Ax=b$ Using LU

If:

$$A = LU,$$

then:

$$LUx = b.$$

Define:

$$y = Ux.$$

Then solve two triangular systems:

$$Ly = b$$

and

$$Ux = y.$$

Since L is lower triangular, solve $Ly=b$ from top to bottom.

Since U is upper triangular, solve $Ux=y$ from bottom to top using back substitution.

Again, a hard system is transformed into a sequence of simple systems.

At the comparison level, QR and Gaussian elimination both aim to reach an upper triangular problem, but they use different simple transformations. QR uses orthogonal matrices together with an upper triangular factor; Gaussian elimination uses lower triangular row-operation matrices to produce an upper triangular matrix.

Gaussian Elimination as Row Operations

[Exam note in transcript]

The instructor connects LU factorization to row operations from homework zero.

The goal of Gaussian elimination is to eliminate entries below the diagonal to obtain an upper triangular matrix.

For example, suppose the first column has entries below the first pivot. To eliminate them, one might use operations such as:

$$R_2 \rightarrow R_2 - 2R_1$$

and

$$R_3 \rightarrow R_3 - 3R_1.$$

After these operations, the entries below the first pivot are zero. This is described as the “first victory” toward an upper triangular matrix.

Then one proceeds to the next column. To eliminate the entry below the second pivot, use row 2 rather than row 1, because using row 1 again may destroy zeros already created.

The instructor emphasizes the algorithmic nature of the row replacement notation.

Row Operations as Left Multiplication

The instructor makes an important conceptual point:

Row operations are matrix multiplications.

Specifically, row operations correspond to left multiplication by another matrix.

Reason:

- A row operation forms new rows as linear combinations of old rows.
- Left multiplication takes linear combinations of rows.
- Right multiplication would take linear combinations of columns.

So Gaussian elimination can be expressed as:

$E A,$

where E is an elimination matrix.

Example of an Elimination Matrix

For the operations:

$R_2 \rightarrow R_2 - 2R_1$

and

$R_3 \rightarrow R_3 - 3R_1,$

the corresponding left-multiplying matrix has rows:

```
\begin{bmatrix}
1 & 0 & 0 \\
-2 & 1 & 0 \\
-3 & 0 & 1
\end{bmatrix}.
```

The first row $[1,0,0]$ means row 1 is unchanged.

The second row $[-2,1,0]$ means:

$\text{new } R_2 = -2 R_1 + R_2.$

The third row $[-3,0,1]$ means:

$\text{new } R_3 = -3 R_1 + R_3.$

This matrix is lower triangular.

Causality Rule in Gaussian Elimination

The instructor describes a “causality” rule in Gaussian elimination.

Standard elimination uses earlier rows, with lower row index, to modify later rows, with higher row index.

For example:

- Use row 1 to eliminate entries in rows 2 and 3.
- Then use row 2 to eliminate entries in lower rows.
- Do not use a lower row to change an earlier row during ordinary forward elimination.

This rule is what makes the elimination matrices lower triangular.

In a lower triangular left-multiplying matrix:

- Row 1 can use only row 1.

- Row 2 can use rows 1 and 2.
- Row 3 can use rows 1, 2, and 3.

There are no entries above the diagonal because earlier output rows do not use later input rows.

This is a key relationship:

Gaussian elimination row operations $\leftrightarrow$ lower triangular elimination matrices.

From Elimination Matrices to LU

Through Gaussian elimination, one applies a sequence of lower triangular elimination matrices:

$$E_k \cdots E_2 E_1 A = U.$$

Here U is upper triangular.

This equation is not yet written as a factorization of A . To express A itself, move the elimination matrices to the other side by taking inverses:

$$A = E_1^{-1} E_2^{-1} \cdots E_k^{-1} U.$$

The product:

$$L = E_1^{-1} E_2^{-1} \cdots E_k^{-1}$$

is lower triangular, so:

$$A = LU.$$

Why L Is Lower Triangular

The instructor gives two structural facts:

1. The inverse of a lower triangular matrix is lower triangular.
2. The product of lower triangular matrices is lower triangular.

Therefore, when we multiply inverses of elimination matrices together, the result is still lower triangular.

The elimination matrices used in the lecture are also unit lower triangular: their diagonal entries are 1. This reflects that a row is kept with coefficient 1 while multiples of earlier rows are added or subtracted below the diagonal. Their inverses remain unit lower triangular.

The transcript also notes that for the special elimination matrices used in Gaussian elimination, the signs of the below-diagonal multipliers flip when taking inverses. For example, elimination coefficients that appear as negative values in E become positive entries in E^{-1} and hence in L .

This is why Gaussian elimination using lower triangular elimination matrices leads to an LU factorization.

Row Exchanges and Permutation Matrices

Sometimes Gaussian elimination requires exchanging rows, either at the beginning or during the process. This is often needed to avoid a zero or bad pivot.

Row exchanges are represented by permutation matrices.

A row-swap permutation matrix is orthogonal:

$$P^T P = I.$$

Example: a permutation matrix that swaps rows 1 and 2 makes:

- the first row of the output equal to the second row of the input,
- the second row of the output equal to the first row of the input.

Because of row exchanges, a more general factorization may involve a permutation matrix together with L and U, rather than a plain $A=LU$.

The transcript mentions left and right multiplication by permutation matrices; for row exchanges specifically, the relevant operation is left multiplication.

Lower Triangular Matrices and Causal Systems

The instructor highlights a system-theoretic interpretation of lower triangular matrices.

If the indices represent time, a lower triangular matrix corresponds to a causal system.

For example:

$$y = Lx.$$

For a three-sample causal system, the lower triangular form is:

```
\begin{bmatrix}
y_1 \\
y_2 \\
y_3
\end{bmatrix}
=
\begin{bmatrix}
h_{11} & 0 & 0 \\
h_{21} & h_{22} & 0 \\
h_{31} & h_{32} & h_{33}
\end{bmatrix}
\begin{bmatrix}
x_1 \\
x_2 \\
x_3
\end{bmatrix}.
```

With a lower triangular L:

y_1

depends only on:

x_1 .

Similarly:

y_2

can depend on:

x_1, x_2 .

And:

y_3

can depend on:

x_1, x_2, x_3 .

But no output at time i depends on a future input x_j with $j > i$.

This is exactly the causality condition:

present output does not depend on future input.

The instructor says this is one reason lower triangular matrices have a special place in system theory.

Homework Remarks

[Exam note in transcript]

The instructor refers to “homework zero” when discussing Gaussian elimination row operations.

The instructor also says “do homework” twice near the discussion of lower triangular matrices and causal systems, implying that the related homework reinforces these ideas and should work out if done.

Factorization Summary From This Lecture

The lecture’s main conceptual chain is:

1. Start with a hard linear system:

$$\mathbf{Ax} = \mathbf{b}.$$

2. Recognize that some matrices make systems easy:

- Diagonal matrices give uncoupled equations.
- Triangular matrices can be solved by substitution.
- Orthogonal matrices are inverted by transposition.
- Permutation matrices just reorder rows or columns.

3. Factor a hard matrix into simple factors:

$$\mathbf{A} = \mathbf{A}_1 \mathbf{A}_2 \cdots \mathbf{A}_k.$$

4. Replace the hard system with a sequence of simple systems.

5. This viewpoint leads to QR, LU, SVD, and diagonalization-type factorizations.

Next Lecture Preview

The instructor says the next lecture will continue this story.

The next question will be whether one can use smart transformations to convert a matrix into diagonal form. Diagonal form is especially desirable because diagonal systems are the easiest to solve.

This foreshadows diagonalization and related matrix decompositions.

The instructor also mentions that there will be no class on Tuesday and asks students to attend or register for the symposium. The instructor plans to send the symposium website.

Source and Coverage Note

These notes were created only from `corrected/lecture7_corrected.md`. They preserve the lecture’s chronological flow and include the transcript’s marked exam notes, definitions, proof ideas, examples, instructor remarks, warnings about terminology, software comments, homework reminders, and relationships among the concepts. Some noisy transcript phrases were interpreted in context, especially “green space” as column/range space and “alternate” as orthogonal matrix; where the wording was uncertain, the surrounding explanation was retained rather than compressed away.

Lecture 08 Notes

1. Big Picture: Linear Systems Through Matrix Factorization

The lecture begins by continuing the philosophy for solving systems of linear equations.

The instructor emphasized that some systems are easy to solve because their coefficient matrices have simple structure:

- **Diagonal form:** the simplest case; equations decouple completely.
- **Triangular form:** easy to solve by forward or back substitution.
- **Orthogonal form:** easy because orthogonal transformations preserve lengths and have simple inverses.

For a general system of linear equations, the strategy is to convert the original system into a sequence of easier systems involving these simple matrix types. This conversion is equivalent to factoring the original matrix into a product of simple matrices.

This viewpoint explains many famous linear algebra factorizations:

- If $A = QR$, where Q is orthogonal and R is upper triangular, then solving $Ax = b$ becomes solving a sequence involving an orthogonal matrix and an upper triangular matrix. The instructor noted that the solver used in the course/platform is based on QR factorization.
- Gaussian elimination is also a matrix factorization. The row operations in Gaussian elimination can be viewed as multiplication from the left by lower triangular matrices.
- Gaussian elimination uses lower triangular matrices to convert a matrix into upper triangular form, producing an LU -type factorization.
- Sometimes row exchanges are needed, so a permutation matrix P is included. A permutation matrix is orthogonal, so the factorization still uses simple matrices.

The unifying idea is:

$$\text{hard matrix} = \text{product of simple matrices.}$$

Then solving one hard system is replaced by solving several easier systems.

2. Diagonalization as a Guiding Theme

Likely exam topic

The instructor then introduced the main storyline of the lecture: the desire to convert matrices into diagonal form.

Diagonal matrices are the simplest matrices, so the diagonalization principle has been central in the development of algorithms. The lecture will explore what can and cannot be done when trying to turn a matrix into diagonal form.

The main related ideas are:

- basis change,
- representation of linear transformations under a new basis,
- eigenvalues and eigenvectors,
- diagonalizable and non-diagonalizable matrices,
- triangularization as a weaker but always possible goal for square matrices,
- singular value decomposition as a more flexible diagonalization framework.

The instructor described the situation as having:

- some good news,
- some bad news,
- and some consolation results,

eventually leading toward SVD.

3. Basis Change for Vectors

The first step is to understand basis change in a vector space.

By default, vectors in $\mathbb{R}^2$ or $\mathbb{R}^n$ are represented with respect to the **standard basis**. In $\mathbb{R}^2$, this means

$$x = x_1 e_1 + x_2 e_2.$$

The coordinates x_1, x_2 are standard coordinates, and e_1, e_2 are the standard coordinate vectors. These standard basis vectors are unit-norm vectors aligned with the coordinate axes, giving the default coordinate system.

However, we can choose a different basis. The new basis vectors do not have to be orthogonal. The only requirements are:

- they span the vector space,
- they are linearly independent.

For example, in $\mathbb{R}^2$, choose basis vectors t_1, t_2 . Then the same vector x can be represented as

$$x = \tilde{x}_1 t_1 + \tilde{x}_2 t_2.$$

The coordinates $\tilde{x}_1, \tilde{x}_2$ are the coordinates of the same vector x with respect to the new basis.

[Exam note]

The instructor emphasized the matrix multiplication interpretation: if basis vectors are placed as columns of a matrix, then multiplying that matrix by a coordinate vector forms the corresponding linear combination of the columns.

Define the basis matrix

$$T = [t_1 \ t_2].$$

Then

$$x = T\tilde{x}.$$

Here:

- x is the vector in standard coordinates,
- T is the basis matrix,
- $\tilde{x}$ is the coordinate vector in the new basis.

Because T has linearly independent columns and is square, it is full rank and invertible. Therefore,

$$\tilde{x} = T^{-1}x.$$

So:

- To go from new coordinates to standard coordinates, multiply by T .
- To go from standard coordinates to new coordinates, multiply by T^{-1} .

This is only a **representation change**. No linear transformation has been applied yet.

The same construction extends directly to $\mathbb{R}^n$. For an n -dimensional basis $t_1, \dots, t_n$, define

$$T = [t_1 \ t_2 \ \cdots \ t_n].$$

Then

$$x = T\tilde{x}, \quad \tilde{x} = T^{-1}x.$$

4. Basis Change for Linear Transformations

The discussion then moves from representing vectors to representing linear transformations.

The lecture focuses first on square matrices, because the immediate goal is diagonalization of square matrices. A square matrix represents a mapping from $\mathbb{R}^n$ to $\mathbb{R}^n$. Since input and output live in spaces of the same dimension, it makes sense to ask whether the same basis can be used for both.

For a linear mapping

$$y = Ax,$$

suppose x and y are represented in standard coordinates. Now choose a new basis T and represent both input and output in that same basis:

$$x = T\tilde{x}, \quad y = T\tilde{y}.$$

The actual mapping does not change. The same point still maps to the same point. What changes is the matrix representation of that mapping.

Substitute the new-coordinate expressions into $y = Ax$:

$$T\tilde{y} = AT\tilde{x}.$$

Multiply by T^{-1} on the left:

$$\tilde{y} = T^{-1}AT\tilde{x}.$$

Therefore the new matrix representation is

$$\tilde{A} = T^{-1}AT.$$

This matrix represents the same linear transformation, but with respect to the new basis.

The relationship

$$\tilde{A} = T^{-1}AT$$

is the central basis-change formula for square matrices when the same basis is used for input and output.

5. The Diagonalization Question

Once the basis-change formula is available, the natural question is:

Can we choose a smart basis T such that $T^{-1}AT$ is diagonal?

If such a basis exists, then the linear transformation is represented by a diagonal matrix in that basis.

The instructor called this the diagonalization trick, but also stressed that this is a **restricted** version of diagonalization because the same basis is being used for both input and output.

This restriction matters:

- If the same basis must be used for input and output, not every square matrix can be diagonalized.
- If different bases are allowed for input and output, then the situation becomes more flexible, and every matrix can be represented in a diagonal-type form. This leads to singular value decomposition.

For now, the main question is:

Given a square matrix A , can we find an invertible T such that $T^{-1}AT$ is diagonal?

This question leads directly to eigenvalues and eigenvectors.

[Exam note]

The instructor said to “put this in your pocket” because the same question will return repeatedly: diagonalization by basis change is ultimately tied to eigenvectors.

6. Eigenvectors: Geometric Motivation

For a square matrix A , the mapping $x \mapsto Ax$ sends a vector in $\mathbb{R}^n$ back into the same space.

Usually, Ax points in a different direction from x , and it may also have a different length.

The eigenvector question is:

Are there special directions x such that Ax points in the same or opposite direction as x ?

Opposite direction is allowed because that is still the same line, just scaled by a negative number.

If Ax lies in the same direction as x , then A acts like scalar multiplication on that vector:

$$Ax = \lambda x.$$

Here:

- $x \neq 0$ is an eigenvector,
- λ is the corresponding eigenvalue.

The zero vector is excluded because it satisfies $A0 = \lambda 0$ trivially for every λ , which carries no directional information.

7. Algebraic Derivation of the Eigenvalue Condition

Start with

$$Ax = \lambda x.$$

Move the right-hand side to the left:

$$Ax - \lambda x = 0.$$

The two terms are not in the same matrix-multiplication form yet: Ax is a matrix-vector product, while λx is scalar multiplication.

Use the identity matrix to rewrite scalar multiplication:

$$\lambda x = \lambda Ix.$$

Then

$$Ax - \lambda Ix = 0,$$

so

$$(A - \lambda I)x = 0.$$

The question becomes:

Can we find λ such that $A - \lambda I$ has a nonzero null-space vector?

Equivalently:

- x must be a nonzero vector in the null space of $A - \lambda I$,
- therefore $A - \lambda I$ must have a nontrivial null space,
- therefore $A - \lambda I$ must be rank deficient.

The instructor explained this as subtracting the same scalar λ from all diagonal entries of A and asking whether the resulting matrix becomes rank deficient.

If A is already rank deficient, then $\lambda = 0$ is an eigenvalue, because $A - 0I = A$ already has a nontrivial null space. The null-space vectors of A are eigenvectors corresponding to eigenvalue 0.

For square matrices, rank deficiency is equivalent to determinant zero. Therefore the eigenvalue condition is

$$\det(A - \lambda I) = 0.$$

The lecture also used

$$\det(\lambda I - A) = 0.$$

For the purpose of finding roots, either form can be used; the sign convention changes only by a possible factor depending on dimension.

8. Determinant as Volume Scaling

To motivate why determinant zero corresponds to rank deficiency, the instructor reviewed determinant as volume scaling.

For a 2×2 diagonal mapping, one basis direction might be scaled by 2 and the other by 4. A unit square is mapped to a rectangle with area

$$2 \cdot 4 = 8.$$

For a diagonal matrix, this area scale is the product of the diagonal entries, which is the determinant.

The instructor also gave the geometric reason that the image of the square is determined by the images of its edge vectors. Points on an edge can be written as convex combinations of the edge endpoints, and a linear map sends those convex combinations to the same convex combinations of the mapped endpoints. This is why the transformed square becomes the rectangle or parallelogram spanned by the images of the basis directions.

More generally:

- A square region maps to a parallelogram.
- A circular region maps to an ellipse; in the same diagonal example, its area is also scaled by the determinant factor 8.
- In higher dimensions, a unit volume maps to a parallelepiped.
- The determinant gives the signed volume-scaling factor.

If the determinant is 1, the mapped parallelogram or parallelepiped has the same area or volume as the original unit region, even though its shape may have changed.

If a mapping is rank deficient, it collapses the input space into a lower-dimensional subspace. For example, a two-dimensional region may be mapped into a one-dimensional line. In the ambient two-dimensional output space, the area is then zero.

Thus:

$$A \text{ rank deficient} \iff \det(A) = 0$$

for square matrices.

This explains why the condition for eigenvalues is a determinant equation.

9. Characteristic Polynomial

Likely exam topic

[Exam note]

The instructor said that in homework one can show that

$$\det(\lambda I - A)$$

is an n -degree polynomial in λ for an $n \times n$ matrix A .

This polynomial is called the **characteristic polynomial** of A :

$$p_A(\lambda) = \det(\lambda I - A).$$

It is a **monic polynomial**, meaning the coefficient of the highest-degree term λ^n is 1.

The eigenvalue problem can therefore be rephrased:

Find the roots of the characteristic polynomial.

Those roots are the eigenvalues of A .

The instructor connected this to the historical problem of solving polynomial equations:

- Degree 1 equations are simple.
- Degree 2 equations have the quadratic formula.
- Degree 3 and degree 4 also have formulas.
- For degree 5 and higher, there is no general formula in radicals for all polynomial roots.

However, this does not mean roots do not exist.

By the **fundamental theorem of algebra**, every degree n polynomial has n complex roots counted with multiplicity. Some roots may repeat. Even if A is a real matrix, its eigenvalues can be complex.

Therefore, over the complex numbers, every square matrix has eigenvalues.

Once an eigenvalue λ_i is found, the corresponding eigenvectors are the nonzero vectors in

$$\text{null}(\lambda_i I - A)$$

or equivalently in

$$\text{null}(A - \lambda_i I).$$

This null space is called the **eigenspace** associated with λ_i .

If x is an eigenvector, then any nonzero scalar multiple of x is also an eigenvector for the same eigenvalue. For example, $5x$ and $10x$ are also eigenvectors. Thus each eigenspace contains infinitely many eigenvectors unless it is trivial, and the eigenvectors are the nonzero vectors in that eigenspace.

10. Connecting Eigenvectors Back to Diagonalization

Now suppose A is an $n \times n$ matrix and we choose n eigenvectors

$$t_1, t_2, \dots, t_n.$$

For each i ,

$$At_i = \lambda_i t_i.$$

At this point the λ_i 's do not have to be distinct. Repeated eigenvalues are allowed; the only assumption is that the selected columns t_i are eigenvectors of A .

[Exam note]

The instructor deliberately reused the notation t_i because earlier t_i denoted basis vectors. The goal is to connect eigenvectors to the basis matrix T .

Put the eigenvectors into a matrix as columns:

$$T = [t_1 \ t_2 \ \cdots \ t_n].$$

Then left-multiply by A :

$$AT = A[t_1 \ t_2 \ \cdots \ t_n].$$

Using column-partitioned matrix multiplication:

$$AT = [At_1 \ At_2 \ \cdots \ At_n].$$

Since each t_i is an eigenvector:

$$AT = [\lambda_1 t_1 \ \lambda_2 t_2 \ \cdots \ \lambda_n t_n].$$

This can be written as

$$AT = T\Lambda,$$

where

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}.$$

The instructor highlighted the column-scaling interpretation:

- Left-multiplying T by A applies A to each column.
- Right-multiplying T by a diagonal matrix scales each column by the corresponding diagonal entry.

If T is invertible, then multiply $AT = T\Lambda$ on the left by T^{-1} :

$$T^{-1}AT = \Lambda.$$

So the original diagonalization question is equivalent to:

Can we find n linearly independent eigenvectors of A ?

Because T is invertible exactly when its columns form a linearly independent set.

11. Diagonalizable and Non-Diagonalizable Matrices

[Exam note]

The instructor restated the key requirement:

To diagonalize A by a basis change, we need an invertible matrix T whose columns are eigenvectors of A . Equivalently, we need n linearly independent eigenvectors.

This leads to a classification of square matrices:

- **Diagonalizable matrices:** matrices for which we can find n linearly independent eigenvectors.
- **Non-diagonalizable matrices:** matrices for which we cannot find n linearly independent eigenvectors.

The answer to the diagonalization question is not always positive. Some matrices are diagonalizable; some are not.

Non-Diagonalizable Example

The instructor gave an “innocent looking” 2×2 example. From the characteristic calculation in the transcript, the matrix is the standard nilpotent Jordan block:

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

Then

$$\lambda I - A = \begin{bmatrix} \lambda & -1 \\ 0 & \lambda \end{bmatrix}.$$

The characteristic polynomial is

$$\det(\lambda I - A) = \lambda^2.$$

So the only eigenvalue is

$$\lambda = 0,$$

and it appears twice as a root.

The eigenvectors are found from the null space of A . The instructor described null-space vectors as vectors orthogonal to every row of the matrix. For this example, the zero row is orthogonal to every vector, while the row $[0 \ 1]$ forces the second component of the vector to be zero.

A vector

$$\begin{bmatrix} r \\ 0 \end{bmatrix}$$

is in the null space for any scalar r from the field being used, real or complex. The eigenspace is therefore spanned by

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

This eigenspace is one-dimensional. Therefore there is only one linearly independent eigenvector direction, not two. Since a 2×2 matrix would need two linearly independent eigenvectors to be diagonalizable, this matrix is not diagonalizable.

12. Algebraic and Geometric Multiplicity

The example motivates two types of multiplicity.

The **algebraic multiplicity** of an eigenvalue is the number of times it appears as a root of the characteristic polynomial.

The **geometric multiplicity** of an eigenvalue is the dimension of its eigenspace:

$$g_i = \dim \text{null}(\lambda_i I - A).$$

In the example

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix},$$

the eigenvalue 0 has:

- algebraic multiplicity 2,
- geometric multiplicity 1.

This mismatch is the core reason the matrix is not diagonalizable.

For a general $n \times n$ matrix with k distinct eigenvalues, let:

- a_i be the algebraic multiplicity of eigenvalue λ_i ,
- g_i be the geometric multiplicity of eigenvalue λ_i .

The algebraic multiplicities always add to n :

$$\sum_{i=1}^k a_i = n.$$

The geometric multiplicities do not necessarily add to n . For diagonalization, we want enough eigenspace dimensions to build a full basis:

$$\sum_{i=1}^k g_i = n.$$

When this happens, A is diagonalizable.

The instructor did not go into a full if-and-only-if theorem in detail, but the practical condition stated in the lecture is:

A is diagonalizable if we can find n linearly independent eigenvectors.

Distinct Eigenvalues

If an $n \times n$ matrix has n distinct eigenvalues, then it is diagonalizable. Each distinct eigenvalue contributes an independent eigenspace direction, giving enough independent eigenvectors.

However, the converse is not true:

n distinct eigenvalues $\implies$ diagonalizable,

but

diagonalizable $\not\Rightarrow n$ distinct eigenvalues.

The identity matrix is the instructor's example of the converse failing. The identity matrix has one repeated eigenvalue, but it is already diagonal and is certainly diagonalizable.

13. Similarity, Triangularization, Schur/Jordan Direction, and SVD

The basis-change expression

$$T^{-1}AT$$

is called a **similarity transformation**. The question “Can A be diagonalized by a basis change?” is the question of whether A is similar to a diagonal matrix.

The answer is no in general: not every square matrix is diagonalizable.

The reason diagonal matrices are desirable is that they make many problems easier, including:

- systems of linear equations,
- systems of linear differential equations,
- other computations involving repeated action of a linear map.

The instructor then proposed a less ambitious target. If diagonalization cannot always be guaranteed, can we at least represent a square matrix as triangular after a basis change?

The question is:

Can we find T such that $T^{-1}AT$ is triangular?

The answer stated in lecture:

Every square matrix can be triangularized by a basis change.

The transcript renders the name unclearly, but this is the direction of the Schur/triangular form discussion. The instructor said this will be discussed later.

The instructor also mentioned Jordan form:

- Jordan form is not always diagonal.
- It is close to diagonal, with possible superdiagonal entries.
- The course will not go deeply into Jordan form.

Then the instructor returned to the restriction that caused the diagonalization problem: using the same basis for input and output.

Using the same basis can be appropriate in some applications, but it is a constraint rather than a necessity in every application.

If we allow different bases:

- one basis for the input space,
- another basis for the output space,

then the diagonal-representation question becomes more flexible.

The instructor stated that with this freedom, the answer is positive: one can represent the linear transformation in diagonal form by choosing smart input and output bases. The special form of this positive result is the **singular value decomposition**.

This also connects to non-square matrices. For a rectangular matrix, input and output dimensions differ, so using the same basis for both spaces does not make sense. In rectangular “diagonal” matrices, only entries on the main diagonal are allowed to be nonzero; all other entries are zero.

14. Transition to Orthogonality and Complex Inner Products

Before continuing with diagonalization, triangularization, and SVD, the instructor said the course needs a deeper treatment of orthogonality.

To discuss orthogonality properly in complex vector spaces, the inner product definition must be extended from real vectors to complex vectors.

15. Real Euclidean Norm Review

[Exam note]

For a real vector $x \in \mathbb{R}^n$, the Euclidean norm is

$$\|x\|_2 = \sqrt{x^T x} = \sqrt{\sum_{i=1}^n x_i^2}.$$

This is the standard extension of the length concept to n -dimensional real vectors.

16. Complex Magnitude and Conjugate Transpose

For complex vectors, ordinary transpose is not enough. The instructor introduced the **conjugate transpose**, also called the **Hermitian transpose**.

Notation used:

$$x^H$$

or sometimes

$$x^*$$

depending on convention.

The conjugate transpose does two things:

1. It transposes the vector or matrix.
2. It conjugates every complex entry.

For a complex scalar a , its magnitude is its distance from the origin in the complex plane. If

$$a = u + iv,$$

then

$$|a| = \sqrt{u^2 + v^2}.$$

This can be written using conjugation:

$$|a| = \sqrt{\bar{a}a}.$$

The reason conjugation is needed is that multiplying a complex number by itself does not generally produce its squared magnitude. For example,

$$(1 + i)^2 = 2i,$$

which is complex, while

$$\overline{(1 + i)}(1 + i) = (1 - i)(1 + i) = 2.$$

So multiplying by the conjugate produces a nonnegative real magnitude-squared value.

For a complex vector x , define the Euclidean norm by

$$\|x\|_2 = \sqrt{x^H x}.$$

Expanding this:

$$x^H x = \overline{x_1}x_1 + \overline{x_2}x_2 + \cdots + \overline{x_n}x_n = |x_1|^2 + |x_2|^2 + \cdots + |x_n|^2.$$

Thus

$$\|x\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2}.$$

This gives a nonnegative real number.

17. MATLAB Warning: Transpose vs Conjugate Transpose

[Exam note]

The instructor gave a practical MATLAB warning.

In MATLAB:

- $\mathbf{x}'$ is the conjugate transpose.
- For real vectors, $\mathbf{x}'$ behaves like an ordinary transpose because conjugation has no effect.
- For complex vectors, $\mathbf{x}'$ also conjugates entries.
- To take the plain transpose without conjugating, use $\mathbf{x}.'$.
- To conjugate without transposing, use `conj(x)`.

In mathematical notation, the plain transpose is written x^T , while the conjugate/Hermitian transpose is written x^H or x^* , depending on convention.

The instructor warned that many people mistakenly use $'$ thinking it is only transpose. This can cause debugging problems in complex-valued computations.

18. Complex Inner Product Convention

The lecture then defined the complex inner product.

The instructor's convention is linear in the first argument and conjugate-linear in the second argument:

$$\langle x, y \rangle = y^H x = \sum_{i=1}^n x_i \overline{y_i}.$$

With this convention, the norm is still obtained from the inner product of a vector with itself:

$$\|x\|_2 = \sqrt{\langle x, x \rangle} = \sqrt{x^H x}.$$

For real vectors, the inner product is symmetric:

$$\langle x, y \rangle = \langle y, x \rangle.$$

For complex vectors, this is no longer true in general. Instead:

$$\langle x, y \rangle = \overline{\langle y, x \rangle}.$$

So the two reversed inner products are complex conjugates of each other.

Example from the lecture:

If

$$\langle x, y \rangle = 3 + 5i,$$

then

$$\langle y, x \rangle = 3 - 5i.$$

19. Scaling Rules for the Complex Inner Product

Because of conjugation, scaling behaves differently depending on which argument is scaled.

With the lecture's convention:

$$\langle \alpha x, y \rangle = \alpha \langle x, y \rangle.$$

Scaling the first vector scales the inner product by the same scalar.

But if the second vector is scaled:

$$\langle x, \beta y \rangle = \overline{\beta} \langle x, y \rangle.$$

The scalar is conjugated because the second argument is the conjugated one in this convention.

For real scalars, this distinction disappears because

$$\overline{\beta} = \beta.$$

For complex scalars, the distinction matters.

20. Orthogonal and Orthonormal Sets

[Exam note]

The instructor extended the definition of orthogonality to complex vectors.

Let

$$S \subseteq \mathbb{C}^m.$$

The set S is an **orthogonal set** if and only if every distinct pair of vectors in S is orthogonal:

$$\langle u, v \rangle = 0 \quad \text{for all distinct } u, v \in S.$$

Orthogonality is a property of the set: any pair chosen from the set must have zero inner product.

The standard basis in three dimensions is an example:

$$e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad e_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad e_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Any pair has inner product zero, so the standard basis is orthogonal.

A set is an **orthonormal set** if:

1. it is orthogonal,
2. every vector in the set has unit norm.

The standard basis is orthonormal, because each standard basis vector has norm 1.

The instructor also contrasted this with an orthogonal set whose vectors are not unit norm. Such a set is orthogonal but not orthonormal. For example, scaled coordinate vectors such as $\{2e_1, 3e_2, e_3\}$ still have zero pairwise inner products, but not every vector has norm 1.

21. Preview of Next Lecture

The next lecture will focus on **orthogonal projection**, which the instructor described as a key operation.

The projection discussion will include:

- projecting a vector onto a vector,
- projecting a vector onto a subspace,
- extending the geometric idea of projection into the linear algebra framework.

Source and Coverage Note

These notes were created only from C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture8_correct

They cover the lecture chronologically, including the transcript's exam-marked topics, definitions, proof ideas, examples, warnings, instructor remarks, and stated links to later topics. No other lecture transcript was processed.

Lecture 09 Notes

1. Review: Factorizations and Simpler Matrix Forms

The lecture opens by returning to the idea that solving many linear algebra problems is essentially helped by factorizing a matrix A into simpler matrices.

Important examples mentioned:

- QR factorization, to be covered later: write $A = QR$, where Q is orthogonal/unitary and R is upper triangular.
- LU factorization: write a matrix using lower triangular and upper triangular factors. Gaussian elimination is essentially a process of triangularizing a matrix into lower and upper triangular structure.
- Permutation matrices appear when row exchanges are needed in elimination. The instructor groups them with the “simple” factors and notes that a permutation matrix is an orthogonal matrix.
- Diagonal matrices are especially simple, so it is natural to ask when a given matrix can be converted into diagonal form.

The main relationship emphasized is:

If A can be replaced by a product of simpler matrices, then one difficult problem involving A can often be replaced by a sequence of simpler problems.

2. Basis Change, Similarity, and Diagonalization

For a square matrix A , the matrix can be interpreted as the representation of a linear transformation in the standard basis. If the basis is changed to a new basis collected in an invertible matrix T , the same transformation is represented by a new matrix

$$\tilde{A} = T^{-1}AT.$$

This operation is called a similarity transformation.

The central question from the previous lecture was:

Given a matrix A that is not diagonal in the standard basis, can we find a clever basis T such that $T^{-1}AT$ becomes diagonal?

This leads to the classification of square matrices into:

- diagonalizable matrices: there exists an invertible T such that $T^{-1}AT$ is diagonal;
- non-diagonalizable matrices: no such invertible T exists.

The instructor stressed that not all matrices are diagonalizable.

3. Similar Matrices Preserve Eigenvalues

The instructor asked what A and its similarity-transformed version $\tilde{A} = T^{-1}AT$ have in common.

One answer was rank, but the more important answer was:

Similar matrices have the same eigenvalues.

Proof Using the Characteristic Polynomial

Start from the characteristic polynomial of $\tilde{A}$:

$$p_{\tilde{A}}(\lambda) = \det(\lambda I - \tilde{A}).$$

Substitute $\tilde{A} = T^{-1}AT$:

$$\det(\lambda I - T^{-1}AT).$$

Use $I = T^{-1}T$ to rewrite λI compatibly:

$$\lambda I - T^{-1}AT = T^{-1}(\lambda I - A)T.$$

Then

$$\det(T^{-1}(\lambda I - A)T) = \det(T^{-1}) \det(\lambda I - A) \det(T).$$

Since

$$\det(T^{-1}) = \frac{1}{\det(T)},$$

the determinant factors cancel:

$$\det(T^{-1}) \det(T) = 1.$$

Therefore

$$p_{\tilde{A}}(\lambda) = \det(\lambda I - A) = p_A(\lambda).$$

So A and $\tilde{A}$ have the same characteristic polynomial and hence the same eigenvalues.

4. Diagonalization, Triangularization, and SVD Preview

Although not every square matrix can be diagonalized by a similarity transformation, the instructor stated that every matrix can be triangularized in an appropriate basis:

$$T^{-1}AT$$

can always be made triangular for a suitable invertible T .

The lecture also previews an important contrast:

- Similarity transformations use the same basis for input and output spaces.
- If we allow different bases for the input and output spaces, then a much more flexible representation is possible.

In similarity, the same basis-change matrix appears on both sides: $T^{-1}AT$. If the input and output bases are allowed to differ, the representation has different left and right basis changes, for example $S^{-1}AT$. That extra freedom is what makes a diagonal representation available in the SVD setting even when a similarity diagonalization is not available.

Likely exam topic If we are flexible about choosing different bases for input and output spaces, then a linear transformation can be represented by a diagonal matrix. This is the idea behind singular value decomposition (SVD), which will be covered later.

This connects eigenvalues/eigenvectors, characteristic polynomials, diagonalization, triangularization, and SVD into one larger theme:

Matrix factorization and basis choice are ways to turn a difficult transformation into a simpler one.

5. Complex Inner Products and Conjugate Transpose

Before entering orthogonal/unitary matrices, the lecture reviews how Euclidean inner products extend from real vector spaces to complex vector spaces.

For real vectors, the Euclidean norm is based on expressions like

$$x^T x.$$

For complex vectors, using $x^T x$ is not appropriate because squaring complex entries can produce complex values. Instead, use the conjugate transpose:

$$x^* x.$$

Here x^* denotes the conjugate transpose, also called the Hermitian transpose. For a column vector, x^* is a row vector whose entries are the complex conjugates of the entries of x .

Instructor notation remark: the star notation is useful because, for scalars, it reduces to ordinary complex conjugation, while for vectors and matrices it also includes the transpose operation. Thus a column vector becomes a row vector with conjugated entries.

The reason for conjugation is that

$$\overline{x_k} x_k = |x_k|^2,$$

which is real and nonnegative. This gives the proper squared norm.

Inner Product Convention

The lecture uses the convention that the second vector is conjugate transposed:

$$\langle x, y \rangle = y^* x.$$

With this convention:

- exchanging the two inputs conjugates the result:

$$\langle x, y \rangle = \overline{\langle y, x \rangle};$$

- scaling the first input by a complex scalar scales the inner product by the same scalar;
- scaling the second input by a complex scalar scales the inner product by the conjugate of that scalar.

The real symmetric property of the inner product therefore becomes Hermitian symmetry in the complex case.

6. Orthogonal and Orthonormal Sets

A set S is orthogonal if any pair of distinct vectors in the set has inner product zero.

A set S is orthonormal if:

- it is orthogonal;
- every vector in the set has unit norm.

So:

An orthogonal set with unit-norm elements is an orthonormal set.

This is the foundation for the projection and unitary-matrix material that follows.

7. Orthogonal Projection of a Vector Onto Another Vector

Likely exam topic The instructor calls orthogonal projection a very basic but very important operation related to orthogonality and inner products.

Instructor remark: the same projection idea later extends to estimation and random variables after the inner product is generalized. The geometry here is simple, but the concept is foundational.

The first projection problem is:

Given a vector x , project it onto another vector y .

The projection vector is denoted $\hat{x}$. It lies on the line determined by y , meaning

$$\hat{x} = \alpha y$$

for some scalar α .

Because this is an orthogonal projection, the projection error

$$x - \hat{x}$$

must be orthogonal to y :

$$\langle x - \hat{x}, y \rangle = 0.$$

Substitute $\hat{x} = \alpha y$:

$$\langle x - \alpha y, y \rangle = 0.$$

Distribute the inner product:

$$\langle x, y \rangle - \alpha \langle y, y \rangle = 0.$$

Equivalently,

$$\langle x, y \rangle = \langle \hat{x}, y \rangle.$$

So the original vector and its projection have the same inner product with the vector being projected onto.

Therefore

$$\alpha = \frac{\langle x, y \rangle}{\langle y, y \rangle}.$$

So the projection of x onto the line spanned by y is

$$\hat{x} = \frac{\langle x, y \rangle}{\langle y, y \rangle} y.$$

[Exam note] This coefficient is the projection coefficient: take the inner product of the vector being projected with the vector being projected onto, then divide by the inner product of the vector with itself.

Geometric Interpretation in Real Euclidean Space

Since

$$\langle y, y \rangle = \|y\|^2,$$

the projection formula can be written as

$$\hat{x} = \frac{\langle x, y \rangle}{\|y\|^2} y = \frac{\langle x, y \rangle}{\|y\|} \frac{y}{\|y\|}.$$

The vector

$$\frac{y}{\|y\|}$$

is the unit vector in the direction of y .

For real two- or three-dimensional vectors,

$$\langle x, y \rangle = \|x\| \|y\| \cos \theta.$$

Then

$$\frac{\langle x, y \rangle}{\|y\|} = \|x\| \cos \theta.$$

So the projection has:

- length $\|x\| \cos \theta$ along the direction of y ;
- direction $y/\|y\|$.

The instructor connects this to the right-triangle picture where $\|x\|$ is the hypotenuse and the projected length is $\|x\| \cos \theta$.

Orthogonal vs. Oblique Projection

The instructor notes that “projection” here means orthogonal projection by default.

There is also a different operation called oblique projection, where the projection direction is tilted rather than perpendicular to the target space. Oblique projection is mentioned but not covered in this part of the lecture.

8. Orthogonal Bases and Coordinates

Now consider an n -dimensional subspace V inside an m -dimensional ambient space, such as $\mathbb{C}^m$.

Let

$$\{q_1, q_2, \dots, q_n\}$$

be an orthogonal basis for V .

Since this is a basis, any vector $x \in V$ can be written as

$$x = \alpha_1 q_1 + \alpha_2 q_2 + \dots + \alpha_n q_n.$$

The key claim is:

If the basis is orthogonal, the coordinates α_k are obtained by projecting x onto each basis vector separately.

Proof of the Coordinate Formula

Take the inner product of x with a basis vector q_k :

$$\langle x, q_k \rangle = \left\langle \sum_{i=1}^n \alpha_i q_i, q_k \right\rangle.$$

Using linearity in the first argument:

$$\langle x, q_k \rangle = \sum_{i=1}^n \alpha_i \langle q_i, q_k \rangle.$$

Because the basis vectors are orthogonal,

$$\langle q_i, q_k \rangle = 0 \quad \text{when } i \neq k.$$

Only the $i = k$ term survives:

$$\langle x, q_k \rangle = \alpha_k \langle q_k, q_k \rangle.$$

Therefore

$$\alpha_k = \frac{\langle x, q_k \rangle}{\langle q_k, q_k \rangle}.$$

[Exam note] This is the same projection coefficient as in the vector-on-vector projection formula.

Why Orthogonal Bases Are Convenient

With an orthogonal basis, each coordinate is found independently:

$$\alpha_k = \frac{\langle x, q_k \rangle}{\langle q_k, q_k \rangle}.$$

You do not need to solve a coupled system of equations.

For an arbitrary non-orthogonal basis, simply taking inner products with basis vectors does not directly give the coordinates. The basis vectors are coupled, so one must solve a system of linear equations.

[Exam note] With an orthogonal basis, to find the coordinate in the direction of q_k , just project x onto q_k . With a general basis, this shortcut does not work.

Orthonormal Basis Simplification

If the basis is orthonormal, then

$$\langle q_k, q_k \rangle = 1.$$

So the coordinate formula becomes

$$\alpha_k = \langle x, q_k \rangle.$$

Thus

$$x = \sum_{k=1}^n \langle x, q_k \rangle q_k.$$

Each term

$$\langle x, q_k \rangle q_k$$

is the projection of x onto the one-dimensional direction q_k .

The whole vector is reconstructed by summing these individual projections.

Two-Dimensional Picture

The instructor describes a two-dimensional picture with basis vectors q_1 and q_2 :

- project x onto q_1 to get the component along q_1 ;
- project x onto q_2 to get the component along q_2 ;
- these give α_1 and α_2 .

[Exam note] This picture is important because it shows coordinates as projection coefficients.

9. Student Question: Rotation, Reflection, and Orthonormal Bases

A student asks whether an orthonormal basis is just a rotation of the standard basis.

The instructor answers:

- orthonormal is a special case of orthogonal, where the vectors also have unit length;
- in the real domain, one can move from one orthogonal/orthonormal basis to another using geometric operations such as rotations and reflections;

- a reflection is with respect to some reflection axis;
- combinations of rotations and reflections can convert the standard basis into an arbitrary orthogonal basis in the real setting.

The instructor says the forms of these operations will be studied later.

10. Outer Products and Rank-One Projection Matrices

For an orthonormal basis vector q_ℓ , the projection of x onto q_ℓ can be written as

$$\langle x, q_\ell \rangle q_\ell.$$

Using the lecture's convention,

$$\langle x, q_\ell \rangle = q_\ell^* x.$$

So

$$\langle x, q_\ell \rangle q_\ell = q_\ell (q_\ell^* x) = (q_\ell q_\ell^*) x.$$

The matrix

$$q_\ell q_\ell^*$$

is a projection matrix onto the one-dimensional space spanned by q_ℓ .

Likely exam topic The instructor emphasizes that this object is a matrix, not a scalar.

Inner Product vs. Outer Product

Compare the two orders:

$$q_\ell^* q_\ell$$

is a row vector times a column vector, so it is a scalar inner product.

But

$$q_\ell q_\ell^*$$

is a column vector times a row vector, so it is a matrix. This is called an outer product.

If $q_\ell \in \mathbb{C}^m$, then

$$q_\ell q_\ell^*$$

is an $m \times m$ matrix.

Rank of an Outer Product

The instructor asks for the rank of an outer product matrix.

For a general outer product

$$xy^*,$$

where x and y do not even need to have the same dimension, the columns of xy^* are scaled copies of x . Therefore the column space is contained in

$$\text{span}\{x\}.$$

If $x \neq 0$ and $y \neq 0$, the rank is one.

So an outer product of a column vector and a row vector is a rank-one matrix.

[Exam note] This is an important concept: outer products create rank-one matrices because all columns point in the same direction.

One-Dimensional Projection Matrix

For a unit vector q_ℓ ,

$$q_\ell q_\ell^*$$

takes any input vector x and projects it onto

$$\text{span}\{q_\ell\}.$$

The range of this projection matrix is exactly that one-dimensional subspace.

11. Summing Rank-One Projections Over an Orthonormal Basis

If

$$\{q_1, \dots, q_n\}$$

is an orthonormal basis for a subspace $V \subseteq \mathbb{C}^m$, then each rank-one matrix

$$q_\ell q_\ell^*$$

projects onto the one-dimensional direction q_ℓ .

The sum

$$\sum_{\ell=1}^n q_\ell q_\ell^*$$

combines all these one-dimensional projections.

Define the basis matrix

$$Q_V = [q_1 \ q_2 \ \cdots \ q_n].$$

This matrix is $m \times n$, usually tall if $n < m$. Its columns are the orthonormal basis vectors for V .

Then

$$\sum_{\ell=1}^n q_{\ell} q_{\ell}^* = Q_V Q_V^*.$$

Dimension/rank warning from the board discussion:

- Q_V is $m \times n$;
- Q_V^* is $n \times m$;
- $Q_V Q_V^*$ is $m \times m$, but when $n < m$ its rank is n , not m .

Therefore $Q_V Q_V^*$ cannot be the full identity matrix on $\mathbb{C}^m$ unless V is the whole ambient space. The identity that does hold for orthonormal columns is

$$Q_V^* Q_V = I_n.$$

For $x \in V$,

$$Q_V Q_V^* x = x.$$

So $Q_V Q_V^*$ acts like the identity on vectors already inside V .

But $Q_V Q_V^*$ is not generally the identity matrix on the whole ambient space. If $x \notin V$, then $Q_V Q_V^* x$ is not equal to x ; it is the projection of x onto V .

This resolves the instructor's point:

The matrix $Q_V Q_V^*$ behaves like identity on the subspace V , but it is a projection operator on the full ambient space.

12. Orthogonal Projection of a Vector Onto a Subspace

Likely exam topic The lecture then moves from projecting onto one vector to projecting onto a vector space.

Let V be an n -dimensional subspace of an m -dimensional ambient space. Let x be an arbitrary vector in the ambient space. The vector x may or may not lie in V .

The orthogonal projection of x onto V is the point $\hat{x} \in V$ such that

$$x - \hat{x}$$

is orthogonal to every vector in V .

Equivalently:

- $\hat{x}$ must be a member of V ;
- the error vector $x - \hat{x}$ must be orthogonal to V .

The instructor notes that this projection point also has a minimization property:

It is the closest point in V to x , with distance measured using the norm induced by the inner product.

With the complex Euclidean inner product used here, that norm is

$$\|z\| = \sqrt{z^*z}.$$

The optimization viewpoint will be discussed later, so here the focus remains on the orthogonality condition.

13. Deriving the Projection Onto a Subspace

The original definition of projection onto V depends only on the subspace, not on a particular basis. However, a basis is a tool for computing the projection.

Assume an orthonormal basis

$$\{q_1, \dots, q_n\}$$

for V .

Since $\hat{x} \in V$, write

$$\hat{x} = \alpha_1 q_1 + \dots + \alpha_n q_n.$$

The unknowns are the coefficients $\alpha_1, \dots, \alpha_n$.

The orthogonality condition says

$$\langle x - \hat{x}, q_k \rangle = 0$$

for every basis vector q_k . This gives n equations for n unknowns.

Using the same projection-coefficient reasoning:

$$\alpha_k = \frac{\langle x, q_k \rangle}{\langle q_k, q_k \rangle}.$$

For an orthonormal basis,

$$\langle q_k, q_k \rangle = 1,$$

so

$$\alpha_k = \langle x, q_k \rangle.$$

Therefore

$$\hat{x} = \sum_{k=1}^n \langle x, q_k \rangle q_k.$$

This is the sum of the projections of x onto the individual orthonormal basis directions.

14. Orthogonal Projection Matrix Onto a Subspace

Using the matrix

$$Q = [q_1 \ q_2 \ \cdots \ q_n],$$

the projection formula becomes

$$\hat{x} = QQ^*x.$$

So the orthogonal projection matrix onto V is

$$P_V = QQ^*.$$

Likely exam topic Projection matrices are a central object in this lecture. The matrix P_V depends on the subspace V , maps ambient vectors back into $\text{range}(Q) = V$, and behaves differently on vectors inside and outside V .

[Exam note] To compute P_V for a given subspace:

1. Find any orthonormal basis for V .
2. Put the basis vectors into the columns of Q .
3. Compute Q^* .
4. Multiply:

$$P_V = QQ^*.$$

This matrix maps an m -dimensional vector to another m -dimensional vector, but the output lies in $V = \text{range}(Q)$.

If $x \in V$, then

$$P_V x = x.$$

If $x \notin V$, then $P_V x$ is the point in V obtained by orthogonally projecting x onto V .

15. Projection Matrix Does Not Depend on the Chosen Orthonormal Basis

A student asks whether the projection matrix is the same for all orthonormal bases of the same subspace.

The instructor answers yes:

The projection matrix depends on the subspace, not on the particular orthonormal basis chosen for that subspace.

Proof idea given in class:

Suppose another orthonormal basis matrix for the same subspace can be written as

$$\tilde{Q} = QT,$$

where T is unitary. Then

$$\tilde{Q}\tilde{Q}^* = (QT)(QT)^* = QTT^*Q^*.$$

Since T is unitary,

$$TT^* = I.$$

Therefore

$$\tilde{Q}\tilde{Q}^* = QQ^*.$$

So all orthonormal bases of the same subspace produce the same orthogonal projection matrix.

The instructor notes that the fact that one orthonormal basis is obtained from another by multiplying by a unitary matrix will be shown more fully later.

16. Properties of Orthogonal Projection Matrices

Let

$$P_V = QQ^*$$

where the columns of Q are orthonormal.

Property 1: Hermitian

The conjugate transpose of P_V is

$$P_V^* = (QQ^*)^* = QQ^* = P_V.$$

The proof uses the rule $(AB)^* = B^*A^*$ and the fact that $(Q^*)^* = Q$.

So an orthogonal projection matrix is Hermitian:

$$P_V^* = P_V.$$

[Exam note] Hermitian matrices are an important family. Later topics include positive semidefinite matrices, which sit under the broader Hermitian-matrix family.

Property 2: Idempotent

The square of P_V is

$$P_V^2 = (QQ^*)(QQ^*) = Q(Q^*Q)Q^*.$$

Since the columns of Q are orthonormal,

$$Q^*Q = I.$$

Therefore

$$P_V^2 = QIQ^* = QQ^* = P_V.$$

So

$$P_V^2 = P_V.$$

A matrix satisfying this property is called idempotent.

Geometric reason:

- $P_V x$ is already in V .
- Projecting it onto V again does not change it.
- Therefore $P_V(P_V x) = P_V x$ for every x .

17. Projection Matrices vs. Orthogonal Projection Matrices

The instructor distinguishes two related ideas:

- A matrix satisfying

$$P^2 = P$$

is a projection matrix.

- If, in addition,

$$P^* = P,$$

then it is an orthogonal projection matrix.

If a projection matrix is idempotent but not Hermitian, it corresponds to an oblique projection. In that case, the projection direction is tilted rather than orthogonal.

The instructor says oblique projections are not covered at this point.

18. Projection Using a Non-Orthonormal Basis

The lecture briefly gives the more general projection formula when the basis is not orthonormal.

Suppose A has linearly independent columns

$$a_1, \dots, a_n$$

that form a basis for the target subspace. These columns are not necessarily orthogonal.

Because these columns are a basis, A^*A is invertible in this full-column rank case, so the correction factor below is well-defined.

Then the orthogonal projection matrix onto $\text{range}(A)$ is

$$P_A = A(A^*A)^{-1}A^*.$$

The middle factor

$$(A^*A)^{-1}$$

is the correction term that accounts for the fact that the basis vectors are not orthonormal.

If the columns are orthonormal, then

$$A^* A = I,$$

and the formula reduces to

$$P_A = AA^*,$$

which matches the earlier QQ^* formula.

[Exam note] The simple formula QQ^* is special to the orthonormal-basis case. With a general basis, the correction term $(A^* A)^{-1}$ is needed.

The instructor says this general formula will be revisited when discussing least squares.

19. Least Squares Connection

The lecture connects projection matrices to the system

$$Ax = b.$$

The columns of A span the column space, or range space, of A :

$$\text{range}(A).$$

Solving $Ax = b$ asks whether b can be written as a linear combination of the columns of A .

If

$$b \notin \text{range}(A),$$

then there is no exact solution.

This is where the earlier existence-and-uniqueness discussion would stop: outside the column space, the equation $Ax = b$ is inconsistent.

But in applications and research, we often do not stop there. Instead, we look for a nearby solution:

Find the point in $\text{range}(A)$ that is as close as possible to b .

That closest point is the orthogonal projection of b onto $\text{range}(A)$.

This is the geometric meaning of the least squares solution.

Different Least-Squares Viewpoints Mentioned

The instructor describes several interpretations:

- Standard least squares “blames” b : b should have been in the range space of A , but measurement noise or error pushed it outside. We project b back onto $\text{range}(A)$.
- A data/tilted-space version “blames” A : b is treated as correct, but A is noisy, so the range space of A is tilted until it contains or better matches b .
- Total least squares “blames” both: both b and the space may be moved to meet each other.

[Exam note] The instructor gives the “mountain and mouse” analogy:

- least squares moves/projects the vector b to the space;
- the tilted-space version moves the space toward b ;

- total least squares moves both.

These versions will be discussed later.

20. Summary of Orthogonality Topics Covered

The instructor explicitly summarizes the orthogonality material covered in this lecture:

- orthogonality;
- orthogonal sets;
- orthonormal sets;
- orthogonal projection of a vector onto another vector;
- orthogonal projection of a vector onto a vector space;
- projection matrices.

These are described as important and foundational topics.

21. Unitary Matrices

After projection matrices, the next special matrix family introduced is the unitary matrices.

Unitary matrices are the complex version of real orthogonal matrices.

A matrix $U \in \mathbb{C}^{n \times n}$ is unitary if

$$U^*U = I$$

and equivalently

$$UU^* = I.$$

This means

$$U^{-1} = U^*.$$

So the inverse of a unitary matrix is simply its conjugate transpose.

The real version is called an orthogonal matrix.

Why Unitary Matrices Are Useful

Systems involving unitary matrices are easier to solve. If

$$Ux = b,$$

then

$$x = U^{-1}b = U^*b.$$

No general matrix inversion is needed. One only takes the conjugate transpose of U , which means transpose the matrix and conjugate its entries, then multiply by b .

In the instructor's words, forming the inverse is just swapping rows and columns and changing the signs of the imaginary parts, because $U^{-1} = U^*$.

22. Columns and Rows of a Unitary Matrix Are Orthonormal Bases

The condition

$$U^*U = I$$

means the columns of U form an orthonormal set.

Reason:

- write U by columns;
- U^*U forms all pairwise inner products of the columns;
- the identity matrix says each column has norm one and distinct columns are orthogonal.

Because U is square, these columns form an orthonormal basis for $\mathbb{C}^n$.

The condition

$$UU^* = I$$

means the rows of U also form an orthonormal basis.

So for a unitary matrix:

- columns are an orthonormal basis;
- rows are an orthonormal basis;
- the inverse is the conjugate transpose.

Likely exam topic Orthonormal bases and unitary matrices are directly linked: unitary matrices are exactly the square matrices whose columns and rows are orthonormal bases.

23. Forward Look

The instructor says unitary matrices have many interesting and useful properties, and the course will continue with them after the break.

Upcoming special matrix families:

- unitary matrices;
- Hermitian matrices;
- important subsets of Hermitian matrices, including positive semidefinite matrices.

Source and Coverage Note

Source used: C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture9_corrected.md.

Coverage: These notes cover only Lecture 09. They preserve the chronological flow of the transcript, including the review of matrix factorizations and similarity, the proof that similar matrices preserve eigenvalues, the complex inner-product recap, orthogonal and orthonormal sets, vector and subspace projection formulas, projection matrices and their properties, least-squares connections, unitary matrices, instructor remarks, student questions, warnings, and all exam-relevant markers present in the transcript. No other lecture transcripts were used.

Lecture 10 Notes: Unitary and Real Orthogonal Matrices

Big Picture: Turning Linear Systems Into Simple Problems

The lecture begins by returning to a broad strategy from earlier lectures: convert an arbitrary system of linear equations into a sequence of simpler systems. The simple matrix types emphasized are:

- Diagonal matrices.
- Triangular matrices.
- Orthogonal/unitary matrices.

These are easy to solve with compared to a general matrix. The practical goal is to factorize a general matrix A as a product of matrices from these simple classes.

Several factorizations fit into this same framework:

- Schur factorization uses orthogonal/unitary and triangular structure.
- QR factorization converts a problem into one involving an orthogonal/unitary factor and a triangular factor.
- Gaussian elimination is a special case of the same idea, leading to LU decomposition. In this view, a matrix is represented using lower and upper triangular matrices, sometimes with a permutation/orthogonal matrix P .

Diagonalization, Basis Change, and Triangularization

For square matrices, a matrix can be understood as the representation of a linear mapping. If the basis changes, the matrix representing the same mapping also changes.

With the usual coordinate convention, if $x = T\tilde{x}$, then the same linear mapping is represented in the new coordinates by:

$$\tilde{A} = T^{-1}AT.$$

The exact left/right placement depends on the coordinate convention, but the important point in the lecture was that using the same basis for input and output produces the same transformation and its inverse on the two sides of A .

The earlier question was whether one can choose a basis so that the matrix becomes diagonal. This is the diagonalization problem, studied through eigenvalues and eigenvectors.

Important distinction:

- Not every square matrix is diagonalizable.
- Square matrices are partitioned into diagonalizable and non-diagonalizable matrices.
- If the same basis must be used for the input and output spaces, diagonalization is not always possible.
- If two different bases are allowed for input and output spaces, the situation changes. The transformed representation has the form $T_{\text{out}}^{-1}AT_{\text{in}}$, or in the instructor's wording two different matrices such as T_1 and T_2^{-1} , rather than the same T and T^{-1} . In that broader input/output basis-change setting, the instructor noted that any matrix can be diagonalized or reduced to a diagonal canonical form. This topic is postponed.

Exam Note: Every Square Matrix Can Be Triangularized

Even though not every matrix is diagonalizable, every square matrix can be triangularized. That is, for any square matrix, one can find a transformation matrix T so that the transformed representation is triangular.

The important extra property is that the transformation matrix used for this triangularization can be chosen to be orthogonal/unitary. Thus a general square matrix can be written in terms of simple matrix types: orthogonal/unitary and triangular matrices.

This is connected to Schur form, which will be a future topic. The instructor emphasized:

- Not all matrices are diagonalizable.
- But all square matrices are triangularizable.
- The triangularizing basis can be chosen orthogonal/unitary.

Review: Orthogonal Projection Onto a Vector

The previous lecture introduced orthogonal projection. The first case was projecting a vector onto another vector.

If a vector x is projected onto a vector y , the projection is a scaled version of y . The scaling coefficient is the ratio of two inner products:

$$\text{proj}_y(x) = \frac{\langle x, y \rangle}{\langle y, y \rangle} y.$$

The numerator measures the interaction between x and y . The denominator normalizes by the size of the vector being projected onto.

Review: Orthogonal Projection Onto a Subspace

The more general case is projection of a vector onto a subspace.

To do this, choose a basis for the subspace and solve for the coefficients of the projected point. If the basis is orthonormal and its columns are collected in a tall matrix $\hat{Q}$, then the orthogonal projection matrix has the form:

$$P = \hat{Q}\hat{Q}^*.$$

Here:

- $\hat{Q}$ is tall.
- $\hat{Q}^*$ is fat.
- The columns of $\hat{Q}$ form an orthonormal basis for the subspace.
- The resulting projection matrix is generally rank deficient because it projects onto a lower-dimensional subspace.

Projection matrices satisfy:

$$P^2 = P.$$

This idempotence property is true for projection matrices in general.

For orthogonal projection matrices, there is an additional property:

$$P^* = P.$$

This means the projection matrix is self-adjoint/Hermitian in the complex case. The instructor contrasted orthogonal projections with oblique projections:

- Orthogonal projections satisfy $P^* = P$.
- Oblique projections generally do not.
- Oblique projections will not be covered in this course.

Definition: Unitary Matrices

A unitary matrix is a square complex matrix U satisfying:

$$U^*U = I \quad \text{and} \quad UU^* = I.$$

Here U^* denotes the conjugate transpose, also called the Hermitian transpose.

Therefore:

$$U^{-1} = U^*.$$

This is the complex generalization of a real orthogonal matrix.

Why Unitary Systems Are Easy To Solve

If a system of linear equations has a unitary coefficient matrix,

$$Ux = b,$$

then solving is easy:

$$x = U^*b.$$

No general matrix inversion or elimination is needed; one only takes the conjugate transpose.

The operation U^*b can also be interpreted as taking inner products between b and the columns of U , because conjugating and transposing the columns turns them into rows.

Rows and Columns of a Unitary Matrix

The defining equalities imply two orthonormality facts:

- $U^*U = I$ says the columns of U form an orthonormal basis of $\mathbb{C}^n$.
- $UU^* = I$ says the rows of U also form an orthonormal basis of $\mathbb{C}^n$.

This links unitary matrices directly to orthonormal bases.

Unitary Matrices Preserve Euclidean Norm

Consider the linear mapping

$$x \mapsto Ux$$

where U is unitary and $x \in \mathbb{C}^n$. The input and output spaces have the same dimension.

The Euclidean norm is induced by the inner product:

$$\|x\|_2 = \sqrt{x^*x}.$$

For the transformed vector:

$$\|Ux\|_2^2 = (Ux)^*(Ux).$$

Using the rule $(Ux)^* = x^*U^*$:

$$(Ux)^*(Ux) = x^*U^*Ux = x^*Ix = x^*x = \|x\|_2^2.$$

Therefore:

$$\|Ux\|_2 = \|x\|_2.$$

Important Warning About Norms

Unitary matrices preserve the Euclidean norm, also called the 2-norm for vectors. They do not necessarily preserve every possible norm.

Some other norms may also be preserved in special cases, but the guaranteed property discussed here is preservation of the Euclidean norm.

Energy Preservation

The squared Euclidean norm is:

$$\|x\|_2^2 = \sum_{i=1}^n |x_i|^2.$$

The instructor referred to this as the energy of the vector or signal x .

Since unitary matrices preserve $\|x\|_2$, they also preserve energy:

$$\|Ux\|_2^2 = \|x\|_2^2.$$

Thus a unitary matrix defines an energy-preserving linear system.

Relationship to Electrical Engineering and All-Pass Systems

In electrical engineering and signal processing, the energy of a signal in the Fourier domain is often written as an integral of the squared magnitude of the Fourier transform.

The limits depend on the setting:

- For discrete-time Fourier transforms, the frequency interval is typically from $-\pi$ to π .
- For continuous-time Fourier transforms, the interval is from $-\infty$ to ∞ .

Energy-preserving linear time-invariant systems are called all-pass systems.

The instructor emphasized an important distinction:

- A unitary matrix gives an energy-preserving linear system.
- But it is not necessarily time invariant.
- To correspond to a linear time-invariant system, the matrix must have the special convolution/shift structure associated with LTI systems.

Toeplitz Structure and LTI Systems

In an earlier homework, an LTI system was represented by a convolution operator. When written as a matrix, this gives a special structured matrix.

For an LTI system, each row is a shifted version of the previous row. This is the Toeplitz-type structure the instructor referred to.

Therefore:

- A unitary matrix without this shift structure is only a linear system.
- A unitary Toeplitz/convolution matrix would represent a linear time-invariant energy-preserving system.

Distances and Inner Products Are Preserved

Because unitary matrices preserve Euclidean norms, they preserve distances. For two points x and y , the distance is:

$$\|x - y\|_2.$$

Under the unitary mapping:

$$\|Ux - Uy\|_2 = \|U(x - y)\|_2 = \|x - y\|_2.$$

The Euclidean inner product is also preserved:

$$\langle Ux, Uy \rangle = \langle x, y \rangle.$$

Proof idea:

$$(Uy)^*(Ux) = y^*U^*Ux = y^*x.$$

Thus unitary matrices preserve:

- Euclidean norm.
- Energy.
- Distance.
- Euclidean inner product.
- Angles, in the real orthogonal case. The instructor noted that angle in the complex case requires more care and will not be emphasized here.

Since the norm is induced by the inner product, preservation of inner products implies preservation of norms.

Geometric Interpretation: Shape Preservation

Unitary matrices preserve shapes in the sense that distances and inner products among components are preserved.

In the real orthogonal case, this geometric picture becomes especially clear. Real orthogonal transformations are built from two fundamental geometric operations:

- Rotations.
- Reflections.

The instructor noted that a general unitary/orthogonal transformation can be understood as a combination of such operations, with the most transparent geometric interpretation in the real case.

Eigenvalues of Unitary Matrices

Theorem: Eigenvalues Lie on the Unit Circle

If U is unitary and $x \neq 0$ is an eigenvector satisfying:

$$Ux = \lambda x,$$

then:

$$|\lambda| = 1.$$

Proof idea:

Since unitary matrices preserve norm,

$$\|Ux\|_2 = \|x\|_2.$$

Substitute $Ux = \lambda x$:

$$\|\lambda x\|_2 = \|x\|_2.$$

Using the scalar property of norms:

$$|\lambda| \|x\|_2 = \|x\|_2.$$

Because x is an eigenvector, $x \neq 0$, so $\|x\|_2 \neq 0$. Divide by $\|x\|_2$:

$$|\lambda| = 1.$$

So every eigenvalue of a unitary matrix is a complex number on the unit circle.

Determinant of a Unitary Matrix

The determinant is not necessarily 1, but its absolute value is 1:

$$|\det(U)| = 1.$$

The instructor answered a student question here: the determinant can be any point on the complex unit circle, not only 1.

Proof idea:

Unitary matrices are diagonalizable, and the lecture states this will be discussed more generally later. If:

$$U = T\Lambda T^{-1},$$

then:

$$\det(U) = \det(T) \det(\Lambda) \det(T^{-1}).$$

Since $\det(T^{-1}) = 1/\det(T)$, those factors cancel:

$$\det(U) = \det(\Lambda) = \prod_i \lambda_i.$$

Each eigenvalue λ_i lies on the unit circle, so:

$$|\det(U)| = \prod_i |\lambda_i| = 1.$$

Orthogonality of Eigenspaces for Distinct Eigenvalues

Likely Exam Topic

If U is unitary and $\lambda \neq \mu$ are two distinct eigenvalues, then the eigenspaces corresponding to λ and μ are orthogonal.

Let:

$$Ux = \lambda x, \quad Uy = \mu y.$$

Using preservation of inner products:

$$\langle Ux, Uy \rangle = \langle x, y \rangle.$$

But using the eigenvalue equations:

$$\langle Ux, Uy \rangle = \langle \lambda x, \mu y \rangle.$$

With the lecture's inner product convention, the scalar multiplying the first argument comes out as λ , while the scalar multiplying the second argument comes out conjugated as μ^* . Hence:

$$\langle \lambda x, \mu y \rangle = \lambda \mu^* \langle x, y \rangle.$$

Therefore:

$$\lambda \mu^* \langle x, y \rangle = \langle x, y \rangle.$$

Move terms:

$$(1 - \mu^* \lambda) \langle x, y \rangle = 0.$$

Because μ is on the unit circle,

$$\mu^* = \frac{1}{\mu}.$$

So:

$$1 - \mu^* \lambda = 1 - \frac{\lambda}{\mu}.$$

If $\lambda \neq \mu$, then:

$$1 - \frac{\lambda}{\mu} \neq 0.$$

Thus:

$$\langle x, y \rangle = 0.$$

Therefore eigenvectors from distinct eigenspaces are orthogonal, and the eigenspaces themselves are orthogonal.

Relationship to Normal Matrices

This orthogonality property is not true for arbitrary matrices. It is true for unitary matrices.

The instructor also stated that the same property extends to a broader class of matrices, including Hermitian matrices. Matrices whose eigenspaces corresponding to distinct eigenvalues are orthogonal are part of the class called normal matrices.

Relationship:

- Unitary matrices are a subset of normal matrices.
- Hermitian matrices also belong to the same broader family.
- Normal matrices will be discussed later.

At this point the lecture used the eigenspace-orthogonality property as the preview. The formal normal-matrix condition will be developed later.

Orthonormal Basis Coordinates

The lecture then returned to orthonormal bases and coordinates.

Suppose:

$$x = \sum_{k=1}^n \alpha_k u_k$$

where $\{u_1, \dots, u_n\}$ is a basis.

For an arbitrary non-orthonormal basis, finding the coefficients α_k requires solving a system of linear equations. If the basis vectors are put into a matrix U , then:

$$x = U\alpha.$$

For a general basis, one must solve for:

$$\alpha = U^{-1}x.$$

But if the basis is orthonormal, finding the coordinates is simple.

Take the inner product of both sides with u_k :

$$\langle x, u_k \rangle = \sum_i \alpha_i \langle u_i, u_k \rangle.$$

All terms vanish except the $i = k$ term because the basis is orthonormal:

$$\langle u_i, u_k \rangle = 0 \quad (i \neq k),$$

and:

$$\langle u_k, u_k \rangle = 1.$$

Therefore:

$$\alpha_k = \langle x, u_k \rangle.$$

Projection Interpretation

Finding the coordinate α_k is the same as projecting x onto the basis element u_k . Since u_k has norm 1, the usual projection denominator is 1.

Matrix interpretation:

If the orthonormal basis vectors are columns of U , then U is unitary and:

$$\alpha = U^{-1}x = U^*x.$$

The entries of U^*x are inner products of x with the basis vectors:

$$\alpha_1 = \langle x, u_1 \rangle, \quad \alpha_2 = \langle x, u_2 \rangle, \quad \dots$$

So the algebraic fact $U^{-1} = U^*$ matches the geometric fact that coordinates in an orthonormal basis are obtained by projection.

Warning: Non-Orthogonal Bases Do Not Work This Way

The instructor emphasized this warning several times:

- If the basis is not orthogonal, one cannot simply project x onto each basis vector and sum the projections.
- Non-orthogonal basis vectors have overlapping components.
- Projecting separately onto non-orthogonal directions double-counts shared components.
- In that case, each coefficient generally depends on inner products with all basis elements, and a full system must be solved.

If the basis vectors are orthogonal, individual projections are valid:

- Project x onto one orthogonal basis direction.
- Project x onto the other orthogonal basis directions.
- Sum those orthogonal components to reconstruct x .

Real Orthogonal Matrices

A real orthogonal matrix is a real version of a unitary matrix.

For a real matrix Q , conjugate transpose becomes ordinary transpose, so:

$$Q^T Q = I, \quad Q Q^T = I, \quad Q^{-1} = Q^T.$$

The instructor noted that the name “real orthogonal matrix” is historically standard, even though one could argue that “real orthonormal matrix” might be more descriptive because the columns form an orthonormal basis.

Properties:

- Columns of Q form an orthonormal basis for $\mathbb{R}^n$.
- Rows of Q form an orthonormal basis for $\mathbb{R}^n$.
- Multiplication by Q corresponds geometrically to combinations of rotations and reflections.

Two-Dimensional Rotation Matrix

The instructor revisited a homework-style example: rotating a vector in $\mathbb{R}^2$ counterclockwise by an angle θ .

If $x = (x_1, x_2)^T$, then the rotated vector y is given by:

$$y_1 = x_1 \cos \theta - x_2 \sin \theta,$$

$$y_2 = x_1 \sin \theta + x_2 \cos \theta.$$

The instructor said this can be derived using trigonometric identities, but skipped the full derivation because it had already appeared in an early homework.

$$y = Qx,$$

where:

$$Q = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

This matrix is real orthogonal:

$$QQ^T = I.$$

So a two-dimensional rotation is an example of a real orthogonal transformation.

Reflections Across a Hyperplane

The second fundamental real orthogonal operation discussed was reflection.

In two dimensions, reflection across a line can be interpreted as taking the mirror image of a vector with respect to that line. In higher dimensions, the reflecting object is a hyperplane through the origin.

Suppose a hyperplane is defined by a normal vector v . The hyperplane is:

$$H = \{x : v^T x = 0\}.$$

The instructor assumes v may be unit norm for simplicity, though the formula can be written without that assumption.

Projection Step

To reflect x across the hyperplane, first project x onto the normal direction v . In the real case:

$$\text{proj}_v(x) = \frac{v^T x}{v^T v} v.$$

Equivalently:

$$\text{proj}_v(x) = \frac{vv^T}{v^T v} x.$$

The matrix:

$$\frac{vv^T}{v^T v}$$

is the orthogonal projection matrix onto the one-dimensional subspace spanned by v .

Orthogonal Component in the Hyperplane

Subtracting the projection onto v removes the normal component:

$$x - \frac{vv^T}{v^T v} x = \left(I - \frac{vv^T}{v^T v} \right) x.$$

This gives the component of x lying in the hyperplane.

In the lecture's geometry, this hyperplane component is still not the reflected point. It is only the projection of x onto the mirror hyperplane. The reflection keeps this hyperplane component and flips the normal component to the other side.

Reflection Formula

For the reflection itself, one must go twice the normal projection distance in the opposite direction. Thus:

$$x_{\text{reflected}} = x - 2 \frac{vv^T}{v^T v} x.$$

So the reflection matrix is:

$$R = I - 2 \frac{vv^T}{v^T v}.$$

If v is unit norm, $v^T v = 1$, and:

$$R = I - 2vv^T.$$

The instructor remarked that this was essentially one of the homework problems and that he had “sort of solved it” in lecture.

Application: Rigid Motion

Real orthogonal matrices appear naturally in rigid motion, especially in mechanical engineering and computer vision.

The instructor remarked that modern deep networks may hide many of these details in applications, but the underlying geometry is still useful background.

A rigid object changes position and orientation but does not change shape. For example, a triangle in two-dimensional space may move and rotate, but its side lengths and internal geometry remain unchanged.

Rigid motion can be written as:

$$b = Qa + t.$$

Here:

- a is a point before motion.
- b is the corresponding point after motion.
- Q is a rotation/orthogonal matrix.
- t is a translation vector.

This mapping is not linear because of the translation term. It is an affine mapping. If $t = 0$, the mapping would be linear.

Relationship:

- Q determines the rotation/orientation change.
- t determines the translation/location change.
- Q satisfies the real orthogonality constraint $QQ^T = I$.

Conceptually, the motion can be viewed as rotating the object, for example relative to its center of mass, and then translating the rotated object to its new location.

Image Registration and Motion Detection

The instructor described image registration:

- Register points on the original object or image.
- Observe the corresponding points after motion.
- Use these correspondences to estimate Q and t .

Example:

$$b_1 = Qa_1 + t,$$

$$b_2 = Qa_2 + t,$$

and so on for m registered points.

The motion detection problem is to determine Q and t from the observed point correspondences.

Because of noise and imperfect registration, the equations may not have an exact solution. Then one solves a least squares version of the problem.

Matrix Form of Registered Point Equations

Collect the original points as columns of a matrix:

$$A = [a_1 \ a_2 \ \cdots \ a_m],$$

and the moved points as:

$$B = [b_1 \ b_2 \ \cdots \ b_m].$$

Then:

$$B = QA + t\mathbf{1}^T.$$

Here $A, B \in \mathbb{R}^{d \times m}$, $Q \in \mathbb{R}^{d \times d}$, $t \in \mathbb{R}^d$, and $\mathbf{1} \in \mathbb{R}^m$. The vector $\mathbf{1}$ is a column vector of ones, so $\mathbf{1}^T$ is a row vector. Multiplying t by $\mathbf{1}^T$ replicates the translation vector t across all columns.

Exam Warning About Broadcasting

The instructor warned that MATLAB may allow writing something like:

$$B = QA + t$$

and automatically replicate t across columns through broadcasting.

This is acceptable in MATLAB programming behavior but not acceptable in the course's mathematical notation or on the exam. In EE 545 notation, the replicated translation must be written explicitly:

$$B = QA + t\mathbf{1}^T.$$

Three-Dimensional Motion Parameters

If the registered points are three-dimensional:

- Q is 3×3 , so it has 9 entries.
- t is 3×1 , so it has 3 entries.
- Together there are 12 scalar unknowns.

The orthogonality constraint means those 9 entries of Q are not freely independent in the final rotation estimate, but the instructor counted the raw scalar quantities before emphasizing the constraint.

More observations generally improve the quality of the estimate, but the quality of the point registration is critical.

The instructor also mentioned projective geometry: in images, two-dimensional measurements often come from projections of a three-dimensional world. For the lecture discussion, this complication was set aside by assuming the relevant three-dimensional coordinates are known.

Orthogonality Constraints in Estimation

The motion estimation problem is not just an unconstrained least squares problem, because Q should satisfy:

$$QQ^T = I.$$

If the point correspondences are perfect, solving without explicitly imposing the constraint may still produce an orthogonal Q .

If the data are noisy, the constraint is useful:

- It encodes the knowledge that Q is a rotation/orthogonal matrix.
- It can reduce the effect of noise.

One practical approach:

1. Ignore the orthogonality constraint and solve for Q and t .
2. The resulting Q may not be exactly orthogonal.
3. Project that Q to the closest real orthogonal matrix.

The instructor said better methods also exist. One can formulate the estimation as an optimization problem and impose the orthogonality constraint throughout the iterative updates.

Closest Orthogonal Matrix Problem

The lecture introduced the problem:

Given a square matrix Z that is not orthogonal, what is the closest orthogonal matrix to Z ?

This raises the question: what does “closest” mean for matrices?

For vectors, distance is already defined through norms. For matrices, the course will extend vector distance concepts to matrix norms.

Frobenius Norm

The Frobenius norm is the matrix analogue of the Euclidean norm.

For a vector, the Euclidean norm is:

$$\|x\|_2 = \sqrt{\sum_i |x_i|^2}.$$

For a matrix, the Frobenius norm is:

$$\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}.$$

That is:

- Take the absolute square of every matrix entry.
- Sum them.
- Take the square root.

The closest orthogonal matrix problem can then be posed as minimizing an error such as:

$$\|Z - Q\|$$

over orthogonal Q , with the particular answer depending on the chosen matrix norm.

The instructor noted:

- The problem is hard in general.
- For some special norms, it becomes easy.
- Singular value decomposition will be used later to solve such problems.

Orthogonality Constraints in Machine Learning

The instructor briefly mentioned that orthogonality constraints appear in modern machine learning as well.

Examples and remarks:

- Some methods impose orthogonality constraints on matrices during optimization.
- There are applications involving special directions in gradient search.
- The instructor mentioned “Moo iterations” or a similar term in the transcript; the exact reference was unclear, but the point was that orthogonal matrices appear in machine learning optimization contexts.

Returning to Energy-Preserving Systems

The lecture returned to the signal/vector interpretation.

A vector can be viewed as a signal. If:

$$y = Ux$$

with U unitary, then:

$$\sum_i |y_i|^2 = \sum_i |x_i|^2.$$

So the input signal energy equals the output signal energy.

Fourier Transform as a Linear Transformation

The discrete Fourier transform can be written as a matrix multiplication:

$$X = Fx.$$

Here:

- x is a finite-duration discrete-time signal.
- X is its discrete Fourier transform.
- F is the Fourier transform matrix.

The Fourier transform is therefore a linear transformation of special form.

With the standard DFT convention discussed in the lecture, for $k = 0, \dots, N - 1$:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}.$$

So each Fourier coefficient is one row of F applied to the signal vector x .

Parseval Relationship and Scaling

The instructor connected this to Parseval’s identity.

If F were unitary, then energy in the time domain and energy in the Fourier domain would match directly:

$$\|X\|_2^2 = \|x\|_2^2.$$

However, the common definition used in standard signal processing books, such as Oppenheim and Schaffer, does not make the DFT matrix unitary. It gives a scaled unitary transformation.

In that convention, the inverse Fourier transform includes a factor $1/n$. This scaling appears because the Fourier matrix has orthogonal but not orthonormal rows and columns.

Equivalently, under this convention:

$$\|X\|_2^2 = N\|x\|_2^2,$$

so Parseval's relation is written as:

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

The $1/N$ factor is exactly the scaling effect the instructor pointed to.

DFT Matrix Rows as Complex Exponentials

The DFT coefficient $X[k]$ is computed as an inner product of the signal x with a complex exponential sequence.

The minus sign in the DFT exponential appears because the complex exponential basis vector is conjugated in the inner product.

For example, if $w_k[n] = e^{j2\pi kn/N}$, then:

$$\langle x, w_k \rangle = w_k^* x = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}.$$

Thus:

- Each row of the DFT matrix is the conjugate transpose of a complex exponential signal.
- Multiplying F by x takes inner products of x with these complex exponentials.
- This is like projecting x onto complex exponentials, except the normalization is delayed to the inverse transform.

DFT Matrix Is Orthogonal but Not Orthonormal Under the Standard Scaling

The DFT matrix has orthogonal rows and columns, but the rows and columns do not have norm 1. Instead, each row/column has norm:

$$\sqrt{n}.$$

Therefore:

$$FF^* = nI.$$

The matrix is not unitary because a unitary matrix would satisfy:

$$FF^* = I.$$

Consequently:

$$F^{-1} = \frac{1}{n} F^*.$$

This explains the $1/n$ factor in the inverse DFT.

It also explains the $1/n$ factor in the Parseval relation under that convention: the DFT matrix is a scaled unitary matrix rather than a unitary matrix.

Preview of Next Topics

The lecture will continue with unitary matrices next.

Then it will move to Hermitian matrices, which belong to the same broader family as unitary matrices and are also very important. Later topics will include positive semidefinite matrices.

Instructor Remark About Homework and AI

The instructor closed with a learning warning about homework.

The main concern is not framed as academic honesty, but as learning:

- Students learn the material by solving problems themselves.
- Understanding a provided solution is not the same as understanding the concepts.
- Real understanding comes from struggling through and solving the problem independently.
- Students should not use AI to solve homework problems for them.
- It is acceptable to ask AI conceptual questions, but using it to bypass the problem-solving process undermines the purpose of the homework.

The instructor specifically warned that it is tempting to upload a screenshot of a problem to an AI system and ask for the answer, but students should spend substantial time thinking through the problem themselves.

Exam-Relevant Items Explicitly Flagged in the Transcript

The transcript explicitly marked the following as likely exam topics or exam notes:

- Unitary and orthogonal matrices.
- Eigenvalues of unitary matrices.
- Orthogonal eigenspaces for distinct eigenvalues.
- Every square matrix can be triangularized, and the triangularizing matrix can be chosen orthogonal/unitary.
- Orthogonal projection onto a subspace using an orthonormal basis matrix.
- Definition of unitary matrices and $U^{-1} = U^*$.
- Unitary matrices preserve Euclidean norm and energy.
- LTI systems require convolution/shift/Toeplitz structure; a unitary matrix alone is not necessarily time invariant.
- Coordinates in an orthonormal basis are found by inner products/projections.
- Projection onto non-orthogonal basis vectors cannot be done independently.
- Two-dimensional rotation matrices are real orthogonal.
- Reflection across a hyperplane uses projection onto the normal direction.
- The reflection formula $I - 2vv^T/(v^T v)$ was essentially a homework problem.
- In exam notation, do not rely on MATLAB broadcasting; write $B = QA + t\mathbf{1}^T$.
- Under the standard DFT scaling used in Oppenheim/Schafer-style notation, F is scaled unitary rather than unitary, $FF^* = nI$, $F^{-1} = \frac{1}{n}F^*$, and Parseval has a $1/n$ factor.
- Homework should be solved independently for learning; AI may be used for conceptual questions but not as a shortcut to solutions.

Source and Coverage Note

Source: C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture10_corrected.md.

Coverage: These notes follow only Lecture 10 and preserve the chronological development from matrix factorizations and projection review through unitary matrices, eigenvalue properties, orthonormal coordinates, real orthogonal rotations/reflections, rigid motion, closest orthogonal matrix motivation, Frobenius norm, Fourier/Parseval scaling, and the closing homework/AI learning remark. No other lecture transcript was processed.

Lecture 11 Notes

1. Context From Previous Lectures

The lecture begins by tying together several earlier themes in matrix analysis and systems of linear equations.

Matrix Rank and Matrix Shape

- The rank of a matrix is the dimension of both its column space and row space.
- Earlier conclusions depended on matrix shape:
 - tall matrices,
 - square matrices,
 - fat matrices.
- These shape-based cases lead to different conclusions about linear systems and matrix mappings.

Solving Linear Systems Through “Simple Problems”

The instructor frames the methods for solving systems of linear equations under one broad strategy:

- Some systems are easy to solve when the coefficient matrix is one of a few simple types:
 - diagonal,
 - triangular,
 - orthogonal/unitary.
- The general strategy is to convert a difficult arbitrary system into a sequence of such simple systems.
- Matrix factorization is the algebraic version of this idea:
 - instead of solving directly with an arbitrary matrix A ,
 - write A as a product of simpler matrices,
 - then solve through the corresponding simpler steps.

This is the organizing principle behind later decompositions such as Schur decomposition and singular value decomposition.

Basis Change and Diagonalization

For a change of basis, the same linear transformation is represented by

$$T^{-1}AT.$$

This representation was previously analyzed through eigenvalues and eigenvectors.

Important conclusion:

- Not every matrix is diagonalizable.
- Some matrices admit a basis in which the transformation is diagonal.
- Some matrices do not.
- Lecture 11 introduces a weaker but universal result:
 - every square matrix can be triangularized by a basis change.

This leads into Schur’s theorem.

Orthogonality, Inner Products, and Projections

Before this lecture, the course extended the real Euclidean inner product to complex inner products.

Previously discussed ideas include:

- projecting one vector onto another vector,
- projecting a vector onto a subspace,
- orthogonal projection matrices,

- describing a projection by choosing a basis for the target subspace.

For projection onto a subspace:

- If an orthonormal basis is chosen, the projection matrix has a simpler form.
- If an arbitrary basis is chosen, the projection matrix becomes more involved.
- The instructor said this more general formula was not derived at that time, but will return later in least squares.
- The formula can be derived independently.

2. Review of Unitary Matrices

LIKELY EXAM TOPIC

A unitary matrix U is a complex matrix whose inverse equals its conjugate transpose:

$$U^{-1} = U^*.$$

Here U^* denotes the Hermitian transpose, i.e. transpose plus complex conjugation.

Why Unitary Systems Are Simple

Unitary systems are one of the “simple” classes of linear systems because solving

$$Ux = b$$

is easy:

$$x = U^{-1}b = U^*b.$$

The inverse is obtained directly by conjugate transposition.

Orthonormal Columns and Rows

For an $n \times n$ unitary matrix:

- the columns form an orthonormal basis for $\mathbb{C}^n$,
- the rows also form an orthonormal basis for $\mathbb{C}^n$.

In the real case, a real unitary matrix is called a real orthogonal matrix. Its rows and columns form orthonormal bases for $\mathbb{R}^n$.

Properties Reviewed

Unitary matrices preserve:

- Euclidean norm,
- inner product.

Consequences:

- eigenvalues of a unitary matrix lie on the complex unit circle,
- the determinant of a unitary matrix also lies on the unit circle,
- eigenvectors corresponding to distinct eigenvalues are orthogonal.

Equivalently, eigenspaces associated with distinct eigenvalues are orthogonal.

The instructor emphasized that this last eigenspace property will later be used to define a larger class of matrices, namely normal matrices.

3. Applications of Unitary Matrices

Rigid Motion and Rotation

The most basic application of unitary or orthogonal matrices is rigid motion, especially rotation.

Because unitary matrices preserve norm and inner product, they preserve geometric structure such as lengths and angles.

Lossless Linear Systems

Another application is modeling lossless linear systems or lossless linear transformations.

“Lossless” means energy-preserving.

Energy is defined using the Euclidean norm, often as the squared norm:

$$\|x\|^2.$$

If a system is represented by a unitary matrix, then the input and output have the same Euclidean norm, hence the same energy.

4. LTI Systems, Toeplitz Matrices, and All-Pass Systems

The instructor then connects unitary matrices to linear time-invariant systems.

Linear Time-Invariant Systems

An LTI system is a linear time-invariant system.

The output sequence can be represented as an infinite vector, and each output sample is a weighted combination of input samples.

For a causal discrete-time system:

$$y_0 = h_0x_0 + h_1x_{-1} + h_2x_{-2} + \cdots.$$

[EXAM NOTE]

Causality means the output at a given time depends only on:

- the input at that same time,
- previous inputs.

It does not depend on future inputs.

For example:

- y_0 depends on $x_0, x_{-1}, x_{-2}, \dots$,
- y_1 depends on $x_1, x_0, x_{-1}, \dots$

The instructor’s point was not only the single equation for y_0 , but the repeated row pattern:

$$y_1 = h_0x_1 + h_1x_0 + h_2x_{-1} + \cdots,$$

and similarly for every later time. The same weights are reused; only the input indices shift.

Time Invariance

Time invariance means the formula relating output to current and past inputs does not change over time.

For example:

- y_0 relates to x_0 through coefficient h_0 ,
- y_1 relates to x_1 through the same coefficient h_0 ,
- the same coefficients $h_0, h_1, h_2, \dots$ appear at each time, shifted in position.

If the coefficients changed from row to row, the system could still be linear, because each output would still be a weighted combination of inputs, but it would not be time-invariant.

Matrix Structure

For a causal LTI system, the input-output relation can be written as a matrix-vector product.

The matrix has two important structural properties:

- lower triangular due to causality,
- shifted rows due to time invariance.

A matrix whose diagonals are constant is called a Toeplitz matrix.

Therefore, a causal LTI system is represented by a lower triangular Toeplitz matrix.

A one-sided finite display of the same structure is

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \end{bmatrix} = \begin{bmatrix} h_0 & 0 & 0 & \cdots \\ h_1 & h_0 & 0 & \cdots \\ h_2 & h_1 & h_0 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \end{bmatrix}.$$

The lower-triangular zeros encode causality, and the constant diagonals encode time invariance.

Convolution

The general input-output formula for a causal LTI system is

$$y_n = \sum_{k=0}^{\infty} h_k x_{n-k}.$$

This is the convolution operation.

The coefficients h_k are the impulse response of the system.

The infinite matrix representation of this convolution operation is called a convolution matrix.

Homework Connection: Finite and Circular Convolution

[EXAM NOTE]

In the current homework, the instructor said students will examine finite-dimensional sequences rather than infinite-dimensional discrete-time sequences. This gives a finite convolution matrix.

[EXAM NOTE]

The homework uses circular convolution. In that case:

- the convolution matrix is not Toeplitz,
- it is circulant.

In a circulant matrix, rows are rotated rather than simply shifted:

- an entry shifted past the right end comes back to the beginning.

The instructor noted this as a homework topic and avoided giving the full spoiler.

Losslessness for LTI Systems

The lower triangular Toeplitz structure comes from causality and time invariance.

Losslessness adds another property:

- in the complex case, the convolution matrix should be unitary,
- in the real-coefficient or real-signal case, it should be real orthogonal.

For a unitary convolution matrix:

$$H^* H = I.$$

In terms of the impulse response, this corresponds to a convolution relation involving h and the conjugated time-reversed impulse response, giving a Kronecker delta:

$$\sum_k h_k \overline{h_{k-m}} = \delta[m]$$

or, equivalently, the impulse response convolved with its conjugate-reversed version gives the delta function. The instructor left this derivation for students to inspect.

Fourier-Domain Interpretation

The Fourier transform turns convolution into multiplication. The instructor emphasized this as one reason Fourier analysis is so useful.

The frequency response is the Fourier transform of the impulse response h_k :

If

$$H(e^{j\omega})$$

is the Fourier transform of the impulse response, then the lossless/unitary condition implies

$$|H(e^{j\omega})|^2 = 1$$

for all frequencies.

Thus

$$|H(e^{j\omega})| = 1$$

for all ω .

Interpretation:

- a lossless LTI system passes every frequency with magnitude one,
- it does not attenuate any frequency,
- it does not amplify any frequency.

Such a system is called an all-pass system.

Relationship:

- arbitrary unitary matrices represent lossless finite-dimensional linear transformations,
- when the linear transformation is also time-invariant and has a frequency response, the unitary property appears as the all-pass property.
- the impulse-response orthogonality condition transforms into $H(e^{j\omega})\overline{H(e^{j\omega})} = 1$, which is exactly the magnitude-one condition.

5. Schur Theorem and Unitary Triangularization

LIKELY EXAM TOPIC

The next main theorem is Schur's theorem.

The instructor describes it as a useful consolation after the failure of universal diagonalization.

Motivation

Previously:

- the class tried to find a basis that makes a given square matrix diagonal,
- this is possible only for diagonalizable matrices,
- not every matrix is diagonalizable.

Schur's theorem says something weaker but always possible for square matrices:

- every square matrix can be represented by a triangular matrix in a suitable orthonormal basis.
- this lecture's theorem is explicitly for square matrices; rectangular/non-square decompositions are deferred to later material, especially the SVD discussion.

Statement

For every square matrix A , there exists a unitary matrix U such that

$$U^*AU = T,$$

where T is upper triangular.

Equivalently,

$$A = UTU^*.$$

This is the Schur decomposition.

Because $U^{-1} = U^*$, the expression U^*AU is a similarity transformation:

$$U^{-1}AU.$$

Interpretation:

- Schur's theorem gives a basis change,
- the new basis can be chosen orthonormal,
- in that basis the linear transformation is triangular.

The instructor's plain-language summary:

- all square matrices are unitarily triangularizable.

A lower triangular version is also possible.

Relation to SVD

The instructor compares the theorem to singular value decomposition.

Schur decomposition:

$$A = UTU^*$$

uses the same unitary basis on the input and output side because A is square and the domain/codomain dimensions match.

SVD, to be discussed later, has the spirit of

$$A = U\Sigma V^*$$

and can handle non-square matrices. It uses different bases for input and output spaces.

Thus:

- Schur: same unitary basis on both sides, triangular middle factor, square matrices.
- SVD: generally different unitary bases on each side, diagonal/singular-value middle factor, can handle rectangular matrices.

6. Proof Idea for Schur Theorem

The proof is constructive and iterative.

Step 1: Choose a Unit-Norm Eigenvector

Start with an eigenvalue-eigenvector pair of A :

$$Au_1 = \lambda_1 u_1.$$

Choose u_1 with unit norm:

$$\|u_1\| = 1.$$

This is allowed because any nonzero eigenvector can be normalized.

Step 2: Extend to an Orthonormal Basis

Extend u_1 to an arbitrary orthonormal basis:

$$u_1, u'_2, u'_3, \dots, u'_n.$$

The vectors $u'_2, \dots, u'_n$:

- are not assumed to be eigenvectors,
- are chosen only to complete an orthonormal basis,
- are orthogonal to u_1 ,
- have unit norm.

There are generally infinitely many ways to choose them.

Place these vectors into the columns of a matrix:

$$U_1 = \begin{bmatrix} u_1 & u'_2 & \cdots & u'_n \end{bmatrix}.$$

Because its columns are orthonormal, U_1 is unitary:

$$U_1^* U_1 = I.$$

Instructor remark:

- U_1 is not yet the final U from Schur's theorem.
- It is an intermediate unitary matrix.

Step 3: Apply a Similarity Transformation

Consider

$$U_1^* A U_1.$$

This is a basis change. It is the first iteration of the triangularization process.

Partition the right multiplication by columns:

$$A U_1 = \begin{bmatrix} A u_1 & A u'_2 & \cdots & A u'_n \end{bmatrix}.$$

Since u_1 is an eigenvector,

$$A u_1 = \lambda_1 u_1.$$

Therefore, the first column of $A U_1$ is $\lambda_1 u_1$.

For the other columns $A u'_2, \dots, A u'_n$, no eigenvector simplification is available. Those basis vectors were chosen only to complete an orthonormal basis, so their images can be collected into unspecified entries.

Now multiply on the left by U_1^* . The entries in the first column are inner products between basis vectors and $\lambda_1 u_1$:

- first entry:

$$u_1^* (\lambda_1 u_1) = \lambda_1 u_1^* u_1 = \lambda_1,$$

because $\|u_1\| = 1$;

- lower entries:

$$(u'_j)^* (\lambda_1 u_1) = 0$$

because u'_j is orthogonal to u_1 .

So the transformed matrix has the block form

$$U_1^* A U_1 = \begin{bmatrix} \lambda_1 & q^* \\ 0 & A_2 \end{bmatrix}.$$

Here:

- q^* represents entries not controlled in the first row,
- A_2 is an $(n-1) \times (n-1)$ submatrix,
- the first column already has the upper-triangular pattern.

Instructor wording:

- this first step does not finish triangularization,
- it makes progress by fixing the first column,
- subsequent steps will fix the second column, then the third, and so on.

7. Eigenvalues of the Submatrix A_2

[EXAM NOTE]

The instructor reminded the class that $U_1^*AU_1$ is similar to A , so it has the same eigenvalues as A .

Proof:

$$\det(\lambda I - U_1^*AU_1)$$

can be rewritten using $I = U_1^*U_1$:

$$\det(U_1^*(\lambda I - A)U_1).$$

Then

$$\det(U_1^*) \det(\lambda I - A) \det(U_1).$$

For a unitary matrix, $\det(U_1^*) \det(U_1) = 1$, so this equals

$$\det(\lambda I - A).$$

Therefore A and $U_1^*AU_1$ share the same characteristic polynomial and eigenvalues.

Now use the block form:

$$U_1^*AU_1 = \begin{bmatrix} \lambda_1 & q^* \\ 0 & A_2 \end{bmatrix}.$$

Then

$$\lambda I - U_1^*AU_1 = \begin{bmatrix} \lambda - \lambda_1 & * \\ 0 & \lambda I - A_2 \end{bmatrix}.$$

The determinant of this block upper triangular matrix is

$$(\lambda - \lambda_1) \det(\lambda I - A_2).$$

Thus the characteristic polynomial of A factors as

$$\det(\lambda I - A) = (\lambda - \lambda_1) \det(\lambda I - A_2).$$

Conclusion:

- the eigenvalues of A_2 are the remaining eigenvalues of A , counting multiplicity,
- the eigenspaces are not the same, because A_2 acts on an $(n - 1)$ -dimensional space.

8. Continuing the Schur Iteration

The first step gives

$$U_1^* A U_1 = \begin{bmatrix} \lambda_1 & q^* \\ 0 & A_2 \end{bmatrix}.$$

Equivalently,

$$A = U_1 \begin{bmatrix} \lambda_1 & q^* \\ 0 & A_2 \end{bmatrix} U_1^*.$$

Now apply the same procedure to A_2 .

Since A_2 is $(n - 1) \times (n - 1)$, choose a unitary matrix U_2 such that

$$U_2^* A_2 U_2 = \begin{bmatrix} \lambda_2 & * \\ 0 & A_3 \end{bmatrix}.$$

Insert this into the block matrix by using the block diagonal unitary matrix

$$\begin{bmatrix} 1 & 0 \\ 0 & U_2 \end{bmatrix}.$$

Concretely, the second step applies

$$\begin{bmatrix} 1 & 0 \\ 0 & U_2^* \end{bmatrix} \begin{bmatrix} \lambda_1 & q^* \\ 0 & A_2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & U_2 \end{bmatrix} = \begin{bmatrix} \lambda_1 & q^* U_2 \\ 0 & U_2^* A_2 U_2 \end{bmatrix}.$$

Since $U_2^* A_2 U_2$ has first column $(\lambda_2, 0, \dots, 0)^T$, the second column of the overall matrix now also satisfies the upper-triangular pattern.

This block diagonal matrix is unitary because:

- 1 is a 1×1 unitary block,
- U_2 is unitary,
- multiplying the block diagonal matrix by its conjugate transpose gives the identity.

The product of unitary matrices is also unitary.

After the second iteration:

- the first two columns have the upper-triangular structure,
- the remaining lower-right block is A_3 , now $(n - 2) \times (n - 2)$,
- A_3 has the remaining eigenvalues of A .

Continue this process.

After enough iterations, the whole matrix becomes upper triangular:

$$U^* A U = T.$$

The instructor said one can think of this as working column by column. In practice, after $n - 1$ main reductions, the final remaining scalar entry is already determined, so the process terminates.

Handling Eigenvalue Multiplicity

In response to a question, the instructor noted that algebraic or geometric multiplicity does not cause difficulty in this construction.

If λ_1 has a multidimensional eigenspace:

- choose one unit eigenvector u_1 from that eigenspace,
- complete it to an orthonormal basis,
- the same eigenvalue may still appear in A_2 .

The method does not require distinct eigenvalues or one-dimensional eigenspaces.

Conceptual Meaning

Schur decomposition achieves the course's main factorization goal:

$$A = UTU^*.$$

This writes any square matrix as a product of simple matrices:

- a unitary matrix,
- a triangular matrix,
- another unitary matrix, specifically the conjugate transpose of the first.

This is a major example of reducing arbitrary matrices to simple matrix factors.

9. Transition to Hermitian and Normal Matrices

The next topic is broader than unitary matrices.

The instructor introduced a bigger family called normal matrices.

Important big-picture relationships:

- unitary matrices are normal,
- Hermitian matrices are normal,
- normal matrices have orthogonal eigenspaces corresponding to distinct eigenvalues.

The lecture has not yet fully developed normal matrices, but the instructor used them to place unitary and Hermitian matrices in context.

Intersection of Unitary and Hermitian Matrices

The intersection of unitary and Hermitian matrices is nontrivial.

Example:

$$I$$

is both unitary and Hermitian.

Why:

- $I^{-1} = I = I^*$, so I is unitary,
- $I^* = I$, so I is Hermitian.

The instructor said the class will return to the larger picture and further partition Hermitian matrices later.

10. Hermitian Matrices

LIKELY EXAM TOPIC

A Hermitian matrix A satisfies

$$A^* = A.$$

That is, the conjugate transpose of the matrix equals the matrix itself.

Contrast With Unitary Matrices

For a unitary matrix:

$$A^* = A^{-1}.$$

For a Hermitian matrix:

$$A^* = A.$$

Thus Hermitian matrices are not “simple” in the same linear-system-solving sense as unitary matrices, because taking the conjugate transpose does not directly give the inverse. However, Hermitian matrices have many important structural properties and many applications.

[EXAM NOTE]

Hermitian matrices are very important. The instructor specifically mentioned later applications in:

- optimization,
- stochastic processes,
- random vectors.

11. Entrywise Structure of Hermitian Matrices

For a Hermitian matrix A , the entries satisfy

$$a_{ij} = \overline{a_{ji}}.$$

Equivalently:

- across the diagonal, entries are conjugates of each other,
- not necessarily equal to each other.

Example:

If

$$a_{21} = 1 + i,$$

then

$$a_{12} = 1 - i.$$

Real Symmetric Matrices as a Special Case

If the matrix is real, conjugation does nothing.

Then the Hermitian condition becomes

$$a_{ij} = a_{ji}.$$

So a real Hermitian matrix is exactly a real symmetric matrix.

Example form:

$$\begin{bmatrix} 1 & 5 \\ 5 & 2 \end{bmatrix}$$

is real symmetric, hence Hermitian.

Diagonal Entries Are Real

For diagonal entries:

$$a_{ii} = \overline{a_{ii}}.$$

A number equal to its own complex conjugate must be real.

Therefore, every diagonal entry of a Hermitian matrix is real.

12. Quadratic Forms From Hermitian Matrices

LIKELY EXAM TOPIC

The instructor emphasized that a major reason Hermitian matrices are important is that they define real-valued quadratic functions of complex vectors.

For a complex vector $x \in \mathbb{C}^n$, consider

$$x^*Ax.$$

Here x^*x is only the special norm-squared case obtained when $A = I$. The quadratic form discussed in this section is x^*Ax .

Dimensions:

- x^* is $1 \times n$,
- A is $n \times n$,
- x is $n \times 1$.

Therefore

$$x^*Ax$$

is a 1×1 scalar.

For a general complex matrix A , this scalar may be complex.

If A is Hermitian, then x^*Ax is real for every x .

Two-Dimensional Example

Let

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

and let

$$A = \begin{bmatrix} a_{11} & a_{12} \\ \overline{a_{12}} & a_{22} \end{bmatrix}$$

be Hermitian.

Then

$$x^* = \begin{bmatrix} \overline{x_1} & \overline{x_2} \end{bmatrix}.$$

Expanding,

$$x^*Ax = |x_1|^2 a_{11} + \overline{x_1}x_2 a_{12} + \overline{x_2}x_1 \overline{a_{12}} + |x_2|^2 a_{22}.$$

The two cross terms are conjugates of each other:

$$\overline{x_1}x_2 a_{12} \quad \text{and} \quad \overline{x_2}x_1 \overline{a_{12}}.$$

Adding a complex number and its conjugate gives twice its real part. Thus

$$x^*Ax = |x_1|^2 a_{11} + |x_2|^2 a_{22} + 2 \operatorname{Re}(\overline{x_1}x_2 a_{12}).$$

Each term is real:

- $|x_1|^2$ and $|x_2|^2$ are real,
- a_{11} and a_{22} are real because Hermitian diagonal entries are real,
- the cross terms combine into a real number.

The instructor used this example to show why the expression is called a quadratic form:

- it contains squared magnitudes of vector components,
- it contains products of vector components and conjugates.

13. General Proof That x^*Ax Is Real for Hermitian A

The elementwise expansion is not necessary. The property follows directly from conjugate transposition.

Let

$$z = x^*Ax.$$

This is a scalar. Its conjugate is

$$\overline{z} = z^*.$$

Now compute:

$$(x^* Ax)^* = x^* A^* x.$$

If A is Hermitian, then

$$A^* = A.$$

Therefore

$$(x^* Ax)^* = x^* Ax.$$

So the scalar equals its own conjugate. Hence it is real.

Conclusion:

- Hermitian matrices define real-valued quadratic forms on complex vector spaces.

14. Quadratic Functions and Optimization

The instructor connected Hermitian quadratic forms to optimization.

Optimization often represents nonlinear functions locally by quadratic approximations. This is the idea behind Newton-type methods and related approaches.

For real multivariable functions, if $x \in \mathbb{R}^n$, then a quadratic form has the form

$$x^T Ax,$$

where A is real symmetric, i.e. the real version of Hermitian.

In two variables, such expressions include terms like

$$a_{11}x_1^2 + 2a_{12}x_1x_2 + a_{22}x_2^2.$$

The instructor noted that this is the untranslated version. A more general quadratic function can include:

- translation in x , such as $x - x_0$,
- vertical translation by adding a scalar b .

A typical general form is conceptually like

$$(x - x_0)^T A(x - x_0) + b.$$

The untranslated form has its reference point at the origin and no vertical offset. The translated form moves the center/reference point to x_0 and shifts the function value by b . Whether this gives a minimum, maximum, or mixed saddle-like shape depends on the matrix A , which motivates classifying Hermitian matrices by eigenvalue signs.

[EXAM NOTE]

Quadratic functions are important because a general nonlinear surface can be approximated near a point x_0 by a quadratic function.

The Hermitian or symmetric matrix determines the shape of the quadratic function.

In one dimension, a quadratic is simply:

- parabola up,

- or parabola down.

In multiple dimensions, the shape can be mixed:

- upward curvature in some directions,
- downward curvature in other directions.

This motivates classifying Hermitian matrices by eigenvalue signs, which the instructor said will be discussed later.

15. Eigenvalues of Hermitian Matrices Are Real

The next theorem:

For a Hermitian matrix A , every eigenvalue is real.

Hermitian matrices may have complex entries, but their eigenvalues lie on the real line.

Proof

Let $x \neq 0$ be an eigenvector of A with eigenvalue λ :

$$Ax = \lambda x.$$

Multiply on the left by x^* :

$$x^* Ax = x^* (\lambda x).$$

Since λ is a scalar,

$$x^* Ax = \lambda x^* x.$$

Now:

- $x^* Ax$ is real because A is Hermitian,
- $x^* x = \|x\|^2$ is real and strictly positive because $x \neq 0$.

Therefore

$$\lambda = \frac{x^* Ax}{x^* x}$$

is a real number.

Comparison With Unitary Matrices

For unitary matrices:

- eigenvalues lie on the complex unit circle.

For Hermitian matrices:

- eigenvalues lie on the real line.

The instructor related this to normal matrices:

- unitary and Hermitian matrices are both normal,
- among normal matrices, unitary matrices are characterized by eigenvalues on the unit circle,
- among normal matrices, Hermitian matrices are characterized by eigenvalues on the real line.

This full normal-matrix picture was flagged as a later topic.

16. Orthogonality of Hermitian Eigenspaces

The final property shown is that eigenvectors of a Hermitian matrix corresponding to distinct eigenvalues are orthogonal.

This was already shown earlier for unitary matrices, and the instructor emphasized it is part of the larger normal-matrix pattern.

Statement

Let

$$Ax = \lambda x,$$

and

$$Ay = \mu y,$$

where $\lambda \neq \mu$.

If A is Hermitian, then

$$y^* x = 0.$$

So x and y are orthogonal.

Equivalently:

- eigenspaces corresponding to distinct eigenvalues are orthogonal.

Proof

Start with

$$y^* Ax.$$

Using $Ax = \lambda x$,

$$y^* Ax = y^* (\lambda x) = \lambda y^* x.$$

Now evaluate the same expression another way.

Because A is Hermitian,

$$A^* = A.$$

Also, since

$$Ay = \mu y,$$

and Hermitian eigenvalues are real, $\mu = \bar{\mu}$.

Taking conjugate transposes gives

$$(Ay)^* = (\mu y)^*$$

so

$$y^* A^* = \bar{\mu} y^*.$$

Using $A^* = A$ and $\bar{\mu} = \mu$,

$$y^* A = \mu y^*.$$

Therefore

$$y^* Ax = \mu y^* x.$$

The two expressions for $y^* Ax$ must be equal:

$$\lambda y^* x = \mu y^* x.$$

Thus

$$(\mu - \lambda)y^* x = 0.$$

Because $\lambda \neq \mu$, it follows that

$$y^* x = 0.$$

So the eigenvectors are orthogonal.

17. Instructor Remarks and Reading Suggestion

The instructor ended by recommending the book *Matrix Analysis* by Horn and Johnson, noting that the lecture is following it closely while trying to present the material with a more accessible perspective.

The lecture will continue on Tuesday.

18. Exam and Homework Cues Collected Chronologically

- **LIKELY EXAM TOPIC** Unitary matrices: definition, inverse as conjugate transpose, orthonormal rows/columns, preservation of norm and inner product, unit-circle eigenvalues, orthogonality of eigenspaces.
- **[EXAM NOTE]** Causality: output at a time depends only on current and previous inputs.
- **[EXAM NOTE]** Homework uses finite-dimensional sequences, giving finite convolution matrices.
- **[EXAM NOTE]** Homework uses circular convolution, giving circulant rather than Toeplitz matrices; rows rotate rather than only shift.
- **LIKELY EXAM TOPIC** Schur factorization theorem: every square matrix is unitarily triangularizable.
- **[EXAM NOTE]** $U_1^* A U_1$ is a similarity transformation and has the same eigenvalues as A .
- **LIKELY EXAM TOPIC** Hermitian matrices: definition $A^* = A$, entrywise conjugate symmetry, real diagonal entries, relation to real symmetric matrices.
- **[EXAM NOTE]** Hermitian matrices are important for optimization, stochastic processes, and random vectors.
- **LIKELY EXAM TOPIC** Real-valued quadratic forms $x^* A x$ for Hermitian A .

- [EXAM NOTE] Quadratic functions are important because nonlinear surfaces are locally approximated by quadratic functions; matrix eigenvalue signs determine multivariable shape.

Source and Coverage Note

These notes were generated only from C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture11_corr
They preserve the lecture order and include the stated concepts, definitions, theorem statements, proof ideas, examples, instructor remarks, homework/exam cues, and relationships between unitary, Schur, Hermitian, quadratic-form, LTI, and normal-matrix ideas.

Lecture 12 Notes

1. Course Storyline: Simple Matrices, Factorizations, and Basis Changes

The lecture begins by placing the current topic inside the overall storyline of the course:

- Start with linear systems that are easy to solve.
- Use these easy cases as motivation for factorizations and matrix structure.
- Then study what happens for a general matrix by trying to convert it into one of the easier forms.

The simple matrix classes mentioned are:

- **Diagonal systems:** easy because each equation separates.
- **Triangular systems:** easy because they can be solved by forward or backward substitution.
- **Orthogonal or unitary systems:** easy because the inverse is obtained by transpose or conjugate transpose.

For a general matrix A , the main idea is to represent it as a product of simple matrices. This gives different matrix factorizations.

Another major idea is basis change:

- A matrix A defines a linear transformation.
- One can try to choose bases in the input and output spaces so that the representation of this transformation becomes diagonal.
- This leads to diagonalization.

A key limitation was recalled:

- Not every matrix is diagonalizable.
- Some matrices can be diagonalized, but others cannot.
- This motivates more flexible replacements, especially triangularization.

The diagonalization question is closely tied to eigenvalues and eigenvectors.

2. Orthogonality, Projection, and Complex Inner Products

The lecture recalls earlier material on geometry:

- Orthogonality.
- Orthogonal projection of a vector onto another vector.
- Orthogonal projection of a vector onto a subspace.
- Orthogonal projection matrices and their properties.

Likely exam topic Projection and orthogonality are part of the course's central set of structured matrix examples.

The instructor also recalls that the Euclidean inner product was extended to complex vectors:

- For complex vectors, the conjugate transpose appears in the inner product.
- This complex inner product is the setting for unitary matrices and Hermitian matrices.

The instructor remarks that later the course will generalize the inner product further:

- Alternative inner products will be introduced.
- These can be viewed as alternative geometries.

The structured matrices seen so far include:

- Diagonal matrices.
- Triangular matrices.
- Orthogonal matrices.
- Orthogonal projection matrices.

- Hermitian matrices.

The instructor emphasizes that understanding what each structure implies is at the core of the analysis.

3. Review of Unitary Matrices

A unitary matrix is the complex analogue of a real orthogonal matrix.

Definition:

$$U^*U = UU^* = I,$$

so

$$U^{-1} = U^*.$$

This means the inverse is obtained effortlessly by taking the conjugate transpose.

Solving a system involving a unitary matrix is therefore simple:

$$Ux = v$$

implies

$$x = U^*v.$$

Important properties of unitary matrices:

- They preserve Euclidean norm:

$$\|Ux\| = \|x\|.$$

- They preserve inner products:

$$\langle Ux, Uy \rangle = \langle x, y \rangle.$$

- Their eigenvalues lie on the unit circle:

$$|\lambda| = 1.$$

- Their determinant also lies on the unit circle, because it is the product of eigenvalues.

[Exam note] If a unitary matrix has two distinct eigenvalues, then the corresponding eigenspaces are orthogonal to each other. The instructor says this property will be generalized in this lecture.

This same distinct-eigenvalue orthogonality property also holds for Hermitian matrices.

4. Right and Left Eigenvectors

The standard, or right, eigenvalue equation is:

$$Ax = \lambda x.$$

A left eigenvector is written as a row vector multiplying the matrix from the left:

$$y^T A = \lambda y^T.$$

The instructor connects this to the idea of left null spaces:

- A left null space condition involves multiplying from the left by a row vector.
- The left null space of A is the null space of A^T .

Taking transposes relates left eigenvectors of A to right eigenvectors of A^T .

Instructor warning:

- The right-eigenvector equation and left-eigenvector equation are the same eigenvalue idea written on different sides of the matrix.
- A left eigenvector is not a new geometric object detached from the usual eigenvector concept; after transposition it becomes an eigenvector condition for A^T .
- The instructor briefly framed this as a trick question to make students distinguish column-vector multiplication from row-vector multiplication.

5. Orthogonal and Unitary Matrices in Applications

Real orthogonal matrices can model rigid motion.

Example:

- A triangle moving in two-dimensional space can be modeled by multiplication by a real orthogonal matrix, together with a translation.

The lecture also recalls unitary transformations:

- A unitary transformation is essentially an orthogonal basis change in complex space.
- It preserves the geometry determined by the Euclidean inner product.

6. Fourier Transform as a Unitary Transformation

[Exam note] The Fourier transform is described as an orthogonal/unitary transformation.

The Fourier basis appears naturally because it diagonalizes circulant matrices.

Important relationship:

- Circulant matrix diagonalization by the Fourier basis means that convolution in the time domain becomes multiplication in the frequency domain.
- This is the major significance of the Fourier transform in the linear algebra viewpoint.

The instructor mentions that the homework studies why the Fourier basis emerges.

7. Lossless Systems, Energy, and All-Pass Systems

Another application of unitary matrices is the modeling of lossless systems.

The instructor notes that “lossless” can mean different things depending on what notion of loss is used. In this lecture, losslessness is defined in terms of energy.

For a vector or signal x , energy is defined as:

$$\sum_i |x_i|^2.$$

The words vector and signal are used interchangeably in this discussion.

A system is lossless if:

$$\text{input energy} = \text{output energy}.$$

For a linear system, preserving this energy can be modeled by a unitary matrix.

If the system is also time invariant, this leads to all-pass systems:

- The frequency response has magnitude equal to one.
- At every frequency, the system neither amplifies nor attenuates the input.

[Exam note] The convolution matrix for a linear time-invariant causal system has several structures at once:

- It is **lower triangular** because the system is causal.
- It is **Toeplitz** because it represents convolution.
- It is **unitary** if the system is lossless.

The rows of this convolution matrix contain the impulse response of the system.

The Fourier transform of the impulse response is the frequency response.

For a lossless time-invariant system, the frequency response has magnitude one at every frequency.

8. Schur Factorization as a Consolation for Non-Diagonalizability

The lecture recalls the main result from the previous lecture: Schur factorization.

Since not every matrix can be diagonalized, Schur factorization provides a weaker but always-available result.

The theorem says:

For any square matrix A , there exists a unitary matrix U such that

$$A = UTU^*,$$

where T is upper triangular.

The columns of U form an orthonormal basis.

Interpretation:

- We may not be able to find a basis in which A is diagonal.
- But we can always find an orthonormal basis in which A is upper triangular.

[Exam note] The proof starts with one eigenvalue and one eigenvector of A , then extends that eigenvector to an orthonormal basis.

This result is important because it will be used to prove properties of normal matrices.

9. Hermitian Matrices

A Hermitian matrix satisfies:

$$A^* = A.$$

The instructor contrasts this with unitary matrices:

- For a unitary matrix, $A^* = A^{-1}$.
- For a Hermitian matrix, $A^* = A$.

So the conjugate transpose does not directly give the inverse for a Hermitian matrix. Hermitian matrices are therefore not “simple” in the same solve-a-linear-system sense as unitary matrices.

However, Hermitian matrices are very important because they define real-valued quadratic functions.

For a complex vector x :

$$x^* x$$

is always real and nonnegative.

For a Hermitian matrix A , the scalar

$$x^* A x$$

is real-valued even when x is complex.

This expression defines a real-valued quadratic function of a complex vector.

[Exam note] Understanding Hermitian matrix properties is important because these properties partition the space of Hermitian matrices into useful subclasses.

Using the real-valued quadratic form property, one can show that every eigenvalue of a Hermitian matrix is real.

Comparison with unitary matrices:

- Unitary matrices have eigenvalues on the unit circle.
- Hermitian matrices have eigenvalues on the real line.

10. Matrices That Are Both Hermitian and Unitary

The identity matrix is both Hermitian and unitary:

$$I^* = I,$$

and

$$I^{-1} = I.$$

Its eigenvalues are all 1, which lie both on the real axis and on the unit circle.

The instructor notes that the identity is not the only example.

A matrix can be both Hermitian and unitary when:

- Its eigenvalues are constrained to 1 and -1 .
- It satisfies the relevant orthogonality structure between eigenspaces.

This will connect to the later normal-matrix viewpoint.

11. Real Symmetric and Skew-Hermitian Matrices

The real-valued version of a Hermitian matrix is a symmetric matrix.

If the matrix entries are real, conjugation does not change anything, so:

$$A^* = A$$

becomes

$$A^T = A.$$

A skew-symmetric real matrix satisfies:

$$A^T = -A.$$

The complex analogue is a skew-Hermitian matrix:

$$A^* = -A.$$

The instructor names this form as **skew-Hermitian**, not “Hermitian skew.”

Entrywise interpretation:

- An off-diagonal entry is the negative conjugate of the transposed entry.
- In the real skew-symmetric case, the $(1, 5)$ entry is the negative of the $(5, 1)$ entry.
- On the diagonal, the skew-Hermitian condition reduces to a scalar satisfying $\bar{a}_{ii} = -a_{ii}$, so diagonal entries are purely imaginary or zero.
- In the real skew-symmetric case, the diagonal entries must be zero.

The instructor asks students to check the eigenvalue property for skew-Hermitian matrices.

For Hermitian matrices:

$$A^* = A$$

implies eigenvalues are real.

For skew-Hermitian matrices:

$$A^* = -A$$

implies eigenvalues are purely imaginary.

Thus skew-Hermitian eigenvalues have the form:

$$i\lambda_r,$$

where λ_r is real.

The instructor gives a useful one-dimensional analogy:

- A 1×1 Hermitian matrix is just a real number.

- A 1×1 skew-Hermitian matrix is just a purely imaginary number.

So Hermitian and skew-Hermitian matrices can be viewed as matrix generalizations of real and imaginary numbers.

12. Normal Matrices: The Common Family

Hermitian matrices, unitary matrices, and skew-Hermitian matrices all belong to a larger family called normal matrices.

Normal matrices are square matrices.

The instructor describes a conceptual Venn diagram:

- Normal matrices contain unitary matrices.
- Normal matrices contain Hermitian matrices.
- Normal matrices contain skew-Hermitian matrices.
- Hermitian and unitary matrices can intersect.
- The diagram is only conceptual, not a literal low-dimensional geometric picture.
- There is also a large collection of square matrices outside the normal class.

The defining equation for a normal matrix is:

$$AA^* = A^*A.$$

This says that A commutes with its conjugate transpose.

The instructor remarks that matrix multiplication is not generally commutative, so this is a special property.

Warning:

- The equation $AA^* = A^*A$ does not immediately make the eigenspace orthogonality property obvious.
- A main goal of the lecture is to explain why this commutation condition is equivalent to unitary diagonalization and to orthogonal eigenspaces for distinct eigenvalues.

Why unitary matrices are normal:

$$U^*U = UU^* = I.$$

Why Hermitian matrices are normal:

If $A^* = A$, then

$$AA^* = A^2$$

and

$$A^*A = A^2.$$

Why skew-Hermitian matrices are normal:

If $A^* = -A$, then both products reduce consistently, so the order does not matter.

The instructor jokes that matrices outside the normal set could be called “abnormal,” but the mathematical name is normal.

Important interpretation:

- The word normal is connected to eigenspaces being normal, or orthogonal, to each other.

- For normal matrices, eigenspaces corresponding to distinct eigenvalues are orthogonal.

Eigenvalue locations distinguish the major subclasses:

- Hermitian matrices: eigenvalues on the real line.
- Unitary matrices: eigenvalues on the unit circle.
- Skew-Hermitian matrices: eigenvalues on the imaginary axis.

The list of structured matrices seen so far includes:

- Diagonal matrices.
- Triangular matrices.
- Hankel matrices from homework.
- Toeplitz matrices from homework.
- Orthogonal matrices.
- Orthogonal projection matrices.
- Hermitian matrices.
- Skew-Hermitian matrices.

[Exam note] Hankel and Toeplitz matrices were explicitly tied to homework.

13. Equivalent Definitions of Normal Matrices

The instructor gives several equivalent definitions. Each can be used as a definition because they imply each other.

The instructor notes that the choice of starting definition is partly pedagogical:

- He starts from $AA^* = A^*A$ because it connects directly to the earlier stories $A^* = A$ for Hermitian matrices and $U^* = U^{-1}$ for unitary matrices.
- He could instead have started from the energy equality or from unitary diagonalizability and then treated the other statements as properties.
- Showing how the definitions imply each other is described as an algebraically rich and useful exercise.

Definition A: Commutation with the Conjugate Transpose

$$AA^* = A^*A.$$

This is the starting definition in the lecture.

Definition B: Unitary Diagonalizability

A matrix is normal if and only if it is unitarily diagonalizable:

$$A = UDU^*,$$

where:

- U is unitary.
- D is diagonal.
- The diagonal entries of D are the eigenvalues of A .

[Exam note] Not every matrix is diagonalizable, but every normal matrix is diagonalizable. More specifically, every normal matrix is diagonalizable by a unitary matrix.

Definition C: Matrix Energy Equals Eigenvalue Energy

A matrix is normal if and only if:

$$\sum_{i,j} |a_{ij}|^2 = \sum_i |\lambda_i|^2.$$

The left side is the sum of squared magnitudes of all matrix entries.

The square root of the left side is the Frobenius norm:

$$\|A\|_F = \left(\sum_{i,j} |a_{ij}|^2 \right)^{1/2}.$$

Without the square root, the instructor calls it the energy of the matrix through its elements.

The right side is the energy of the eigenvalues.

Likely exam topic The equality between matrix-entry energy and eigenvalue energy is one of the equivalent ways to recognize normal matrices.

Definition D: Orthonormal Eigenbasis

A matrix is normal if and only if its eigenvectors can form an orthonormal basis for $\mathbb{C}^n$.

The instructor clarifies what is nontrivial here:

- Within a single eigenspace, one can always choose an orthonormal basis.
- The key issue is whether eigenspaces for distinct eigenvalues are orthogonal to each other.
- For normal matrices, distinct eigenspaces are orthogonal, so the eigenvectors from all eigenspaces can be assembled into one orthonormal basis.

This connects back to the earlier property for Hermitian and unitary matrices.

14. Proof Idea: Orthonormal Eigenbasis Implies Unitary Diagonalization

The instructor shows that Definition D implies Definition B.

Assume A has eigenvectors

$$u_1, u_2, \dots, u_n$$

that form an orthonormal basis.

Put these vectors into the columns of a matrix:

$$U = [u_1 \ u_2 \ \cdots \ u_n].$$

Because the columns are orthonormal, U is unitary.

Now multiply A by U :

$$AU = [Au_1 \ Au_2 \ \cdots \ Au_n].$$

Since each u_i is an eigenvector,

$$Au_i = \lambda_i u_i.$$

So:

$$AU = [\lambda_1 u_1 \ \lambda_2 u_2 \ \cdots \ \lambda_n u_n].$$

This can be written as:

$$AU = U\Lambda,$$

where

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}.$$

The instructor emphasizes the side of multiplication:

- Multiplying by a diagonal matrix from the right scales columns.
- Multiplying by a diagonal matrix from the left scales rows.

Here we need to scale the columns of U , so the diagonal matrix appears on the right:

$$AU = U\Lambda.$$

Multiplying on the right by U^* gives:

$$A = U\Lambda U^*.$$

Equivalently,

$$U^*AU = \Lambda.$$

Thus A is unitarily diagonalizable.

15. Trace as a Useful Algebraic Tool

The instructor introduces trace as a tool for proving the energy equivalence.

Definition:

$$\text{tr}(A)$$

is the sum of the diagonal entries of A .

[Exam note] Trace is a simple but very useful operator.

Trace is defined for square matrices. In this lecture it is applied to products such as A^*A and AA^* , which are square.

Properties of trace:

- It ignores off-diagonal entries.
- It is linear:

$$\text{tr}(\alpha A) = \alpha \text{tr}(A),$$

and

$$\text{tr}(A + B) = \text{tr}(A) + \text{tr}(B).$$

- It satisfies the cyclic two-factor property:

$$\text{tr}(AB) = \text{tr}(BA),$$

when the products are defined.

The instructor says trace is useful because it can replace messy index-based summations with matrix expressions.

16. Trace and the Energy of a Matrix

The instructor derives:

$$\text{tr}(A^*A) = \sum_{i,j} |a_{ij}|^2.$$

Let A be partitioned into columns:

$$A = [a_1 \ a_2 \ \cdots \ a_n].$$

Then:

$$A^*A$$

has diagonal entries:

$$a_1^*a_1, \ a_2^*a_2, \ \dots, \ a_n^*a_n.$$

Taking the trace gives:

$$\text{tr}(A^*A) = \sum_j a_j^*a_j = \sum_j \|a_j\|^2.$$

Each column norm expands into the sum of squared magnitudes of entries in that column:

$$\sum_j \|a_j\|^2 = \sum_{i,j} |a_{ij}|^2.$$

The same energy can also be obtained using $\text{tr}(AA^*)$.

This establishes that the entrywise energy of a matrix can be written compactly using trace.

Important status of this step:

- At this point the instructor has not yet proved the normal-matrix energy equivalence.
- He has proved the reusable identity that converts the entrywise sum into a trace expression.
- That identity is then plugged into the unitary diagonalization argument.

17. Proof Idea: Unitary Diagonalization Implies Energy Equality

Assume A is unitarily diagonalizable:

$$A = U\Lambda U^*.$$

Then:

$$A^* = U\Lambda^*U^*.$$

Using the trace expression for matrix energy:

$$\sum_{i,j} |a_{ij}|^2 = \text{tr}(A^*A).$$

Substitute the diagonalization:

$$\text{tr}(A^*A) = \text{tr}(U\Lambda^*U^*U\Lambda U^*).$$

Because $U^*U = I$,

$$\text{tr}(A^*A) = \text{tr}(U\Lambda^*\Lambda U^*).$$

Using the trace property $\text{tr}(AB) = \text{tr}(BA)$, move U^* cyclically to combine with U :

$$\text{tr}(U\Lambda^*\Lambda U^*) = \text{tr}(\Lambda^*\Lambda U^*U) = \text{tr}(\Lambda^*\Lambda).$$

Since Λ is diagonal, $\Lambda^*\Lambda$ is diagonal with entries:

$$|\lambda_1|^2, |\lambda_2|^2, \dots, |\lambda_n|^2.$$

Therefore:

$$\text{tr}(\Lambda^*\Lambda) = \sum_i |\lambda_i|^2.$$

So:

$$\sum_{i,j} |a_{ij}|^2 = \sum_i |\lambda_i|^2.$$

Instructor clarification:

- The result uses magnitude squares, not ordinary squares.
- This is because each eigenvalue is multiplied by its complex conjugate.

18. Proof Idea: Normality Implies Unitary Diagonalization

The instructor next discusses why:

$$AA^* = A^*A$$

implies unitary diagonalizability.

The instructor remarks that trying to prove this directly from the eigenvector definition would be possible but nontrivial. The previous lecture's Schur factorization makes the proof much cleaner.

The key tool is Schur factorization.

For any square matrix A , write:

$$A = UTU^*,$$

where U is unitary and T is upper triangular.

The goal is to show that if A is normal, then T is not merely triangular but actually diagonal.

Since T is upper triangular, it has the eigenvalues of A on its diagonal. Its possible nonzero entries above the diagonal are the only obstruction to diagonal form.

Substitute the Schur form into the normality equation:

$$AA^* = A^*A.$$

Using:

$$A = UTU^*,$$

and

$$A^* = UT^*U^*,$$

the normality equation becomes:

$$UTT^*U^* = UT^*TU^*.$$

Multiplying by U^* on the left and U on the right gives:

$$TT^* = T^*T.$$

Now use the fact that T is upper triangular.

Compare the $(1,1)$ entry of both sides.

For TT^* , the $(1,1)$ entry is:

$$|t_{11}|^2 + |t_{12}|^2 + \cdots + |t_{1n}|^2.$$

For T^*T , the $(1,1)$ entry is:

$$|t_{11}|^2,$$

because the first column of an upper triangular matrix has no entries below t_{11} .

Equality forces:

$$|t_{12}|^2 + \cdots + |t_{1n}|^2 = 0.$$

Since each term is nonnegative, all of them must be zero:

$$t_{12} = t_{13} = \cdots = t_{1n} = 0.$$

Then one repeats the same argument for the $(2, 2)$ entry, then the $(3, 3)$ entry, and so on.

At each step, the remaining off-diagonal entries in that row must be zero.

Therefore T is diagonal.

Thus:

$$A = UTU^*$$

is actually a unitary diagonalization.

Conclusion:

- Schur factorization says every square matrix is unitarily triangularizable.
- Normality forces the triangular factor to be diagonal.
- Therefore every normal matrix is unitarily diagonalizable.

19. Normal Matrices Classified by Eigenvalue Geometry

The instructor summarizes the big picture:

Normal matrices are exactly the unitarily diagonalizable matrices.

Within this family, subclasses are distinguished by where their eigenvalues lie.

Examples:

- If the eigenvalues lie on the unit circle, the matrix is unitary.
- If the eigenvalues lie on the real line, the matrix is Hermitian.
- If the eigenvalues lie on the imaginary axis, the matrix is skew-Hermitian.

This explains why these classes share properties:

- They all sit inside the normal matrix family.
- They are all unitarily diagonalizable.
- Their eigenspaces for distinct eigenvalues are orthogonal.

What changes from one subclass to another is the eigenvalue constraint.

20. Focusing on Hermitian Matrices by Eigenvalue Signs

The lecture then turns to a finer classification inside the Hermitian matrices.

Hermitian matrices have real eigenvalues. Since the eigenvalues are real, their signs become meaningful.

The instructor says the course will partition Hermitian matrices according to whether their eigenvalues are:

- Positive.
- Negative.
- Zero.
- Mixed positive and negative.

This partition is not arbitrary notation: it is motivated by the quadratic form x^*Ax and by the geometry of multivariable quadratic functions.

This classification is important because of the quadratic function:

$$x^*Ax.$$

[Exam note] The sign of the eigenvalues is critical for understanding the shape of this quadratic function.

21. Quadratic Forms and Surface Shapes

For a two-variable quadratic function, think of:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

and the function:

$$x^*Ax.$$

If A is Hermitian with positive eigenvalues, the quadratic surface looks like an upward-opening paraboloid.

Optimization interpretation:

- This is a convex function.
- It has a unique minimum.
- It is easier to optimize.
- The instructor calls this one of the “dream” cases for optimization.

If A has negative eigenvalues, the surface looks like a downward-opening paraboloid.

Optimization interpretation:

- This is a concave function.
- It is the negative analogue of the convex case.

If A has mixed eigenvalues, with some positive and some negative, the surface has a saddle structure.

Geometric interpretation:

- In one direction, it curves upward.
- In an orthogonal direction, it curves downward.
- The instructor compares it to a horse saddle.

Optimization and machine learning remark:

- Saddle structures are difficult in nonconvex optimization.
- The instructor mentions terminology such as rideable and non-rideable saddles.

- Mixed-sign eigenvalues lead to functions that are neither convex nor concave.

22. Probability Connection: Covariance and Correlation Matrices

The instructor asks where Hermitian matrices arise in probability.

The answer is:

- Correlation matrices.
- Covariance matrices.

These matrices are Hermitian.

The instructor adds that they always have nonnegative eigenvalues.

This property will be derived later.

This previews the connection between:

- Hermitian matrices.
- Positive semidefinite matrices.
- Covariance and correlation.

23. Zero Matrix and Intersections of Classes

The zero matrix is both Hermitian and skew-Hermitian:

$$0^* = 0$$

and

$$0^* = -0.$$

Its eigenvalues are all zero.

Zero lies both on the real axis and on the imaginary axis.

The instructor uses this to explain that the class diagrams should touch at the origin.

The zero matrix also belongs to both the nonnegative and nonpositive eigenvalue classes, because all its eigenvalues are zero.

24. Positive, Negative, Semidefinite, Definite, and Indefinite Matrices

The lecture introduces names for subclasses of Hermitian matrices based on eigenvalue signs.

The instructor remarks that the names may sound more intimidating than the definitions: they are simply labels for Hermitian matrices whose real eigenvalues satisfy sign constraints.

Positive Definite

A Hermitian matrix A is positive definite if all eigenvalues are strictly positive:

$$\lambda_i > 0 \quad \text{for all } i.$$

Example:

- The identity matrix is positive definite because all eigenvalues are 1.

Positive Semidefinite

A Hermitian matrix A is positive semidefinite if all eigenvalues are nonnegative:

$$\lambda_i \geq 0 \quad \text{for all } i.$$

Zero eigenvalues are allowed.

Negative Definite

A Hermitian matrix A is negative definite if all eigenvalues are strictly negative:

$$\lambda_i < 0 \quad \text{for all } i.$$

Negative Semidefinite

A Hermitian matrix A is negative semidefinite if all eigenvalues are nonpositive:

$$\lambda_i \leq 0 \quad \text{for all } i.$$

Zero eigenvalues are allowed.

Indefinite

A Hermitian matrix is indefinite if it has a mixture of positive and negative eigenvalues.

This corresponds to saddle-shaped quadratic forms.

Likely exam topic The instructor explicitly highlights positive definite, negative definite, positive semidefinite, negative semidefinite, and indefinite matrices.

25. Meaning of “Semidefinite” and Connection to Rank

A student asks about the word “definite.”

The instructor connects definiteness to rank and invertibility:

- If all eigenvalues are strictly nonzero, the matrix is invertible.
- If zero eigenvalues are present, the matrix is not invertible.

For positive or negative definite matrices:

- All eigenvalues are strictly positive or strictly negative.
- No eigenvalue is zero.
- The matrix is invertible.

For semidefinite matrices:

- Some eigenvalues may be zero.
- The matrix may be non-invertible.

The instructor says he is not certain of the exact historical origin of the word “definite,” but his interpretation is that when all eigenvalues are strictly nonzero, the related problem can be solved without ambiguity.

26. Alternative Definition of Positive Definite Matrices

The first definition of positive definiteness is spectral:

$$A \text{ is positive definite} \iff \lambda_i > 0 \text{ for all } i.$$

The instructor then gives an equivalent quadratic-form definition.

A Hermitian matrix A is positive definite if and only if:

$$x^*Ax > 0$$

for every nonzero vector x .

At the origin:

$$x = 0$$

gives:

$$x^*Ax = 0.$$

[Exam note] The expression x^*Ax is a real-valued quadratic function of x . For positive definite matrices, it is always positive except at the origin.

Why the spectral definition can lead to the quadratic-form definition:

- Since A is Hermitian, it is normal and therefore unitarily diagonalizable.
- Thus one can write $A = U\Lambda U^*$, where Λ is real diagonal.
- If A is positive definite, the diagonal entries of Λ are positive.
- The postponed proof will use this structure to show $x^*Ax > 0$ for every $x \neq 0$.

The instructor notes that the equivalence between the eigenvalue definition and the quadratic-form definition can be shown both ways:

- Positive eigenvalues imply $x^*Ax > 0$ for all nonzero x .
- The positivity of x^*Ax for all nonzero x implies all eigenvalues are positive.

He postpones the full proof to the next lecture because it would take time and he does not want to confuse the class at the end of the session.

27. Relationships to Preserve

The lecture repeatedly links concepts through the following relationships:

- Easy-to-solve matrix structures motivate factorizations.
- Failure of universal diagonalization motivates Schur factorization.
- Schur factorization gives universal unitary triangularization.
- Normality forces the Schur triangular factor to become diagonal.
- Therefore normal matrices are exactly unitarily diagonalizable matrices.
- Unitary, Hermitian, and skew-Hermitian matrices are all normal.
- These subclasses are distinguished by eigenvalue location:
 - Unit circle for unitary matrices.
 - Real line for Hermitian matrices.
 - Imaginary axis for skew-Hermitian matrices.
- Orthogonality of eigenspaces for distinct eigenvalues is shared by normal matrices.
- Hermitian matrices define real-valued quadratic forms.

- The signs of Hermitian eigenvalues determine the geometry of the quadratic form.
- Positive eigenvalues correspond to convex upward paraboloid behavior.
- Negative eigenvalues correspond to concave downward paraboloid behavior.
- Mixed signs correspond to saddle behavior.
- Covariance and correlation matrices are Hermitian and will later be shown to be positive semidefinite.

28. Instructor Remarks, Warnings, and Exam Cues

The instructor explicitly flags or emphasizes:

- Projection and orthogonality are important structured-matrix material.
- The orthogonality of eigenspaces for distinct eigenvalues, previously seen for unitary and Hermitian matrices, is generalized by normal matrices.
- Fourier transform and Fourier bases are important because they diagonalize circulant matrices.
- Convolution matrices combine multiple structures: lower triangular, Toeplitz, and unitary in the lossless causal LTI case.
- Schur factorization is important and was proved by starting with an eigenvector and extending it to an orthonormal basis.
- Hermitian matrices are important because they define real-valued quadratic functions.
- Normal matrices are important because they are unitarily diagonalizable.
- The Frobenius norm and the energy equality between entries and eigenvalues are important equivalent descriptions of normality.
- Trace is simple but powerful, especially for replacing index-heavy expressions.
- The commutation condition $AA^* = A^*A$ should not be treated as self-evidently geometric; the lecture's proof chain explains why it yields unitary diagonalization and orthogonal eigenspaces.
- The normal-matrix equivalences are not just facts to memorize; the instructor treats the implications between them as useful algebra practice.
- The sign of eigenvalues is critical for understanding the quadratic form x^*Ax .
- Positive definite, semidefinite, negative definite, negative semidefinite, and indefinite matrices are likely important classifications.
- The proof that positive eigenvalues are equivalent to $x^*Ax > 0$ for all nonzero x is explicitly postponed to the next lecture.

The instructor also gives informal remarks:

- The name “normal” can sound distracting, but it is connected to eigenspaces being normal to each other.
- The Venn diagram picture of matrix classes is conceptual, not an exact geometric picture.
- The labels positive semidefinite, negative semidefinite, definite, and indefinite are just the named sign-based subsets of Hermitian matrices.
- The term “definite” may be understood through the absence of zero eigenvalues and the resulting invertibility, though the instructor does not claim this as the historical origin.
- Saddle structures are a major difficulty in nonconvex optimization and machine learning.

Source and Coverage Note

These notes were created from the corrected Lecture 12 transcript only:

`C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture12_corrected.md`

Coverage follows the transcript chronologically, including the opening review, instructor examples and remarks, exam cues, proof ideas, normal matrix equivalences, trace/Frobenius arguments, and the closing transition into positive definiteness. The transcript ends with the instructor postponing the proof of the positive-definite quadratic-form equivalence to the next lecture.

Lecture 13 Notes

Opening Context: Hermitian Matrices and Quadratic Functions

The lecture begins by continuing the discussion of special subsets of the normal matrix family. The instructor remarks that some subsets, such as some materials or regions in the unit circle picture from the previous lecture, may not have special names. The main named family of interest here is the Hermitian family.

Hermitian matrices are important because they define real-valued quadratic forms. The lecture focuses on multivariate quadratic functions and explains how the eigenvalue structure of a Hermitian matrix determines the shape of those functions.

Within the Hermitian matrices, the instructor divides matrices according to the signs of their eigenvalues:

- all eigenvalues positive: positive definite;
- all eigenvalues nonnegative: positive semidefinite;
- all eigenvalues negative: negative definite;
- all eigenvalues nonpositive: negative semidefinite;
- a mixture of positive and negative eigenvalues: indefinite.

The remaining case, where a Hermitian matrix has both positive and negative eigenvalues, is called indefinite. These sign classes are not just algebraic labels; they determine the geometry and optimization behavior of the associated quadratic function.

Hermitian Matrices and the Quadratic Form x^*Ax

Let A be Hermitian. Then

$$A = A^*$$

and the quadratic form

$$x^*Ax$$

is always real-valued, even when x is a complex vector.

Hermitian matrices are also normal matrices. Since normal matrices are unitarily diagonalizable, a Hermitian matrix can be written as

$$A = U\Lambda U^*$$

where:

- U is unitary;
- the columns of U are unit-norm eigenvectors of A ;
- Λ is diagonal;
- the diagonal entries λ_i are the eigenvalues of A ;
- for Hermitian matrices, all λ_i are real.

The instructor connects this to earlier results:

- Hermitian matrices are a special case of normal matrices.
- Normal matrices are unitarily diagonalizable.
- Distinct eigenvalues of Hermitian matrices have orthogonal eigenspaces.
- Because of this, one can choose an orthonormal eigenbasis and build U from those eigenvectors.

Coordinate Interpretation of U^*x

The lecture emphasizes the geometric meaning of

$$U^*x.$$

If

$$U = [u_1 \ u_2 \ \cdots \ u_n],$$

then

$$U^*x = \begin{bmatrix} u_1^*x \\ u_2^*x \\ \vdots \\ u_n^*x \end{bmatrix}.$$

Each entry u_i^*x is an inner product of x with a unit eigenvector u_i . Therefore U^*x gives the coordinates of x with respect to the orthonormal eigenbasis of A .

The instructor denotes

$$y = U^*x.$$

Then

$$x^*Ax = x^*U\Lambda U^*x = y^*\Lambda y.$$

Since U is unitary, it preserves norm:

$$\|U^*x\| = \|x\|.$$

Thus, if $x \neq 0$, then $y = U^*x \neq 0$. Equivalently, U^* has trivial null space because it is full rank.

Positive Definite Matrices

A Hermitian matrix A is positive definite if all its eigenvalues are strictly positive:

$$\lambda_i > 0 \quad \text{for all } i.$$

The identity matrix is an example of a positive definite matrix.

The lecture states the equivalent quadratic-form definition:

$$A \succ 0 \quad \Longleftrightarrow \quad x^*Ax > 0 \text{ for every } x \neq 0.$$

The condition $x \neq 0$ is essential. At the origin,

$$0^*A0 = 0,$$

so strict positivity cannot hold for every vector including 0.

Proof Idea: Eigenvalues Positive Implies Quadratic Form Positive

Assume A is Hermitian and all eigenvalues are positive. Use the unitary diagonalization

$$A = U\Lambda U^*.$$

Let

$$y = U^*x.$$

Then

$$x^*Ax = y^*\Lambda y.$$

Because Λ is diagonal,

$$y^*\Lambda y = \sum_{i=1}^n \lambda_i |y_i|^2.$$

Here:

- every $\lambda_i > 0$;
- every $|y_i|^2 \geq 0$;
- if $x \neq 0$, then $y \neq 0$, so at least one $|y_i|^2 > 0$.

Therefore the sum is strictly positive:

$$x^*Ax > 0 \quad \text{for every } x \neq 0.$$

The instructor notes that this proves one implication: positive eigenvalues imply positivity of the quadratic form. The reverse implication can be checked by evaluating the quadratic form along eigenvector directions.

Eigenvector Direction Check

Let u_i be the eigenvector corresponding to λ_i . In matrix form,

$$u_i = Ue_i,$$

where e_i is the standard basis vector with a 1 in the i -th position and zeros elsewhere.

Multiplying U by e_i picks out the i -th column of U , which is u_i .

If $x = u_i = Ue_i$, then

$$y = U^*x = U^*Ue_i = e_i.$$

So

$$x^*Ax = e_i^*\Lambda e_i = \lambda_i.$$

Thus, along an eigenvector direction, the value of the quadratic form is exactly the corresponding eigenvalue. This explains why the eigenvalue signs directly determine the sign behavior of x^*Ax .

The instructor remarks that there was some notation confusion during this derivation, but the key idea is: choosing x in an eigenvector direction isolates the corresponding eigenvalue.

Positive Semidefinite Matrices

A Hermitian matrix A is positive semidefinite if all eigenvalues are nonnegative:

$$\lambda_i \geq 0 \quad \text{for all } i.$$

Equivalently,

$$x^* Ax \geq 0 \quad \text{for every } x.$$

The instructor warns that this is not the same as positive definite. The slide or expression with eigenvalue condition $\lambda_i \geq 0$ corresponds to positive semidefinite, not positive definite.

The difference is that positive semidefinite matrices may have zero eigenvalues. If A has a zero eigenvalue, then there can be nonzero vectors in the null space of A . For such nonzero vectors,

$$x^* Ax = 0.$$

Therefore, in the positive semidefinite definition, one does not require $x \neq 0$ and strict positivity. The statement is simply nonnegativity for all x .

Relationships:

- Positive definite matrices are a subset of positive semidefinite matrices.
- Positive definite matrices are exactly the full-rank, invertible positive semidefinite matrices.
- A zero eigenvalue means rank deficiency.
- A zero eigenvalue corresponds to a nontrivial null space.
- Positive semidefinite matrices can be non-invertible.

Negative Definite and Negative Semidefinite Matrices

The negative counterparts are defined by reversing the signs.

A Hermitian matrix is negative definite if all eigenvalues are strictly negative:

$$\lambda_i < 0 \quad \text{for all } i.$$

A Hermitian matrix is negative semidefinite if all eigenvalues are nonpositive:

$$\lambda_i \leq 0 \quad \text{for all } i.$$

Here, nonpositive means each eigenvalue is either negative or zero.

Relationships:

- Negative definite matrices are a subset of negative semidefinite matrices.
- Negative definite matrices are invertible negative semidefinite matrices.
- Negative semidefinite matrices may have zero eigenvalues and therefore may be non-invertible.

Indefinite Hermitian Matrices

If a Hermitian matrix has a mixture of positive and negative eigenvalues, it is indefinite.

An example is a Hermitian matrix with eigenvalues 1 and -1 , such as

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

This matrix is neither positive semidefinite nor negative semidefinite, because it has both signs among its eigenvalues.

The instructor asks why these subclasses matter. The answer is that eigenvalue signs determine the geometry of multivariable quadratic functions.

Single-Variable Quadratic Functions as Motivation

The lecture then reviews a single-variable quadratic function:

$$f(x) = ax^2 + bx + c.$$

The coefficient a , which multiplies x^2 , is the critical parameter for the general shape of the function:

- $a > 0$: parabola opens upward;
- $a < 0$: parabola opens downward;
- $a = 0$: the function becomes a line $bx + c$.

The coefficients b and c affect shifts:

- b changes the horizontal location of the optimum;
- c shifts the function vertically;
- together they affect where the minimum or maximum occurs and what value it takes.

But the main parabola-up or parabola-down shape is determined by a .

Convex and Concave Functions

For $a > 0$, the quadratic function is convex. It has a global minimum.

For $a < 0$, the quadratic function is concave. It has a global maximum.

The instructor gives the geometric chord definitions:

- A function is convex if, when any two points on the function are connected by a line segment, that line segment lies above the function.
- A function is concave if the line segment between any two points lies below the function.

For $a = 0$, the function is affine:

$$f(x) = bx + c.$$

A line is both convex and concave, because the chord between any two points on the line lies exactly on the function.

Convex and concave quadratics are important in optimization because they have simple global behavior. A convex quadratic has a single global minimum. A concave quadratic has a single global maximum.

Quadratic Approximation and Taylor Series

The instructor relates quadratic functions to optimization of general nonlinear functions. A general nonlinear function can have many peaks, valleys, and local structures. To optimize it, one often builds a local quadratic approximation around a point.

Taylor series expansion is one way to obtain such a local approximation. Suppose the original function is $g(x)$ and we approximate it around x_0 . The Taylor expansion uses derivative information at x_0 :

- the function value at x_0 ;
- the first derivative at x_0 , which gives the local slope;
- the second derivative at x_0 , which gives curvature;
- higher derivatives if included.

The instructor assumes, for the Taylor series discussion, that g is infinitely differentiable.

For a single-variable function, the local expansion around x_0 has the form

$$g(x) = g(x_0) + g'(x_0)(x - x_0) + \frac{1}{2}g''(x_0)(x - x_0)^2 + \cdots$$

Keeping terms through the second derivative gives a quadratic approximation. The approximation is built entirely from local derivative information at x_0 , so its reliability is tied to how close x stays to x_0 .

The first derivative gives the slope of the tangent line. At a local minimum or local maximum, the tangent slope is zero.

The second derivative gives curvature:

- positive second derivative: locally parabola-up behavior, convex approximation;
- negative second derivative: locally parabola-down behavior, concave approximation.

Taylor approximations are local and are most accurate for small deviations around x_0 . The instructor notes that one can also fit a quadratic approximation over a chosen region if one wants a better approximation on that region rather than only an infinitesimal neighborhood.

Warning About Zero Slope and Saddle-Like Points

A zero first derivative does not automatically mean a local minimum or maximum. The instructor gives the idea of a point with zero slope but curvature behavior changing around it. Such a point is saddle-like rather than a true local optimum.

In the instructor's sketch, the slope at the point is zero, but one side has negative curvature while the other side has positive curvature. The warning is that the first derivative test alone is not enough: zero slope can occur at points that are neither minima nor maxima.

The lecture mentions that, in multivariable settings, saddle points become important. In optimization, there are terms such as attraction to saddles, and one often wants to avoid converging to saddle points when searching for maxima or minima.

Two-Variable Real Quadratic Functions

The lecture then moves to real-valued quadratic functions of two real variables:

$$f(x_1, x_2) = ax_1^2 + bx_1x_2 + cx_2^2 + ex_1 + fx_2 + d.$$

The terms are:

- ax_1^2 : square term in x_1 ;
- cx_2^2 : square term in x_2 ;

- bx_1x_2 : cross term or product term;
- $ex_1 + fx_2$: linear terms;
- d : constant term.

The constant term d shifts the graph up or down. It does not change the shape.

The linear terms ex_1 and fx_2 shift the function in the domain. They affect where the optimum is located, but they do not determine the basic quadratic shape.

To focus on the shape, the instructor first removes the linear terms and studies the quadratic part. The instructor also first studies the case where the cross term coefficient b is zero.

Two-Variable Quadratic With No Cross Term

Assume $b = 0$ and ignore the linear terms. Then

$$f(x_1, x_2) = ax_1^2 + cx_2^2 + d.$$

This can be written in matrix notation as

$$f(x) = x^T Ax + d,$$

where

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad A = \begin{bmatrix} a & 0 \\ 0 & c \end{bmatrix}.$$

This matrix is real symmetric, hence Hermitian. Its eigenvalues are simply a and c .

Case 1: $a > 0, c > 0$

If both a and c are positive, then A is positive definite.

The corresponding quadratic surface is parabola-up in every direction. Any slice through the surface in any direction looks like an upward-opening parabola. The function is convex and has bowl-like geometry.

The instructor notes that if one walks in any direction on the surface, one sees parabola-up behavior in that direction.

Case 2: $a < 0, c < 0$

If both a and c are negative, then A is negative definite.

The quadratic surface is concave, shaped like an upside-down bowl or dome. It has a single optimal point, now a maximum rather than a minimum.

Case 3: One Eigenvalue Positive and One Negative

If one of a, c is positive and the other is negative, then A is indefinite.

In one coordinate direction, the function behaves like a parabola up. In the other coordinate direction, it behaves like a parabola down. This creates a saddle surface.

Such a function is neither convex nor concave.

The instructor emphasizes that this saddle structure is a new richness that appears in multivariable quadratic functions and does not occur in the same way for single-variable quadratics.

Instructor Remark on Generated Plots

The instructor remarks that some surface plots shown in lecture were generated by asking an early GPT model, around GPT-3 or GPT-3.5 and around late 2022, to write Python code for a convex two-variable quadratic plot. This remark was not mathematically central, but it explained how the plotted surfaces were produced.

Two-Variable Quadratic With a Cross Term

Now consider $b \neq 0$:

$$f(x_1, x_2) = ax_1^2 + bx_1x_2 + cx_2^2.$$

The cross term bx_1x_2 adds coupling between the two coordinate directions.

The instructor asks what b does. A student suggests that it should rotate the oval or contour. The instructor agrees: the cross term corresponds to a rotation of the principal axes in the domain.

However, b also affects the eigenvalues. When $b = 0$, the eigenvalues are simply a and c . When $b \neq 0$, the eigenvalues depend on all three coefficients a, b, c , not just a and c .

The instructor also warns that a plot can hide this distinction if the curvatures are too close to each other. A larger eigenvalue means the surface rises faster in that principal direction; a smaller eigenvalue means a flatter parabola-up direction. The cross term changes the principal directions and can change those curvature values.

Main effects of the cross term:

- it rotates the principal axes;
- the principal axes are no longer aligned with the coordinate axes;
- the curvatures are determined by the eigenvalues of the associated symmetric matrix;
- those eigenvalues depend on a, b, c .

Matrix Representation of the Cross Term

The quadratic form can be represented by a symmetric matrix:

$$A = \begin{bmatrix} a & b/2 \\ b/2 & c \end{bmatrix}.$$

Then

$$x^T Ax = ax_1^2 + \frac{b}{2}x_1x_2 + \frac{b}{2}x_2x_1 + cx_2^2 = ax_1^2 + bx_1x_2 + cx_2^2.$$

The two off-diagonal terms each contribute half of the cross term.

The instructor points out that this symmetric choice is not the only possible matrix representation. For the same quadratic expression, the off-diagonal entries only need to sum to b . For example, one could choose one off-diagonal entry to be b and the other to be 0. There are infinitely many matrices that give the same quadratic form.

Equivalently, the allowable off-diagonal choices form a line of possibilities: if the $(1, 2)$ entry is r and the $(2, 1)$ entry is s , then any pair with $r + s = b$ produces the same bx_1x_2 term. The symmetric choice $r = s = b/2$ is selected for analysis, not because the quadratic expression forces it uniquely.

But choosing the symmetric matrix is helpful because:

- it is Hermitian in the real case;

- its eigenvalues are real;
- it is diagonalizable by an orthogonal matrix;
- the eigenvectors give meaningful principal directions;
- the analysis becomes easier.

Symmetric and Skew-Symmetric Decomposition

The instructor gives an important decomposition for any real square matrix A :

$$A = \frac{A + A^T}{2} + \frac{A - A^T}{2}.$$

The first part,

$$\frac{A + A^T}{2},$$

is symmetric.

The second part,

$$\frac{A - A^T}{2},$$

is skew-symmetric, because its transpose equals its negative.

This decomposition is algebraically simple because adding the two parts cancels the A^T terms and leaves A .

Why the Skew-Symmetric Part Does Not Matter for $x^T Ax$

Let

$$K = \frac{A - A^T}{2}.$$

Then $K^T = -K$, so K is skew-symmetric.

For the quadratic form,

$$x^T K x$$

is a scalar. A scalar equals its own transpose:

$$x^T K x = (x^T K x)^T.$$

But

$$(x^T K x)^T = x^T K^T x = x^T (-K) x = -x^T K x.$$

Therefore

$$x^T K x = -x^T K x,$$

so

$$x^T K x = 0.$$

Thus the skew-symmetric component contributes nothing to the quadratic form. Only the symmetric component matters:

$$x^T A x = x^T \left(\frac{A + A^T}{2} \right) x.$$

This justifies always choosing the symmetric matrix representation for real quadratic forms.

Orthogonal Diagonalization of the Symmetric Matrix

For a real symmetric matrix A , there exists a real orthogonal matrix Q such that

$$A = Q \Lambda Q^T.$$

Here:

- Q is real orthogonal;
- $Q^T Q = I$ and $Q Q^T = I$;
- the columns of Q are orthonormal eigenvectors of A ;
- Λ is the diagonal matrix of eigenvalues.

This is the real version of unitary diagonalization. In the real symmetric case, the unitary matrix becomes a real orthogonal matrix.

Change of Coordinates to the Eigenbasis

Substitute

$$A = Q \Lambda Q^T$$

into the quadratic form:

$$x^T A x = x^T Q \Lambda Q^T x.$$

Define

$$z = Q^T x.$$

Then

$$x^T A x = z^T \Lambda z.$$

As with $U^* x$ earlier, $Q^T x$ gives coordinates of x in the orthonormal eigenbasis.

If

$$Q = [q_1 \ q_2 \ \cdots \ q_n],$$

then

$$Q^T x = \begin{bmatrix} q_1^T x \\ q_2^T x \\ \vdots \\ q_n^T x \end{bmatrix}.$$

Each entry $q_i^T x$ is the inner product of x with a unit eigenvector q_i , so it is the projection coefficient of x along q_i . These entries are the coordinates of x with respect to the normalized eigenbasis.

In the new coordinates,

$$z^T \Lambda z = \lambda_1 z_1^2 + \lambda_2 z_2^2 + \cdots + \lambda_n z_n^2.$$

There are no cross terms. The change of basis to eigenvector coordinates eliminates cross terms.

Geometric meaning:

- the original x_1, x_2 axes are the natural coordinate axes in which the quadratic was first written;
- the q_1, q_2 axes are eigenvector directions;
- in the eigenvector coordinates, the curvature in direction q_i is λ_i ;
- if $\lambda_i > 0$, the surface curves upward in direction q_i ;
- if $\lambda_i < 0$, the surface curves downward in direction q_i .

The instructor notes that in a drawing, the eigenvectors may not visually look orthogonal or unit length because of the perspective of the picture, but mathematically they are assumed to be unit vectors at 90° .

Example: Matrix With Rotated Principal Directions

The lecture uses the matrix

$$A = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}.$$

The off-diagonal entries are 4, so in the convention

$$A = \begin{bmatrix} a & b/2 \\ b/2 & c \end{bmatrix},$$

this corresponds to a mixed-term coefficient $b = 8$ in $x^T A x$. The instructor describes the off-diagonal entries as the cross-term entries.

Eigenvector $[1, 1]^T$

Compute:

$$A \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5+4 \\ 4+5 \end{bmatrix} = \begin{bmatrix} 9 \\ 9 \end{bmatrix} = 9 \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Therefore

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

is an eigenvector with eigenvalue 9.

Eigenvector $[1, -1]^T$

Compute:

$$A \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 - 4 \\ 4 - 5 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Therefore

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

is an eigenvector with eigenvalue 1.

The normalized versions of these vectors form the orthonormal eigenbasis.

Curvature Interpretation

In the $[1, 1]^T$ direction, the curvature is 9.

In the $[1, -1]^T$ direction, the curvature is 1.

So the function rises much faster in the $[1, 1]^T$ direction than in the $[1, -1]^T$ direction. The principal curvature directions are rotated relative to the original coordinate axes.

Level Sets and Contours of Quadratic Functions

The lecture then introduces level sets.

For a function f , the level set at value α is

$$\{x : f(x) = \alpha\}.$$

It is the set of domain points that produce the same function value. The instructor also calls these:

- contours;
- preimages of a function value;
- regions in the domain where the function achieves a constant output.

Geometrically, for a two-variable function whose graph is a surface in three dimensions, choosing a level value α is like slicing the surface with a plane parallel to the domain. The intersection corresponds to the points whose function value is α . Looking down at the domain gives the contour or level set.

For positive definite or negative definite quadratic functions, level sets are ellipses in two dimensions and ellipsoids or hyperellipses in higher dimensions, when the chosen level value is compatible with the sign of the function. For a positive definite quadratic form, negative levels are empty and the zero level is only the origin. For a negative definite quadratic form, the analogous nonempty ellipses occur for negative levels, with the zero level again at the origin when no shifts are included.

Level Set Example: Equal Eigenvalues

Consider

$$A = \begin{bmatrix} 9 & 0 \\ 0 & 9 \end{bmatrix}.$$

Then

$$f(x_1, x_2) = 9x_1^2 + 9x_2^2.$$

The level set equation is

$$9x_1^2 + 9x_2^2 = \alpha.$$

If $\alpha < 0$, the level set is empty, because $f(x_1, x_2) \geq 0$ for every point.

If $\alpha = 0$, the only point in the level set is the origin:

$$(x_1, x_2) = (0, 0).$$

If $\alpha > 0$, divide by 9:

$$x_1^2 + x_2^2 = \frac{\alpha}{9}.$$

This is a circle centered at the origin with radius

$$\frac{\sqrt{\alpha}}{3}.$$

The intersections with the coordinate axes occur at

$$x_1 = \pm \frac{\sqrt{\alpha}}{3}$$

when $x_2 = 0$, and similarly for x_2 when $x_1 = 0$.

Because the eigenvalues are equal, the curvature is the same in both coordinate directions and the level sets are circles.

Level Set Example: Different Eigenvalues

Now consider a diagonal positive definite matrix with eigenvalues 9 and 1:

$$f(x_1, x_2) = 9x_1^2 + x_2^2.$$

The level set equation is

$$9x_1^2 + x_2^2 = \alpha.$$

For $x_2 = 0$,

$$9x_1^2 = \alpha,$$

so

$$x_1 = \pm \frac{\sqrt{\alpha}}{3}.$$

For $x_1 = 0$,

$$x_2^2 = \alpha,$$

so

$$x_2 = \pm\sqrt{\alpha}.$$

The level set is not a circle. It is an ellipse. The axis in the x_2 direction is longer because the coefficient/eigenvalue in that direction is smaller. Smaller curvature allows the level set to extend farther before reaching the same function value.

Principal Semi-Axes of an Ellipse

The instructor defines principal semi-axes using distance from the center of the ellipse.

The first principal semi-axis is the direction from the center in which the ellipse reaches the largest distance, or largest radius.

The second principal semi-axis is the next orthogonal direction in which the ellipse reaches the largest distance after the first direction is fixed.

The orthogonality requirement matters. Without requiring the next direction to be orthogonal, many nearby directions could also have large distances from the center.

For a diagonal positive definite matrix, the ellipse's principal semi-axes align with the coordinate axes.

For a non-diagonal matrix, the ellipse is rotated. Its principal semi-axes no longer align with the original coordinate axes. Instead, they align with the eigenvectors of the symmetric matrix defining the quadratic form.

The instructor says this will be derived and examined more fully in the next lecture.

Relationships Between the Main Concepts

Hermitian matrices connect algebra to geometry through quadratic forms:

$$A = A^* \quad \Rightarrow \quad x^*Ax \in \mathbb{R}.$$

Unitary diagonalization connects eigenvalues to the quadratic form:

$$A = U\Lambda U^*, \quad y = U^*x, \quad x^*Ax = y^*\Lambda y.$$

The coordinate vector $y = U^*x$ is the representation of x in the orthonormal eigenbasis.

Eigenvalue signs determine quadratic-form signs:

- all $\lambda_i > 0$: $x^*Ax > 0$ for $x \neq 0$;
- all $\lambda_i \geq 0$: $x^*Ax \geq 0$ for all x ;
- all $\lambda_i < 0$: $x^*Ax < 0$ for $x \neq 0$;
- all $\lambda_i \leq 0$: $x^*Ax \leq 0$ for all x ;
- mixed signs: the quadratic form can be positive in some directions and negative in others.

Eigenvalue signs determine shape:

- positive definite: convex bowl;
- negative definite: concave dome;
- indefinite: saddle;
- semidefinite: flat directions may occur because of zero eigenvalues.

Eigenvectors determine directions:

- in the original coordinates, cross terms may appear;
- in the eigenvector coordinates, cross terms disappear;
- the eigenvectors give the principal directions;
- the eigenvalues give curvature along those directions.

Level sets reflect the same structure:

- equal positive eigenvalues produce circular level sets;
- unequal positive eigenvalues produce ellipses;
- non-diagonal symmetric matrices produce rotated ellipses;
- eigenvectors align with principal semi-axes;
- eigenvalues determine how stretched the ellipse is in each principal direction.

Instructor Warnings and Study Remarks

The instructor specifically warns about the distinction between positive definite and positive semidefinite:

- positive definite requires $\lambda_i > 0$ and $x^*Ax > 0$ for $x \neq 0$;
- positive semidefinite allows $\lambda_i = 0$ and only requires $x^*Ax \geq 0$.

The instructor also emphasizes that zero eigenvalues mean rank deficiency and non-invertibility.

Another important warning is that a quadratic form can be represented by many matrices if the matrix is not required to be symmetric. For analysis, use the symmetric or Hermitian representative because the skew-symmetric part contributes zero to $x^T Ax$.

The instructor asks students to review positive definite matrices, semidefinite matrices, and quadratic functions before the next lecture so the class can continue with the derivation and geometry of ellipses and ellipsoids.

Source and Coverage Note

These notes were created only from C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture13_correc
They cover the full lecture 13 transcript in chronological order, including definitions, proof ideas, examples, instructor remarks, warnings, and concept relationships. No other lecture transcript was processed, and no audit or exam files were created.

Lecture 14 Notes

Normal Matrices and Eigenvalue-Based Subclasses

The lecture opened by connecting several matrix classes through normal matrices.

- Normal matrices are diagonalizable by unitary matrices.
- A key reason this diagonalization is useful is that eigenspaces of a normal matrix are orthogonal to each other.
- Because of that orthogonal eigenspace structure, one can organize subclasses of normal matrices according to where their eigenvalues lie.

For a general normal matrix, eigenvalues may be arbitrary complex numbers. Important subclasses arise by restricting the eigenvalues:

- Eigenvalues on the unit circle correspond to unitary matrices.
- Eigenvalues on the real axis correspond to Hermitian matrices.
- Eigenvalues on the imaginary axis correspond to the skew case discussed in lecture, mathematically the skew-Hermitian type.

The lecture then focused on the Hermitian case. Hermitian matrices have real eigenvalues, so it becomes meaningful to classify them by the signs of their eigenvalues.

Hermitian Sign Classes

For Hermitian matrices, the important sign-based classes are:

- Positive definite matrices.
- Negative definite matrices.
- Positive semidefinite matrices.
- Negative semidefinite matrices.
- Indefinite matrices.

The difference among these is determined by the sign pattern of the real eigenvalues.

Positive Definite

There are two equivalent ways emphasized for defining a positive definite matrix.

1. Eigenvalue definition:
 - A is Hermitian.
 - All eigenvalues of A are positive.
2. Quadratic-form definition:
 - The quadratic form defined by A is always positive away from the origin:

$$x^*Ax > 0 \quad \text{for every } x \neq 0.$$

The second definition is often easier to use in proofs.

Positive Semidefinite

The semidefinite case relaxes strict positivity:

- Eigenvalues may be positive or zero.
- No eigenvalue is negative.
- The quadratic form satisfies

$$x^*Ax \geq 0.$$

Zero eigenvalues correspond to directions where the quadratic form can be flat.

Negative Semidefinite and Negative Definite

For negative semidefinite matrices:

- Eigenvalues are less than or equal to zero.
- No eigenvalue is positive.
- The quadratic form is always nonpositive:

$$x^*Ax \leq 0.$$

For negative definite matrices, the “semi” is removed:

- Eigenvalues are strictly negative.
- For every nonzero x ,

$$x^*Ax < 0.$$

The instructor described this as removing the equality/zero allowance from the semidefinite case.

Indefinite

If a Hermitian matrix has a mixture of positive and negative eigenvalues, then the quadratic form is indefinite.

- In some directions the form behaves like a parabola opening up.
- In other directions it behaves like a parabola opening down.
- Geometrically this leads to saddle-type behavior.

Motivation: Quadratic Functions

The motivation for organizing Hermitian matrices by eigenvalue signs is the behavior of quadratic functions.

Single-Variable Quadratics

For a single-variable quadratic, the coefficient of the x^2 term determines the basic shape:

- Positive coefficient: parabola opening upward.
- Negative coefficient: parabola opening downward.
- Zero coefficient: the quadratic part disappears, leaving a linear structure.

Multivariate Quadratics

In the multivariate case, a quadratic function can be written in shifted form as

$$f(x) = (x - x_0)^*A(x - x_0) + d.$$

Here:

- x_0 shifts the center/location of the quadratic function.
- d shifts the function vertically.
- The matrix A determines the shape.

The sign class of A controls the geometry:

- If A is positive definite Hermitian, the function is convex and parabola-up.
- If A is negative definite Hermitian, the function is concave and parabola-down.
- If A is indefinite, the function has saddle structure.

The instructor remarked that an AI-generated example of an indefinite quadratic function naturally produced a function with mixed eigenvalue signs, even without explicitly asking for eigenvalue conditions.

Cross Terms, Rotation, and Principal Directions

The lecture contrasted diagonal and non-diagonal quadratic forms.

When the matrix is diagonal, the main directions of the quadratic function align with the original coordinate axes. For example:

- In a saddle picture, the parabola-down direction might align with the x -axis.
- The parabola-up direction might align with the y -axis.

This is the case where the cross term is zero.

When a cross term such as bx_1x_2 is present with $b \neq 0$, it effectively rotates the quadratic function. The principal axes no longer align with the standard coordinate axes.

The remedy is a change of coordinates using eigenvectors:

- The eigenvectors of A determine the principal directions.
- In the new eigenvector coordinates, the quadratic form has no cross terms.
- This is the geometric meaning of diagonalizing A .

Main Example: $A = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$

The lecture returned to the matrix

$$A = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}.$$

This matrix has cross terms in the standard-coordinate quadratic form and is positive definite.

The eigenvalues are:

$$\lambda_1 = 9, \quad \lambda_2 = 1.$$

Interpretation:

- In the $(1, 1)$ direction, the eigenvalue is 9, so the quadratic is highly curved.
- In the $(1, -1)$ or perpendicular direction, the eigenvalue is 1, so the function is flatter.

Level Curves of Quadratic Functions

The previous lecture had introduced level curves of quadratic functions. Lecture 14 continued that discussion.

For a quadratic form, an α -level set is the set of domain points satisfying

$$x^*Ax = \alpha.$$

In the positive definite or negative definite case, these level sets are ellipse-like.

- For a convex quadratic, the level curves are ellipses around the center.
- For a concave quadratic, one can similarly look at intersections with horizontal levels and obtain corresponding level sets.
- For indefinite quadratic forms, the level sets are generally hyperbolic rather than elliptic. The instructor said that the hyperbolic indefinite case would not be covered further there.

Circular and Elliptic Cases

If

$$A = \begin{bmatrix} 9 & 0 \\ 0 & 9 \end{bmatrix},$$

then

$$9x_1^2 + 9x_2^2 = \alpha$$

is a circle.

If one eigenvalue changes, for example to 1, the level set becomes an ellipse.

For diagonal matrices, the principal semi-axes align with the coordinate axes.

For non-diagonal matrices, the ellipse rotates. The level sets rotate because the quadratic function has rotated.

The principal semi-axes are the directions of maximum deviation from the origin or center of the ellipse. They are determined by the eigenvectors of A .

Eigenvalue Decomposition of the 5, 4; 4, 5 Example

The formal procedure starts with the characteristic equation:

$$\det(\lambda I - A) = 0.$$

For

$$A = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix},$$

we have

$$\lambda I - A = \begin{bmatrix} \lambda - 5 & -4 \\ -4 & \lambda - 5 \end{bmatrix}.$$

The determinant is

$$(\lambda - 5)^2 - 16.$$

Thus

$$|\lambda - 5| = 4,$$

which gives

$$\lambda = 9 \quad \text{or} \quad \lambda = 1.$$

Eigenspace for $\lambda = 9$

To find the eigenspace, plug $\lambda = 9$ into $\lambda I - A$. This produces a rank-deficient matrix. One row is the negative of the other, so the null space is nontrivial.

Vectors with equal entries are annihilated:

$$\begin{bmatrix} \beta \\ \beta \end{bmatrix}.$$

So the eigenspace for $\lambda = 9$ is in the direction of

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

A unit eigenvector is

$$q_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Because the matrix is normal/Hermitian, eigenspaces corresponding to distinct eigenvalues are orthogonal. Therefore, a unit eigenvector for the other eigenspace can be chosen as

$$q_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

The instructor noted that the negative of a unit eigenvector could also have been chosen.

Putting the eigenvectors into a matrix Q , with q_1 and q_2 as columns, gives an orthonormal eigenvector matrix. The decomposition is

$$A = Q\Lambda Q^T$$

in the real case, or $A = Q\Lambda Q^*$ in the complex notation.

The eigenvectors form an orthonormal basis:

- They are orthogonal.
- They each have unit norm.

Change of Coordinates and Cross-Term Removal

The eigenvector matrix gives a change of basis.

- x contains coordinates with respect to the standard basis.
- z contains coordinates with respect to the eigenvector basis.

In standard coordinates, the quadratic form is

$$x^T A x = 5x_1^2 + 5x_2^2 + 8x_1x_2.$$

The $8x_1x_2$ term is the cross term.

In eigenvector coordinates, the quadratic form becomes

$$z^T \Lambda z = 9z_1^2 + z_2^2.$$

There is no cross term. The coordinate axes have been rotated intelligently so that they align with the eigenvectors.

Geometrically:

- q_1 points at a 45-degree angle and corresponds to $\lambda = 9$.
- q_2 is perpendicular and corresponds to $\lambda = 1$.
- These eigenspaces define the principal semi-axes of the ellipse.

Larger Eigenvalue Means Narrower Axis for $x^T A x = \alpha$

For the level set

$$9z_1^2 + z_2^2 = \alpha,$$

if $z_2 = 0$, then

$$9z_1^2 = \alpha$$

and therefore

$$|z_1| = \frac{\sqrt{\alpha}}{3}.$$

If $z_1 = 0$, then

$$|z_2| = \sqrt{\alpha}.$$

Thus, for the level set $x^T A x = \alpha$, the ellipse is narrower in the direction associated with the larger eigenvalue.

The instructor warned that some people instead write ellipses using A^{-1} , such as $x^T A^{-1} x$. In that convention the eigenvalues are reciprocals, so the visual relation between large eigenvalues and axis lengths reverses.

For $x^T A x = \alpha$:

- Large eigenvalue direction -> narrower semi-axis.
- Small eigenvalue direction -> wider semi-axis.

The key relationship remains: the eigenvectors of A define the principal semi-axes of the ellipse.

Higher-Dimensional Level Sets

In three dimensions, if

$$A = I,$$

then

$$x^T A x = 1$$

gives a sphere.

If one diagonal entry is changed, for example from 1 to 2, the level set becomes an ellipsoid.

In higher dimensions:

- The boundary level set is called a hyperellipse in the lecture's terminology.
- If the solid interior is included, it is a hyperellipsoid.

The instructor emphasized that so far the lecture had been constructing multivariate quadratic functions based on Hermitian matrices, and then moved to why they are useful.

Application: Quadratic Approximation

One important application area is quadratic approximation of general nonlinear functions.

Single-Variable Taylor Approximation

Near a point x_0 , a general nonlinear function can be approximated using Taylor series.

The instructor stressed that the Taylor series itself is not an approximation if all infinitely many terms are kept. It becomes an approximation when only finitely many terms are retained.

Keeping terms through second order gives

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2.$$

Interpretation:

- Keeping only the value gives a constant local approximation.
- Keeping the value and first-derivative term gives an affine/tangent approximation.
- Adding the second-derivative term gives a quadratic approximation.

If the original function has positive curvature around x_0 , then the quadratic approximation is convex and parabola-up in that region.

Multivariate Taylor Approximation

For a multivariate function, the second-order approximation around x_0 has the form

$$f(x) \approx f(x_0) + \nabla f(x_0)^T (x - x_0) + \frac{1}{2}(x - x_0)^T H(x_0)(x - x_0),$$

where $H(x_0)$ is the Hessian matrix.

The parts have the following meanings:

- $f(x_0)$ is the function value at the expansion point.
- $\nabla f(x_0)^T (x - x_0)$ is the affine part.
- $(x - x_0)^T H(x_0)(x - x_0)$ is the quadratic curvature part.

The gradient contains partial derivatives with respect to the coordinate directions. Taking the inner product of the gradient with a direction vector gives a directional derivative in that direction.

The Hessian matrix contains second-order derivative information:

- Diagonal entries are second derivatives with respect to individual variables, such as $\partial^2 f / \partial x_1^2$.
- Off-diagonal entries are mixed second derivatives and correspond to cross terms.

The lecture assumed regularity so that the order of differentiation does not matter. Under that assumption, the Hessian is symmetric.

The eigenvalue sign pattern of the Hessian determines the local quadratic shape:

- Positive definite Hessian: locally parabola-up/convex.
- Negative definite Hessian: locally parabola-down/concave.
- Mixed positive and negative eigenvalues: local saddle structure.

A student asked whether this connects to gradient descent or Newton's method. The instructor answered that this kind of quadratic approximation is the basis of Newton-like algorithms for optimization, and that this may be discussed later.

Positive Semidefinite Geometry: Flat Directions

The instructor then discussed what happens when the Hessian or matrix is positive semidefinite but not positive definite.

Example:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

This matrix is positive semidefinite because its eigenvalues are 1 and 0.

The quadratic form is

$$x^T A x = x_1^2.$$

Consequences:

- In the x_1 direction, the function is parabola-up.
- In the x_2 direction, the curvature is zero.
- The value of x_2 does not affect the quadratic form.
- Along zero-eigenvalue directions, the function is flat.

The instructor described the shape as like a parabolic sheet or rolled paper: it curves in one direction and is flat in the other.

The same idea persists for shifted or rotated semidefinite quadratics. A shift changes where the parabolic sheet is centered, and a non-diagonal positive semidefinite matrix can rotate the curved and flat directions away from the standard coordinate axes. The zero-eigenvalue directions remain the directions with no quadratic curvature.

Properties of Positive Definite Matrices

Before continuing with applications, the lecture gave useful algebraic properties of positive definite and negative definite matrices.

Positive Linear Combinations

Suppose

$$A_1, \dots, A_m$$

are positive definite matrices, and

$$c_1, \dots, c_m > 0$$

are positive scalars.

Then

$$\sum_{i=1}^m c_i A_i$$

is also positive definite.

Proof idea using quadratic forms:

Take any nonzero x . Then

$$x^* \left(\sum_{i=1}^m c_i A_i \right) x = \sum_{i=1}^m c_i x^* A_i x.$$

Each $x^* A_i x$ is strictly positive because A_i is positive definite and $x \neq 0$. Each c_i is also positive. Therefore every term in the sum is positive, so the total sum is positive.

The instructor noted that proving this directly from eigenvalues of the sum would be harder, while the quadratic-form definition makes it easy.

Notation Warning: $A > 0$

The notation $A > 0$ or $A \geq 0$, when used for matrices in this context, does not mean that all entries of A are positive or nonnegative.

It means positive definite or positive semidefinite in the eigenvalue/quadratic-form sense.

Some authors use curved notation such as $A \succ 0$ or $A \succeq 0$ to avoid confusion with elementwise inequalities. The instructor said they would continue using the regular inequality notation with the understanding that it refers to definiteness.

Positive Integer Powers

If A is positive definite, then every positive integer power A^k is also positive definite.

Proof idea:

Use the unitary eigenvalue decomposition

$$A = U \Lambda U^*,$$

where all diagonal entries of Λ are positive eigenvalues.

Then

$$A^2 = U \Lambda U^* U \Lambda U^* = U \Lambda^2 U^*.$$

Similarly,

$$A^k = U \Lambda^k U^*.$$

Since positive numbers remain positive under positive integer powers, all eigenvalues of A^k are positive.

For a negative definite matrix, parity matters:

- Even powers are positive definite.
- Odd powers are negative definite.

This happens because even powers of negative eigenvalues become positive, while odd powers remain negative.

Diagonal Dominance as a Sufficient Condition

The lecture introduced a sufficient condition for positive semidefiniteness and positive definiteness.

For a Hermitian matrix with positive diagonal entries, if each diagonal entry dominates the sum of the absolute values of the other entries in that row,

$$a_{ii} \geq \sum_{j \neq i} |a_{ij}|,$$

then the matrix is positive semidefinite.

The positive-diagonal condition is not cosmetic. The right side is nonnegative, so this dominance condition implicitly requires the relevant diagonal entries to be nonnegative, and in the strict positive definite version they must be positive.

If the inequality is strict for each row,

$$a_{ii} > \sum_{j \neq i} |a_{ij}|,$$

then the matrix is positive definite.

Important warning:

- This is a sufficient condition, not a necessary condition.
- A matrix may fail this diagonal dominance condition and still be positive semidefinite or positive definite.

The instructor connected this to the earlier matrix

$$\begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix},$$

which satisfies strict diagonal dominance because $5 > 4$ in each row.

For negative definiteness, one can apply the positive definite condition to $-A$. Equivalently, the negative of a negative definite matrix is positive definite.

In row-dominance language, a negative definite sufficient condition is obtained by having negative diagonal entries whose absolute values dominate the off-diagonal row sums. This is just the same positive definite dominance test applied after multiplying the matrix by -1 .

Similarity Versus Star-Congruence

The lecture then introduced an important relationship for Hermitian matrices: star-congruence.

Similarity

Similarity had appeared earlier in the context of changing basis for linear transformations.

If two matrices are related by a similarity transformation, they represent the same linear transformation in different bases. Similar matrices have the same eigenvalues.

In symbolic form, the relationship is of the type

$$A = TBT^{-1}$$

for an invertible T .

Key property:

- Similar matrices have exactly the same eigenvalues.

Star-Congruence

For Hermitian matrices, define a different relationship:

$$A = SBS^*$$

where S is invertible.

The instructor emphasized:

- S is not assumed to be unitary.
- It only needs to be nonsingular/invertible.
- This relationship is called star-congruence.

Star-congruence does not preserve the actual eigenvalues. Instead, it preserves the sign pattern of eigenvalues.

For example:

- If all eigenvalues of A are positive, all eigenvalues of B are positive.
- If five eigenvalues of A are positive, then five eigenvalues of B are positive.
- The numerical eigenvalues may differ, but the counts of positive, negative, and zero eigenvalues agree.

Inertia

The preserved sign pattern is called the inertia of a Hermitian matrix.

For a Hermitian matrix, inertia is the ordered triple

$$(n_+, n_-, n_0),$$

where:

- n_+ is the number of positive eigenvalues.
- n_- is the number of negative eigenvalues.
- n_0 is the number of zero eigenvalues.

Eigenvalues are counted with multiplicity.

Examples

For

$$\begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix},$$

the eigenvalues are 9 and 1. Therefore its inertia is

$$(2, 0, 0).$$

For the 3×3 identity matrix, all three eigenvalues are 1. Counting multiplicity, the inertia is

$$(3, 0, 0).$$

For a matrix with two positive eigenvalues, one negative eigenvalue, and no zero eigenvalues, the inertia is

$$(2, 1, 0).$$

The instructor was asked about why the term “inertia” is used. The instructor did not give a definitive historical answer, but noted that these ideas arise in contexts involving Hermitian matrices and energy functions, such as mechanical systems and Lagrangian-type formulations.

Sylvester's Law of Inertia

The lecture stated the key connection between star-congruence and inertia.

For Hermitian matrices A and B ,

$$A = SBS^*$$

for some nonsingular S if and only if A and B have the same inertia.

This is Sylvester's law of inertia.

The instructor emphasized the contrast:

- Similarity preserves eigenvalues.
- Star-congruence preserves inertia, meaning only the sign counts of eigenvalues.

The proof was skipped in lecture and left for students to think about in both directions.

Corollary: Positive Definite Matrices Are Star-Congruent to Identity

An important corollary follows directly from Sylvester's law of inertia.

A matrix is positive definite if and only if it is star-congruent to the identity matrix.

Reason:

- The $n \times n$ identity matrix has inertia $(n, 0, 0)$.
- Any $n \times n$ positive definite matrix also has inertia $(n, 0, 0)$.
- Therefore a positive definite matrix and the identity have the same inertia.
- By Sylvester's law, they are star-congruent.

Thus, if A is positive definite, there exists a nonsingular matrix S such that

$$A = SIS^* = SS^*.$$

This leads to the notion of a matrix square root.

Matrix Square Roots in the Broad SS^* Sense

If

$$A = SS^*,$$

then S is called a square root of the positive definite matrix A in the lecture's broad sense.

The instructor motivated this concept with applications:

- Generating random vectors with arbitrary correlation.
- Whitening signals in communication systems.

Non-Uniqueness of Square Roots

The square root is not unique.

Suppose S is one square root:

$$SS^* = A.$$

Let T be any unitary matrix, so

$$TT^* = I.$$

Define

$$S_2 = ST.$$

Then

$$S_2 S_2^* = (ST)(ST)^* = STT^* S^* = SIS^* = SS^* = A.$$

Therefore S_2 is also a square root of A .

Since there are infinitely many unitary matrices, a positive definite matrix has infinitely many square roots in this sense.

The instructor also pointed out the converse parameterization idea: once one square root S is known, the whole family of square roots can be described by right-multiplying S by unitary matrices. If $RR^* = A$ and $SS^* = A$, then $R = SU$ for a unitary $U = S^{-1}R$.

1×1 Example

In the 1×1 case, positive definite matrices are positive real numbers.

For example, if $A = 25$, then $S = 5$ is a square root because

$$5 \cdot 5^* = 25.$$

But 1×1 unitary matrices are complex numbers on the unit circle, $e^{j\phi}$. Therefore

$$5e^{j\phi}$$

is also a square root in the $SS^* = A$ sense, because

$$(5e^{j\phi})(5e^{j\phi})^* = 25.$$

Notation Warning

The instructor writes something like

$$S = A^{1/2}$$

for a square root, but warned that this notation is ambiguous unless a particular square root is specified.

It may simply mean “choose one matrix S satisfying $SS^* = A$.”

The conjugate-transpose of a chosen square root must also be read relative to that choice. The instructor mentioned a shorthand style that saves parentheses, such as writing a conjugate-square-root notation for $(A^{1/2})^*$, but the warning is the same: without choosing the particular square root, the notation is not identifying a unique matrix.

A student suggested interpreting the notation as a set of all square roots and then choosing an element from that set. The instructor said that is a possible viewpoint, but the lecture notation would stick with choosing one square root.

Positive Definite Square Root via Eigenvalue Decomposition

The lecture then gave a concrete way to find one square root.

Let

$$A = U\Lambda U^*$$

be the eigenvalue decomposition of a positive definite Hermitian matrix.

Because A is positive definite, all eigenvalues in Λ are positive.

Define

$$A^{1/2} = U\Lambda^{1/2}U^*,$$

where $\Lambda^{1/2}$ is the diagonal matrix whose entries are the positive square roots of the eigenvalues.

This gives the positive definite square root.

Properties:

- It is Hermitian.
- It is positive definite.
- Its conjugate transpose equals itself.
- Multiplying it by its conjugate transpose gives back A :

$$A^{1/2}(A^{1/2})^* = A.$$

This is one distinguished square root among the infinitely many possible square roots.

Cholesky Factorization: Lower Triangular Square Root

The lecture stated another square-root theorem for positive definite matrices: Cholesky factorization.

If A is positive definite, then there exists a lower triangular matrix L with positive diagonal entries such that

$$A = LL^*.$$

This L is a lower triangular square root of A .

The instructor connected this to Gaussian elimination:

- Whenever lower triangular matrices appear, think of Gaussian elimination.
- Earlier LDU factorization was essentially based on Gaussian elimination.
- For positive definite matrices, the upper triangular factor can be related to the conjugate transpose of the lower triangular factor.

Cholesky Proof Sketch: Block Partition

The instructor began a proof sketch for Cholesky factorization.

Partition the positive definite matrix A by separating its first row and first column:

$$A = \begin{bmatrix} \alpha & v^* \\ v & M \end{bmatrix},$$

where:

- α is the first diagonal entry.
- v is a column vector.
- v^* appears because A is Hermitian.
- M is an $(n-1) \times (n-1)$ Hermitian block.

Why $\alpha > 0$

Since A is positive definite,

$$x^*Ax > 0$$

for every nonzero x .

Choose $x = e_1$, the first standard basis vector. Then

$$e_1^*Ae_1 = \alpha.$$

Therefore

$$\alpha > 0.$$

The same idea shows that every diagonal entry of a positive definite matrix is positive.

Why M Is Positive Definite

Take a vector of the form

$$x = \begin{bmatrix} 0 \\ \beta \end{bmatrix},$$

where $\beta \neq 0$.

Then

$$x^*Ax = \beta^*M\beta.$$

Since A is positive definite, this quantity is positive for every nonzero β . Therefore

$$M$$

is positive definite.

At this point:

- $\alpha > 0$.
- M is positive definite.
- v has not yet been characterized.

Eliminating the First Column

The next step is to eliminate the entries below α in the first column, similar to Gaussian elimination.

Use the lower triangular matrix

$$S = \begin{bmatrix} 1 & 0 \\ -v/\alpha & I \end{bmatrix}.$$

Left multiplication by S performs row operations:

- The first row is preserved.
- Scaled multiples of the first row are subtracted from the lower rows.
- The lower entries in the first column are canceled.

This gives an intermediate block upper triangular form.

Then multiply on the right by S^* :

$$SAS^*.$$

Right multiplication by S^* performs the corresponding column operation and cancels the upper-right block. The result is a block diagonal matrix:

$$SAS^* = \begin{bmatrix} \alpha & 0 \\ 0 & M - \frac{vv^*}{\alpha} \end{bmatrix}.$$

This is block diagonal, not fully diagonal.

The matrix S is invertible because it is triangular with ones on its diagonal. Thus A and the block diagonal matrix are star-congruent.

The Schur Complement Block

The lower-right block

$$M - \frac{vv^*}{\alpha}$$

is the important remaining block.

The instructor analyzed

$$\frac{vv^*}{\alpha}.$$

Since $\alpha > 0$, division by α preserves positive semidefiniteness.

Properties of vv^*

The matrix vv^* is Hermitian:

$$(vv^*)^* = vv^*.$$

It is an outer product, so it has rank one unless $v = 0$.

The vector v is an eigenvector:

$$vv^*v = v(v^*v) = \|v\|^2v.$$

Thus the one possible nonzero eigenvalue is

$$\|v\|^2.$$

All remaining eigenvalues are zero. Therefore vv^* is positive semidefinite, and so is vv^*/α .

Important Warning

Even though M is positive definite and vv^*/α is positive semidefinite, it is not automatically obvious from the expression alone that

$$M - \frac{vv^*}{\alpha}$$

is positive definite.

In general, subtracting a positive semidefinite matrix from a positive definite matrix can destroy positive definiteness if the subtracted matrix is large enough.

The instructor said that the expression-by-itself route is not the easiest way to prove positivity.

Instead, use star-congruence:

- SAS^* is star-congruent to A .
- A is positive definite.
- Star-congruence preserves inertia.
- Therefore SAS^* is also positive definite.
- Since the block diagonal matrix is positive definite and $\alpha > 0$, the lower-right block

$$M - \frac{vv^*}{\alpha}$$

must be positive definite.

This is the key proof idea that allows the Cholesky proof to continue recursively.

Instructor Remarks and Study Warnings

The instructor made several remarks that are important for studying this lecture:

- The hyperbolic level sets of indefinite quadratic forms were mentioned but not covered further.
- Taylor series itself has infinitely many terms; approximation comes from truncating it.
- The symmetry of the Hessian assumes mixed partial derivatives commute.
- Quadratic approximations are a foundation for Newton-like optimization methods.
- For positive definite notation, $A > 0$ is not an elementwise statement.
- Diagonal dominance is sufficient but not necessary.

- Similarity and star-congruence are different: one preserves eigenvalues, the other preserves inertia.
- The proof of Sylvester's law of inertia was left to students to think about in both directions.
- The Cholesky proof was not finished in this lecture; the instructor said it would continue on Thursday.
- The instructor specifically asked students to review star-congruence, Sylvester's law of inertia, and square roots. These algebraic tricks will be used repeatedly.

Source and Coverage Note

Source used: C:\Users\mohdh\Downloads\New folder (2)\lectures\corrected\lecture14_corrected.md.

Coverage: These notes follow the lecture chronologically and include the concepts, definitions, examples, proof ideas, instructor remarks, warnings, and relationships present in the lecture 14 transcript. No other lecture transcript was processed.

Lecture 15 Notes

Opening Review: Story Line Summary

The lecture begins with the instructor placing the current work inside the course narrative.

The course has been building toward solving $Ax = b$ efficiently by writing A as a product of simple matrices. The full hierarchy covered so far:

- All square matrices can be triangularized using a unitary basis: **Schur factorization** $A = UTU^*$, where U is unitary and T is upper triangular.
- When T is diagonal, we have **unitary diagonalization** $A = U\Lambda U^*$. Matrices that can be diagonalized this way are the **normal matrices** ($A^*A = AA^*$).
- Within normal matrices, **Hermitian** matrices ($A^* = A$) have real eigenvalues, and **unitary** matrices ($U^*U = I$) have eigenvalues on the unit circle.
- Within Hermitian matrices, the sign of eigenvalues determines the class: positive definite, positive semidefinite, negative definite, negative semidefinite, or indefinite.

The instructor then reviews properties of positive definite matrices established in the previous lecture:

1. Positive linear combinations with positive scalar weights: $\sum c_i A_i$ is positive definite if each A_i is PD and each $c_i > 0$.
2. Diagonal entries of a PD matrix are positive (proved by setting $x = e_i$).
3. **Principal submatrix property**: If A is PD and we select a submatrix using the same row and column indices I (e.g., rows/columns 1, 3, 5), that submatrix is also PD. Proof: choose a vector x that is zero outside index set I ; then $x^*Ax > 0$ restricts to the submatrix.
4. **Star-congruence**: $A = SBS^*$ for invertible S preserves inertia (the sign pattern of eigenvalues). This is Sylvester's law: star-congruent Hermitian matrices have the same inertia.
5. The identity matrix has inertia $(n, 0, 0)$. Any PD matrix also has inertia $(n, 0, 0)$. Therefore any PD matrix is star-congruent to the identity, meaning it can be written $A = SS^*$ for some invertible S . The instructor calls S a **matrix square root** of A .
6. There are infinitely many matrix square roots: if S works, so does ST for any unitary T , because $(ST)(ST)^* = S(TT^*)S^* = SS^* = A$.
7. The **positive definite square root** is $A^{1/2} = U\Lambda^{1/2}U^*$ where $A = U\Lambda U^*$.
8. The **lower triangular square root** exists by Cholesky factorization (the focus of this lecture).

The instructor pauses to clarify notation in this review. The principal submatrix statement is about selecting the same indexed rows and columns from one matrix. The positive linear combination statement is different: $A_1, A_2, A_3, \dots$ are separate positive definite matrices, and the coefficients c_i are arbitrary positive scalars. The index i in the sum is not the same as the row/column index set I .

Cholesky Proof Completion

The Cholesky proof started in L14 and was left at a critical point. The instructor resumes.

Recap of Where We Were

Partition positive definite A as:

$$A = \begin{bmatrix} \alpha & v^* \\ v & M \end{bmatrix},$$

where: - $\alpha > 0$ (proved in L14 by setting $x = e_1$ in $x^*Ax > 0$). - M is $(n-1) \times (n-1)$ PD (proved by setting the first entry of x to zero).

Block Gaussian elimination by left-multiplying by the lower triangular matrix:

$$S = \begin{bmatrix} 1 & 0 \\ -v/\alpha & I \end{bmatrix}$$

and right-multiplying by S^* (conjugate transpose):

$$SAS^* = \begin{bmatrix} \alpha & 0 \\ 0 & M - \frac{vv^*}{\alpha} \end{bmatrix}.$$

The lower-right block $M - vv^*/\alpha$ is the **Schur complement** of α .

Why the Schur Complement Is Positive Definite

The critical question: is $M - vv^*/\alpha$ positive definite?

At first glance this is not obvious. We know M is PD and vv^*/α is positive semidefinite (it is a scaled outer product). Subtracting a PSD matrix from a PD matrix does not automatically give a PD matrix — the subtracted piece could be large enough to destroy positive definiteness.

The instructor also notes the outer-product intuition: for nonzero v , vv^* is rank one and its nonzero eigenvalue is $\|v\|^2$. That still does not make the subtraction safe.

The argument that resolves this is **star-congruence**:

1. S is invertible (it is lower triangular with ones on the diagonal, so $\det(S) = 1 \neq 0$).
2. SAS^* is star-congruent to A .
3. A is PD, so $\text{inertia}(A) = (n, 0, 0)$.
4. Star-congruence preserves inertia.
5. Therefore SAS^* also has inertia $(n, 0, 0)$, i.e., it is PD.
6. The block diagonal matrix is PD iff each diagonal block is PD.
7. The upper-left block is $\alpha > 0$ (positive definite, trivially).
8. Therefore the lower-right block $M - vv^*/\alpha$ must also be PD.

Instructor emphasis: This is the right way to see it. Trying to prove positivity of the Schur complement directly from the expression $M - vv^*/\alpha$ is harder. The star-congruence argument is clean and powerful.

Inductive/Recursive Step

Once we know the Schur complement $M - vv^*/\alpha$ is PD and $(n-1) \times (n-1)$, we can apply the same partition-and-eliminate procedure to it. Two equivalent ways to continue:

Recursive approach: Apply block diagonalization to the Schur complement block, and then again to its Schur complement, and so on. After n steps, the original PD matrix has been converted to a diagonal matrix with all positive diagonal entries, through star-congruent transformations.

Inductive approach: Assume Cholesky factorization holds for $(n-1) \times (n-1)$ PD matrices (base case $n = 1$ is trivial: the Cholesky factor of $\alpha > 0$ is $\sqrt{\alpha}$). Then apply it to the Schur complement: $M - vv^*/\alpha = L_{n-1}L_{n-1}^*$. Reconstruct:

$$SAS^* = \begin{bmatrix} \alpha & 0 \\ 0 & L_{n-1}L_{n-1}^* \end{bmatrix} = \underbrace{\begin{bmatrix} \sqrt{\alpha} & 0 \\ 0 & L_{n-1} \end{bmatrix}}_{L_{\text{new}}} \underbrace{\begin{bmatrix} \sqrt{\alpha} & 0 \\ 0 & L_{n-1}^* \end{bmatrix}}_{L_{\text{new}}^*}.$$

Since $A = S^{-1}(L_{\text{new}}L_{\text{new}}^*)(S^*)^{-1}$, and multiplying through the triangular factors of S^{-1} and L_{new} gives a lower triangular product, we recover $A = LL^*$ for a lower triangular L . The positive diagonal entries of L come from the square roots of positive pivots at each recursive step.

What Cholesky Is Not

The instructor notes the connection to Gaussian elimination: when Gaussian elimination is applied to a PD matrix, the LDU factorization has positive diagonal entries in D , and the Cholesky factorization is essentially combining L and $D^{1/2}$ into the Cholesky factor.

Application of Cholesky: Solving Positive Definite Systems

Suppose $Px = y$ where P is positive definite.

Strategy using Cholesky:

1. Factor $P = LL^*$ (Cholesky factorization, computed once).
2. The system becomes $LL^*x = y$.
3. Set $z = L^*x$. Then $Lz = y$: a lower triangular system, solved by **forward substitution**.
4. Once z is known, solve $L^*x = z$: an upper triangular system, solved by **back substitution**.

Two simple triangular system solves instead of one expensive inversion.

Application to Causal Estimators

The instructor mentions another important use of the lower triangular Cholesky factor that connects to systems theory.

A lower triangular matrix corresponds to a **causal linear system**: the output at time n depends only on inputs at times $\leq n$. This is the linear systems interpretation of lower triangularity.

In estimation theory, when the observation covariance matrix must be factored, using the Cholesky factor (lower triangular) gives a **causal Wiener filter** — a causal linear estimator. The non-causal Wiener filter uses the full matrix; the causal version requires the Cholesky factorization. This is mentioned as a topic from a separate estimation theory course; the lecture does not go into details.

The Least Squares Problem

The instructor now asks: in practice, when does a positive definite system arise?

Setup

Consider $Ax = b$ where: - A is $m \times n$ with $m > n$ (tall matrix, more equations than unknowns). - A has full column rank ($\text{rank}(A) = n$).

What is the problem?

- Null space of A is trivial (full column rank), so uniqueness is guaranteed if a solution exists.
- However, the column space of A is an n -dimensional subspace of $\mathbb{R}^m$. Since $n < m$, not every $b \in \mathbb{R}^m$ lies in $\mathcal{R}(A)$.
- In practice, b may be a noisy measurement that doesn't exactly lie in the column space of A .

The instructor's attitude: Instead of giving up when $b \notin \mathcal{R}(A)$, “blame it on b .” Assume the noise in b has pushed it out of the column space. Find the nearest vector $\hat{b}$ inside the column space, and use that.

This is the **least squares problem**:

$$\min_x \|Ax - b\|_2^2.$$

We minimize the squared Euclidean distance between Ax and b over all choices of x .

Geometric Interpretation

The best $\hat{b} = A\hat{x}$ inside the column space is the **orthogonal projection** of b onto $\mathcal{R}(A)$.

The error vector $b - \hat{b} = b - A\hat{x}$ must be **orthogonal to the column space** of A . This is the **projection theorem** (or orthogonality principle).

Formally: $b - A\hat{x} \perp \mathcal{R}(A)$, meaning for every column a_j of A :

$$a_j^*(b - A\hat{x}) = 0.$$

Collecting all columns:

$$A^*(b - A\hat{x}) = 0.$$

Rearranging:

$$A^*A\hat{x} = A^*b.$$

These are the **normal equations**.

The transcript writes this derivation using A^T while discussing real least squares, because the rows of A^T are the columns of A . In complex notation the same orthogonality condition is written with A^* .

Normal Equations

$A^*A\hat{x} = A^*b.$

The solution is:

$$\hat{x} = (A^*A)^{-1}A^*b \quad (\text{when } A^*A \text{ is invertible}).$$

The matrix $(A^*A)^{-1}A^*$ is called the **Moore-Penrose left pseudoinverse** of A .

A^*A Is Positive Definite for Full Column Rank A

The instructor proves a key result, generalizing from the specific least squares setup to a broader statement.

Theorem: Z^*Z Is Always PSD

For any matrix Z :

Step 1: Z^*Z is Hermitian: $(Z^*Z)^* = Z^*(Z^*)^* = Z^*Z$. ✓

Step 2: Positive semidefiniteness: for any vector x :

$$x^*(Z^*Z)x = (Zx)^*(Zx) = \|Zx\|_2^2 \geq 0.$$

Therefore $Z^*Z \succeq 0$ for any Z . No conditions needed. The same style of argument also shows $ZZ^* \succeq 0$; the lecture says the PSD/Hermitian conclusion does not depend on whether Z is tall, fat, square, full rank, or rank deficient.

When Is Z^*Z Positive Definite?

If Z has full column rank, then $Zx = 0 \iff x = 0$ (trivial null space). Therefore:

$$x \neq 0 \implies Zx \neq 0 \implies \|Zx\|^2 > 0 \implies x^*(Z^*Z)x > 0.$$

So $Z^*Z \succ 0$ when Z has full column rank. If Z is merely tall but not full column rank, then Z^*Z is still PSD but not PD, because some nonzero x lies in the null space and gives $x^*Z^*Zx = 0$. For ZZ^* , the analogous PD condition is full row rank.

Application: For our tall full-rank A , $A^*A \succ 0$. Therefore the normal equations have a **unique solution**.

The instructor asks students to think about the general case: even if Z is not tall (it could be square or fat), as long as it has full column rank, $Z^*Z \succ 0$. The condition is purely about the column rank, not the shape.

Solving Least Squares via Cholesky: Theoretically Correct, Numerically Inadvisable

Since $A^*A \succ 0$, apply Cholesky to solve the normal equations:

1. Factor $A^*A = LL^*$.
2. Solve $Lz = A^*b$ by forward substitution.
3. Solve $L^*\hat{x} = z$ by back substitution.

The instructor's warning: "This is good in theory, but bad in practice. Don't do that."

The reason involves **numerical conditioning**. If A has condition number $\kappa(A) = \sigma_{\max}/\sigma_{\min}$, then A^*A has condition number $\kappa(A)^2$. Forming A^*A squares the condition number, making the system much harder to solve in finite-precision arithmetic. The rounding errors accumulate.

What to do instead: MATLAB's backslash operator (and Python's `numpy.linalg.lstsq`) uses QR decomposition or SVD-based approaches that operate directly on A without forming A^*A . These methods maintain the original condition number.

The instructor explicitly says: "If b is not in the range space of A , MATLAB doesn't do this. It uses other methods." QR-based and SVD-based least squares solutions will be discussed in later lectures.

The theoretical purpose of the Cholesky approach is to establish the connection between least squares and positive definite systems. Practically, prefer QR or SVD.

Application: Generating Correlated Random Vectors

The instructor introduces an application of matrix square roots (not restricted to the Cholesky or positive definite square root specifically) that is practically important in signal processing.

The problem: A graduate student's advisor says: "Write code to generate random vectors whose elements have a specific correlation structure." The advisor gives a desired covariance matrix C_y . MATLAB's `randn` generates uncorrelated (white) random vectors. How do you produce correlated ones?

More specifically, the lecture frames the computational task as generating L samples of a random vector with prescribed mean m_x and prescribed covariance C_x . A call like `randn(n,L)` gives L columns, each an n -dimensional Gaussian sample. The entries are independent Gaussian variables; for Gaussian variables, independence and uncorrelatedness coincide. The instructor emphasizes that the goal here is the covariance structure, not a detailed discussion of Gaussian versus non-Gaussian distributions.

Background: Random Vectors

The lecture reviews the basics for completeness.

Mean Vector

For a random vector $x = (x_1, \dots, x_n)^T$:

$$\mu_x = \mathbb{E}[x] = \begin{bmatrix} \mathbb{E}[x_1] \\ \vdots \\ \mathbb{E}[x_n] \end{bmatrix}.$$

The mean vector contains the individual means of each component.

For continuous random vectors, the instructor expands the definition using the joint PDF:

$$\mathbb{E}[x] = \int x f_X(x_1, \dots, x_n) dx_1 \cdots dx_n.$$

Looking at the first component, this integral becomes:

$$\int x_1 f_X(x_1, \dots, x_n) dx_1 \cdots dx_n = \int x_1 f_{X_1}(x_1) dx_1 = \mathbb{E}[x_1],$$

because integrating the joint PDF over the other variables produces the marginal PDF of x_1 . The same argument applies component by component, so $\mathbb{E}[x]$ is the vector of scalar expectations.

Correlation and Covariance Between Two Scalar Random Variables

The **correlation** between x_i and x_j is $\mathbb{E}[x_i \bar{x}_j]$ (real case: $\mathbb{E}[x_i x_j]$). The instructor calls this a potentially confusing point: two variables being uncorrelated does **not** mean this raw correlation is zero in general. It means the raw correlation equals the product of the means:

$$\mathbb{E}[x_i \bar{x}_j] = \mathbb{E}[x_i] \overline{\mathbb{E}[x_j]}.$$

Only in the zero-mean case does “uncorrelated” reduce to raw correlation equal to zero.

The **covariance** subtracts the product of means:

$$\text{Cov}(x_i, x_j) = \mathbb{E}[(x_i - \mu_i) \overline{(x_j - \mu_j)}].$$

This is the correlation of the **centralized** variables $x_i - \mu_i$ and $x_j - \mu_j$. Centralizing means subtracting the mean, shifting the PDF so the new variable has mean zero. Expanding the product gives:

$$\text{Cov}(x_i, x_j) = \mathbb{E}[x_i \bar{x}_j] - \mu_i \bar{\mu}_j.$$

Distinction: Two random variables are **uncorrelated** if $\text{Cov}(x_i, x_j) = 0$, equivalently if their raw correlation equals the product of their means.

Geometric/Intuitive Meaning of Covariance

The instructor explains covariance by asking students to imagine a scatter plot of n independent sample points $(x_1^{(k)}, x_2^{(k)})$ drawn from the joint PDF. The point of the picture is intuition, not a probability-one theorem.

When the cloud has a positive trend, most points above the x_1 mean also sit above the x_2 mean, but not all of them must. The instructor phrases this as a bet that is usually right but can sometimes lose.

Case 1: Positive covariance. The cloud of points has a positive slope — when x_1 is above its mean, x_2 tends to be above its mean. Then $(x_1 - \mu_1)$ and $(x_2 - \mu_2)$ tend to have the same sign, so their product is positive, and the expectation of the product is positive.

Case 2: Negative covariance. The cloud slopes downward — when x_1 is above its mean, x_2 tends to be below its mean. Opposite signs $\rightarrow$ negative product $\rightarrow$ negative expectation.

Case 3: Zero covariance. The cloud is circular — knowing x_1 gives no linear information about x_2 . However, the instructor notes: zero covariance does NOT mean x_1 contains no information about x_2 . A circular cloud can have a nonlinear relationship; zero covariance only means there is no **linear** relationship.

The zero-covariance picture need not literally be circular. The transcript's point is broader: a symmetric or nonlinear cloud can have no clear positive or negative linear trend. Knowing x_1 may still restrict x_2 to a smaller nonlinear region, but a linear estimator cannot exploit that structure. A nonlinear estimator would be needed, and estimation theory is not pursued here.

Practical meaning: If someone tells you $x_1 > \mu_1$ and the covariance is positive, you would bet that $x_2 > \mu_2$. If covariance is zero, knowing x_1 gives you no useful linear prediction of x_2 .

MMSE Linear Estimator: If you want to estimate x_2 as a linear function of x_1 (i.e., $\hat{x}_2 = a(x_1 - \mu_1) + \mu_2$ minimizing the mean squared error), the optimal coefficient is:

$$a = \frac{\text{Cov}(x_1, x_2)}{\text{Var}(x_1)}.$$

Equivalently, for an affine estimator $\hat{x}_2 = ax_1 + b$, the offset is $b = \mu_2 - a\mu_1$. The instructor labels this estimator discussion as extra intuition “in case you are curious.”

If the covariance is zero, $a = 0$, and your best linear estimator of x_2 is just its mean μ_2 — x_1 provides no linear benefit. The instructor notes: “this looks like an inner product divided by an inner product — in fact, covariances can be viewed as inner products,” a connection to be developed later.

Variance: $\text{Var}(x_i) = \text{Cov}(x_i, x_i) = \mathbb{E}[|x_i - \mu_i|^2]$. Expanding gives $\text{Var}(x_i) = \mathbb{E}[|x_i|^2] - |\mu_i|^2$. This is the diagonal entry of the covariance matrix.

Autocorrelation Matrix

$$R_x = \mathbb{E}[xx^*].$$

Autocorrelation is for one random vector, not two different random vectors. It is a library of all pairwise correlations between components of that vector. Entry $(R_x)_{ij} = \mathbb{E}[x_i \bar{x}_j]$.

Key point: Even though each sample xx^* is a rank-one matrix, the expectation $\mathbb{E}[xx^*]$ is generally full rank (it is a weighted sum of many rank-one matrices and can have arbitrary rank).

For a discrete random vector taking values $x^{(k)}$ with probabilities p_k :

$$R_x = \sum_k p_k x^{(k)} (x^{(k)})^*.$$

Each $x^{(k)} (x^{(k)})^*$ is rank one, but the sum need not be rank one.

Covariance Matrix

$$C_x = \mathbb{E}[(x - \mu_x)(x - \mu_x)^*] = R_x - \mu_x \mu_x^*.$$

Entry $(C_x)_{ij} = \text{Cov}(x_i, x_j)$. The diagonal entries are variances.

The terminology “white” and “colored” comes from random processes, which the instructor says are outside this course. A random process can be viewed as a random vector with infinitely many components. Pairwise correlations across time form a correlation function; the Fourier transform of that function is a spectrum. If the samples are uncorrelated, the spectrum is constant across frequencies, analogous to white light. If some pairs are correlated, engineers call the vector or process colored.

White vs. colored random vectors: - **White random vector:** elements are pairwise uncorrelated $\rightarrow C_x$ is diagonal. Analogy: white light has a flat power spectrum; white random vectors have a flat (diagonal) covariance. - **Colored random vector:** elements have nonzero covariances $\rightarrow C_x$ has nonzero off-diagonal entries.

Autocorrelation and Covariance Matrices Are Hermitian and Positive Semidefinite

The instructor explicitly connects the random-vector definitions back to Hermitian and positive definite matrices. Both R_x and C_x are Hermitian and PSD; C_x is the autocorrelation matrix of the mean-centered vector $x - \mu_x$.

For the autocorrelation matrix:

$$a^* R_x a = \mathbb{E}[a^* x x^* a] = \mathbb{E}[|a^* x|^2] \geq 0.$$

Here a is deterministic, so it can be moved inside the expectation, and $a^* x$ is a scalar random variable.

Hermitian: $C_x^* = \mathbb{E}[(x - \mu)(x - \mu)^*]^* = C_x$. ✓

Positive semidefinite: For any vector a :

$$a^* C_x a = \mathbb{E}[a^* (x - \mu)(x - \mu)^* a] = \mathbb{E}[|a^* (x - \mu)|^2] \geq 0.$$

The expected value of a nonnegative quantity is nonneg. Therefore $C_x \succeq 0$.

Positive definite? If C_x has full rank (all eigenvalues positive), then it is PD. If the random vector lies in a lower-dimensional subspace with probability 1, then C_x is only PSD.

In most practical scenarios with a full-rank random vector, $C_x \succ 0$. The instructor states this as: a full-rank PSD matrix cannot have zero eigenvalues, so its eigenvalues must all be positive.

Coloring: Generating a Correlated Random Vector

Goal: Given desired covariance $C_y \succ 0$ and a white random vector z with $\mathbb{E}[zz^*] = I$ (e.g., columns from MATLAB's `randn(n,L)`), construct a zero-mean y with covariance C_y . Then add the desired mean if the requested random vector is nonzero mean.

Algorithm:

1. Find any matrix square root S satisfying $SS^* = C_y$.
 - Options: Cholesky factor L (lower triangular), or positive definite square root $C_y^{1/2} = U\Lambda^{1/2}U^*$.
 - For the coloring application, it does not matter which square root is used.
2. Set $y = Sz$.

3. If the final desired mean is m_x , set $x = y + m_x$. For L samples stored as columns, write $X = SZ + m_x \mathbf{1}_L^T$. Mean shifting does not change covariance, so the zero-mean construction is enough for the covariance part.

Verification:

$$\mathbb{E}[yy^*] = \mathbb{E}[Szz^*S^*] = S\mathbb{E}[zz^*]S^* = SIS^* = SS^* = C_y. \checkmark$$

Why it works intuitively: When we multiply z by S , we take weighted sums of the uncorrelated components of z . The elements of $y = Sz$ each depend on the same pool of uncorrelated sources, creating linear relationships between them. The square root structure ensures that the pairwise covariances are exactly as specified by C_y .

The key insight for why we use the square root: When computing $\mathbb{E}[yy^*] = \mathbb{E}[Szz^*S^*] = S \cdot I \cdot S^* = SS^* = C_y$, the S appears once on each side, so we need the factor that multiplied by its conjugate gives C_y — i.e., the square root. The instructor explicitly explains this to a student asking why we use the square root.

Note on unitary coloring matrix: If instead of a general square root we used a unitary matrix U , then $y = Uz$ would still have identity covariance: $\mathbb{E}[yy^*] = U\mathbb{E}[zz^*]U^* = UIU^* = I$. So using a unitary matrix does not create correlation; you still get a white vector. The non-unitary part of the square root is what introduces the correlation.

Whitening: Reversing the Process

Goal: Given a colored random vector y with covariance C_y , produce a white vector q with identity covariance.

Algorithm: Set $q = S^{-1}y$, where S is any invertible square root satisfying $SS^* = C_y$. If S is denoted $C_y^{1/2}$, this is often written $q = C_y^{-1/2}y$: multiply by the inverse square root.

Verification:

$$\mathbb{E}[qq^*] = S^{-1}\mathbb{E}[yy^*](S^{-1})^* = S^{-1}C_yS^{-*} = S^{-1}SS^*S^{-*} = I. \checkmark$$

The transcript has a student correction during this derivation: the vector being whitened should be the colored vector y with covariance C_y , and the matrix should be the inverse square root of C_y . The instructor agrees that the notation became confusing and says it will be clarified in the next lecture.

Applications of whitening: Blind source separation and signal pre-processing in communications. The idea is to remove the correlation structure so that subsequent algorithms can assume uncorrelated inputs.

Instructor Remarks and Study Guidance

- The Cholesky proof's key step is **star-congruence preserves inertia** $\rightarrow$ the Schur complement is PD. Do not try to prove Schur complement positivity from the formula alone.
- **Normal equations** $A^*A\hat{x} = A^*b$ arise from the projection theorem: the least squares error must be orthogonal to the column space of A . This is the geometric meaning.
- Z^*Z is always PSD and PD when Z has full column rank. These are frequently examined facts.
- Numerically, **do not solve least squares via normal equations + Cholesky**; the condition number gets squared. Use QR or SVD.
- The **coloring and whitening operations** use the matrix square root. The choice of which square root (PD vs. Cholesky vs. other) does not matter for the coloring application; it matters for applications like causal estimation where lower triangularity is required.

- **Covariance as inner product:** The expression $\text{Cov}(x_i, x_j)/\text{Var}(x_i)$ looks like an inner product divided by a squared norm — the same structure as the projection coefficient. This connection will be made explicit when inner products are formalized.
- For random variables, **uncorrelated means covariance zero**, not necessarily raw correlation zero. Raw correlation equals the product of means; in the zero-mean case these statements coincide.
- The whitening derivation has a notation caveat in the transcript. Remember the clean version: if $C_y = SS^*$, then $q = S^{-1}y$ has covariance I .

Source and Coverage Note

Source: `corrected/lecture15_corrected.md`.

Coverage: Full lecture covered chronologically. Topics: opening summary of all factorizations and matrix classes, review of PD properties (positive linear combinations, diagonal entries, principal submatrices, star-congruence, Sylvester's law, SS^* form), Cholesky proof completion (Schur complement PD by star-congruence, inductive step), causal estimator application of lower triangular square root, solving PD systems via Cholesky, least squares problem (geometric setup, orthogonality condition, normal equations, pseudoinverse), Z^*Z always PSD and PD for full column rank (proof), numerical warning about normal equations, covariance intuition (scatter plot, positive/negative/zero covariance, MMSE linear estimator, covariance as inner product, variance), autocorrelation and covariance matrices (Hermitian and PSD proofs), white vs. colored random vectors, coloring algorithm (any square root of C_y), why unitary coloring does not create correlation, whitening algorithm, applications.

Audit addendum: Notes now also preserve the instructor's notation clarification for principal submatrices versus sums of separate PD matrices, the joint-PDF/marginal-PDF derivation of $\mathbb{E}[x]$, raw correlation versus covariance, centralization, trend exceptions in covariance scatter plots, the nonlinear-information caveat for zero covariance, the discrete weighted-sum expression for R_x , the Random Processes origin of white/colored terminology, PSD proofs for both R_x and C_x , multiple-sample coloring with mean shift, and the whitening notation correction.

Lecture 16 Notes

Opening: Mixed Feelings About Coverage

The instructor opened the lecture reflectively, saying that sometimes it feels like the course has not covered much, and other times it feels like a great deal has been covered — “I have mixed feelings.” The hope expressed is that students feel they have gone beyond basic undergraduate linear algebra and have picked up concepts that are genuinely useful, and that the motivations for these subjects will become even clearer as later material builds upon them.

This lecture is primarily a **mid-course review** (the midterm is approaching) followed by a detailed treatment of **linear time-invariant (LTI) systems, circulant matrices, the DFT, and the FFT**, ending with a preview of QR factorization. The instructor organizes the entire course so far into two intertwined story lines and walks through them slowly.

Story Line 1: Analysis of $Ax = b$ (Existence and Uniqueness)

The first story line started with the problem $Ax = b$ and the observation that there are fundamental questions about **existence** and **uniqueness** of solutions. To address these analysis problems, the course adopted **vector space methods**, which generalize our two- and three-dimensional intuition to higher-dimensional spaces.

The chronological build-up was:

1. **Vector spaces and subspaces.** The course introduced abstract vector spaces and subspaces. The instructor recalled interesting examples such as the **subspace of zero-trace matrices** — examples meant to show that “vectors” need not be arrows; matrices can form vector spaces too.
2. **Span.** Given a set of vectors, the span is the minimum linear space (smallest subspace) that contains them.
3. **Linear independence.** Whether every vector in a set carries valuable information, or whether one of them can be generated as a linear combination of the others.
4. **Basis and dimension.** Given a vector space, find a minimum set of vectors whose span equals that space. The number of vectors in such a minimal spanning set is the **dimension** of the space.
5. **Geometric tools beyond linear structure.** On top of the vector-space skeleton, the course added geometric structure. So far only the **Euclidean norm** has been used as the standard norm — the instructor emphasizes that up to this point **no other way of measuring the size of a vector has been introduced** (this changes in the norm lectures coming up). The Euclidean norm squares the entries, sums them, and takes the square root: the Euclidean distance to the origin.
6. **Euclidean inner product.** This gives angle information. In two and three dimensions the inner product corresponds to the geometric angle; the course extended the angle notion to high dimensions using the **Cauchy-Schwarz inequality**, which says the inner product, normalized by the norms, always lies between -1 and $+1$. That bound is exactly what lets us keep calling it a cosine of an angle in high dimensions.
7. **Orthogonality.** Orthogonality of vectors, orthonormal sets.
8. **Hyperplanes and half-spaces.** Described as really important concepts, e.g., in machine learning — a single neuron is essentially a half-space classifier that tells you which half-space your data lies in.
9. **Four fundamental subspaces.** Moving into the analysis of $Ax = b$, the course built the four fundamental subspaces by viewing the matrix as a collection of column vectors versus a collection of row vectors: **column space** vs. **row space**, plus the spaces orthogonal to them (left null space and null space).

10. **Rank.** The instructor notes he didn't list it explicitly above, but **rank** — the common dimension of the column space and the row space (proved equal) — is central. The existence/uniqueness analysis depends on **two factors**: the **shape** of A (square, tall, fat) and the **rank** of A . Based on those, conclusions about existence and uniqueness follow.
-

Story Line 2: Solving $Ax = b$ via Factorizations

The second story line uses $Ax = b$ as an excuse to introduce a sequence of powerful concepts. The strategy:

- Identify which A matrices make $Ax = b$ **easy** to solve: **diagonal, triangular, and orthogonal (unitary)** matrices.
- The basic strategy is to convert a general $Ax = b$ into a **sequence of simple problems**, which is equivalent to writing A as a **product of simple matrices**.

Along this story line the course developed:

- **Basis change and similarity transformation** as a trick to convert a given matrix into diagonal form. This led to **eigenvalues and eigenvectors** and the concept of being **diagonalizable**. The course saw this trick may fail — some matrices cannot be represented by a diagonal matrix in any basis (not diagonalizable).
- **Complex vectors and projection operations**: projection of a vector onto another vector, and projection of a vector onto a subspace.
- **Special matrices**: diagonal, triangular, permutation, Hermitian, unitary, (orthogonal) projection matrices, Toeplitz, Hankel (in homework), and **circulant** matrices (also in homework, and revisited in detail today).
- **Normal matrices**, the big family. Within normal matrices: **Hermitian** and **unitary**; the normal family also partitions into unitary, Hermitian, **skew-Hermitian**, and others with mixed eigenvalue structure. A normal matrix is precisely a **unitarily diagonalizable** matrix. The search was for a diagonalizable T ; not all matrices are diagonalizable, but within the diagonalizable ones, the **unitarily** diagonalizable ones are the normal matrices, and their eigenvalue locations give the characteristic subsets:
 - eigenvalues on the unit circle $\rightarrow$ **unitary**;
 - eigenvalues real $\rightarrow$ **Hermitian**;
 - eigenvalues on the imaginary axis $\rightarrow$ **skew-Hermitian**.
- **Hermitian sub-classification by eigenvalue sign**. Hermitian matrices are useful for defining real-valued **quadratic functions** of complex vectors. If eigenvalues are all nonneg: **positive semidefinite**; all positive: **positive definite** (defines a **convex** quadratic); all negative: negative definite (defines a **concave** quadratic); mixed: **indefinite** (saddle structure).
- **Inertia and Sylvester's law**. Inertia is the sign pattern (counts of positive/negative/zero eigenvalues). **Star-congruence preserves inertia** (Sylvester's law of inertia): if two Hermitian matrices are star-congruent, they have the same inertia.
- **Matrix square root**. Because every PD matrix has inertia $(n, 0, 0)$, it is star-congruent to the identity, so any PD matrix can be written $A = SS^*$ with invertible S . There are infinitely many such square roots (multiply S by any unitary). Generic notation is used when the specific structure doesn't matter (e.g., random vector generation). Uses include solving PD systems via the lower triangular (Cholesky) square root and, in estimation theory, factoring the observation covariance to build **causal estimators (Wiener filters)** — the causal Wiener filter needs the lower triangular square root because lower triangular matrices are causal linear operators; the non-causal Wiener filter has no causality constraint.

Catalog of Factorizations Seen So Far

The instructor lists the factorizations encountered, all instances of “write a matrix as a product of simple matrices”:

- **PLU**: permutation $\times$ lower triangular $\times$ upper triangular.
 - **Eigenvalue decomposition**: $A = T\Lambda T^{-1}$ — T is not necessarily unitary in general, but for **normal** matrices T becomes a unitary (simple) matrix.
 - **Schur decomposition**: you may not be able to diagonalize a square matrix, but you can **triangularize** it using a unitary matrix: $A = UTU^*$. Still in the “product of simple matrices” form.
 - **Cholesky**: for PD matrices, the LU factorization specializes to Cholesky, a symmetric (lower triangular $\times$ its conjugate transpose) factorization.
 - **QR**: to be started next, with detail.
 - **SVD (Singular Value Decomposition)**: described as “the queen of factorizations” and “the best thing that happened to us.” It writes any matrix using **two** unitary matrices (unlike Schur, which uses one) and a **real, nonneg diagonal** matrix in between, and it applies to **non-square** matrices. Geometric reasoning will be given in later lectures.
-

Recap: Coloring and Whitening (with Student Q&A)

The instructor revisits last lecture’s random-vector application, since it ties matrix square roots to a concrete computation.

Before the construction, he recalls the meaning of a covariance matrix: it is a **library of pairwise covariances** for all pairs of random variables inside a vector. Correlation/covariance is being used to capture the **linear relationship** between two random variables. For a random vector, the covariance matrix collects all such pairwise relationships in one matrix.

Setup. `randn(n, L)` in MATLAB (with a Python counterpart) generates an $n \times L$ matrix that can be viewed as L independent samples of an n -dimensional **uncorrelated** Gaussian vector z with zero mean and identity covariance. By default you get **white** (uncorrelated) vectors.

Coloring. To produce a vector y with a desired covariance C_y , factor $C_y = SS^*$ (any square root — not restricted to PD or lower triangular) and set $y = Sz$.

Why correlation appears. Writing the coloring matrix as A , the first element $y_1 = A_{11}z_1 + A_{12}z_2 + \dots$, the second $y_2 = A_{21}z_1 + A_{22}z_2 + \dots$. Because the y_i are built from the **same** pool of underlying random variables $z_1, z_2, \dots$, it is logical to expect them to become correlated. The instructor stresses this is the intuition for *why* mixing creates correlation.

When mixing does NOT create correlation — student question. A student effectively asks which A keeps the output uncorrelated. Two cases: - If A is **diagonal**, each y_i depends only on z_i , so they remain uncorrelated (trivially). - But there are **non-diagonal** A that still leave y uncorrelated. Compute the correlation of the colored vector:

$$\mathbb{E}[yy^*] = \mathbb{E}[Azz^*A^*] = A\mathbb{E}[zz^*]A^* = AA^*.$$

For this to equal the identity we need $AA^* = I$, i.e., A **unitary**. **Therefore if you use a unitary coloring matrix you are not actually coloring** — despite y_1 and y_2 being built from the same sources, a unitary mixing keeps them uncorrelated. (A student also raised the idea of using a *random* matrix; the instructor set that aside as more complicated and chose to keep A fixed.)

Why the square root (student asked directly). A student said they didn’t understand why the square root is needed. The instructor re-derived it: since the correlation involves y **and** its conjugate transpose,

$$\mathbb{E}[yy^*] = \mathbb{E}[C_y^{1/2}zz^*C_y^{*/2}] = C_y^{1/2}I C_y^{*/2} = C_y.$$

The coloring matrix appears **twice** (once as itself, once conjugate-transposed). That is exactly why you need the *square root* of C_y and not C_y itself.

Final step. Add the desired mean. Up to this point y is zero-mean; you then shift by whatever mean you want.

A new homework covering these random-vector concepts will be assigned **after the midterm**.

LTI Systems from a Linear-Algebra Viewpoint

The instructor now reinterprets undergraduate **Signals and Systems** material through linear algebra, aiming to give intuition. (He notes he shared review notes for the EE 301-type material and recommends Oppenheim & Willsky's book — “a bit thick, but a very good book.”)

Linearity

An **LTI system** has two properties: **linearity** plus **time invariance**.

Linearity is the **superposition** property, which combines: - **Scaling (homogeneity)**: if you scale the input by α , the output scales by α . - **Additivity**: if input x_1 produces output y_1 and input x_2 produces y_2 , then $x_1 + x_2$ produces $y_1 + y_2$ (for a linear, not necessarily time-invariant, system).

Decomposition consequence. Suppose the input can be written as a weighted sum of special basis signals u_k :

$$x = \sum_{k=1}^m a_k u_k.$$

Then by linearity the output is the **same** weighted combination of the individual responses y_k :

$$y = \sum_{k=1}^m a_k y_k,$$

where y_k is the system's response to u_k . The instructor stresses the practical power: **run an experiment** feeding each basis signal u_k into the system and **record** its output y_k ; afterward you no longer need the system. As long as your basis signals are “rich” (any input of interest lies in their span), you can compute the output of any new input from the recorded y_k 's.

Time Invariance

Time invariance means: if you **shift the input in time, the output shifts by the same amount** and otherwise keeps the same shape. The system's behavior does not depend on *when* the input is applied.

Illustration. Apply an input (e.g., a pulse over samples 0 to 1) and get some output shape. Now apply the same input **delayed by 5 samples**. A time-invariant system produces the **same output shape, delayed by 5 samples**. The shape of input and output never changes; the system is not sensitive to the time of application. (Note: time invariance by itself can hold for linear *or* nonlinear systems.)

Finite-Length Signals and Circular Convolution

To stay within linear algebra (avoiding functional analysis), the lecture restricts to **finite-length signals** represented by N -dimensional vectors. (The full Signals and Systems course handles infinite-extent and continuous signals, which require vector-space/functional-analysis machinery.) The instructor says students already saw the Fourier-transform idea in homework; here he wants to give more intuition and formalize why taking the Fourier transform helps.

With input and output both constrained to length N , LTI systems are represented by **circular convolution**. Using r as the sample index to avoid overloading the length symbol,

$$y[r] = \sum_{k=0}^{N-1} h[k] x[(r - k) \bmod N].$$

The $\bmod N$ (modulo- N) operation always returns an index in $\{0, 1, \dots, N - 1\}$, which makes the shift **circular** (a rotation): entries that “fall off” one end **wrap around** to the other end.

The Circular Shift Operator Z

The instructor demonstrates the circular shift on a concrete 4-vector. Starting from

$$h = \begin{bmatrix} 2 \\ -1 \\ 1 \\ -2 \end{bmatrix},$$

a one-step circular (downward) shift moves every entry down by one and the **bottom entry wraps to the top**. This rotation is itself a **linear operation**, represented by a matrix Z that is a **rotated/shuffled identity matrix**:

$$Z = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

For the displayed vector this gives

$$Zh = \begin{bmatrix} -2 \\ 2 \\ -1 \\ 1 \end{bmatrix}.$$

Reading off the rows: row 2 of Z picks row 1’s entry, row 3 picks row 2’s, etc., and the top row picks the bottom entry — exactly the wrap-around. Thus a one-step circular shift is Zh ; a k -step circular shift is $Z^k h$ (because $Z^2 h = Z(Zh)$, and so on). Z is a **permutation matrix**.

Circular Convolution as a Circulant Matrix

Circular convolution is a **weighted sum of circularly shifted copies** of h , weighted by the input entries:

$$y = x[0] h + x[1] Zh + x[2] Z^2 h + \dots + x[N - 1] Z^{N-1} h = \left(\sum_{m=0}^{N-1} x[m] Z^m \right) h.$$

The index of each x entry tells you **how much shift** to apply to h : $x[0]$ multiplies the un-shifted h , $x[1]$ the one-shift Zh , $x[2]$ the two-shift $Z^2 h$, up to $x[N - 1]$ multiplying $Z^{N-1} h$. Equivalently, collecting these shifted copies of h as columns produces the **circulant matrix** H_h , so the output is a matrix-vector product:

$$y = H_h x, \quad H_h = [h \quad Zh \quad Z^2 h \quad \dots \quad Z^{N-1} h].$$

For the downward-shift convention above, the same matrix can be written as a polynomial in the shift matrix,

$$H_h = \sum_{m=0}^{N-1} h[m] Z^m = \begin{bmatrix} h_0 & h_{N-1} & h_{N-2} & \dots & h_1 \\ h_1 & h_0 & h_{N-1} & \dots & h_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_{N-1} & h_{N-2} & h_{N-3} & \dots & h_0 \end{bmatrix}.$$

Because $y = H_h x$, this is a **linear system**; its special time-invariance (with respect to *circular* shifts) forces the **circulant** structure on H_h . Circular convolution is commutative, so the expression $(\sum_m x[m] Z^m) h$ gives the same vector while emphasizing the instructor’s “weighted shifted copies of h ” viewpoint.

Multiplication Count for a General Linear System

How many multiplications does computing $y = Hx$ require for a **general** (non-circulant) $N \times N$ matrix? Each output entry y_i needs N multiplications, and there are N entries, so N^2 **multiplications** (additions are cheaper; multiplications are the costly operations counted). A general, unstructured linear-system output costs N^2 . The point of the rest of the lecture is to **exploit the circulant structure** to do far better — and that exploitation is tied to the Fourier transform.

Complex Exponentials Are Eigenvectors of Circulant Matrices

Consider a **structured input**: a complex exponential. Define the family of vectors f_k (indexed by k) whose r -th entry is

$$(f_k)[r] = e^{j2\pi kr/N}, \quad r = 0, \dots, N-1.$$

where k/N is the **frequency** of the complex exponential. So f_k is the “state vector” at frequency k/N : its entries are $1, e^{j2\pi k/N}, e^{j2\pi k \cdot 2/N}, \dots, e^{j2\pi k(N-1)/N}$.

Key fact. When you pass f_k through the circulant system, you get the **same vector back, scaled by a constant** that depends only on the frequency:

$$Hf_k = \hat{h}_k f_k.$$

Therefore each f_k is an **eigenvector** of H , with eigenvalue $\hat{h}_k$. The instructor frames the discovery as a question to the class — “what would you call this?” — answer: an eigenvector, since the input comes back in the same direction, merely scaled.

The eigenvalue is the DFT of h . The scaling constant is the **inner product** of the impulse response h with the complex exponential vector:

$$\hat{h}_k = \sum_{r=0}^{N-1} h[r] e^{-j2\pi kr/N},$$

i.e., the Fourier transform of h evaluated at frequency k/N under the forward-DFT convention clarified later in the lecture. The minus sign appears because the DFT is F^*h when F is written with positive-exponent columns.

The shift-matrix proof is the cleanest way to see the scaling. Since

$$Zf_k = e^{-j2\pi k/N} f_k,$$

we get

$$H_h f_k = \left(\sum_{m=0}^{N-1} h[m] Z^m \right) f_k = \left(\sum_{m=0}^{N-1} h[m] e^{-j2\pi km/N} \right) f_k.$$

Thus the complex exponential vector comes back unchanged in direction and scaled by the DFT coefficient of h . The instructor does not grind through every row multiplication, but he says this can be verified from the shift structure and connects it to what students already saw in the homework.

Diagonalizing a Circulant Matrix: the DFT Matrix

Stacking the eigenvector equations for all frequencies $k = 0, \dots, N-1$ as columns:

$$H [f_0 \ f_1 \ \cdots \ f_{N-1}] = [f_0 \ f_1 \ \cdots \ f_{N-1}] \Lambda,$$

where multiplying the matrix of eigenvectors **from the right** by a diagonal matrix Λ scales each column by its eigenvalue. Calling the matrix of complex-exponential columns the **Fourier matrix** F (columns f_k) and $\Lambda = \text{diag}(\hat{h}_0, \dots, \hat{h}_{N-1})$:

$$HF = F\Lambda \implies H = F\Lambda F^{-1}.$$

The eigenvalues on the diagonal of Λ are the **DFT of** h evaluated at the N frequencies.

F Is Orthogonal but Not Orthonormal

The columns of F are **orthogonal** to each other. The instructor asks what that says about F^{-1} ; the answer “conjugate transpose” is *almost* right — but you must include a factor of $1/N$ because the columns are orthogonal but **not orthonormal**.

Compute F^*F : a typical entry is the inner product of one complex-exponential column with another. For example, in F^* a row is $1, e^{-j2\pi/N}, \dots, e^{-j2\pi(N-1)/N}$ and in F a column is $1, e^{j2\pi/N}, \dots$. The diagonal inner product gives: $1 \cdot 1 = 1$, then $e^{-j2\pi/N}e^{j2\pi/N} = 1$ (conjugate pairs each give 1), and so on — summing N ones gives N . Off-diagonal inner products sum to 0. Hence

$$F^*F = FF^* = NI.$$

So F is a **scaled unitary** matrix, and

$$F^{-1} = \frac{1}{N}F^*.$$

(The instructor notes the 4-dimensional version of this orthogonality was already verified in homework.)

The Normalized (Unitary) DFT Matrix

The squared norm of each column of F is N , so each column has norm $\sqrt{N}$. Dividing F by $\sqrt{N}$ gives a genuinely **unitary** matrix:

$$\tilde{F} = \frac{1}{\sqrt{N}}F, \quad \tilde{F}^*\tilde{F} = I.$$

Then the circulant matrix diagonalizes as

$$H = \tilde{F}\Lambda\tilde{F}^*,$$

a unitary $\times$ diagonal $\times$ unitary-conjugate decomposition. **Therefore every circulant matrix is unitarily diagonalizable — it is a normal matrix.** But its eigenvalues (the DFT values $\hat{h}_k$) are **not** restricted to the unit circle or the real line; they sit wherever the Fourier transform of h lands. So **circulant (circular convolution) matrices are a special case of normal matrices.**

Why This Matters: the FFT and Fast Algorithms

A Historical Aside (Cold War)

The instructor recommends a Veritasium episode on how the fast computation of the Fourier transform emerged during the **Cold War**. The U.S. wanted to detect whether the Soviets were conducting **underground nuclear tests**, so sensors were placed and the measurements analyzed for characteristic frequencies. Fast Fourier computation was critical, and **Tukey** — who co-developed the fast procedure — was on the U.S. scientific committee involved in this detection effort. (Cooley–Tukey FFT.)

Operation Count Without the FFT

Computing $y = Hx$ directly costs N^2 multiplications. Using the factorization $H = \tilde{F}\Lambda\tilde{F}^*$ (i.e., $H = \frac{1}{N}F\Lambda F^*$) means: 1. Compute the frequency response/eigenvalues $\hat{h}_k$, i.e. the DFT of h : naively N^2 multiplications if done as dense Fourier-matrix multiplication. 2. Multiply x by the Fourier matrix (take the **DFT** of x): naively N^2 multiplications (an $N \times N$ matrix times a vector). Each output entry is an inner product of x

with a complex exponential — i.e., the DFT. 3. Multiply by the **diagonal** Λ : only N multiplications. This is the step that **converts convolution into elementwise multiplication** — the values being multiplied are the coordinates of x in the Fourier (orthogonal) basis, and the diagonal entries are the output's coordinates. 4. Multiply by $\frac{1}{N}F$ (the **inverse DFT**) to return to the original coordinates: another N^2 .

So done naively, the transform route costs about $3N^2 + N$ if $\hat{h}$ is not already available — **more** expensive than the direct N^2 multiply. By itself, changing to the Fourier basis is not a win.

Operation Count With the FFT

The payoff is that the multiplication by F (or F^*) need **not** be done as a dense N^2 matrix-vector product. Tukey's **FFT** exploits the special structure of the Fourier matrix (the **butterfly structure**) to compute the DFT in $O(N \log N)$ instead of $O(N^2)$. The instructor does not derive the butterfly but emphasizes the scale of the saving:

- DFT of h to get $\hat{h}_k$: $N \log N$ (or precompute once if the system is fixed).
- Forward DFT of x : $N \log N$.
- Elementwise/diagonal multiply: N (negligible at this scale).
- Inverse DFT: $N \log N$.

Total $\approx 3N \log N$ (plus negligible terms) for one convolution including the transform of h , or about $2N \log N$ if $\hat{h}$ was precomputed. This replaces N^2 . For $N = 1000$: direct is $\sim$ one million operations; the FFT route is on the order of $1000 \times \log(1000)$ — a few thousand. “A huge saving,” especially in the **big-data era** where N can be in the millions.

The Inverse DFT and the $1/N$ Factor

The $1/N$ in the inverse DFT comes directly from $F^{-1} = \frac{1}{N}F^*$. The inverse transform is

$$x[r] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kr/N}.$$

This is why the inverse Fourier transform carries the $1/N$ factor and the forward one does not.

Displacement Structure (Generalization)

Circulant matrices are one example of structured matrices that admit fast multiplication. The instructor mentions the broader concept of **displacement structure** (there is a book on it): for various structured matrices, you try to convert the N^2 matrix multiplication into a faster $O(N \log N)$ (or other) form by exploiting structure. These are the **fast algorithms**. Regular (unstructured) matrix multiplication needs N^2 ; structured matrices like circulants can be sped up dramatically.

DFT Conventions and Sign of Exponents (Student Q&A)

A student asked about the DFT formula and the placement of minus signs. The instructor clarifies the bookkeeping:

- Writing F^*x gives the **DFT** (forward); writing $\frac{1}{N}F X$ gives the **inverse DFT**. Since F is not unitary, its inverse is $\frac{1}{N}F^*$, not just F^* .
- In the F matrix as written, the entries carry **no minus sign** in the exponent. In F^* (conjugate), the columns become rows and the exponents pick up **minus signs**. So when you multiply F^*x you get terms $\sum_{r=0}^{N-1} x[r]e^{-j2\pi kr/N}$ — the familiar forward DFT with the **minus sign**. The Fx version has $e^{+j2\pi kr/N}$.
- The summation $1/N$ factor in the inverse appears because F is **scaled unitary**, not unitary.

Symmetric (Normalized) Convention

If instead you use the **normalized** basis vectors (dividing each by $\sqrt{N}$), then the matrix is unitary and its conjugate is its inverse, making the forward and inverse transforms **symmetric** in their scaling:

$$\tilde{X}[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N-1} x[r] e^{-j2\pi kr/N}, \quad x[r] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \tilde{X}[k] e^{+j2\pi kr/N}.$$

The instructor calls $\tilde{X}$ the **normalized Fourier transform**. The choice is “not a big deal,” but it explains why the asymmetric convention has a $1/N$ in the inverse and nothing in the forward.

Preview: QR Factorization

The instructor closes with one sentence on what comes next. **QR factorization** writes a matrix as a **unitary matrix times an upper triangular matrix**:

$$A = QR,$$

with Q unitary and R upper triangular — both simple matrices. He muses that he doesn’t know why it’s called “Q” (perhaps Q for unitary?) and jokes that maybe it should be called “**UT factorization**” (unitary–triangular). The claim, to be proved next: **any square matrix can be written as a unitary matrix times an upper triangular matrix**, and there are different ways to obtain it. “We will meet in the evening. See you tonight.”

Instructor Remarks and Study Guidance

- The course is two intertwined story lines: **analyzing** $Ax = b$ (vector spaces, four fundamental subspaces, existence/uniqueness from shape + rank) and **solving** $Ax = b$ (factor A into simple matrices).
- All factorizations (PLU, eigendecomposition, Schur, Cholesky, QR, SVD) are instances of “write A as a product of simple matrices.” SVD is the “queen” — two unitaries and a real nonneg diagonal, works for any matrix.
- A covariance matrix is a library of pairwise covariance/correlation information; coloring starts from white Gaussian samples and uses a square root of the target covariance.
- **A unitary coloring matrix does no coloring:** $AA^* = I$ keeps the output white. The square root appears because the covariance computation puts the coloring matrix on both sides ($\mathbb{E}[yy^*] = SS^*$).
- **Complex exponentials are eigenvectors of circulant matrices;** the eigenvalues are the DFT of the impulse response. Hence circulant matrices are normal and unitarily diagonalized by the (normalized) DFT matrix.
- The (unnormalized) Fourier matrix satisfies $F^*F = NI$: scaled unitary, so $F^{-1} = \frac{1}{N}F^*$. This is the origin of the $1/N$ in the inverse DFT.
- The FFT reduces the DFT cost from N^2 to $O(N \log N)$ via the butterfly structure; including the transform of h , the circular convolution workflow is about $3N \log N$, or about $2N \log N$ if the frequency response is precomputed.
- A new random-vector homework comes after the midterm; reread the EE 301 / Oppenheim–Willsky background if Signals and Systems is rusty.

Source and Coverage Note

Source: `corrected/lecture16_corrected.md`.

Coverage: Opening reflection; full chronological mid-course review of both story lines (vector spaces, span, linear independence, basis/dimension, Euclidean norm and inner product, Cauchy-Schwarz, orthogonality, hyperplanes/half-spaces, four fundamental subspaces, rank, existence/uniqueness via shape + rank; easy $Ax =$

b cases, factorization strategy, basis change/similarity, eigenvalues, diagonalizability, special matrices, normal matrices and their subclasses, Hermitian sign classes, inertia, Sylvester's law, matrix square roots, catalog of factorizations including SVD preview); covariance as a library of pairwise relationships; coloring/whitening recap with the unitary-matrix and square-root student Q&A; LTI systems (linearity/superposition, basis-signal decomposition, time-invariance illustration); finite-length signals and circular convolution; the circular shift operator Z as a permutation matrix with the worked 4-vector example and shifted vector; circulant matrix $H_h = [h \ Zh \ \cdots \ Z^{N-1}h] = \sum h[m]Z^m$, its explicit row structure, and the N^2 multiplication count; complex exponentials as eigenvectors with eigenvalue = forward DFT of h , including the Zf_k proof step; diagonalization $H = F\Lambda F^{-1}$; $F^*F = NI$, scaled-unitary nature, normalized unitary DFT matrix, circulant matrices as normal matrices; FFT motivation (Cold War / Tukey), operation-count comparison N^2 vs $O(N \log N)$, including the DFT of h , displacement structure; DFT sign/convention student Q&A and the symmetric normalized convention; QR preview.

Lecture 17 Notes

Opening: Where We Are in the Factorization Story

The instructor reminds the class that the current theme is **matrix factorization**, the major tool for solving systems of linear equations by writing a matrix as a product of simple matrices. So far the course has covered:

- **PLU** (permutation $\times$ lower triangular $\times$ upper triangular),
- **Eigenvalue decomposition**,
- **Schur factorization**,
- **Cholesky factorization**, and
- the general **square-root factorization** for positive definite matrices.

Now the focus is **QR factorization**: writing a matrix as a product of a matrix with **orthonormal columns** (Q) and an **upper triangular** matrix (R):

$$A = QR.$$

The instructor again jokes about the naming — he doesn't know whether “QR” comes from someone's initials — and notes that since Q is unitary and R is upper triangular, “UT factorization” would be a more descriptive name.

Reduced vs. Full QR

- **Reduced (economical) QR**: $\hat{Q}$ has the **same size** as A (an $m \times n$ matrix with orthonormal columns) and $\hat{R}$ is an $n \times n$ **square** upper triangular matrix.
- **Full QR**: add extra columns to $\hat{Q}$ to make it a genuine **unitary** $m \times m$ matrix Q . Then R becomes $m \times n$ — a **rectangular** upper triangular matrix whose entries below the main diagonal are zero (the bottom rows are all zero). The added columns of Q are “silenced” by the zero rows of R .

Why QR matters: it is numerically stable for solving least squares because it operates on A directly without forming A^*A (which would square the condition number). MATLAB's backslash uses QR/SVD-based methods.

Review: Gram-Schmidt Process

The instructor recaps the Gram-Schmidt construction from the previous lecture.

Problem Statement

View the columns $a_1, \dots, a_n$ of A as spanning some vector space. Gram-Schmidt finds an **orthonormal basis** $q_1, \dots, q_n$ for that space — but with a **causal constraint**:

$$\text{span}\{q_1, \dots, q_k\} = \text{span}\{a_1, \dots, a_k\}, \quad k = 1, \dots, n.$$

This “causality” (each q_k is built only from $a_1, \dots, a_k$) is exactly what produces the **upper triangular** R : “all the theory is reflected in the triangular structure.”

Step-by-Step

Step 1. $q_1 = a_1 / \|a_1\|$ — just normalize the first column; the span constraint is trivial.

Step 2. Find the component of a_2 orthogonal to q_1 by orthogonal projection:

$$\tilde{a}_2 = a_2 - (q_1^* a_2) q_1, \quad q_2 = \frac{\tilde{a}_2}{\|\tilde{a}_2\|}.$$

The subtracted term is the projection of a_2 onto q_1 ; the leftover error vector is orthogonal to q_1 (because it is an orthogonal projection), and we normalize it.

Step 3. Find the component of a_3 orthogonal to **both** previous directions:

$$\tilde{a}_3 = a_3 - (q_1^* a_3) q_1 - (q_2^* a_3) q_2, \quad q_3 = \frac{\tilde{a}_3}{\|\tilde{a}_3\|}.$$

Because a_1, a_2 span the same space as q_1, q_2 , removing the q_1, q_2 components is the same as removing the a_1, a_2 components.

General step k :

$$\tilde{a}_k = a_k - \sum_{i=1}^{k-1} (q_i^* a_k) q_i, \quad q_k = \frac{\tilde{a}_k}{\|\tilde{a}_k\|}.$$

Cross-product analogy. The instructor recalls the 3-D picture: given two vectors, the cross product gives a vector orthogonal to the plane they span. Gram-Schmidt does the analogous thing in **high dimensions** — find the component of a new vector orthogonal to the span of the previous ones. The cross product is the special low-dimensional case.

Why R Is Upper Triangular

Inverting the construction, each a_k is a combination of only the first k orthonormal vectors:

$$a_k = \sum_{i=1}^k (q_i^* a_k) q_i.$$

Defining $R_{ik} = q_i^* a_k$ for $i \leq k$ and $R_{ik} = 0$ for $i > k$ gives $A = QR$ with R upper triangular. The diagonal entries $R_{kk} = \|\tilde{a}_k\|$ (the norms before normalization) are positive when the columns are linearly independent.

Linear Dependence Case

If at step k the vector a_k lies in the span of $a_1, \dots, a_{k-1}$, it carries **no new dimension / no new information** (the instructor adds: “not in the Shannon sense,” but a genuine linear relationship). After subtracting projections, $\tilde{a}_k = 0$. The number of q -vectors produced is then $k' = \text{rank}(A) < n$. To complete a **full** unitary QR, add $m - k'$ extra orthonormal columns to Q (orthogonal to all the constructed q 's) and append corresponding **zero rows** to R to silence them. The instructor recalls having worked an explicit example of this in the previous lecture and does not repeat it.

Modified Gram-Schmidt (Triangular Orthogonalization)

The next step is the **Modified Gram-Schmidt (MGS)** procedure — “a variation on Gram-Schmidt.” The mathematical result is identical in exact arithmetic, but the **order of operations** differs, which matters numerically.

The Key Difference

Classical Gram-Schmidt: to find q_k , project the *original* a_k onto each previous q_i and subtract all at once.

Modified Gram-Schmidt: as soon as q_1 is generated, **immediately subtract its component from ALL remaining vectors**, so that every other vector becomes orthogonal to q_1 right away:

$$q_1 = \frac{a_1}{\|a_1\|}, \quad a_j^{(1)} = a_j - (q_1^* a_j) q_1 \quad \text{for all } j = 2, \dots, n.$$

After this first pass, all vectors are orthogonal to q_1 (but not yet to each other). Then take $q_2 = a_2^{(1)} / \|a_2^{(1)}\|$ (already orthogonal to q_1), and subtract the q_2 -component from all remaining **updated** vectors:

$$a_j^{(2)} = a_j^{(1)} - (q_2^* a_j^{(1)}) q_2 \quad \text{for all } j = 3, \dots, n,$$

and continue. In words: **once you generate an orthonormal basis element, you kill the component of all remaining vectors in that direction immediately**, then move on. You progressively build the orthonormal basis.

Matrix Interpretation

Each MGS pass is a **right-multiplication of A by an upper triangular matrix**. For example, the first pass multiplies A by an upper triangular R_1 whose first column is $[1/\|a_1\|, 0, \dots]^\top$ (normalizing column 1) and whose later columns encode “subtract the q_1 -projection.” For three columns, the instructor’s board calculation is:

$$[a_1 \ a_2 \ a_3] \begin{bmatrix} 1/\|a_1\| & -\frac{a_1^* a_2}{a_1^* a_1} & -\frac{a_1^* a_3}{a_1^* a_1} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [q_1 \ a_2^{(1)} \ a_3^{(1)}],$$

where $a_j^{(1)}$ means “the q_1 -component has been removed.” The second and third columns keep their original vector with coefficient 1, then subtract a multiple of a_1 ; entries below the diagonal are zero because earlier columns are the only ones used.

The second pass leaves q_1 untouched, normalizes $a_2^{(1)}$, and removes only the q_2 -direction from the remaining updated columns:

$$[q_1 \ a_2^{(1)} \ a_3^{(1)}] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1/\|a_2^{(1)}\| & -\frac{(a_2^{(1)})^* a_3^{(1)}}{(a_2^{(1)})^* a_2^{(1)}} \\ 0 & 0 & 1 \end{bmatrix} = [q_1 \ q_2 \ a_3^{(2)}].$$

The instructor stresses that there is “no business with q_1 ” in this later projection: the q_1 -component was already eliminated. After this pass, $a_3^{(2)}$ is orthogonal to the span of the first two directions but is not yet unit norm; the next pass normalizes it and removes its direction from any remaining columns.

Because each step only touches components of vectors at or before the current index, the multiplying matrix is upper triangular. Successive passes give

$$AR_1 R_2 \cdots = \hat{Q} \implies A = \hat{Q} (\cdots R_2^{-1} R_1^{-1}) = \hat{Q} \hat{R},$$

and since inverses and products of upper triangular matrices are upper triangular, $\hat{R}$ is upper triangular.

The instructor’s name for this: triangular orthogonalization — *use triangular matrices to convert a given matrix into orthogonal form*. The matrix A is multiplied from the right by triangular matrices until its columns are orthonormal.

Why MGS Is Numerically Better

In floating point, classical Gram-Schmidt reuses the original a_k and accumulates rounding errors. MGS always subtracts from an **already partially orthogonalized** vector $a_j^{(i)}$ (with $q_1, \dots, q_i$ components already removed), so each subtraction acts on a “cleaner” vector and rounding errors do not compound as badly.

Householder Reflections (Orthogonal Triangularization)

The instructor introduces the **dual** of triangular orthogonalization:

- **MGS / triangular orthogonalization:** multiply A from the **right** by upper triangular matrices to obtain an orthogonal matrix.
- **Householder / orthogonal triangularization:** multiply A from the **left** by **unitary** matrices to obtain an upper triangular matrix.

He draws the analogy to LU: in Gaussian elimination (LU) we multiply A from the left by **lower triangular** matrices to make it upper triangular. Here we instead multiply from the left by **unitary** matrices to make it upper triangular. So LU and QR are “similar” in that both have an upper triangular right factor; QR’s other factor is unitary instead of lower triangular.

Goal of Each Step

Triangularize column by column, zeroing entries **below the diagonal**. Step 1: find a unitary Q'_1 that maps the first column a_1 to a multiple of e_1 :

$$Q'_1 a_1 = r_{11} e_1 = \begin{bmatrix} r_{11} \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Since Q'_1 is unitary, it **preserves the Euclidean norm**, so

$$|r_{11}| = \|a_1\|_2.$$

We know the **target vector** exactly: $\|a_1\| e_1$ (or $-\|a_1\| e_1$ — either sign is allowed, “it’s up to me, nothing harmful”).

Rotation vs. Reflection

Geometrically (2-D toy picture) we want to move a_1 onto the first coordinate axis while preserving its length. Two kinds of length-preserving maps do this: - a **rotation** (Givens rotation), which involves cosines and sines, or - a **reflection** across a hyperplane (the **Householder** form).

The instructor chooses the **reflection** because it is **simpler** (the rotation/Givens approach with cosines and sines is “left to homework or exam”). He defines a hyperplane and reflects a_1 onto the target $r_{11}e_1$; this is the **Householder form**.

Constructing the Reflection

Define the **direction vector** (the difference between a_1 and its target):

$$z = a_1 - r_{11} e_1, \quad r_{11} = \|a_1\|_2,$$

which is the **normal of the reflection hyperplane**. The geometric reasoning the instructor walks through:

1. Project a_1 onto z : the projection coefficient is $\frac{z^* a_1}{z^* z}$.
2. Subtracting **once** lands you on the hyperplane (the foot of the projection).
3. To reach the **reflection** (the mirror image), you must move **twice** as far in the $-z$ direction.

A student questioned the “twice” step; the instructor clarified there is an implicit factor of z : the scalar $\frac{z^* a_1}{z^* z}$ is just the **reflection coefficient**, which then multiplies the vector z . (“I’m more excited about the reflection coefficient than the reflection itself.”) Carrying this out:

$$Q'_1 a_1 = a_1 - 2 \frac{z z^* a_1}{z^* z} = \left(I - 2 \frac{z z^*}{z^* z} \right) a_1.$$

So the **Householder reflection matrix** is

$$H = I - 2 \frac{z z^*}{z^* z}, \quad z = a_1 - \|a_1\| e_1.$$

Properties: - **Hermitian:** $H^* = H$. - **Unitary:** $H^* H = I$ — reflecting twice returns the identity. - Maps a_1 onto $\|a_1\| e_1$ (by construction).

Applying Householder Iteratively (Shrinking Columns)

After step 1, $Q'_1 A$ has first column $r_{11}e_1$: entries below the diagonal in column 1 are zero, and the rest is

$$Q'_1 A = \begin{bmatrix} r_{11} & * & \cdots \\ 0 & & \\ \vdots & & \tilde{A} \\ 0 & & \end{bmatrix}.$$

Now focus on the second column **within the $(m-1)$ -dimensional subspace** (ignoring the already-finished first row/column). Build an $(m-1) \times (m-1)$ Householder reflection Q'_2 that sends that shorter vector to its own $\|\cdot\|e_1$, and embed it as a **block-diagonal** unitary

$$Q'_2 = \begin{bmatrix} 1 & 0 \\ 0 & Q''_2 \end{bmatrix},$$

(identity in the top-left, Householder in the bottom-right). This is unitary and zeros the entries below the diagonal in column 2 without disturbing column 1. Continue for columns 3, 4, ..., each time working in a one-dimension-smaller space:

$$Q'_k \cdots Q'_2 Q'_1 A = R.$$

Since each Q'_i is unitary, its inverse is its conjugate transpose (very simple), so

$$A = (Q'_1)^* (Q'_2)^* \cdots (Q'_k)^* R = QR,$$

with $Q = (Q'_1)^* \cdots (Q'_k)^*$ unitary. (The instructor notes the sign choice for r_{11} corresponds to two possible reflection hyperplanes; in practice the sign is chosen to keep z large and avoid numerical cancellation.)

Gram Matrix (Student Question)

A student asks about the **Gram matrix**. Given a set of vectors $p_1, \dots, p_n$, the Gram matrix collects **all pairwise inner products**:

$$G_{ij} = \langle p_i, p_j \rangle = p_i^* p_j.$$

Putting the vectors as columns of $P = [p_1 \cdots p_n]$,

$$G = P^* P.$$

Properties: - **Hermitian:** $G^* = P^* P = G$. - **Positive semidefinite:** $x^* G x = \|Px\|^2 \geq 0$. - **Positive definite** iff the vectors are linearly independent (iff P has full column rank).

Connection to least squares / projection. The instructor reminds students that the projection onto the range of a general (non-orthonormal) A used the inverse $(A^* A)^{-1}$ — and $A^* A$ is exactly the **Gram matrix** of the columns of A . This will reappear in the least squares normal equations.

Recap of the Two Dual Procedures

- **Triangular orthogonalization** (Gram-Schmidt / MGS): right-multiply A by triangular matrices to orthogonalize it.
- **Orthogonal triangularization** (Householder): left-multiply A by unitary matrices to triangularize it.

Both yield the same $A = QR$. All of this still revolves around the very first lecture's question: *what are the simple matrices?* (diagonal, triangular, orthogonal) — and the goal of writing A in terms of them. “But the story is not over.”

Property	Classical GS	Modified GS	Householder
Mechanism	right $\times$ triangular	right $\times$ triangular (reordered)	left $\times$ unitary
Produces directly	orthonormal columns Q	orthonormal columns Q	upper triangular R
Conceptual name	—	triangular orthogonalization	orthogonal triangularization
Numerical stability	least stable	better	most stable

Preview: Polar Decomposition

The instructor previews an upcoming factorization (to be developed with the SVD): the **polar decomposition**, motivated by the **polar form of a complex number**.

A nonzero complex number c can be written

$$c = r e^{j\theta},$$

where $r = |c| > 0$ is the magnitude (distance to the origin) and $e^{j\theta}$ is a unit-magnitude complex number on the unit circle (with $\cos \theta, \sin \theta$ the real and imaginary parts). The matrix generalization: any **non-singular** $n \times n$ matrix A can be written

$$A = PT,$$

where P is a **positive definite** matrix (the analog of the positive magnitude r) and T is a **unitary** matrix (the analog of $e^{j\theta}$ on the unit circle). This is the **polar decomposition**.

The instructor notes PT is not quite a “simple-matrix” factorization, but since a positive definite matrix is itself normal (= unitary $\times$ diagonal $\times$ unitary), the polar decomposition connects to the SVD. It has applications such as **finding the closest unitary matrix** to a given matrix. (A brief exchange about left/right inverses: since A here is square and invertible, its inverse is unique — the left-inverse notion was for tall full-rank matrices, not relevant for this invertible square case.)

Beginning of Block LDU Factorization (Carried into L18)

With time remaining, the instructor begins the next topic, warning it is “**a bit algebraically involved but very useful**” — it underlies major algorithms in **adaptive filtering, optimization, and control**. The topic is **block LDU factorization**: lower–diagonal–upper triangular factorization, but in terms of **block** submatrices.

Block Partition

Partition a non-singular square matrix A of size $(m+n) \times (m+n)$ into blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

where A_{11} is $m \times m$, A_{12} is $m \times n$, A_{21} is $n \times m$, and A_{22} is $n \times n$. The full A , A_{11} , and A_{22} are assumed non-singular. The goal: perform **Gaussian elimination at the block level**.

Block Elimination

To convert A into **block upper triangular** form, eliminate the A_{21} block by left-multiplying by a **block lower triangular** matrix:

$$\begin{bmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}.$$

The first block row is preserved (multiplied by identity). The $(2, 1)$ block becomes $-A_{21}A_{11}^{-1}A_{11} + A_{21} = 0$, and the $(2, 2)$ block becomes

$$A_{22} - A_{21}A_{11}^{-1}A_{12},$$

the **Schur complement** (of A_{11}). The instructor gives a memory aid for reconstructing the formula: start at index **2-2**, then go **2-1**, then invert **1-1**, then **1-2** — i.e., $A_{22} - A_{21}A_{11}^{-1}A_{12}$.

The next step (eliminating A_{12} by right-multiplying with a block upper triangular matrix to reach block **diagonal** form) is deferred to the next lecture. “Please look at this so that we can pick up from here.”

Instructor Remarks and Study Guidance

- The **causal/span constraint** $\text{span}\{q_1, \dots, q_k\} = \text{span}\{a_1, \dots, a_k\}$ is exactly what forces R to be upper triangular — the central structural fact of QR.
- **MGS vs. classical GS**: identical in exact arithmetic; MGS subtracts each new q -component from *all* remaining vectors immediately, which is more stable. The matrix view is **triangular orthogonalization** (right-multiply by triangular matrices).
- **Householder is the dual** (orthogonal triangularization, left-multiply by unitaries). The reflector $H = I - 2zz^*/z^*z$ with $z = a_1 - \|a_1\|e_1$ is Hermitian and unitary, and reflects a_1 onto $\|a_1\|e_1$. It is the most numerically stable QR method. The **Givens rotation** (cosine/sine) is the alternative, left to homework.
- The reflection requires moving **twice** the projection distance along z — that factor of 2 is the source of the 2 in H .
- The **Gram matrix** $G = P^*P$ is Hermitian PSD (PD iff columns independent) and is exactly the A^*A that appears in least squares / projection.
- **Polar decomposition** $A = PT$ (PD $\times$ unitary) is the matrix analog of $c = re^{j\theta}$; developed alongside the SVD.
- **Block LDU** begins here and continues in L18; the $(2, 2)$ block after block elimination is the **Schur complement** $A_{22} - A_{21}A_{11}^{-1}A_{12}$.

Source and Coverage Note

Source: `corrected/lecture17_corrected.md`.

Coverage: QR overview (reduced and full, naming aside, why QR matters); Gram-Schmidt review (causal/span constraint, step-by-step, cross-product analogy, upper triangular R , diagonal entries, linear-dependence case and completion to full unitary QR); Modified Gram-Schmidt (immediate elimination of each new direction from all remaining vectors, explicit first- and second-pass right-multiplication matrices, matrix interpretation as triangular orthogonalization via upper triangular matrices, numerical-stability reasoning); Householder reflections (dual orthogonal triangularization, left-multiplication by unitaries, LU analogy, norm preservation and target vector, rotation vs. reflection choice, full geometric derivation of the reflection coefficient and the factor of 2, the reflector $H = I - 2zz^*/z^*z$, Hermitian and unitary properties, iterative shrinking-column application via block-diagonal embedding, sign choice); Gram matrix (definition, P^*P , Hermitian/PSD/PD, connection to least squares); recap of the two dual procedures and comparison table; polar decomposition preview (complex-number analogy, $A = PT$, closest-unitary application, left/right-inverse aside); beginning of block LDU factorization (block partition with block sizes, block lower triangular elimination, Schur complement and its memory aid, deferral of the right-elimination step to L18).

Lecture 18 Notes

Opening: Continuing Block LDU

The lecture resumes the block matrix factorization started at the end of L17. The instructor stresses this material “leads to a key identity in many signal processing algorithms” and asks for patience: “after some algebra we will be all happy.” The plan: take a block-partitioned invertible matrix, perform **block Gaussian elimination** (block LDU), extract the **Schur complement**, the **determinant** and **inverse** formulas, derive the **Matrix Inversion Lemma (Woodbury identity)**, apply it to **recursive least squares**, and finally specialize to **Hermitian** matrices to connect to **inertia**, **positive definiteness**, and **linear matrix inequalities**.

Setup

Partition an invertible matrix A into four blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

with A_{11} of size $m \times m$, A_{22} of size $n \times n$. Assume A is invertible and A_{11} (and A_{22}) invertible. The goal is to convert A into block **diagonal** form using a block row operation on the left and a block column operation on the right.

Block LDU Factorization

Step 1 — Eliminate the Lower-Left Block (block row operation)

To make the $(2, 1)$ block zero **without touching the first block row**, left-multiply by a block **lower triangular** matrix:

$$\begin{bmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}.$$

The first block row is preserved (multiplied by I); the new $(2, 1)$ block is $-A_{21}A_{11}^{-1}A_{11} + A_{21} = 0$; and the new $(2, 2)$ block picks up the correction $-A_{21}A_{11}^{-1}A_{12}$. This is exactly Gaussian elimination at the block level — “multiply from the left by a lower triangular matrix to get an upper triangular form.”

Step 2 — Eliminate the Upper-Right Block (block column operation)

Now right-multiply the block upper triangular result by a block **upper triangular** matrix to clear the $(1, 2)$ block, **without touching the first block column**:

$$\begin{bmatrix} A_{11} & A_{12} \\ 0 & S_{11} \end{bmatrix} \begin{bmatrix} I & -A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix} = \begin{bmatrix} A_{11} & 0 \\ 0 & S_{11} \end{bmatrix},$$

where $S_{11} = A_{22} - A_{21}A_{11}^{-1}A_{12}$. The first block column is preserved; the $(1, 2)$ block becomes $-A_{11}A_{11}^{-1}A_{12} + A_{12} = 0$; and the $(2, 2)$ block is unaffected because it is multiplied by the zero block. We have reached **block diagonal** form by sandwiching A between a block lower triangular matrix (left) and a block upper triangular matrix (right).

Reconstruction

The triangular factors are easy to invert — just **flip the sign** of the off-diagonal block (you can verify this). Moving them to the right side:

$$A = \underbrace{\begin{bmatrix} I & 0 \\ A_{21}A_{11}^{-1} & I \end{bmatrix}}_{\text{block } L} \underbrace{\begin{bmatrix} A_{11} & 0 \\ 0 & S_{11} \end{bmatrix}}_{\text{block } D} \underbrace{\begin{bmatrix} I & A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix}}_{\text{block } U}.$$

This is the **block LDU factorization** — “diagonalization à la Gaussian elimination, applied at the block level.” The instructor cautions to be careful about block **sizes and order**, since matrix multiplication is not commutative.

The Schur Complement of A_{11}

The crucial new quantity is the **Schur complement** of A_{11} :

$$S_{11} = A_{22} - A_{21}A_{11}^{-1}A_{12}.$$

It is the “other corner” A_{22} with a correction subtracted off. **Memory aid** (the instructor’s): start at index **2-2**, transition to **2-1**, take the inverse of **1-1**, transition back from **1 to 2** (i.e., A_{12}) — giving $A_{22} - A_{21}A_{11}^{-1}A_{12}$.

Determinant Formula (Application 1)

Because $\det(\text{product}) = \prod \det$, and the block triangular factors have determinant 1 (triangular with identity diagonal blocks), while the block diagonal factor has determinant equal to the product of its diagonal blocks’ determinants:

$$\det(A) = \det(A_{11}) \cdot \det(S_{11}).$$

Intuition. If A were genuinely **block diagonal**, $\det(A) = \det(A_{11}) \det(A_{22})$. For a **non-diagonal** block matrix it is *not* $\det(A_{11}) \det(A_{22})$ but $\det(A_{11})$ times the determinant of the **Schur complement** of A_{11} . The dual (block UDL) gives $\det(A) = \det(A_{22}) \det(S_{22})$ with $S_{22} = A_{11} - A_{12}A_{22}^{-1}A_{21}$.

Scalar check. For $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the Schur complement of a is $d - ca^{-1}b$, and $a(d - ca^{-1}b) = ad - bc$ — the familiar 2×2 determinant.

Matrix Inverse via Block Factorization (Application 2)

Once $A = LDU$ (block), the inverse is easy because **each factor is easy to invert** — invert in **reverse order**:

$$A^{-1} = U^{-1}D^{-1}L^{-1}.$$

The block diagonal D inverts by inverting each diagonal block; the triangular factors invert by negating their off-diagonal block. Expanding gives the explicit **block inverse (from LDU)**:

$$A^{-1} = \begin{bmatrix} A_{11}^{-1} + A_{11}^{-1}A_{12}S_{11}^{-1}A_{21}A_{11}^{-1} & -A_{11}^{-1}A_{12}S_{11}^{-1} \\ -S_{11}^{-1}A_{21}A_{11}^{-1} & S_{11}^{-1} \end{bmatrix}.$$

Dual: Block UDL

Reversing the roles (eliminate A_{12} first using A_{22} as the pivot) yields a **block UDL** factorization with the **Schur complement of A_{22}** :

$$S_{22} = A_{11} - A_{12}A_{22}^{-1}A_{21},$$

the determinant $\det(A) = \det(A_{22}) \det(S_{22})$, and a second **block inverse (from UDL)**:

$$A^{-1} = \begin{bmatrix} S_{22}^{-1} & -S_{22}^{-1}A_{12}A_{22}^{-1} \\ -A_{22}^{-1}A_{21}S_{22}^{-1} & A_{22}^{-1} + A_{22}^{-1}A_{21}S_{22}^{-1}A_{12}A_{22}^{-1} \end{bmatrix}.$$

We now have **two equal but different-looking expressions** for A^{-1} .

Matrix Inversion Lemma (Woodbury Identity)

Equating the two alternative inverses (in particular their $(1, 1)$ blocks) and doing the algebra (which the instructor does not write out in full) yields a fundamental identity, called the **matrix inversion lemma** or **Woodbury identity**:

$$(A + BCD)^{-1} = A^{-1} - A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1}.$$

It inverts “a matrix plus a product of three matrices.” The result looks complicated but is enormously useful.

Instructor emphasis: “This is the only equation that I want you to memorize in this course.” It underlies adaptive signal processing, Bayesian estimation, and control.

Key use case. When A is large with a **known** inverse and BCD is a **low-rank** perturbation: instead of re-inverting the full $n \times n$ matrix, you invert only the small $(C^{-1} + DA^{-1}B)$. For a **rank-one** update (B a column, C a scalar, D a row), the only inverse needed is of a **scalar**.

Application: Sample Autocorrelation and Recursive Least Squares (RLS)

From Theory to Data

In estimation theory, optimal estimators are often built from the **autocorrelation matrix** $R_h = \mathbb{E}[hh^*]$ (the library of pairwise correlations among the entries of a random vector h). But in real applications **nobody hands you** R_h — you must estimate it from data.

Sample autocorrelation (analogy to the sample mean). Just as the mean $\mathbb{E}[x]$ is estimated from samples by averaging ($\frac{1}{N} \sum x_i$), the autocorrelation is estimated by averaging outer products of the observed sample vectors $h_1, \dots, h_N$:

$$\hat{R}^{(N)} = \frac{1}{N} \sum_{i=1}^N h_i h_i^*.$$

The instructor explicitly draws the parallel: we don’t have the expectation, but we have samples, so we average the $h_i h_i^*$ “auto-products.”

The Online Update Problem

In **online** applications data keeps arriving. When sample h_{N+1} appears:

$$\hat{R}^{(N+1)} = \frac{1}{N+1} \sum_{i=1}^{N+1} h_i h_i^* = \frac{N}{N+1} \hat{R}^{(N)} + \frac{1}{N+1} h_{N+1} h_{N+1}^*.$$

This is a **rank-one update** — adding the rank-one outer product $h_{N+1} h_{N+1}^*$ to a scaled copy of the previous estimate, so you reuse $\hat{R}^{(N)}$ instead of re-summing from scratch.

But many estimators need the **inverse** $\hat{R}^{-1}$. Re-inverting a (say) 1000×1000 matrix at every step costs $\sim n^3 \approx 10^9$ operations per sample — impractical for real-time use. The instructor’s idea: **update the inverse directly** rather than the matrix.

Applying Woodbury

The instructor rewrites the update by pulling out the scalar factor:

$$\hat{R}^{(N+1)} = \frac{N}{N+1} \left[\hat{R}^{(N)} + \frac{1}{N} h_{N+1} h_{N+1}^* \right].$$

Inside the brackets this is exactly the Woodbury form $A + BCD$ with

$$A = \hat{R}^{(N)}, \quad B = h_{N+1}, \quad C = \frac{1}{N} \text{ (scalar)}, \quad D = h_{N+1}^*.$$

Woodbury gives $(\hat{R}^{(N+1)})^{-1}$ entirely in terms of the **previous inverse** $(\hat{R}^{(N)})^{-1}$ and the new vector h_{N+1} :

$$(\hat{R}^{(N+1)})^{-1} = \frac{N+1}{N} \left[(\hat{R}^{(N)})^{-1} - \frac{(\hat{R}^{(N)})^{-1} h_{N+1} h_{N+1}^* (\hat{R}^{(N)})^{-1}}{N + h_{N+1}^* (\hat{R}^{(N)})^{-1} h_{N+1}} \right].$$

The denominator comes from $C^{-1} + DA^{-1}B = N + h_{N+1}^* (\hat{R}^{(N)})^{-1} h_{N+1}$, and it is a **scalar** (row $\times$ matrix $\times$ column = 1×1), so the only “inversion” is a scalar division. Everything else is matrix-vector products costing $O(n^2)$. **Net: $O(n^2)$ per update instead of $O(n^3)$.** This is the **Recursive Least Squares (RLS)** algorithm, central to adaptive filtering (adaptive equalizers, noise cancellation). The instructor notes RLS will be revisited when least squares is covered.

Hermitian Case: Star-Congruence and Inertia

Now specialize: let A be **Hermitian**. Then A_{11} and A_{22} are Hermitian and $A_{21} = A_{12}^*$. The same block LDU still applies (it works for any 2×2 block partition); the only change is $A_{21} \rightarrow A_{12}^*$. Crucially, the **left and right triangular factors are now conjugate transposes of each other**:

$$A = S \begin{bmatrix} A_{11} & 0 \\ 0 & S_{11} \end{bmatrix} S^*, \quad S = \begin{bmatrix} I & 0 \\ A_{12}^* A_{11}^{-1} & I \end{bmatrix}.$$

Since S is invertible, this is a **star-congruence** between A and the block diagonal matrix.

Inertia Additivity

By **Sylvester’s law of inertia** (star-congruence preserves inertia), and because the inertia of a block diagonal matrix is the **sum** of the inertias of its diagonal blocks:

$$\text{inertia}(A) = \text{inertia}(A_{11}) + \text{inertia}(S_{11}).$$

The number of positive eigenvalues of A equals the positive count of A_{11} plus that of S_{11} , and likewise for negative and zero counts. The dual UDL form gives $\text{inertia}(A) = \text{inertia}(A_{22}) + \text{inertia}(S_{22})$. The instructor calls this “very useful information in many control and estimation problems.”

Positive Definiteness Criterion

$$A \succ 0 \iff A_{11} \succ 0 \text{ and } S_{11} = A_{22} - A_{21} A_{11}^{-1} A_{12} \succ 0.$$

Proof. $A \succ 0$ iff its inertia is all-positive; by additivity that holds iff both A_{11} and S_{11} have all-positive inertia, i.e., both are PD.

Important warning (stated emphatically). Knowing $A_{11} \succ 0$ **and** $A_{22} \succ 0$ is **not** enough to conclude $A \succ 0$. You must check the Schur complement. The Schur complement is A_{22} **minus** a positive semidefinite matrix $A_{21} A_{11}^{-1} A_{12}$; subtracting a PSD matrix from a PD matrix can make it **indefinite**. So PD of the two diagonal corners is **necessary but not sufficient**; PD of one corner *and its Schur complement* is necessary **and** sufficient. (If $A_{12} = 0$, i.e., block diagonal, then $A \succ 0 \iff A_{11} \succ 0$ and $A_{22} \succ 0$.)

The Quiz: Why $A_{21} A_{11}^{-1} A_{12} \succeq 0$

The instructor poses this as a quiz, offering **two approaches**:

1. (*Eigenvalue route*) compute eigenvalues and show they are nonneg — harder.
2. (*Quadratic-form route, preferred*) multiply by a row vector on the left and its conjugate on the right. Write $D = A_{12}x$. Then

$$x^* (A_{12}^* A_{11}^{-1} A_{12}) x = (A_{12}x)^* A_{11}^{-1} (A_{12}x) = D^* A_{11}^{-1} D \geq 0,$$

because $A_{11} \succ 0$ implies $A_{11}^{-1} \succ 0$. It is 0 only when $D = A_{12}x = 0$. Hence $A_{12}^* A_{11}^{-1} A_{12} \succeq 0$ — always positive **semidefinite** — “no need to go into eigenvalue calculations.” This is exactly the PSD term that can drag A_{22} down into indefiniteness.

Linear Matrix Inequality (LMI) via Schur Complement

In applications the criterion is usually used in **reverse**: to handle a **quadratic** matrix inequality, convert it into a **linear** one. Suppose you must show a quadratic-in- H expression like

$$B - HC^{-1}H^* \succ 0 \quad (\text{quadratic in } H),$$

given $C \succ 0$. Via the Schur complement equivalence this is the same as a condition on a larger matrix that is **linear** in H :

$$\begin{bmatrix} C & H^* \\ H & B \end{bmatrix} \succ 0 \iff C \succ 0 \text{ and } B - HC^{-1}H^* \succ 0.$$

The sign and exact placement of the blocks depend on how the original quadratic inequality is written, but the trick is always the same: the quadratic product in H becomes an off-diagonal block where H appears only linearly. The big matrix is called a **linear matrix inequality (LMI)**. **Why this matters:** LMI constraints are **convex**, so problems with LMI constraints are solvable by **semidefinite programming (SDP)**. “You can convert quadratic optimization problems into linear (matrix-inequality) forms using this Schur complement trick” — a standard technique in control theory (Lyapunov stability, H_∞ control).

The instructor summarizes the arc: “we just started from Gaussian elimination” of a block-partitioned matrix, did LDU and UDL, obtained two inverse forms ($\rightarrow$ Woodbury), and in the Hermitian case got star-congruence and inertia results ($\rightarrow$ PD criteria, LMIs).

Transition to Normed Vector Spaces

The next major topic is **normed spaces** — extending the norm concept **beyond the Euclidean norm**, first for n -dimensional vectors and later to arbitrary vector spaces (matrices, functions), each useful in different applications.

A norm is an added structure. The Euclidean norm is **not** part of the definition of a vector space — a vector space is four objects (set of vectors, set of scalars, vector addition, scalar multiplication). A **normed vector space** adds a fifth object: a **norm** that measures how big each vector is. Different choices of that fifth object give different geometries.

Motivation: Combining Several Numbers into One

Take the two-dimensional real vector space; a vector like $(3, 4)$ is a point in the plane (its position). To say “how big” it is, you must combine its several numbers into a **single** number. The Euclidean norm does this by measuring **distance to the origin** — the hypotenuse $\sqrt{3^2 + 4^2} = 5$. (The instructor mentions the hand-drawn “norm” figure his daughter made during Covid, which he still keeps because she likes it.)

But this is just **one** of infinitely many ways to measure size, even in 2-D.

The Taxicab (Manhattan, ℓ_1) Norm

Imagine traveling from the origin to $(3, 4)$: - A **helicopter** can fly the straight Euclidean path: distance 5. - A **pedestrian or taxi** in Manhattan must follow the grid of horizontal and vertical streets (avoiding “Broadway Avenue,” the one strange diagonal road — the instructor jokes a Turkish migrant must have disrupted the perfect grid). Either route covers 3 horizontal + 4 vertical = 7, regardless of which coffee shops you pass (as long as you don’t backtrack).

So the **taxicab / Manhattan / ℓ_1 norm** of $(3, 4)$ is the **sum of absolute values**:

$$\|x\|_1 = \sum_i |x_i| = 3 + 4 = 7.$$

Which is “correct”? Neither — it depends on the application (helicopter $\rightarrow$ Euclidean; taxi $\rightarrow \ell_1$). Note $\|x\|_1 = 7 \geq \|x\|_2 = 5$: the ℓ_1 -norm is always $\geq$ the Euclidean norm. The instructor insists on the Manhattan map because of this “minutes/Manhattan” property.

Why ℓ_1 Became Important: Sparsity

Historically the Euclidean norm dominated 20th-century engineering (people wrote $\|\cdot\|$ and everyone assumed Euclidean). After the **1990s**, the ℓ_1 -norm surged in importance — **not** because of Manhattan travel, but because of **sparse representations**.

The homework problem (and a historically important problem). Consider an **underdetermined** system $Ax = b$ with A **fat** (infinitely many solutions). Among all solutions, pick the one of **minimum norm**. The norm you choose shapes the solution: - Minimizing the ℓ_1 -**norm** subject to $Ax = b$ tends to yield the **sparsest** solution — the x with the **maximum number of zeros** (under suitable conditions).

The truly “right” objective — minimize the **number of nonzero entries** (cardinality, the “ ℓ_0 ” count) — is a **hard, non-convex** problem. Remarkably, minimizing the **convex** ℓ_1 -norm is, under certain conditions, **equivalent** to that hard problem. This non-obvious finding was a **turning point in the 1990s**, spawning enormous literature (“hundreds of thousands of papers”). A vector that is mostly zeros is called **sparse**, and this is why ℓ_1 is so important.

Modern uses: ℓ_1 regularization of neural network weights forces most weights toward zero (a form of regularization); applied to a layer’s **activations**, it forces most activations to zero — echoing the **brain**, where of approximately 80–100 billion neurons only small groups fire at any time (sparse activation). The instructor recommends an “ ℓ_1 -norm magic” video for those who want to explore why ℓ_1 produces sparsity.

Tooling (homework logistics). The last homework problem is computer-oriented: download **CVX**, the convex optimization package from **Stephen Boyd’s** group at Stanford (originally MATLAB, now also Python; **Julia** is another option — the instructor tried Julia, hit bugs, and went back to Python). He strongly recommends taking an **advanced optimization** course after EE 545 and points to Boyd’s YouTube lectures. He predicts students will end up most grateful for learning CVX.

Norm Axioms

Can *any* scalar-valued function of a vector be a norm? No — it must “measure bigness.” Formally, a **norm** is a real-valued function $f : V \rightarrow \mathbb{R}$ on a vector space satisfying:

1. **Non-negativity / definiteness:** $\|x\| \geq 0$, and $\|x\| = 0$ **iff** $x = 0$ (a nonzero vector must have positive norm).
2. **Homogeneity:** $\|\alpha x\| = |\alpha| \|x\|$ — scaling the vector scales its norm by the **absolute value** of the scalar.
3. **Triangle inequality:** $\|x + y\| \leq \|x\| + \|y\|$ — the third side of a triangle is no longer than the sum of the other two.

This is the **positive-definite norm** definition the course adopts.

Aside: Minkowski Functional / Non-Standard “Norms”

The instructor notes you *can* build “weird geometries” that violate these rules. Example: in **relativity**, a spacetime position (x, y, z, t) uses the **Minkowski functional**, where the associated expression can be **zero even when** $(x, y, z, t) \neq 0$ (violating definiteness). Such Minkowski/Lorentzian “norm” spaces appear in EE too — e.g., in H_∞ robust control, some formulations use spaces not governed by the standard norm axioms. The course, however, sticks with the standard positive-definite norms.

The p -Norm Family

Both the Euclidean and taxicab norms are special cases of the **p -norm**. Writing the Euclidean norm as $\|x\|_2 = \sqrt{x^*x} = (\sum_i |x_i|^2)^{1/2}$ (absolute values handle complex entries), generalize to

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{1/p}.$$

- $p = 1$: taxicab / ℓ_1 norm (sum of absolute values). - $p = 2$: Euclidean / ℓ_2 norm. - $p \rightarrow \infty$: the ℓ_∞ (Chebyshev / max) norm — it picks out the **maximum absolute element**:

$$\|x\|_\infty = \max_i |x_i|.$$

For example, $\|(3, -4)\|_\infty = 4$. The p -norms are the most famous family, but **not the only useful norms** — the course will introduce others on $\mathbb{C}^n$ (and on matrices and functions). “All norms are useful, depending on the application.”

The instructor stops here, leaving the detailed norm development to be picked up in the next lecture.

Instructor Remarks and Study Guidance

- The **Schur complement** $S_{11} = A_{22} - A_{21}A_{11}^{-1}A_{12}$ is one of the most important expressions in applied linear algebra — memorize its form (2-2, 2-1, invert 1-1, 1-2).
- $\det(A) = \det(A_{11}) \det(S_{11})$ generalizes $ad - bc$; a non-diagonal block matrix uses the **Schur complement**, not $\det(A_{22})$.
- The **Woodbury / matrix inversion lemma** is “the only equation to memorize.” Its killer app is **rank-one (or low-rank) updates of an inverse** — the basis of **RLS** ($O(n^2)$ per step vs. $O(n^3)$).
- For **Hermitian** A : $A \succ 0$ iff $A_{11} \succ 0$ **and** the Schur complement $S_{11} \succ 0$. Both corners being PD is necessary but **not sufficient**.
- $A_{12}^* A_{11}^{-1} A_{12} \succeq 0$ is shown cleanly by the quadratic-form trick, not eigenvalues.
- The Schur complement converts **quadratic** matrix inequalities into **linear** matrix inequalities (LMIs), which are convex and SDP-solvable — key in control.
- A **norm** is an added structure on a vector space satisfying non-negativity/definiteness, homogeneity, and the triangle inequality. The ℓ_1 (taxicab) norm $= \sum |x_i| \geq \|x\|_2$; minimizing ℓ_1 promotes **sparsity** (1990s breakthrough, neural-net/brain analogy). $\ell_\infty = \max_i |x_i|$. Do the CVX homework early.

Source and Coverage Note

Source: `corrected/lecture18_corrected.md`.

Coverage: Block LDU factorization (block Gaussian elimination, both elimination steps, reconstruction); Schur complement of A_{11} and memory aid; determinant formula and block-diagonal-vs-not intuition with scalar check; block inverse via reverse-order inversion (LDU and UDL forms); block UDL dual and Schur complement of A_{22} ; equating the two inverses to obtain the Matrix Inversion Lemma (Woodbury identity) and its low-rank use case; sample autocorrelation matrix (sample-mean analogy), online rank-one update, RLS derivation via Woodbury ($O(n^2)$ vs $O(n^3)$); Hermitian case star-congruence and inertia additivity (LDU and UDL); positive-definiteness criterion with the emphatic “necessary but not sufficient” warning; the instructor’s quiz showing $A_{12}^* A_{11}^{-1} A_{12} \succeq 0$ via the quadratic-form trick; LMI via Schur complement (quadratic $\rightarrow$ linear, convexity, SDP, control); full introduction to normed vector spaces (norm as added structure, motivation, taxicab/Manhattan/ ℓ_1 norm with helicopter-vs-pedestrian and the daughter’s-map anecdote, 5 vs 7 and $\ell_1 \geq \ell_2$, sparsity history and the underdetermined min-norm/ ℓ_0 -vs- ℓ_1 story, CVX/Boyd homework and optimization-course recommendation, neural-net/brain sparsity, norm axioms, Minkowski-functional/relativity/ H_∞ aside, the p -norm family and $\ell_\infty = \max |x_i|$).

Lecture 19 Notes

Recap: Norms and the p -Norm Family

The lecture continues the normed-spaces topic. Recap from last time: if you are constrained to a rectangular grid (no diagonal moves), the distance to a point is the **absolute sum** of its components — the **one-norm**, also called the **taxicab** or **Manhattan** norm. It is one alternative to the Euclidean norm and, as the homework will show, a very powerful norm for **sparsity** applications.

Norm Axioms (Review)

A **norm** maps the set of vectors to the reals and must satisfy:

1. **Non-negativity:** $\|x\| \geq 0$ (it measures “bigness”).
2. **Definiteness:** $\|x\| = 0 \iff x = 0$ (only the origin has zero norm).
3. **Homogeneity:** $\|\alpha x\| = |\alpha| \|x\|$ (scaling scales the norm by the **absolute value** of the scalar).
4. **Triangle inequality:** $\|x + y\| \leq \|x\| + \|y\|$ (the third side of a triangle is bounded by the sum of the other two).

The Standing (Euclidean) Norm on $\mathbb{C}^n$

For complex vectors, the **Euclidean norm** is

$$\|x\|_2 = \left(\sum_k |x_k|^2 \right)^{1/2} = \sqrt{x^* x}.$$

(The conjugate transpose handles complex entries.) The **one-norm** is the sum of absolute values, $\|x\|_1 = \sum_k |x_k|$; minimizing it tends to produce **sparse** vectors (most entries zero, a few nonzero). Both are members of the **p -norm (L^p) family**:

$$\|x\|_p = \left(\sum_k |x_k|^p \right)^{1/p}, \quad p \geq 1.$$

- $p = 1$: taxicab / Manhattan norm. - $p = 2$: Euclidean norm. (You could name the $p = 3$ norm yourself — there is no standard name.) - $p \rightarrow \infty$: taking the limit picks out the **maximum absolute element**:

$$\|x\|_\infty = \max_k |x_k|.$$

Worked values for $x = (3, -4)$: $\|x\|_1 = 3 + 4 = 7$, $\|x\|_2 = 5$, $\|x\|_\infty = 4$ (the peak magnitude). **As p increases, the norm decreases** — not a coincidence but a general property.

Why ℓ_∞ Matters: Worst-Case / Minimax

If x is an **error vector**, the ∞ -norm is the **maximum error**. Designing for robustness — minimizing the **worst** error — means minimizing $\|e\|_\infty$, a **minimax** optimization.

Application: FIR Filter Design

A clean application illustrating the three norms is **FIR filter design** in the frequency domain.

Setup

On the (positive) frequency axis, suppose we want a **low-pass filter**: desired response = 1 in the passband and = 0 in the stopband. Sample the frequency axis at f points (e.g., 2000 samples) to get the **desired vector**

$$h_d = (\underbrace{1, 1, \dots, 1}_{\text{passband}}, \underbrace{0, 0, \dots, 0}_{\text{stopband}})^\top \in \mathbb{C}^f.$$

A **realizable** FIR filter has a finite impulse response with r coefficients c ; its frequency response is the Fourier transform of c , written as a matrix product $h = Fc$ (F the $f \times r$ Fourier matrix). The realized response will **not** be perfectly flat — it ripples — so there is an **error vector**

$$e = h_d - Fc.$$

Choice of Norm = Design Criterion

$$\min_c \|h_d - Fc\|_p,$$

and different p give different filters:

- $\ell_2 \rightarrow$ **least squares**. $\min_c \|h_d - Fc\|_2$ has a **closed-form algebraic solution** via the pseudoinverse, $\hat{c} = (F^*F)^{-1}F^*h_d$. Easy.
- $\ell_\infty \rightarrow$ **equiripple / minimax**. $\min_c \|h_d - Fc\|_\infty$ minimizes the worst-case deviation. **No formula** — needs an iterative algorithm. MATLAB’s **firpm** (Finite Impulse Response **Parks–McClellan**, “pm”) implements this minimax criterion. (A student asked which MATLAB filter-design command applies; the instructor distinguished special-purpose commands from **firpm**, the general minimax designer.)
- $\ell_1 \rightarrow$ **robust**. $\min_c \|h_d - Fc\|_1$ minimizes total absolute error; no formula, solvable as an LP / via CVX.

The instructor’s point: there is **no universal “best” norm** — it depends on the application and which kind of error matters. “Each norm is easy to solve” only for ℓ_2 (closed form); the others need iteration.

Convexity and Why It Helps

A **convex function** is one where the chord joining any two points on the graph lies **above** the function, and which has **no spurious local minima**. All p -norms ($p \geq 1$) are convex. Minimizing a convex function over a convex search space is **tractable**, even when iterative: tools like **CVX** solve it directly (you state the problem, get the answer), though you don’t get a closed-form formula for ℓ_1/ℓ_∞ . The instructor flags CVX as a tool students will use in the next homework.

Linear-Programming Formulation of ℓ_∞ (and ℓ_1)

ℓ_∞ and ℓ_1 minimization can be cast as **linear programs (LP)** — a special class of convex optimization (an entire industrial-engineering course). The general LP form is

$$\min_x c^\top x \quad \text{s.t.} \quad Ax \leq b \quad (\text{or } \geq),$$

minimizing the **inner product** $c^\top x$ over a **polyhedral region**.

ℓ_∞ **trick**. Introduce a scalar upper bound t on the peak magnitude. Since $|e_k| \leq t \iff -t \leq e_k \leq t$:

$$\min_{c,t} t \quad \text{s.t.} \quad Fc - h_d \leq t\mathbf{1}, \quad -(Fc - h_d) \leq t\mathbf{1}.$$

Stacking F and $-F$ (with $\mp\mathbf{1}$ columns for t) puts it into the standard LP inequality form, and the objective is the inner product picking out t . At the optimum, t equals the maximum absolute element of the error. So ℓ_∞ **minimization = LP**; a similar trick handles ℓ_1 . You don’t *need* the LP form (CVX’s general convex solver works), but historically people were excited to discover ℓ_1/ℓ_∞ reduce to LP.

For a general p -norm objective such as $p = 3$, the LP trick no longer applies, but the problem is still convex as long as $p \geq 1$. In that case the general convex-optimization route (for example CVX) is the natural tool.

Weighted ℓ_2 -Norms

Beyond the p -norm family, another way to measure size is a **weighted** norm.

Diagonal Weighting

For a diagonal **positive** matrix $D = \text{diag}(d_1, \dots, d_n)$, $d_k > 0$:

$$\|x\|_D = \sqrt{x^* D x} = \left(\sum_k d_k |x_k|^2 \right)^{1/2}.$$

Each coordinate's squared magnitude carries a **weight** d_k . This is useful when not all components are equally significant. **In filter design**, for instance, you weight the **stopband** much more than the passband (e.g., 10,000 vs. 1): on the dB scale a given error in the suppression band makes a “lousy filter,” so you prioritize stopband suppression over passband flatness.

Why the weights must be strictly positive. If some $d_k = 0$, then the nonzero vector e_k has $\|e_k\|_D = 0$, violating definiteness. (Positive **semidefinite** is not enough.)

General Positive-Definite Weighting (Student Q&A)

Replace D by any positive definite matrix W :

$$\|x\|_W = \sqrt{x^* W x}.$$

The instructor asks the class what W does **geometrically** when it is not diagonal. Through guided Q&A: - A student guesses a **correlation** connection — the instructor agrees W is often a **correlation matrix** (in fact the **inverse** correlation appears naturally in applications). - Another student suggests **projecting x onto eigenvector directions** — “Exactly.”

Derivation. Since W is Hermitian PD, it is unitarily diagonalizable: $W = U \Lambda U^*$ with positive eigenvalues. Then

$$x^* W x = x^* U \Lambda U^* x = (U^* x)^* \Lambda (U^* x) = \sum_k \lambda_k |z_k|^2, \quad z = U^* x.$$

Here $z_k = u_k^* x$ is the **projection of x onto eigenvector u_k** (its coordinate in the eigenbasis). So:

Instead of weighting the **coordinate axes** (the diagonal case), a general W weights the **eigenvector directions** of W : you change basis to the eigenvectors $u_1, u_2, \dots$, and the eigenvalues $\lambda_1, \lambda_2, \dots$ become the weights along those (possibly rotated) directions.

Geometrically: drop the projections $z_1 = u_1^* x$, $z_2 = u_2^* x$ onto the eigenvectors; each squared projection is scaled by its eigenvalue.

Weighted ℓ_2 -Norm in Statistics: the Multivariate Gaussian

The PD-weighted 2-norm appears naturally in statistical estimation (the subject of a 530-type course; here just the connection).

Scalar and Independent Gaussians

A single Gaussian has density $\propto \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ — a bell curve centered at μ , with σ controlling width. For **jointly Gaussian** random variables, the instructor notes a special fact: if they are uncorrelated, then they are independent. That implication is **not true for arbitrary random variables**.

For two independent zero-mean Gaussians with variances σ_1^2 and σ_2^2 , the joint density is the product of marginals:

$$p(x_1, x_2) \propto \frac{1}{2\pi\sigma_1\sigma_2} \exp\left(-\frac{1}{2}\left(\frac{x_1^2}{\sigma_1^2} + \frac{x_2^2}{\sigma_2^2}\right)\right).$$

If the variances are equal, $\sigma_1 = \sigma_2 = \sigma$, this simplifies to

$$p(x_1, x_2) \propto \exp\left(-\frac{x_1^2 + x_2^2}{2\sigma^2}\right) = \exp\left(-\frac{\|x\|_2^2}{2\sigma^2}\right).$$

So for **uncorrelated, equal-variance** Gaussians the ordinary **2-norm squared** appears in the exponent.

Correlated Gaussians → Weighted 2-Norm

When the components are **correlated**, you need the covariance matrix Σ , and the joint density becomes

$$p(x) = \frac{1}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp\left(-\frac{1}{2} x^\top \Sigma^{-1} x\right).$$

The exponent is a **weighted 2-norm squared** with weight $W = \Sigma^{-1}$, the **precision (inverse covariance) matrix**. So the PD-weighted norm sits “at the top of the Gaussian,” with the inverse covariance as the weight.

Maximum Likelihood = Weighted Least Squares

Maximizing the Gaussian likelihood means **minimizing** the (negated) exponent $\frac{1}{2} x^\top \Sigma^{-1} x$ — a **weighted least squares** problem. The weighting along eigen-directions interpretation applies: directions of large variance (large eigenvalues of Σ) get small precision weight, so they are penalized less.

L^p Norms for Function Spaces

Norms are not restricted to n -dimensional vectors. For appropriately integrable functions on an interval $[a, b]$, define the L^p **norm**:

$$\|f\|_{L^p} = \left(\int_a^b |f(t)|^p dt \right)^{1/p}.$$

Think of $f(t)$ as a “vector” with **uncountably many** entries (one per t); the integral replaces the sum. This lets us measure **how big a function is** and the **distance between two functions** $\|f - g\|_{L^p}$ — treating each function as a **point** in an infinite-dimensional space.

Polynomial Approximation

Approximate an arbitrary target function $g(t)$ by a polynomial $f(t) = a + bt + ct^2$. The polynomial can only take certain shapes, so it deviates from g at various points (big error here, zero there). To measure performance with a **single number**, define the error function $e(t) = g(t) - f(t)$ and minimize a norm of it:

$$\min_{a,b,c} \|g - f\|_{L^p}.$$

- $p = 2$: minimize total squared error (closed-form via inner products with the polynomial basis). - $p = \infty$: minimize the worst-case pointwise deviation (Chebyshev / equiripple approximation).

The big idea: **norms let us carry two- and three-dimensional geometric intuition (distance, closeness, projection) into high- and infinite-dimensional problems**. The instructor cautions, however, that geometric intuition from 2-D/3-D can sometimes **fail** in high dimensions — “there are surprises in high dimensions.”

Norm Balls (Geometric Understanding)

Before matrix norms, the instructor develops **norm balls**, which give geometric insight — especially into the ℓ_1 -sparsity connection. The **unit ℓ_p -ball** is the region of all vectors with $\|x\|_p \leq 1$ (the boundary is where $\|x\|_p = 1$; “unit” is dropped for brevity).

The ℓ_1 Ball in 2-D (Quadrant-by-Quadrant)

We need $|x_1| + |x_2| \leq 1$. The sign of each term changes by quadrant: - **First quadrant** ($x_1, x_2 > 0$): $x_1 + x_2 \leq 1$. The boundary $x_1 + x_2 = 1$ is a line; $x_1 + x_2 = 0.9, 0.8, \dots$ are parallel lines; the region is the triangle below $x_1 + x_2 = 1$. - **Second quadrant** ($x_1 < 0, x_2 > 0$): $-x_1 + x_2 \leq 1$, another line, another triangle. - Continuing through the third and fourth quadrants gives four triangular pieces that assemble into a **diamond** (a rotated square) with corners at $(\pm 1, 0)$ and $(0, \pm 1)$.

So the ℓ_1 ball is a **diamond** — “this is a new geometry; your ball is no longer the round thing.”

The ℓ_2 and ℓ_∞ Balls in 2-D

- ℓ_2 : $x_1^2 + x_2^2 \leq 1$ is the familiar **circular** disk.
- ℓ_∞ : $\max(|x_1|, |x_2|) \leq 1$ means **both** $|x_1| \leq 1$ and $|x_2| \leq 1$ — a **square** (axis-aligned, **not** tilted) with corners at $(\pm 1, \pm 1)$. On the right edge $x_1 = 1$ (with $|x_2| \leq 1$), the peak value is 1; inside, both coordinates are < 1 .

Nesting Comparison

Overlaying the three: the ℓ_1 **diamond is inside the ℓ_2 circle, which is inside the ℓ_∞ square**. This visualizes $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1$: - A point with $\|x\|_1 > 1$ but $\|x\|_2 \leq 1$ sits **outside** the diamond but **inside** the circle (and square). - A point with $\|x\|_1, \|x\|_2 > 1$ but $\|x\|_\infty \leq 1$ sits outside both the diamond and circle but inside the square.

The 3-D Balls and “Orthant” Terminology

In $\mathbb{R}^3$: - ℓ_2 **ball**: $x_1^2 + x_2^2 + x_3^2 \leq 1$ — a **sphere** (solid spherical region) around the origin. - ℓ_∞ **ball**: $\max(|x_1|, |x_2|, |x_3|) \leq 1$ — a **cube**. - ℓ_1 **ball**: $|x_1| + |x_2| + |x_3| \leq 1$ — an **octahedron**.

For the ℓ_1 ball, in the all-positive region the boundary $x_1 + x_2 + x_3 = 1$ is a **hyperplane** — note $x_1 + x_2 + x_3 = \langle x, \mathbf{1} \rangle = 1$, a plane orthogonal to $\mathbf{1}$, shifted off the origin (since the right side is 1). Each of the 2^n sign regions contributes a triangular face, giving the octahedron.

Terminology Q&A. Students try to name the n -dimensional generalization of “quadrant.” In 2-D the axes divide the plane into 4 regions (quadrants); in 3-D into 8 regions; in n -D into 2^n regions. The general name (after some back-and-forth) is **orthant** — each orthant being where all components have fixed signs.

Application: Overdetermined Systems and Norms

One of the most useful parts of the course: using norms to solve linear systems.

Overdetermined (Tall) Systems

An **overdetermined** system has **more equations than unknowns** — A is a **tall** matrix. The potential problem: the column space (range) of A does **not** cover the whole target space, so if b is **not** in the range of A there is **no solution** (the system is **inconsistent**). A “lazy researcher” stops there — but we don’t.

Closest Point in the Range

Instead, find the point $\hat{b} = Ax^*$ in the range of A that is **as close as possible to b** , minimizing the **norm** of the error $b - Ax$. But “close” requires a choice of norm:

$$\min_x \|b - Ax\|.$$

The ℓ_2 Case Has a Formula

If we use the **2-norm** (and A is full rank), the solution is the **orthogonal projection** of b onto the range of A with respect to the Euclidean inner product. Recalling the projection formula for a non-orthonormal basis,

$$\hat{b} = A(A^*A)^{-1}A^*b,$$

and the multiplier of A is the solution

$$x^* = (A^*A)^{-1}A^*b.$$

This closed-form formula is **why people gravitate to the 2-norm**: no iterative algorithm needed. (For ℓ_1/ℓ_∞ there is no such formula — use CVX or LP.)

For ℓ_1 and ℓ_∞ , the same LP reductions discussed earlier can be used; for a general p -norm residual, CVX can still solve the convex problem when $p \geq 1$, but it will not usually be a linear program.

Why the 2-Norm Has a Formula but ℓ_1/ℓ_∞ Don't

The 2-norm is **associated with an inner product**: $\langle x, y \rangle = y^\top x$ (real case), and $\|x\|_2 = \sqrt{\langle x, x \rangle}$. That inner-product structure is what yields the orthogonal-projection solution. The ℓ_1 and ℓ_∞ norms have **no associated inner product** — you cannot write them as $\sqrt{\langle x, x \rangle}$ — so the projection machinery does not apply, and you must optimize numerically. (Inner-product norms will be developed in the inner-product lectures.)

Statistical Justification: Gaussian Noise $\rightarrow$ Least Squares

The 2-norm criterion is also **statistically optimal** under a Gaussian-noise model. Suppose the measurement is

$$b = Ax + n,$$

with n Gaussian noise. If n is **white** (covariance = $\sigma^2 I$), the **maximum-likelihood** estimate minimizes $\|b - Ax\|_2^2$ — exactly **least squares** (the σ^2 scales out). This is why minimizing the sum of squared errors is “justified”: b was really in the range, and only Gaussian noise pushed it out.

Correlated Noise $\rightarrow$ Weighted Least Squares

If the noise is **correlated** with covariance Σ , the ML estimate uses the **weighted** norm with $W = \Sigma^{-1}$:

$$\min_x \|b - Ax\|_{\Sigma^{-1}}^2 = \min_x (b - Ax)^\top \Sigma^{-1} (b - Ax).$$

For uncorrelated noise $\Sigma = \sigma^2 I$ this collapses back to ordinary least squares because the scalar σ^2 only rescales the objective. With known noise statistics you insert Σ^{-1} as the weight; this again weights the eigen-directions of Σ by the inverse eigenvalues. (For non-full-rank A , additional tricks are needed — deferred to later lectures.)

Instructor Remarks and Study Guidance

- **No universally best norm** — the norm encodes which error you care about: ℓ_1 (total absolute, robust, sparsity-promoting), ℓ_2 (total squared, smooth, closed-form), ℓ_∞ (worst-case, equiripple, minimax).
- **As p grows the norm shrinks**: $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1$. Norm balls: ℓ_1 **diamond** $\subset \ell_2$ **circle** $\subset \ell_\infty$ **square** (sphere/cube/octahedron in 3-D). The n -D sign regions are **orthants** (2^n of them).
- **Convexity** ($p \geq 1$) makes all these minimizations tractable; ℓ_1 and ℓ_∞ further reduce to **linear programs**; use **CVX** for general convex p , including cases such as $p = 3$ that are not LPs. CVX is explicitly tied to the next homework.
- **Weighted ℓ_2** with PD weight W weights the **eigenvector directions** of W by its eigenvalues. It is the natural norm for **multivariate Gaussian** estimation, with weight = precision matrix Σ^{-1} .

- **The 2-norm is special** because it comes from an **inner product**, giving the closed-form least-squares / orthogonal-projection solution $x^* = (A^*A)^{-1}A^*b$. ℓ_1 and ℓ_∞ have no associated inner product.
- **Least squares is ML estimation** under white Gaussian noise; correlated noise gives **weighted** least squares with Σ^{-1} .

Source and Coverage Note

Source: `corrected/lecture19_corrected.md`.

Coverage: Recap of norm axioms and the p -norm family ($\ell_1/\ell_2/\ell_\infty$, worked $(3, -4)$ values, monotonicity in p); ℓ_∞ as worst-case/minimax; FIR filter design (low-pass desired vector, $h = Fc$, error vector, three criteria — ℓ_2 least-squares closed form, ℓ_∞ `firpm`/Parks–McClellan equiripple, ℓ_1 robust — with the filter-command Q&A); convex-function definition; LP formulation of ℓ_∞ (scalar bound t , polyhedral region, inner-product objective) and the ℓ_1 analog; weighted ℓ_2 -norm (diagonal weighting with filter-design stopband example, strict-positivity requirement, general PD W with the correlation/eigenvector student Q&A and eigen-decomposition derivation); multivariate Gaussian (scalar/independent $\rightarrow$ 2-norm, correlated $\rightarrow$ weighted 2-norm with precision matrix, MLE = weighted least squares); L^p function-space norms and polynomial approximation; detailed norm-ball geometry (quadrant-by-quadrant ℓ_1 diamond, ℓ_2 circle, ℓ_∞ square, nesting comparison, 3-D sphere/cube/octahedron, hyperplane faces, orthant terminology Q&A); overdetermined systems and norms (tall/inconsistent, closest point, ℓ_2 projection formula $x^* = (A^*A)^{-1}A^*b$, 2-norm $\leftrightarrow$ inner-product connection and why ℓ_1/ℓ_∞ lack one, Gaussian-noise/MLE justification, correlated-noise weighted least squares).

Lecture 20 Notes

Recap and Goal: From Vector Norms to Matrix Norms

The previous lectures developed vector p -norms in $\mathbb{C}^n$ (ℓ_1 taxicab/sparsity, ℓ_2 Euclidean, ℓ_∞ Chebyshev/minimax). This lecture asks: **how do we measure how big a matrix is?** There are two fundamentally different answers.

Two Approaches to Matrix Norms

Approach 1: Flatten the Matrix into a Vector

Take the $m \times n$ matrix, stack its columns into a single mn -dimensional vector (the **vec** notation), and apply a vector norm. This approach **does not care where the entries come from** — it just stuffs them into a vector.

Using the **2-norm** of the flattened vector gives the **Frobenius norm**: the “total energy” of the matrix.

$$\|A\|_F = \left(\sum_{i,j} |a_{ij}|^2 \right)^{1/2} = \sqrt{\text{tr}(A^*A)} = \sqrt{\text{tr}(AA^*)}.$$

(Recall $\text{tr}(A^*A) = \sum_{i,k} |a_{ik}|^2$: the sum of squared magnitudes.)

Approach 2: Induced (Operator) Norms

Other norms **care about the locations** of the entries because a matrix defines a **linear mapping** $A: \mathbb{C}^n \rightarrow \mathbb{C}^m$. Picture an **input space** $\mathbb{C}^n$ and a (possibly different-dimensional) **output space** $\mathbb{C}^m$. We may measure size **differently** in each space — say a p -norm in the input and a q -norm in the output. For a nonzero input x , look at the **gain**

$$\frac{\|Ax\|_q}{\|x\|_p},$$

the factor by which the map changes the norm. Gain > 1 is amplification, < 1 is attenuation — but it depends on x (one direction might give 1.1, another 0.5). The **induced norm** is the **maximum gain** over all directions:

$$\|A\|_{q,p} = \max_{x \neq 0} \frac{\|Ax\|_q}{\|x\|_p} = \max_{\|x\|_p=1} \|Ax\|_q.$$

Scale invariance. Because $\|A(\alpha x)\|_q / \|\alpha x\|_p = \|Ax\|_q / \|x\|_p$, the ratio depends only on the **direction** of x . So we restrict the search to the **boundary of the unit p -norm ball** in the input space and ask: which boundary point produces the maximum output q -norm?

Geometric Calculation Strategy

To compute an induced norm geometrically: 1. Draw the unit $\|\cdot\|_p$ -ball boundary in the input space. 2. Map all boundary points through A to find the **image**. 3. Find the maximum $\|\cdot\|_q$ -norm over that image.

Key lemma (linear maps send line segments to line segments). Let a, b be two points with images $Aa = a'$, $Ab = b'$. Any point on the segment joining them is a **convex combination** $\lambda a + (1 - \lambda)b$ with $\lambda \in [0, 1]$, and

$$A(\lambda a + (1 - \lambda)b) = \lambda Aa + (1 - \lambda)Ab = \lambda a' + (1 - \lambda)b'.$$

So the segment from a to b maps to the segment from a' to b' . **Consequence:** for a **polytope** ball (boundaries are line segments, as for ℓ_1 and ℓ_∞), it suffices to map the **vertices** and connect them.

Induced 1-1 Norm (Worked Example)

Throughout, use the running example

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}.$$

For $\|A\|_{1,1}$ we use the ℓ_1 -norm at both input and output (written just $\|A\|_1$, understood as 1-1). The unit ℓ_1 -ball is a **diamond** with vertices

$$A = (1, 0), \quad B = (0, 1), \quad C = (-1, 0), \quad D = (0, -1).$$

Map the vertices (multiplying by a standard basis vector **picks a column**):

$$A \rightarrow \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad B \rightarrow \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad C \rightarrow \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad D \rightarrow \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

($B = (0, 1)$ picks column 2 = $(2, 2)$; $A = (1, 0)$ picks column 1 = $(1, 0)$.) The diamond maps to a parallelogram; the image is this **polytope** (a bounded region whose boundaries are hyperplanes; an unbounded one is a **polyhedron**, a bounded one a **polytope**). Now take ℓ_1 -norms of the image vertices:

$$\|(1, 0)\|_1 = 1, \quad \|(2, 2)\|_1 = 4, \quad \|(-1, 0)\|_1 = 1, \quad \|(-2, -2)\|_1 = 4.$$

Maximum is at B' and $-D'$: $\boxed{\|A\|_{1,1} = 4}$.

Geometric reading: if the maximum point is visually unclear, imagine expanding centered ℓ_1 -balls in the output space until the largest one still intersects the image. The last-contact points here are the images of B and D , both with ℓ_1 -norm 4.

Shortcut. This was “a heck of a thing” even for a 2×2 . It can be shown (via the Hölder inequality below) that

$$\|A\|_{1,1} = \max_j \|a_j\|_1 \quad (\text{maximum column } \ell_1\text{-norm}).$$

Here columns have ℓ_1 -norms 1 and 4; max is 4. ✓ Essential for large matrices (you cannot draw a 1000-dimensional ℓ_1 -ball).

Frobenius for comparison: $\|A\|_F = \sqrt{1 + 4 + 0 + 4} = \sqrt{9} = 3$.

Induced ∞ - ∞ Norm (Worked Example)

For $\|A\|_{\infty,\infty}$ (written $\|A\|_\infty$), the unit ℓ_∞ -ball is a **square** with vertices $(\pm 1, \pm 1)$. Map them:

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \quad \begin{bmatrix} -1 \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \begin{bmatrix} -1 \\ -1 \end{bmatrix} \rightarrow \begin{bmatrix} -3 \\ -2 \end{bmatrix}, \quad \begin{bmatrix} 1 \\ -1 \end{bmatrix} \rightarrow \begin{bmatrix} -1 \\ -2 \end{bmatrix}.$$

(($(1, 1)$ sums the columns = $(3, 2)$; $(-1, 1)$ subtracts column 1 from column 2 = $(1, 2)$.) The square maps to a parallelogram (“a parallel-parking shape”). Take ℓ_∞ -norms of the image vertices:

$$\|(3, 2)\|_\infty = 3, \quad \|(1, 2)\|_\infty = 2, \quad \|(-3, -2)\|_\infty = 3, \quad \|(-1, -2)\|_\infty = 2.$$

Maximum is $\boxed{\|A\|_{\infty,\infty} = 3}$.

Shortcut (dual to the 1-1 case). Where 1-1 used max **column** ℓ_1 -norm, ∞ - ∞ uses the maximum **row** ℓ_1 -norm:

$$\|A\|_{\infty,\infty} = \max_i \|r_i\|_1.$$

Rows have ℓ_1 -norms $\|(1, 2)\|_1 = 3$ and $\|(0, 2)\|_1 = 2$; max is 3. ✓

The bottom line: the **same matrix** gives **different norm values** for different input/output norm choices, each useful in different applications.

The Star of the Show: Induced 2-2 Norm

Using ℓ_2 at both input and output (written $\|A\|_{2,2}$ or $\|A\|_2$) gives the **maximum Euclidean gain** — “one of the most frequently used norms,” and the gateway to the **SVD**.

The Image of the Unit Sphere Is an Ellipse(oid)

The unit ℓ_2 -ball boundary in 2-D is a **circle**. Unlike the polytope cases (whose images are polytopes), the image of a sphere under a linear map is an **ellipsoid**. For invertible A , set $y = Ax$, so $x = A^{-1}y$, and the condition $\|x\|_2 = 1$ becomes $\|A^{-1}y\|_2 = 1$, i.e.

$$y^* (A^{-1})^* A^{-1} y = y^* (AA^*)^{-1} y = 1.$$

This is the equation of an **ellipse** $y^* M^{-1} y = 1$ with $M = AA^*$. The **principal semi-axes** point along the eigenvectors of M , with lengths $\sqrt{\lambda_i(AA^*)}$. The maximum ℓ_2 -norm on the ellipse is the **longest semi-axis**:

$$\|A\|_{2,2} = \sqrt{\lambda_{\max}(AA^*)} = \sigma_1(A),$$

the **largest singular value** (the nonzero eigenvalues of AA^* and A^*A coincide; either works — to be justified later).

Worked Numbers for the Running Example

For $A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}$:

$$A^*A = \begin{bmatrix} 1 & 2 \\ 2 & 8 \end{bmatrix}, \quad \text{char. eqn: } \lambda^2 - 9\lambda + 4 = 0,$$

$$\lambda = \frac{9 \pm \sqrt{81 - 16}}{2} = \frac{9 \pm \sqrt{65}}{2} \approx 8.53 \text{ and } 0.47.$$

The eigenvectors give the **directions** of the principal semi-axes (e.g., the longest, v_1), and the semi-axis **lengths** are the square roots of the eigenvalues: $\sqrt{8.53} \approx 2.92$ (long axis) and $\sqrt{0.47} \approx 0.69$ (short axis). Drawing a sequence of ℓ_2 -balls of increasing radius over the ellipse, the largest one that still touches the ellipse touches it at the **ends of the longest semi-axis** — confirming that the maximum 2-norm is the longest semi-axis. Hence

$$\|A\|_{2,2} = \sqrt{8.53} \approx 2.92.$$

(The instructor’s running discussion is partly garbled around the numbers, but the eigenvalues 8.53 and 0.47 are exactly those of $A^*A = [1, 2; 2, 8]$.)

The output-side ellipse derivation in the lecture used

$$AA^* = \begin{bmatrix} 5 & 4 \\ 4 & 4 \end{bmatrix},$$

because the ellipse equation was written as $y^*(AA^*)^{-1}y = 1$. This matrix has the same characteristic equation $\lambda^2 - 9\lambda + 4 = 0$, so the same eigenvalues 8.53 and 0.47. If $AA^* = V\Lambda V^*$ and $z = V^*y$, then

$$y^*(AA^*)^{-1}y = 1 \iff \frac{|z_1|^2}{\lambda_1} + \frac{|z_2|^2}{\lambda_2} = 1.$$

So the inverse matrix contributes $1/\lambda_i$ in the ellipse equation, but the actual semi-axis lengths are $\sqrt{\lambda_i}$, not λ_i . The instructor explicitly corrected this point: the long-axis length is $\sqrt{8.53}$, not 8.53. The drawn v_1, v_2 arrows on the slide were directions of principal axes, not eigenvectors already scaled by the eigenvalues, and the picture was not to scale.

Notation Warning

$\|A\|_2$ means the **induced 2-2 norm** $= \sigma_1$, **not** the Frobenius norm. If you mean the root-sum-of-squares-of-entries, you must write $\|A\|_F$. The 2-2 induced norm is also called the **operator norm** or **spectral norm**. In some references “ $\|A\|$ ” with no subscript defaults to the operator norm; in others it defaults to Frobenius — so always check.

The Non-Invertible (Degenerate) Case — Student Question

A student asks what happens if A is **not invertible**. Then the image of the sphere is a **degenerate ellipsoid**: in 2-D with a rank-1 matrix, a line segment; in 3-D with a rank-2 matrix, an ellipse lying in the (rank-2) plane through the origin (the range space). You can reach that subspace via the **QR factorization** of A (whose Q columns give an orthonormal basis for the range) and write the ellipse in restricted coordinates. **Crucially**, the formula $\sqrt{\lambda_{\max}(A^*A)}$ still gives the maximum 2-norm **even when A is singular** — invertibility was only needed to *draw* the full ellipse, not to compute the largest semi-axis. So **invertibility is not required** for $\|A\|_{2,2} = \sigma_1$.

Shortcut Formulas Summary

Induced norm	Shortcut	Geometric meaning
$\ A\ _{1,1}$	$\max_j \ a_j\ _1$ (max column ℓ_1)	ℓ_1 -ball vertex $\rightarrow$ column
$\ A\ _{\infty,\infty}$	$\max_i \ r_i\ _1$ (max row ℓ_1)	ℓ_∞ -ball vertex $\rightarrow$ row
$\ A\ _{2,2}$	$\sigma_1 = \sqrt{\lambda_{\max}(A^*A)}$	longest semi-axis of image ellipsoid
$\ A\ _F$	$\sqrt{\sum_{ij} a_{ij} ^2}$	flatten + ℓ_2

For the running example: $\|A\|_F = 3$, $\|A\|_{1,1} = 4$, $\|A\|_{\infty,\infty} = 3$, $\|A\|_{2,2} = \sqrt{8.53} \approx 2.92$. Different numbers, different meanings — none universally “largest.”

A Few Student Questions on Defining Norms

Affine maps. A student asks whether you can define a norm for an affine map $x \mapsto Ax + b$. The instructor: you typically **lift** the affine map into a linear one by concatenation,

$$Ax + b = \begin{bmatrix} A & b \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix},$$

working in the lifted space (this is how affine expressions are folded into a single matrix, used e.g. in **total least squares**). But the gain of the concatenated matrix $[A \ b]$ is **not** a true norm of the affine map, because the lifted input’s last entry is **fixed at 1** — you don’t have the freedom over that coordinate. Computing the 2-norm of $[A \ b]$ gives an **upper bound** on the affine map’s peak gain (it assumes control over that fixed coordinate), but whether the bound is achieved is unclear. The peak gain of an affine system *can* be defined, but it is a system gain, not a matrix norm.

Averaging two norms. Another student asks whether averaging norms (or “average of 2 and 4”) yields a norm. The instructor hasn’t seen it and is unsure it would satisfy the **homogeneity** (scaling) and **triangle inequality** axioms — but notes that we *do* build norms from singular values (next), so some non-obvious combinations are legitimate norms.

Preview — nuclear norm and the Netflix challenge. One especially important matrix norm to come is the **sum of the principal semi-axis lengths** (the sum of singular values / square roots of the A^*A

eigenvalues) — the **nuclear norm**. It is the key to **low-rank approximation** and powered the **Netflix challenge** (recommendation systems: recommend movies so users keep their subscription). It “burnt the brains” of many researchers in the 1990s–2000s. (The instructor jokes about reusing the same Netflix-challenge story year after year — like a friend told to “change your jokes or change your friends”; luckily the students change each term, so the jokes can repeat.) Details come with the Schatten norms in a later lecture.

Unitary-Invariant Norms

Earlier, **unitary matrices** were introduced as the matrices that **preserve the 2-norm**. Now flip the viewpoint: which **norms** are preserved under unitary maps? The 2-norm is — multiplying a vector by a unitary matrix doesn’t change its 2-norm. This is now read as a **property of the 2-norm**: it is **unitarily invariant**. By contrast, the ℓ_1 and ℓ_∞ norms are **not** unitarily invariant (a unitary rotation changes them).

Frobenius Norm Is Unitarily Invariant

For unitary U, Q :

$$\|U A Q\|_F^2 = \text{tr}((U A Q)^*(U A Q)) = \text{tr}(Q^* A^* U^* U A Q) = \text{tr}(Q^* A^* A Q).$$

Using $U^* U = I$, then cycling the trace ($\text{tr}(Q^* A^* A Q) = \text{tr}(A^* A Q Q^*) = \text{tr}(A^* A)$) and $Q Q^* = I$:

$$\|U A Q\|_F = \sqrt{\text{tr}(A^* A)} = \|A\|_F.$$

So multiplying A by unitary matrices on the left and right leaves the Frobenius norm unchanged.

2-2 (Operator) Norm Is Unitarily Invariant

The 2-2 norm is a **ratio of two 2-norms**, each unitarily invariant. Multiplying A on the right by a unitary changes the input without changing its 2-norm; multiplying on the left by a unitary changes the output without changing its 2-norm. So $\|U A Q\|_{2,2} = \|A\|_{2,2}$.

In formula form, for unitary U, Q :

$$\|U A Q\|_{2,2} = \max_{x \neq 0} \frac{\|U A Q x\|_2}{\|x\|_2} = \max_{x \neq 0} \frac{\|A Q x\|_2}{\|x\|_2}.$$

Let $z = Qx$. Since Q is unitary, $\|z\|_2 = \|x\|_2$ and z ranges over the same input space as x . Hence

$$\|U A Q\|_{2,2} = \max_{z \neq 0} \frac{\|A z\|_2}{\|z\|_2} = \|A\|_{2,2}.$$

Why this matters: both the Frobenius and 2-2 norms depend **only on the singular values** of A (which are themselves unitarily invariant). Indeed $\|A\|_{2,2} = \sigma_1$ and $\|A\|_F = \sqrt{\sigma_1^2 + \cdots + \sigma_r^2}$ (proved after SVD).

Norm Inequalities (Toward Shortcut Justifications)

Cauchy–Schwarz (Recap)

For n -dimensional vectors, the magnitude of the **Euclidean inner product** is bounded by the product of the 2-norms:

$$|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2.$$

This is what lets us define the cosine of the angle (the ratio of inner product to the product of norms). The proof is short.

The Hadamard Product

Define the **Hadamard product** as **elementwise** multiplication: for same-dimension vectors (or matrices) x and y , $(x \odot y)_i = x_i y_i$. In MATLAB this is `.*` (the dot before the star); ordinary `*` is matrix multiplication. In Python (NumPy), elementwise is the default `*`, while **matrix** multiplication needs `@` (or `np.dot`) — “I like Python, but I hate that part.”

A Hölder-Type Inequality

A stronger bound than Cauchy–Schwarz uses the ℓ_1 -norm of the **Hadamard product**:

$$\|x \odot y\|_1 = \sum_i |x_i y_i| \geq |\langle x, y \rangle|,$$

because summing **absolute values** of the term-by-term products is at least the magnitude of their (signed) sum. And this is itself bounded by a product of a p -norm and a q -norm:

$$\|x \odot y\|_1 \leq \|x\|_p \|y\|_q, \quad \frac{1}{p} + \frac{1}{q} = 1.$$

The exponents are **conjugate** ($1/p + 1/q = 1$), not arbitrary. Common special cases: - $p = \infty$, $q = 1$ (the most common). - $p = 1$, $q = \infty$ (the symmetric version, used in the equalization application below). - $p = 2$, $q = 2$ (recovers Cauchy–Schwarz, since $\|x \odot y\|_1 \geq |\langle x, y \rangle|$).

So this Hölder inequality is **stronger** than bounding $|\langle x, y \rangle|$ directly.

Clarification (student Q&A). The inner product here is the **Euclidean** inner product — there is **no inner product that induces the ℓ_1 or ℓ_∞ norm** (only the 2-norm comes from an inner product). Cauchy–Schwarz in its general form holds in any **inner product space** with the **induced** norm; here we are in a normed space bounding the Euclidean inner product by **non-induced** norms (ℓ_p, ℓ_q). The full inner-product-space treatment comes later.

Use: this Hölder inequality is exactly what **justifies the shortcut formulas** for the induced 1-1 and ∞ - ∞ matrix norms (each matrix-norm entry involves a row $\times$ vector inner product, bounded by the relevant p/q norms).

For the 1-1 norm, let a_j be column j . For $\|x\|_1 = 1$,

$$\|Ax\|_1 = \sum_i \left| \sum_j a_{ij} x_j \right| \leq \sum_i \sum_j |a_{ij}| |x_j| = \sum_j |x_j| \|a_j\|_1 \leq \max_j \|a_j\|_1.$$

Equality is achieved by choosing $x = e_j$ for a column with maximum ℓ_1 -norm, so $\|A\|_{1,1} = \max_j \|a_j\|_1$.

For the ∞ - ∞ norm, let r_i be row i . For $\|x\|_\infty = 1$,

$$\|Ax\|_\infty = \max_i |\langle r_i, x \rangle| \leq \max_i \|r_i\|_1 \|x\|_\infty = \max_i \|r_i\|_1.$$

For a row attaining the maximum, choose x_j with signs matching the conjugates/signs of that row’s entries; then the bound is attained. Thus $\|A\|_{\infty,\infty} = \max_i \|r_i\|_1$.

Application: Communication-Channel Equalization

The Hölder/ ℓ_1 - ℓ_∞ inequality has a real engineering use in **channel equalization** (one of the instructor’s research areas).

Setup. In digital communications, bits are sent as a sequence of ± 1 symbols (logic 1 $\rightarrow +1$, logic 0 $\rightarrow -1$). Passing through a physical channel (reflections off “mountains,” multiple delayed copies), the signal

is **scrambled** — this is **inter-symbol interference**, modeled as **convolution** with the channel impulse response h . An ideal channel would be a **delta** (all zeros except one tap): no scrambling.

Equalizer. At the receiver we add a filter and convolve, producing an **overall channel** $g = h * (\text{equalizer})$, under our control. We want g to be as close to a **delta** as possible — i.e., **sparse**, with small ℓ_1 -norm.

The inequality at work. The equalizer output is an **inner product** of the overall channel g with the (bounded) input symbol sequence. By Hölder with $p = 1, q = \infty$: the inner product is bounded by $\|g\|_1 \|\text{symbols}\|_\infty$, and since the symbols are ± 1 (ℓ_∞ -norm = 1), **minimizing the peak (worst-case) output magnitude is equivalent to minimizing the ℓ_1 -norm of the overall channel** — which **sparsifies** g , inverting the channel without training. This connects matrix norms, the Hölder inequality, and sparsity into a practical equalization method.

Toward SVD

The image of the unit sphere under A is an **ellipsoid** whose principal semi-axes have lengths equal to the **singular values** $\sigma_1 \geq \sigma_2 \geq \dots$, point along the **left singular vectors** u_k , and correspond to input directions the **right singular vectors** v_k , with $Av_k = \sigma_k u_k$. This sphere $\rightarrow$ ellipsoid picture is the geometric heart of the **Singular Value Decomposition** — “the best thing that happened to us” — which writes a matrix as **unitary $\times$ diagonal $\times$ unitary** (two unitary matrices, one real nonneg diagonal), gives a clean picture of the four fundamental subspaces, and underlies countless algorithms and proofs. Developed in L21.

Instructor Remarks and Study Guidance

- **Frobenius $\neq$ operator norm.** $\|A\|_F = \sqrt{\sum \sigma_k^2}$ (flatten + ℓ_2); $\|A\|_2 = \sigma_1$ (induced 2-2). Both use ℓ_2 but differently; this is a classic point of confusion. Write $\|A\|_F$ explicitly.
- **Induced norm = maximum gain** over the unit input ball; restrict to the ball boundary, and for polytope balls, to the **vertices**.
- **Shortcuts:** 1-1 = max **column** ℓ_1 ; ∞ - ∞ = max **row** ℓ_1 ; 2-2 = $\sigma_1 = \sqrt{\lambda_{\max}(A^*A)}$, valid even for singular A .
- **Frobenius and 2-2 norms are unitarily invariant** (hence functions of singular values only); ℓ_1 and ℓ_∞ are not.
- **Hölder inequality** $\|x \odot y\|_1 \leq \|x\|_p \|y\|_q$ with $1/p + 1/q = 1$ is stronger than bounding $|\langle x, y \rangle|$; it justifies the matrix-norm shortcut formulas and underlies the channel-equalization sparsity application. Only the 2-norm comes from an inner product.
- **Nuclear norm** (sum of singular values) is coming — the engine of low-rank approximation and the Netflix challenge.

Source and Coverage Note

Source: `corrected/lecture20_corrected.md`.

Coverage: Two approaches to matrix norms (flatten/Frobenius vs. induced); Frobenius definition, trace formulas; induced-norm definition as maximum gain with scale-invariance and unit-ball-boundary reduction; geometric calculation strategy and the line-segment/convex-combination lemma with polytope/polyhedron terminology; worked 1-1 norm for $A = [1, 2; 0, 2]$ (diamond vertices $\rightarrow$ parallelogram, max $\ell_1 = 4$, output ℓ_1 -ball last-contact cue, max-column shortcut) and worked ∞ - ∞ norm (square $\rightarrow$ parallelogram, max $\ell_\infty = 3$, max-row shortcut); induced 2-2 norm (sphere $\rightarrow$ ellipse derivation $y^*(AA^*)^{-1}y = 1$, output-side $AA^* = [5, 4; 4, 4]$, inverse-eigenvalue coordinate equation, longest semi-axis = $\sqrt{\lambda_{\max}} = \sigma_1$, worked eigenvalues 8.53/0.47 of both AA^* and $A^*A = [1, 2; 2, 8]$, drawing/not-to-scale correction, operator/spectral-norm notation warning, non-invertible degenerate case and QR/reduced-form remark, invertibility not required); shortcut-formula summary table; student Q&As (affine-map lifting/total-least-squares upper bound, averaging norms, nuclear-norm/Netflix preview with the recurring-joke anecdote); unitary-invariant norms (2-norm

viewpoint flip, Frobenius invariance via trace proof, 2-2 invariance via explicit maximum-ratio proof, ℓ_1/ℓ_∞ not invariant); norm inequalities (Cauchy–Schwarz recap, Hadamard product with $\odot$ notation, Hölder inequality $\|x \odot y\|_1 \leq \|x\|_p \|y\|_q$ with conjugate exponents and special cases, clarification that only the 2-norm is inner-product-induced, proof sketches for the 1-1 and ∞ - ∞ shortcuts); communication-channel equalization application (ISI as convolution, overall channel sparsification, peak-minimization $\Leftrightarrow \ell_1$ -minimization via Hölder); SVD preview.

Lecture 21 Notes

Recap: Matrix Norms Leading to SVD

The instructor recaps the matrix-norm results from L20:

- One way to define matrix norms is to **vectorize** A and apply a familiar vector p -norm. This treats the matrix as a vector in $\mathbb{C}^{mn}$, not as a linear map.
- **Frobenius norm** (flatten then 2-norm): $\|A\|_F = (\sum_{i,j} |a_{ij}|^2)^{1/2}$ — sums the energies of all entries; does **not** care where the entries sit (no linear-map perspective).
- **Induced norms** treat $A : \mathbb{C}^n \rightarrow \mathbb{C}^m$ as a mapping; $\|A\|_{q,p} = \sup_{x \neq 0} \|Ax\|_q / \|x\|_p = \sup_{\|x\|_p=1} \|Ax\|_q$.
 - $\|A\|_{1,1} = \max \text{column } \ell_1\text{-norm.}$
 - $\|A\|_{\infty,\infty} = \max \text{row } \ell_1\text{-norm.}$
 - $\|A\|_{2,2} = \sqrt{\lambda_{\max}(A^*A)}$ = length of the **longest principal semi-axis** of the ellipsoid that is the image of the unit ℓ_2 -ball.

That last picture — **unit sphere maps to an ellipsoid** — is the geometric center of the **Singular Value Decomposition (SVD)**. This lecture presents the SVD from an **algebraic** view and a **geometric** view, then gives a **formal existence proof**.

SVD: The Algebraic View

The SVD states that **any** matrix A (rectangular allowed) can be written

$$A = U\Sigma V^*,$$

where: - U is $m \times m$ **unitary** (left singular vectors), - V is $n \times n$ **unitary** (right singular vectors), - Σ is $m \times n$ **rectangular diagonal** (the (i,i) entries may be nonzero, all others zero), with **real, non-negative** diagonal entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

The relative sizes follow the shape of A : if A is **fat**, V is the larger unitary; if A is **tall**, U is larger. This fits the course storyline of writing A as a product of **simple matrices** — here **two unitaries and one (real, nonneg) diagonal**.

On the conjugate transpose V^* . It is a convention to write the third factor as V^* rather than V : multiplying by V^* means taking inner products with the right singular vectors. (Geometrically V might read more naturally, but the $U\Sigma V^*$ convention is standard.)

Connection to (and Departure from) Similarity

Recall the **similarity** story for a **square** $A : \mathbb{C}^n \rightarrow \mathbb{C}^n$: pick **one** basis T for **both** input and output; the map's new representation is $T^{-1}AT$; we asked whether we can choose T to make this **diagonal**. Answer: **not for all** matrices (only diagonalizable ones). The restriction was using the **same basis** for input and output.

The key step to SVD: drop that restriction. Use a basis T_1 for the input and a **different** basis T_2 for the output; the representation becomes $T_2^{-1}AT_1$. Question: can we always find T_1, T_2 making this **diagonal**, even for **rectangular** A (where input and output dimensions differ, so the **same** basis is impossible)? **Answer: yes — always — using orthonormal bases.** That is exactly the SVD: choosing one orthonormal basis for the input (V) and one for the output (U) renders **any** linear map diagonal, with real nonneg diagonal entries. “I can represent any linear mapping by a diagonal matrix if I choose my coordinate axes wisely — one set for the input, one for the output.”

SVD: The Geometric View

Image of a Sphere Is an Ellipsoid

Two facts about linear maps: 1. The image of a **line segment** is a line segment (shown earlier for the ℓ_1 -ball). 2. The image of an **ellipsoid** (in particular the unit sphere) under a linear map is an **ellipsoid** — shape preserved.

A **line segment is a degenerate ellipsoid** (one semi-axis has length zero). So degenerate cases fit the same picture: - A **rank-1** 2×2 map sends the circle to a line segment (an ellipse with second semi-axis = 0). - A map from 3-D to 2-D sends a sphere to an ellipse in the plane; if the matrix is not full rank, that ellipse can collapse to a line segment. - A **tall** 3×2 map (rank ≤ 2) sends a 2-D circle to an ellipse lying in a plane through the origin (the range space). If the rank is only 1, the image is again a line segment.

The instructor notes that this geometric discussion is based on the hand-drawn/pre-lecture figure: a sphere/circle maps to an ellipsoid/ellipse, with degenerate cases handled by allowing zero semi-axis lengths.

Student Q&A: What Is an Ellipsoid?

A student asks for the definition. An ellipsoid is the **level set of a quadratic function**:

$$\{x : (x - x_0)^* A (x - x_0) \leq b\},$$

with A **positive definite**. (A positive **semidefinite** A gives a **degenerate** ellipsoid — some dimensions collapse.) Geometrically you “cut” the quadratic at a level and look at the region in the domain.

Assigning Singular Values and Vectors

Accept (formal proof later) that the image of the unit sphere is an ellipsoid. Label its components: - The **first (longest) principal semi-axis** has length σ_1 along the unit vector u_1 : the axis is $\sigma_1 u_1$. Its **pre-image** on the input sphere is the unit vector v_1 . So $Av_1 = \sigma_1 u_1$. - The **second** semi-axis is $\sigma_2 u_2$, with pre-image v_2 : $Av_2 = \sigma_2 u_2$. - For a rank- r matrix there are r nonzero semi-axes $\sigma_1 \geq \dots \geq \sigma_r > 0$ with $Av_k = \sigma_k u_k$, $k = 1, \dots, r$.

Student check: why does such a v_k exist? Because $\sigma_k u_k$ is chosen as a point on the image of the unit sphere under A ; by definition of image, at least one unit input vector maps to that point.

The u_k (semi-axis directions in the **output** space) are **orthogonal by definition** of principal axes. The pre-images v_k (in the **input** space) are also **orthonormal** — not obvious, proved formally below.

Matrix Form (Reduced)

Collecting the pre-images as columns of $\hat{V} = [v_1 \dots v_r]$ ($n \times r$) and the axis directions as $\hat{U} = [u_1 \dots u_r]$ ($m \times r$):

$$A\hat{V} = \hat{U}\hat{\Sigma}, \quad \hat{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_r) \text{ } (r \times r, \text{ positive diagonal}).$$

This relation records the nonzero action of A : the v_k 's generate the principal semi-axes. After the null-space directions are added and the zero actions are accounted for, the zero terms can be removed again to give the **reduced (compact) SVD**

$$A = \hat{U}\hat{\Sigma}\hat{V}^*.$$

Extending to the Full SVD and the Four Fundamental Subspaces

$\hat{V}$ and $\hat{U}$ have only r orthonormal columns (orthonormal, but not unitary). Extend: - Add $n - r$ orthonormal columns to $\hat{V} \rightarrow$ unitary V . The added columns are arbitrary subject to completing an orthonormal basis, and in this construction they span the null space: $Av_k = 0$ for $k > r$ (they are silenced by the zero columns/diagonal positions of Σ). - Add $m - r$ orthonormal columns to $\hat{U} \rightarrow$ unitary U . These added columns are also arbitrary subject to completing U to unitary; they do not contribute to A because their coefficients in Σ are zero.

The $r \times r$ positive-diagonal block thus extends to the $m \times n$ rectangular diagonal Σ (first r diagonal entries nonzero, rest zero). The geometric construction first gives

$$AV = U\Sigma.$$

Multiplying on the right by V^* gives the **full SVD** $A = U\Sigma V^*$. The columns immediately give **orthonormal bases for all four fundamental subspaces**:

Subspace	Basis	SVD columns
Row space $\mathcal{R}(A^*)$	$v_1, \dots, v_r$	first r cols of V (nonzero outputs)
Null space $\mathcal{N}(A)$	$v_{r+1}, \dots, v_n$	last $n - r$ cols of V ($Av_k = 0$)
Column space $\mathcal{R}(A)$	$u_1, \dots, u_r$	first r cols of U
Left null space $\mathcal{N}(A^*)$	$u_{r+1}, \dots, u_m$	last $m - r$ cols of U

So the SVD hands us the **rank** (number of nonzero σ_k) and orthonormal bases for all four subspaces at once — “an excellent analysis tool.”

Outer-Product (Rank-1 Sum) Form

Expanding $U\Sigma V^*$:

$$A = \sum_{k=1}^r \sigma_k u_k v_k^*.$$

The multiplication is: $\hat{U}\hat{\Sigma} = [\sigma_1 u_1 \ \cdots \ \sigma_r u_r]$, while $\hat{V}^*$ has rows $v_1^*, \dots, v_r^*$; multiplying column-by-row gives the sum of outer products.

Each $\sigma_k u_k v_k^*$ is **rank one** (column vector $\times$ row vector). Summing r of them gives a rank- r matrix. Ordered by $\sigma_1 \geq \sigma_2 \geq \dots$, the first term is the “most important.” This is the form you see in many papers; it is an **orthonormal expansion** of A in a basis **adapted to A itself** (derived from A).

The Matrix Inner Product and Orthogonality of Rank-1 Components

The rank-1 terms $\sigma_k u_k v_k^*$ are **orthogonal** with respect to the **matrix inner product** — the extension of the Euclidean inner product to matrices:

$$\langle A, B \rangle = \text{tr}(B^* A).$$

Proof of orthogonality. For $i \neq j$,

$$\langle u_i v_i^*, u_j v_j^* \rangle = \text{tr}((u_j v_j^*)^* (u_i v_i^*)) = \text{tr}(v_j u_j^* u_i v_i^*).$$

Because the left singular vectors are orthonormal, $u_j^* u_i = 0$ for $i \neq j$, so the whole trace is 0. Hence the rank-1 SVD components form an **orthonormal basis** (up to the σ_k scaling) for the subspace they span, in the Frobenius/matrix inner product. (Inner-product spaces for matrices are developed formally in L23.)

Connection to PCA

The SVD is essentially **Principal Component Analysis (PCA)** applied to a **data matrix**. Suppose you observe data vectors $x_1, \dots, x_n$ and form the **sample correlation matrix**

$$R = \frac{1}{n} \sum_{i=1}^n x_i x_i^* = \frac{1}{n} X X^*, \quad X = [x_1 \ \cdots \ x_n] \text{ (data/snapshot matrix).}$$

PCA looks at the **eigenvalue decomposition** of R and asks which eigenvalues (directions) are most significant. But computing the **SVD of the data matrix X directly** yields the **same** eigenvectors: the left singular vectors of X are the eigenvectors of $R = \frac{1}{n}XX^*$. So **PCA = SVD on the data matrix** (rather than eigendecomposition of the correlation matrix). This is the standard, numerically preferable route.

How Singular Values/Vectors Are Computed (Preview)

The singular values and **left/right singular vectors** come from the **eigenvalue decomposition of A^*A or AA^*** (detailed in L22): $\sigma_k = \sqrt{\lambda_k(A^*A)}$, with v_k the eigenvectors of A^*A and u_k of AA^* . There are efficient algorithms; the main conceptual route is via A^*A .

Formal Proof of the SVD (by Induced 2-Norm and Block Diagonalization)

This is the centerpiece of the lecture: an existence proof modeled on the **Schur decomposition** proof (extend a vector to an orthonormal basis, get a block form, recurse). The recalled Schur result was that every square matrix is unitarily triangularizable; here the same extension idea is used to get block diagonal pieces for SVD.

Setup

Let $\sigma_1 = \|A\|_{2,2}$, the induced 2-norm — the **maximum Euclidean gain**, which is the longest principal semi-axis from the geometric picture. By definition of the induced norm, there exist **unit vectors** v_1 (input) and u_1 (output) with

$$Av_1 = \sigma_1 u_1, \quad \|v_1\|_2 = \|u_1\|_2 = 1.$$

Extend to Orthonormal Bases

As in the Schur proof: extend u_1 to an orthonormal basis $u_1, u'_2, \dots, u'_m$ of the output space (forming unitary U_1), and extend v_1 to an orthonormal basis $v_1, v'_2, \dots, v'_n$ of the input space (forming unitary V_1). (The primed vectors are **not** the final singular vectors; they merely complete the bases.)

Form $S_1 = U_1^* A V_1$

Compute $S_1 = U_1^* A V_1$. The first column: $Av_1 = \sigma_1 u_1$, and

$$u_1^*(\sigma_1 u_1) = \sigma_1 \text{ (since } u_1^* u_1 = 1), \quad (u'_k)^*(\sigma_1 u_1) = 0 \text{ (orthogonality)}.$$

So the first column of S_1 is $(\sigma_1, 0, \dots, 0)^\top$. The top row has entries $w_k^* = u_1^* A v'_k$ (call this row vector w^*). Thus

$$S_1 = \begin{bmatrix} \sigma_1 & w^* \\ 0 & B \end{bmatrix},$$

a **block upper-triangular-looking** form. The claim to prove: $w = 0$, which makes S_1 block **diagonal** and lets the proof recurse.

The Key Trick: $w = 0$ from Unitary Invariance of the 2-Norm

The instructor recalls the relevant unitary-invariance facts: the Euclidean vector 2-norm is unitarily invariant, the Frobenius norm is unitarily invariant for matrices, and the induced 2-2 norm is also unitarily invariant. Since $S_1 = U_1^* A V_1$ is A sandwiched by unitaries, S_1 has the **same** 2-norm as A :

$$\|S_1\|_{2,2} = \|A\|_{2,2} = \sigma_1.$$

So for **every** unit vector x , $\|S_1x\|_2 \leq \sigma_1$. Choose the **clever** unit vector built from the first row of S_1 :

$$x = \frac{1}{\sqrt{\sigma_1^2 + \|w\|_2^2}} \begin{bmatrix} \sigma_1 \\ w \end{bmatrix}$$

(its norm is 1 because the denominator is exactly the norm of $[\sigma_1; w]$). Then

$$S_1x = \frac{1}{\sqrt{\sigma_1^2 + \|w\|_2^2}} \begin{bmatrix} \sigma_1^2 + \|w\|^2 \\ Bw \end{bmatrix},$$

so its squared norm is

$$\|S_1x\|_2^2 = \frac{(\sigma_1^2 + \|w\|^2)^2 + \|Bw\|^2}{\sigma_1^2 + \|w\|^2} = (\sigma_1^2 + \|w\|^2) + \frac{\|Bw\|^2}{\sigma_1^2 + \|w\|^2}.$$

This must be $\leq \sigma_1^2$. The second term is **non-negative**, so already

$$\sigma_1^2 + \|w\|^2 \leq \|S_1x\|_2^2 \leq \sigma_1^2 \implies \|w\|^2 \leq 0.$$

A norm cannot be negative, so $\|w\|^2 = 0$, i.e. $w = 0$. Therefore

$$S_1 = \begin{bmatrix} \sigma_1 & 0 \\ 0 & B \end{bmatrix}$$

is **block diagonal**.

Recurse

Now apply the identical argument to the smaller $(m-1) \times (n-1)$ block B : its induced 2-norm is σ_2 ($\leq \sigma_1$), giving a unit u_2, v_2 , another orthonormal extension, and another block diagonalization $\begin{bmatrix} \sigma_2 & 0 \\ 0 & C \end{bmatrix}$. Continuing peels off $\sigma_1 \geq \sigma_2 \geq \dots$ down the diagonal, accumulating unitary factors, and **proves the SVD** $A = U\Sigma V^*$. (The ordering $\sigma_1 \geq \sigma_2 \geq \dots$ follows because each step takes the 2-norm of a submatrix of the previous.) This also confirms the geometric claim that the **pre-images** v_k **are orthonormal**.

SVD vs. Eigenvalue Decomposition (Summary)

Property	Eigenvalue Decomposition	SVD
Form	$A = T\Lambda T^{-1}$	$A = U\Sigma V^*$
Applies to	square only	any (rectangular) matrix
Bases	one basis (same for in/out)	two orthonormal bases
Side matrices	invertible T (unitary iff normal)	unitary U , unitary V
Diagonal entries	complex eigenvalues	real non-negative singular values
Always exists?	no (not all diagonalizable)	yes (always)

Eigendecomposition diagonalizes using the **same** basis (when possible); for **normal** matrices that basis is unitary. The SVD uses **different** orthonormal bases for input and output, works for **any** matrix, and **always exists** — which is why it is so powerful.

Instructor Remarks and Study Guidance

- The conceptual leap to SVD is **dropping the same-basis restriction** of similarity: two orthonormal bases (one in, one out) diagonalize **any** matrix.
- Geometrically, A maps the unit sphere to an **ellipsoid**; σ_k = semi-axis lengths, u_k = axis directions (output), v_k = pre-images (input), with $Av_k = \sigma_k u_k$. Degenerate (rank-deficient) cases give flattened ellipsoids.
- The **rank** is the number of nonzero singular values; the **four fundamental subspaces** read directly off the columns of U and V .
- The rank-1 components $\sigma_k u_k v_k^*$ are **orthogonal in the matrix inner product** $\langle A, B \rangle = \text{tr}(B^* A)$ (proof via $u_i^* u_j = 0$).
- **PCA = SVD of the data matrix** (equivalently eigendecomposition of the correlation matrix).
- The **existence proof**: $\sigma_1 = \|A\|_{2,2}$ gives $Av_1 = \sigma_1 u_1$; extend to orthonormal bases; unitary invariance of the 2-norm + a clever unit vector forces the off-diagonal row $w = 0$, giving a block-diagonal S_1 ; recurse. Modeled on the Schur proof.
- Singular values come from the eigenvalues of $A^* A$ (or AA^*); detailed in L22. The instructor says there are efficient algorithms, but the lecture's main conceptual route is through these eigenvalue decompositions.
- Next lecture will cover uses of SVD and additional matrix norms formed by combining the singular values/principal semi-axis lengths.
- The starting point of SVD is the **2-2 norm** — the instructor notes he has not seen an analogous construction starting from the 1-1 or ∞ - ∞ norm (an open curiosity).

Source and Coverage Note

Source: `corrected/lecture21_corrected.md`.

Coverage: Matrix-norm recap motivating SVD, including vectorization-based matrix norms and the Frobenius example; algebraic view ($A = U\Sigma V^*$, shapes, real nonneg diagonal, V^* convention); connection to and departure from similarity (drop the same-basis restriction $\rightarrow$ two orthonormal bases diagonalize any matrix); geometric view (sphere $\rightarrow$ ellipsoid, line segment as degenerate ellipsoid, rank-deficient and tall/fat degenerate cases, hand-drawn figure remark, ellipsoid-as-quadratic-level-set student Q&A, preimage-existence check, assigning σ_k, u_k, v_k with $Av_k = \sigma_k u_k$); reduced and full SVD with $AV = U\Sigma$, multiplication by V^* , extension to unitary matrices, arbitrary completion columns, null-space zero action, and the four fundamental subspaces table; outer-product/rank-1 sum form with multiplication steps as an A -adaptive orthonormal expansion; matrix inner product $\langle A, B \rangle = \text{tr}(B^* A)$ and proof that the rank-1 components are orthogonal; PCA connection (SVD of data matrix = eigendecomposition of correlation matrix); preview of computing singular values via $A^* A / AA^*$ and efficient algorithms; full formal existence proof (induced 2-norm σ_1 , orthonormal-basis extension à la Schur, Schur triangularization recall, $S_1 = U_1^* A V_1$ block form, unitary-invariance facts + clever-unit-vector argument forcing $w = 0$, recursion); SVD-vs-eigendecomposition comparison table; next-lecture remarks on SVD uses and norms built from singular values; instructor curiosity about whether analogous constructions exist for 1-1 or ∞ - ∞ norms.

Lecture 22 Notes

Recap: Full and Reduced SVD

The instructor recaps the SVD established in L21.

- **Singular values:** the diagonal entries of Σ , real and non-negative; only r of them are nonzero.
- **Left singular vectors:** $u_1, \dots, u_m$ (columns of U); **right singular vectors:** $v_1, \dots, v_n$ (columns of V).
- **Full SVD:** $A = U\Sigma V^*$ with U ($m \times m$) and V ($n \times n$) **unitary** and Σ ($m \times n$) rectangular diagonal.
- **Reduced SVD:** eliminate the zero diagonal entries of Σ (which silence the corresponding columns of U and V); what remains are the **non-degenerate principal semi-axis directions** $u_1, \dots, u_r$ and their pre-images $v_1, \dots, v_r$, giving $A = \hat{U}\hat{\Sigma}\hat{V}^*$ with an $r \times r$ positive diagonal.
- **Outer-product form:** $A = \sum_{k=1}^r \sigma_k u_k v_k^*$, a sum of rank-1 terms that (with respect to the matrix inner product) are **orthogonal**. The instructor describes this as writing A in a matrix basis **adapted to A** : the full family can be completed from singular-vector outer products, while A itself uses only the diagonal/rank-1 SVD terms weighted by σ_k .

The geometric origin: the image of a hypersphere under A is an **ellipsoid**; the σ_k are the principal semi-axis lengths, the u_k the axis directions, the v_k their pre-images.

SVD as the Ultimate Matrix-Analysis Tool

The instructor stresses the SVD is a **perfect matrix / linear-mapping analysis tool** — “like the miracle products they sell on TV,” advertising feature after feature. Given the SVD of A , you can read off:

1. **Rank:** the number of nonzero singular values = r (the number of non-degenerate ellipsoid dimensions = dimension of range = dimension of row space).
2. **Column space $\mathcal{R}(A)$:** orthonormal basis $u_1, \dots, u_r$ (first r columns of U).
3. **Left null space $\mathcal{N}(A^*)$:** $u_{r+1}, \dots, u_m$ (orthogonal to the column space).
4. **Row space $\mathcal{R}(A^*)$:** $v_1, \dots, v_r$ (first r columns of V).
5. **Null space $\mathcal{N}(A)$:** $v_{r+1}, \dots, v_n$.
6. **Induced 2-norm:** $\|A\|_{2,2} = \sigma_1$ (recall the proof started by setting $\sigma_1 = \|A\|_{2,2}$).
7. **Frobenius norm:** $\|A\|_F = \sqrt{\sum_k \sigma_k^2}$ (derived below).

Projecting a vector onto the range of A is then **easy**: take inner products with the orthonormal $u_1, \dots, u_r$ and recombine.

Why Singular Values Are Real and Non-Negative

Geometric reason. In the construction, $\sigma_1 = \|A\|_{2,2}$ is a **norm** (a length of a principal semi-axis), hence real and ≥ 0 . Each subsequent σ_k is the 2-norm of a smaller block in the recursive proof — also a norm. So all σ_k are non-negative reals; off-diagonal entries of Σ are zero. Hence Σ is a **real non-negative diagonal** matrix, and the count of nonzero diagonal entries is the rank.

Algebraic reason (via A^*A). From $A = U\Sigma V^*$,

$$A^*A = V\Sigma^*U^*U\Sigma V^* = V(\Sigma^*\Sigma)V^*,$$

the eigendecomposition of A^*A with eigenvalues σ_k^2 (diagonal of $\Sigma^*\Sigma$) and eigenvectors V . Since $A^*A \succeq 0$ (proved earlier: $x^*A^*Ax = \|Ax\|^2 \geq 0$), the eigenvalues $\sigma_k^2 \geq 0$, so $\sigma_k = \sqrt{\lambda_k(A^*A)}$ is a real non-negative number.

Frobenius Norm from Singular Values

$$\|A\|_F = \sqrt{\text{tr}(A^*A)}.$$

Substitute the SVD. With $A = U\Sigma V^*$ and $A^* = V\Sigma^\top U^*$ (order reverses; Σ is real):

$$\text{tr}(A^*A) = \text{tr}(V\Sigma^\top U^*U\Sigma V^*) = \text{tr}(V\Sigma^\top \Sigma V^*).$$

The $U^*U = I$ cancels; then by the cyclic property $\text{tr}(VXV^*) = \text{tr}(V^*VX) = \text{tr}(X)$:

$$\text{tr}(A^*A) = \text{tr}(\Sigma^\top \Sigma).$$

Now $\Sigma^\top \Sigma$ is diagonal with entries $\sigma_1^2, \dots, \sigma_r^2, 0, \dots$, whose trace is $\sum_k \sigma_k^2$. Therefore

$$\boxed{\|A\|_F = \sqrt{\sigma_1^2 + \dots + \sigma_r^2}}.$$

Alternative (unitary invariance). Frobenius is unitarily invariant, so $\|A\|_F = \|U\Sigma V^*\|_F = \|\Sigma\|_F = \sqrt{\sum_k \sigma_k^2}$ directly. The instructor phrases this as the **energy of the singular-value vector**. Both $\|A\|_{2,2} = \sigma_1$ and $\|A\|_F$ depend **only on the singular values**, because both are unitarily invariant.

Computing the SVD via A^*A or AA^*

This is **not** the numerically preferred algorithm, but it shows the **connection between SVD and eigendecomposition**.

- **Via A^*A :** $A^*A = V(\Sigma^\top \Sigma)V^*$. It is Hermitian and **positive semidefinite** (always), so it has a unitary eigendecomposition. Its eigenvectors are the **right singular vectors** V ; its eigenvalues are σ_k^2 , giving the singular values. Then recover the left singular vectors from $Av_k = \sigma_k u_k$, i.e. $u_k = Av_k/\sigma_k$ — **no second eigendecomposition needed**.
- **Via AA^* :** $AA^* = U(\Sigma \Sigma^\top)U^*$; its eigenvectors are the **left singular vectors** U .

(Shape note: if A is fat, A^* is tall; if A is full-rank fat then AA^* is positive **definite**, but A^*A or AA^* is always at least positive **semidefinite** and Hermitian — hence normal, hence unitarily diagonalizable.)

Numerical warning. In practice forming A^*A is **inadvisable** (it squares the condition number). Built-in routines (MATLAB `svd`, NumPy `numpy.linalg.svd`) work directly on A (bidiagonalization), avoiding A^*A . The instructor doesn't know the exact state-of-the-art algorithm but confirms it avoids the explicit product.

Low-Rank Approximation (Eckart–Young)

A major application area.

Problem. Given A , find a rank- p matrix B as close to A as possible in Frobenius norm:

$$\min_{\text{rank}(B) \leq p} \|A - B\|_F.$$

The transcript states the constraint as “rank p ”; the standard formulation is rank **at most** p . The truncated SVD has rank at most p , and it has rank exactly p when the first p retained singular values are nonzero.

Why it's hard. The objective (a norm) is **convex** — “a pleasure to optimize.” But the **constraint set** (rank- p matrices) is **not convex**: the convex combination of two rank- p matrices can have rank up to $2p$. (Example: two orthogonal rank-1 matrices averaged 0.5/0.5 give a rank-2 matrix.) So the feasible region is non-convex, making the problem hard in general.

Solution (Eckart–Young). Despite non-convexity, the SVD gives the **exact global optimum**. Because the singular values are **ordered** $\sigma_1 \geq \sigma_2 \geq \dots$ (monotonically non-increasing, by construction), simply **truncate** the outer-product sum at p :

$$A_p = \sum_{k=1}^p \sigma_k u_k v_k^*,$$

discarding the $r - p$ smallest singular values. The error is

$$\|A - A_p\|_F = \sqrt{\sum_{k=p+1}^r \sigma_k^2}.$$

The instructor does **not** prove this theorem in lecture; he states the result and moves to implications/applications.

The same truncation is optimal in the induced 2-norm, with error $\|A - A_p\|_{2,2} = \sigma_{p+1}$. The instructor notes he mentions this only after emphasizing the ordering, which is “critical” for this application. For other norms, e.g. the **1-1 norm**, he says he does not know a closed-form solution; changing the norm can force non-convex heuristics. Thus the **choice of norm determines both the meaning of “close” and the tractability** of the problem.

Application: Image Compression

A grayscale image is a matrix; the running example is the **guitar-player** photo. The source aside identifies the musician with Deep Purple, but the transcribed name is garbled. The image began as color/RGB and was converted to a monochrome intensity matrix of size $1236 \times 2060 \approx 2.5$ million pixel values.

Compute the SVD and keep only the first k rank-1 terms: $A_k = \sum_{i=1}^k \sigma_i u_i v_i^*$. **Storage:** instead of mn numbers, store k singular values, km numbers for the u ’s, and kn for the v ’s — total $k(m + n + 1)$. For $m \approx 1000, n \approx 2000$: - **Rank-1** ($k = 1$): you “barely have some idea” what the image is — a poor representation. - **Rank-10**: $\approx 10 \cdot 1000 + 10 \cdot 2000 + 10 \approx 30,000$ numbers vs. 2 million. - **Rank-20**: $\approx 60,000$ vs. 2 million.

The point of the storage count is that we store the **factors** u_i , v_i , and σ_i , not the already-multiplied reconstructed image. The instructor repeatedly frames the comparison as roughly **2 million full pixel values** versus **30,000** (rank 10) or **60,000** (rank 20) stored factor entries.

The error (in Frobenius/MSE sense, since Frobenius measures squared error) is small for modest k because the **singular values decay rapidly** for natural images (the first is large; most are near zero — best seen on a log scale). This is **not** how real image compression is done, but it illustrates the principle: represent the data in a basis where energy concentrates in a few coefficients. (Netflix matrix completion — discussed later — is another low-rank application.)

Closest Unitary Matrix to a Given Matrix

A second SVD application. **Problem:** given a square matrix A (not unitary), find the unitary Q ($QQ^* = I$) closest to it in **Frobenius** norm:

$$\min_{QQ^*=I} \|A - Q\|_F^2.$$

Again the objective is convex but the **set of unitary matrices is non-convex** (the convex combination of two unitaries need not be unitary), so it looks hard. But the SVD gives a **closed-form** solution.

The Frobenius norm is chosen deliberately: the instructor says the problem is hard for other norms, while Frobenius gives a clean analytic formula. He also mentions that unitary approximation appears in some recent neural-network algorithms.

Scalar analogy (1×1). For a complex number $a = re^{j\theta}$, the closest unit-magnitude number is $e^{j\theta}$ — obtained by the **polar decomposition** and replacing r with 1.

Derivation. Write $A = U\Sigma V^*$ and use Frobenius unitary invariance:

$$\|A - Q\|_F = \|U\Sigma V^* - Q\|_F = \|\Sigma - U^*QV\|_F = \|\Sigma - Q'\|_F,$$

where $Q' = U^*QV$ is unitary (product of unitaries). Minimizing the distance from a **non-negative diagonal** Σ to a unitary Q' is solved by $Q' = I$. Hence

$$Q = UV^*,$$

i.e. **find the SVD, set all singular values to 1, and multiply** UV^* — clearly unitary, and the closest unitary to A in Frobenius norm.

Application: Rigid Motion / Image Registration (Computer Vision)

Setup. In one scene a (rigid) object sits at some location; in a second scene it has **moved without changing shape**. We want the motion = **rotation** (a unitary/orthogonal Θ) **plus translation** t . Register matching points $x_1, x_2, \dots$ in the first scene and $y_1, y_2, \dots$ in the second; ideally each $y_i = \Theta x_i + t$. Stacking points as columns of X and Y (and adding t to each column via $t\mathbf{1}^\top$):

$$Y \approx \Theta X + t\mathbf{1}^\top.$$

Why it's not exact: registration noise, and projecting a 3-D world to 2-D. So minimize the **least-squares** error:

$$\min_{\Theta, t} \|Y - \Theta X - t\mathbf{1}^\top\|_F^2,$$

which (as a least-squares problem, revisited under inner-product spaces) has a nice solution — **but** it ignores the **constraint** that Θ be unitary ($\Theta\Theta^* = I$). Two strategies: 1. **Project after solving:** solve the unconstrained least squares to get a general Θ^* , then find the **closest unitary matrix** to Θ^* via UV^* . (Simple but not optimal.) 2. **Gradient on the unitary manifold:** take a gradient step that leaves the unitary set, then **project back** onto the unitary set (via UV^* from the SVD), repeating — moving “a little bit and coming back” along the manifold.

Related ideas appear in **independent component analysis (ICA)** (minimize mutual information over a unitary matrix, after a PCA whitening step). These are mentioned as buzzwords; details deferred.

The projected-gradient picture is not presented as the only or best constrained method. The instructor says there are many algorithms; in practice one may use something other than the raw gradient, and he skips those details.

Polar Decomposition for Matrices

The complex polar form $c = re^{j\theta}$ ($r > 0$, $e^{j\theta}$ on the unit circle) generalizes to matrices. The instructor calls this a **non-trivial decomposition**: a **non-singular square** A can be written

$$A = PT, \quad P \succ 0 \text{ (positive definite)}, \quad T \text{ unitary.}$$

(“ P ” plays the role of $r > 0$; T the role of $e^{j\theta}$.) **Why it's easy from the SVD:** for non-singular A , Σ is positive (full rank). Insert $U^*U = I$:

$$A = U\Sigma V^* = (U\Sigma U^*)(UV^*).$$

Here $U\Sigma U^*$ is **positive definite** (unitary diagonalization with positive diagonal), and UV^* is **unitary**. So $P = U\Sigma U^*$, $T = UV^*$. **Notably, the unitary factor $T = UV^*$ is exactly the closest unitary matrix to A in Frobenius norm** (from the previous section).

1×1 **check and the correspondence.** In the scalar case P is a positive real number ($= r$) and T is a number on the unit circle ($= e^{j\theta}$) — recovering the complex polar form. A **student asks** about a real/imaginary-part decomposition; the instructor clarifies the right correspondence is via **eigenvalues**, not real/imaginary parts: a general complex matrix lives in an n^2 -dimensional space, so you cannot split it into “two dimensions.” The clean analogy is: - **Hermitian** matrices $\leftrightarrow$ **real** numbers (eigenvalues on the real line), - **positive definite** matrices $\leftrightarrow$ **positive real** numbers, - **unitary** matrices $\leftrightarrow$ numbers on the **unit circle** (eigenvalues on the unit circle). So a unitary matrix generalizes a point on the unit circle, and the polar decomposition generalizes $c = re^{j\theta}$.

Schatten p -Norms

New norms defined from the **singular values**. Collect the singular values into a vector $\sigma = (\sigma_1, \dots, \sigma_r)$. The **Schatten p -norm** of A is the **vector p -norm** of σ :

$$\|A\|_{(p)} = \left(\sum_k \sigma_k^p \right)^{1/p}.$$

(The subscript (p) is written on the **left/inside** to distinguish it from the induced norm.) Special cases: - $p = 2$: $\|A\|_{(2)} = \sqrt{\sum_k \sigma_k^2} = \|A\|_F$ — the **Frobenius** norm. - $p = \infty$: $\|A\|_{(\infty)} = \sigma_1 = \|A\|_{2,2}$ — the **operator** norm. - $p = 1$: $\|A\|_{(1)} = \sum_k \sigma_k$ — the **nuclear norm** (below).

So Frobenius and operator norms are both Schatten norms; the new one is $p = 1$.

Nuclear Norm and Low-Rank via Convex Relaxation

The **Schatten-1 norm** — the **sum of singular values** — is “the star of the show,” researched intensively for two decades. It has its own name and notation:

$$\|A\|_* = \sum_k \sigma_k \quad (\text{nuclear norm, a.k.a. trace norm}),$$

sometimes written as the trace of $(A^*A)^{1/2}$.

Why it matters — convex relaxation of rank. Recall ℓ_1 minimization promotes **sparsity** (many zero entries). Applying the same idea to the **singular value vector** σ : minimizing the nuclear norm $\|A\|_*$ drives most σ_k to **zero** — and the number of nonzero singular values is the **rank**. So **minimizing the nuclear norm minimizes the rank** (with appropriate constraints on A ; otherwise the trivial minimizer is the zero matrix). The nuclear norm is the **convex substitute for rank**, just as ℓ_1 is the convex substitute for sparsity. This gives **low-rank solutions** through a tractable convex problem.

Warning / exam-style caution. The instructor explicitly flags the same caution as in ℓ_1 /CVX-style sparsity problems: minimizing the nuclear norm alone gives the zero matrix. It becomes meaningful only with constraints, e.g. matching observed entries in a completion problem.

Application: Matrix Completion and the Netflix Challenge

Who wants low rank? Matrix completion — central to **recommendation systems** and the **Netflix challenge**.

Setup. Rows = customers (tens/hundreds of millions), columns = movies (perhaps hundreds of thousands). Entry (i, j) is customer i ’s rating of movie j , but each customer has rated only a few movies, so the matrix is **mostly incomplete**. Goal: **fill in** the missing entries to predict whether a customer would like an unseen movie.

Low-rank model. Represent the rating matrix as a product of two thin matrices:

$$\text{Ratings} \approx PQ^\top,$$

where each customer is an r -dimensional **preference vector** (a row of P) and each movie is an r -dimensional **feature vector** (a row of Q). The **inner product** of a user-preference vector and a movie-feature vector gives the predicted **score**. The features may correspond to interpretable factors (is it a horror movie? a comedy? a certain actor present?); a user who weights “comedy” highly scores a comedy highly.

Two approaches: 1. **Fixed-rank factorization:** choose rank r (e.g. 100), and minimize the **Frobenius error over the known entries**:

$$\min_{P,Q} \| \text{Ratings} - PQ^\top \|_F^2 \quad (\text{over observed entries}),$$

recovering the customer matrix P and movie matrix Q . This **low-rank decomposition is the approach that won the Netflix challenge**. 2. **Nuclear-norm minimization:** minimize $\|X\|_*$ subject to matching the observed entries — a convex relaxation that produces a low-rank completion without fixing r in advance.

History: the instructor describes the Netflix challenge as an early-2000s event (the transcript says “2003 / beginning of 2000”) where Netflix released ratings data but held out some values; teams who best predicted the hidden values won a **\$1,000,000 prize**, and the winning solutions used such low-rank approximation.

The next topic is **inner product spaces**; the instructor says he will continue this discussion on Thursday. He also ends with an administrative request for students to complete a class/course item (the transcript wording is garbled).

Instructor Remarks and Study Guidance

- The SVD is the **ultimate analysis tool**: rank, orthonormal bases for all four fundamental subspaces, $\|A\|_2 = \sigma_1$, and $\|A\|_F = \sqrt{\sum \sigma_k^2}$ — all read off directly.
- Singular values are **real, non-negative** because they are norms (geometric) and because $\sigma_k^2 = \lambda_k(A^*A) \geq 0$ with $A^*A \succeq 0$ (algebraic).
- $\|A\|_F = \sqrt{\sum \sigma_k^2}$ via the trace cyclic property or unitary invariance.
- **Eckart–Young:** truncating the ordered SVD gives the best rank- p approximation (global optimum despite the non-convex rank constraint), in both Frobenius and 2-norms; image compression is the showcase.
- **Closest unitary matrix** to A (Frobenius) is $Q = UV^*$ (set all $\sigma_k = 1$); used in rigid-motion/registration and ICA.
- **Polar decomposition** $A = (U\Sigma U^*)(UV^*) = PT$ generalizes $c = re^{j\theta}$; the correspondence is via eigenvalues (Hermitian $\leftrightarrow$ real, PD $\leftrightarrow$ positive real, unitary $\leftrightarrow$ unit circle).
- **Schatten p -norms** are p -norms of the singular-value vector: $p = 2 \rightarrow$ Frobenius, $p = \infty \rightarrow$ operator, $p = 1 \rightarrow$ **nuclear norm** ($\sum \sigma_k$), the **convex relaxation of rank** (low-rank analog of ℓ_1 sparsity).
- **Matrix completion / Netflix:** model ratings as low-rank $PQ^\top$; the winning approach minimized Frobenius error over observed entries; nuclear-norm minimization is the convex alternative.
- **Recurring warnings:** changing the norm can destroy the closed-form SVD solution; non-convex constraints appear in both rank- p and unitary-matrix problems; nuclear-norm minimization needs additional constraints or the zero matrix wins.
- **Closing:** continuation on Thursday, then inner product spaces; administrative class/course request at the end.

Source and Coverage Note

Source: `corrected/lecture22_corrected.md`.

Coverage: Recap of full/reduced SVD and outer-product form; SVD as analysis tool (rank, four fundamental subspaces, projection to range, σ_1 , Frobenius); why singular values are real/non-negative (geometric

and algebraic via $A^*A \succeq 0$); Frobenius norm $= \sqrt{\sum \sigma_k^2}$ (trace-cyclic and unitary-invariance derivations, singular-value energy); computing the SVD via A^*A/AA^* eigendecomposition (both directions, $u_k = Av_k/\sigma_k$, PSD/Hermitian/normal structure, numerical warning); low-rank approximation / Eckart-Young (rank p versus rank at most p , convex objective, non-convex rank set with the rank-1+rank-1 example, ordered- σ truncation solution stated without proof, Frobenius and 2-norm errors, no closed form for other norms); image compression (guitar-player/Deep Purple aside with garbled name, RGB-to-monochrome conversion, 1236×2060 , storage $k(m+n+1)$, factor storage rather than full product, rank-1/10/20 comparison, singular-value decay, not real image compression); closest unitary matrix ($Q = UV^*$ via Frobenius invariance, deliberate Frobenius choice, scalar polar analogy, neural-network mention); rigid-motion/image-registration application (rotation+translation, least squares, ignored unitary constraint, project-onto-unitary vs. projected-gradient/manifold method, many algorithms/raw-gradient caveat, ICA/PCA mention); matrix polar decomposition ($A = (U\Sigma U^*)(UV^*)$, nonsingular square assumption, non-triviality, unitary factor = closest unitary, scalar check, eigenvalue correspondence Hermitian/PD/unitary, student Q&A on real/imaginary parts); Schatten p -norms (p -norm of singular-value vector; notation distinction; $p=2$ Frobenius, $p=\infty$ operator, $p=1$ nuclear); nuclear/trace norm as convex relaxation of rank (sparsifying singular values, zero-solution constraint warning); matrix completion and the Netflix challenge (incomplete ratings matrix, low-rank $PQ^\top$ user/movie factorization, fixed-rank Frobenius vs. nuclear-norm approaches, \$1M prize and source-stated early-2000s history); closing continuation on Thursday, inner product spaces, and garbled administrative request.

Lecture 23 Notes

Schatten p -Norms

The **Schatten p -norm** of a matrix applies the vector ℓ_p -norm to the **singular-value vector** $\sigma = (\sigma_1, \dots, \sigma_r)$:

$$\|A\|_{(p)} = \|\sigma\|_p = \left(\sum_{k=1}^r \sigma_k^p \right)^{1/p}.$$

(The subscript notation is the instructor's own, used to distinguish it from induced norms.) Three special cases unify the most important matrix norms:

p	Formula	Name
1	$\sum_k \sigma_k$	Nuclear norm $\ A\ _*$ (trace norm)
2	$\sqrt{\sum_k \sigma_k^2} = \ A\ _F$	Frobenius norm
∞	$\max_k \sigma_k = \sigma_1 = \ A\ _{2,2}$	operator / induced 2-2 norm

All three depend **only on the singular values** — they differ only in how the singular values are aggregated. The Schatten- ∞ norm is σ_1 , which (since singular values are ordered) is the largest singular value — the very norm the SVD construction started from (the induced 2-2 / operator norm = max Euclidean gain).

Nuclear Norm: Convex Relaxation of Rank

The **Schatten-1 norm** — the **sum of singular values** — is “the most famous one” and the focus of two decades of machine-learning and signal-processing research. Its own name and notation:

$$\|A\|_* = \sum_{k=1}^r \sigma_k \quad (\text{nuclear norm, a.k.a. trace norm}).$$

Why it matters. It is the ℓ_1 -norm of the singular-value vector. From the homework experience, minimizing an ℓ_1 -norm has a **sparsifying** effect. So minimizing the nuclear norm drives **most singular values to zero** — and the number of nonzero singular values is the **rank**. Thus:

$$\text{minimize } \|A\|_* \iff \text{minimize the rank of } A.$$

Rank minimization is a **hard** (non-convex) problem; the nuclear norm is its **convex relaxation** (a convex function of the matrix entries), solvable efficiently with convex tools (e.g. SDP, CVX). This is exactly analogous to using ℓ_1 as the convex substitute for sparsity (the ℓ_0 count) in vectors.

Application: Matrix Completion and the Netflix Challenge

Setup. Netflix has a matrix with **customers** as rows and **movies** as columns; entry (i, j) is customer i 's rating of movie j (early ratings were thumbs-up/down or small integer scales). With millions of customers and tens/hundreds of thousands of movies — and each customer having rated only hundreds — **most entries are unknown**. Netflix's **challenge** (around 2006, a seminal event for large-scale data science): they had some known ratings and offered a **\$1,000,000 prize** to whoever best predicted them.

Additional transcript details: the instructor's drawing labels rows and columns informally, but the orientation is not mathematically important; the key object is a partially observed customer-movie rating matrix. The old rating scheme is described loosely as numeric/thumb-style values, and the exact scale is not used in the math. Netflix deliberately removed some ratings it already knew, treated them as unknown, and used prediction of those held-out values to score the challenge.

Low-Rank Model

Approximate the giant rating matrix as a product of two **thin** (low-rank) matrices:

$$A \approx C M, \quad C \in \mathbb{R}^{m \times r} \text{ (customer feature matrix), } M \in \mathbb{R}^{r \times n} \text{ (movie feature matrix).}$$

Each customer is an r -dimensional **feature vector** (a row of C); each movie is an r -dimensional feature vector (a column of M). The predicted rating A_{ij} is the **inner product** of customer i 's vector with movie j 's vector — measuring their **alignment** (e.g. a comedy-loving user against a comedy movie scores high). The feature dimensions may correspond to interpretable factors (horror? comedy? a given actor?).

Two Approaches

1. Fixed-rank factorization. Choose r up front and fit C, M by minimizing the Frobenius error **over the observed entries only** (let Ω be the known index set):

$$\min_{C, M} \sum_{(i, j) \in \Omega} (A_{ij} - (CM)_{ij})^2.$$

The number of unknowns is $r(m + n)$; ideally the number of known entries exceeds this (otherwise add **regularization**; there may be infinitely many solutions). **This low-rank factorization is the approach that won the Netflix challenge.** (It is a bilinear, non-convex problem — typically solved by alternating least squares.)

2. Nuclear-norm minimization (convex). Don't assume a rank; instead minimize the nuclear norm subject to matching the observed entries:

$$\min_B \|B\|_* \quad \text{s.t.} \quad B_{ij} = A_{ij} \quad \forall (i, j) \in \Omega.$$

The objective is convex and the constraints are **linear**, so CVX/SDP solvers handle it. **Advantages:** no need to fix r in advance — the rank emerges **adaptively** from the data (the nuclear norm “pushes r as small as possible” to fit the data). The more observed entries you have, the better constrained the completion is.

More precisely, more observed entries make the completion better constrained. With too few known entries relative to the number of factor parameters, many completions/factorizations can fit the data; regularization or a rank-minimizing objective is then needed to avoid infinitely many plausible solutions.

Either way, once the completed matrix is obtained, you predict each customer's reaction to each movie and recommend accordingly.

Other SVD Applications (Briefly)

The instructor notes there are countless SVD-based algorithms, especially in EE. One named example: **subspace algorithms for direction finding** — an array of antennas receives an impinging electromagnetic wave, and you estimate the **angle of arrival**. Subspace methods (built on the SVD of the data) are the key tool. (Skipped for time.)

This concludes the SVD; the lecture turns to **inner product spaces**.

Abstract Inner Product Spaces

Just as a **norm** was added on top of a vector space to measure “how big” a vector is (generalizing the Euclidean norm), an **inner product** can be added to a vector space — and from it a norm is **induced**. Terminology: a complete **normed** space is a **Banach space**; a complete **inner product** space is a **Hilbert**

space (a vector space equipped with an inner product, plus a technical completeness condition not dwelt on here — in finite dimensions completeness is automatic).

Definition. An inner product $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{C}$ (or $\mathbb{R}$) satisfies the properties the Euclidean inner product has: 1. **Conjugate symmetry:** $\langle x, y \rangle = \overline{\langle y, x \rangle}$. 2. **Linearity in the first argument:** $\langle \alpha x, y \rangle = \alpha \langle x, y \rangle$ and $\langle x + z, y \rangle = \langle x, y \rangle + \langle z, y \rangle$. (Fixing the second argument, it is a **linear** operator — hence “bilinear”/sesquilinear overall; scaling the **second** argument by α pulls out $\bar{\alpha}$.) 3. **Positive definiteness:** $\langle x, x \rangle \geq 0$, with equality iff $x = 0$.

Induced norm. Property 3 makes

$$\|x\| = \sqrt{\langle x, x \rangle}$$

a valid norm. So **defining an inner product simultaneously defines a connected norm**, and the **main utility** of inner product spaces is that the inner product is a **tool for solving the norm-minimization (projection / least-squares) problems** — usually yielding **closed-form** solutions.

Four Important Inner Product Spaces

1. $\mathbb{C}^n$ (Euclidean and Weighted)

$$\langle x, y \rangle = y^* x = \sum_k \bar{y}_k x_k, \quad \|x\| = \sqrt{x^* x} = \|x\|_2.$$

The conjugate is taken on the **second** argument. **Weighted version:** insert a positive definite W ,

$$\langle x, y \rangle_W = y^* W x, \quad \|x\|_W = \sqrt{x^* W x}.$$

2. $L^2([a, b])$ (Square-Integrable Functions)

$$\langle f, g \rangle = \int_a^b f(t) \overline{g(t)} dt, \quad \|f\|_{L^2} = \sqrt{\int_a^b |f(t)|^2 dt}.$$

Fourier transform as an inner product: $\hat{f}(\nu) = \int f(t) e^{-j2\pi\nu t} dt = \langle f, e^{j2\pi\nu(\cdot)} \rangle$ — the Fourier coefficient at frequency ν is the **inner product** of f with the complex exponential at that frequency. **Orthogonality** of functions: $\langle f, g \rangle = 0$. A **weighted** version uses a positive weight $w(t) > 0$: $\langle f, g \rangle_w = \int f \bar{g} w dt$ (weighting each time/frequency differently).

3. Matrices (Frobenius Inner Product)

For $m \times n$ complex matrices,

$$\langle A, B \rangle = \text{tr}(B^* A) = \text{vec}(B)^* \text{vec}(A) = \sum_{i,j} \bar{b}_{ij} a_{ij},$$

i.e. the ordinary complex inner product of the vectorized matrices, where $\text{vec}(\cdot)$ stacks the matrix columns into one long vector. Induced norm $= \|A\|_F$. Two matrices are **orthogonal** if $\text{tr}(B^* A) = 0$. This is how the usual geometric language extends to rectangular complex matrices.

4. Random Variables (Correlation as Inner Product)

For zero-mean complex random variables,

$$\langle X, Y \rangle = \mathbb{E}[X \bar{Y}] = \text{Corr}(X, Y).$$

Two random variables are **orthogonal iff uncorrelated**. The induced norm is

$$\|X\| = \sqrt{\mathbb{E}[|X|^2]} = \text{standard deviation (for zero mean)},$$

a measure of the random variable's **uncertainty/spread**. The instructor asks for **another** uncertainty measure: **entropy** $H = -\sum_x p_x \log p_x$ (Shannon entropy). Standard deviation is the right measure for **linear** MMSE estimation; for a **Gaussian**, entropy is essentially $\log(\text{variance})$, so the two are simply related. (Entropy belongs to information theory, beyond this course.)

This inner product is the **basis of linear stochastic mean-square estimation** — **Wiener** and **Kalman** filters — via the principle “correlation = inner product.”

SVD Rank-1 Components Are Orthonormal in the Frobenius Inner Product

The reduced SVD $A = \sum_{k=1}^r \sigma_k u_k v_k^*$ expresses A in a basis of rank-1 matrices $u_k v_k^*$, which are **orthonormal** in the matrix (Frobenius) inner product.

Proof of orthogonality. For the inner product of $u_i v_i^*$ and $u_j v_j^*$:

$$\langle u_i v_i^*, u_j v_j^* \rangle = \text{tr}((u_j v_j^*)^* (u_i v_i^*)) = \text{tr}(v_j u_j^* u_i v_i^*).$$

Since the left singular vectors are orthonormal, $u_j^* u_i = \delta_{ij}$, so this is $\delta_{ij} \text{tr}(v_j v_i^*)$. The trace of the (column)(row) product $v_j v_i^*$ equals $v_i^* v_j$ (cycling), which is also δ_{ij} . Hence

$$\langle u_i v_i^*, u_j v_j^* \rangle = \delta_{ij} \text{ (Kronecker delta)}.$$

The instructor paused on the notation: δ_{ij} is the **Kronecker delta** - it is 1 when $i = j$ and 0 when $i \neq j$. This follows because the left singular vectors u_i are columns of a unitary matrix (unit norm and mutually orthogonal), and the same is true for the right singular vectors v_i . The trace step uses $\text{tr}(AB) = \text{tr}(BA)$; after cycling, the trace of the resulting 1×1 scalar is just that scalar.

Norm check: $\|u_k v_k^*\|_F^2 = \text{tr}(v_k u_k^* u_k v_k^*) = \text{tr}(v_k v_k^*) = v_k^* v_k = 1$. ✓

Interpretation. The SVD writes A as a linear combination of an **orthonormal basis** of rank-1 matrices, with **coordinates** $\sigma_k = \langle A, u_k v_k^* \rangle$ — the singular value is the projection of A onto its own k -th natural basis element. This is the matrix analogue of a Fourier series (orthonormal basis functions, Fourier coefficients). The catch: this orthonormal basis is **adapted to A itself**.

Orthogonality and Gram–Schmidt in Any Inner Product Space

Once an inner product is fixed, **orthogonality** and **Gram–Schmidt** carry over to **any** vector space — producing an orthonormal basis for the given inner product.

Legendre Polynomials from Gram–Schmidt on Monomials

Take functions over $[-1, 1]$ with $\langle f, g \rangle = \int_{-1}^1 f(t)g(t) dt$. The **monomials** $g_1 = 1, g_2 = t, g_3 = t^2, \dots$ span the polynomials but are **not** orthonormal. Gram–Schmidt:

For degree n , $\text{span}\{g_1, \dots, g_{n+1}\}$ is the space of polynomials of degree at most n . Gram–Schmidt must preserve that span while replacing the monomials by pairwise orthonormal functions $h_1, \dots, h_{n+1}$.

- h_1 : $\|1\|^2 = \int_{-1}^1 1 dt = 2$, so $h_1 = 1/\sqrt{2}$.
- h_2 : project t onto h_1 : $\langle t, h_1 \rangle = \frac{1}{\sqrt{2}} \int_{-1}^1 t dt = 0$ (odd integrand). So t is already orthogonal to the constant; normalize: $\|t\|^2 = \int_{-1}^1 t^2 dt = \frac{2}{3}$, giving $h_2 = \sqrt{3/2} t$.
- h_3 : project t^2 onto h_1 and h_2 . $\langle t^2, h_2 \rangle = 0$ (odd integrand: t^2 and t are orthogonal), but $\langle t^2, h_1 \rangle \neq 0$ (it gives $\frac{2}{3\sqrt{2}}$). Subtract the h_1 component:

$$v_3 = t^2 - \frac{1}{3},$$

then normalize. (The t^2 -vs-1 inner product $\int_{-1}^1 t^2 dt = \frac{2}{3} \neq 0$ is exactly why the monomials are not orthogonal.)

The resulting (un-normalized) polynomials $1, t, t^2 - \frac{1}{3}, \dots$ are the **Legendre polynomials** — an **orthogonal basis** for polynomials on $[-1, 1]$. (Different inner products give different families: e.g. a spherical-geometry weight in electromagnetics yields other classical orthogonal polynomials.)

Why bother? With an **orthonormal** basis, projecting an arbitrary function g onto the polynomial subspace (best polynomial approximation of degree $\leq n$) is just a sum of independent inner products — **no matrix inverse needed** (see the projection theorem below).

If the basis functions are not orthogonal, the closest polynomial is not obtained by projecting onto the monomials one at a time; the coupled Gram-matrix system must be solved instead.

Innovation Sequence / Kalman Filter Interpretation

In estimation language, the span of measurements is the set of linear functions/linear estimators built from those measurements. Preserving the same span means preserving the same total information while repackaging it into uncorrelated pieces.

In the random-variable inner product space, Gram–Schmidt on correlated observations $x_1, x_2, \dots$ produces **uncorrelated** random variables $q_1, q_2, \dots$ with the **same span** — the **innovation sequence**. Geometrically, q_2 is the component of x_2 **orthogonal to x_1** : the **new information** in x_2 that **cannot be predicted** from x_1 . This is the **prediction interpretation** of projection: projecting x_2 onto x_1 is the best linear **prediction** of x_2 from x_1 ; the orthogonal remainder is the **prediction error** (the innovation). The **Kalman filter** does exactly this recursively — building an orthonormal basis on the fly as data arrives (assuming a special structure on the observations), so that estimation reduces to projection onto uncorrelated components. (Details belong to an estimation-theory course.)

The Projection Theorem (Climax)

This is the central theorem of inner product spaces — the reason inner products are introduced and the reason orthonormal bases are valuable.

Statement

Let V be an inner product space with induced norm $\|x\| = \sqrt{\langle x, x \rangle}$. Given vectors $p_1, \dots, p_n \in V$ (spanning a subspace M) and a target $x \in V$ (possibly outside M), consider the **approximation problem**

$$\min_{\hat{x} \in \text{span}\{p_1, \dots, p_n\}} \|x - \hat{x}\|.$$

The norm in this problem is not arbitrary: it is the norm induced by the inner product chosen for the application. That connection is exactly why the inner product can solve the minimization problem. **Projection theorem:** the minimizer $\hat{x}$ is the **orthogonal projection** of x onto M , characterized by the **orthogonality principle**: the error $x - \hat{x}$ is **orthogonal to the entire subspace M** , i.e.

$$\langle x - \hat{x}, p_i \rangle = 0 \quad \text{for all } i = 1, \dots, n.$$

This holds in **any** inner product space — x and the p_i could be vectors, **matrices**, **functions** (monomials), or **random variables**.

Why Orthogonality Is Optimal (Pythagoras)

If the error were **not** orthogonal to M , it would have a nonzero component **inside** M ; by the Pythagorean theorem, removing that component would strictly decrease the error. So at the optimum the error must be orthogonal to M .

Normal Equations and the Gram Matrix

Write the optimal point as a combination of the basis vectors, $\hat{x} = \sum_j \alpha_j p_j$ (n unknown coefficients α_j). The orthogonality conditions $\langle x - \hat{x}, p_i \rangle = 0$ give n equations:

$$\langle x, p_i \rangle = \sum_j \alpha_j \langle p_j, p_i \rangle, \quad i = 1, \dots, n.$$

In matrix form $G\alpha = b$, where $b_i = \langle x, p_i \rangle$ and

$$G_{ij} = \langle p_j, p_i \rangle$$

is the **Gram matrix** of all pairwise inner products of the p_i . Solving this linear system gives the coefficients α .

Orthonormal simplification. If $\{p_i\}$ is **orthonormal**, then $G = I$, and

$$\alpha_i = \langle x, p_i \rangle, \quad \hat{x} = \sum_i \langle x, p_i \rangle p_i.$$

This is **why** we work so hard (Gram–Schmidt) to get orthonormal bases: projection becomes a set of **independent inner products** — no system to solve. (If merely orthogonal but not normalized, G is diagonal — still easy.) This is exactly what the Kalman filter exploits: project onto the on-the-fly orthonormal innovation basis.

Connection to Least Squares

The projection theorem **is** the least-squares solution: given $Hx = y$ with $y \notin \mathcal{R}(H)$ (no exact solution), find the point in the range of H closest to y — the orthogonal projection of y onto $\mathcal{R}(H)$. Variations are obtained simply by **changing the inner product**: - **Weighted least squares** — use a weighted inner product (W -weighted). - **Regularized least squares** — another inner-product-space formulation.

All become the same projection problem in the appropriate inner product space. In the transcript these are named as post-theorem applications rather than fully developed lecture material.

Exam Boundary and Further Reading

The instructor states the exam covers **everything through the projection theorem, inclusive**; material after it (detailed least-squares applications, weighted/regularized least squares as projection, estimation-theory applications, state-space notes) is **not** on the exam but is highly recommended. He specifically recommends the supplementary notes on **estimation of a random variable from multiple observations** (LMMSE), **polynomial approximation**, and **state-space representations** (a current hot topic in machine learning), building on the matrix background from this course.

Summary: Inner Products Unify the Geometry

Concept	In $\mathbb{C}^n$	In a general inner product space
Inner product	$y^* x$	$\langle x, y \rangle$ (space-specific)
Norm	$\ x\ _2 = \sqrt{x^* x}$	$\ x\ = \sqrt{\langle x, x \rangle}$
Orthogonality	$x^* y = 0$	$\langle x, y \rangle = 0$
Projection (ONB)	$\hat{x} = \sum_k (q_k^* x) q_k$	$\hat{x} = \sum_k \langle x, q_k \rangle q_k$
Projection (general)	normal equations $A^* A$	Gram-matrix system $G\alpha = b$
Gram–Schmidt	orthonormalize vectors	orthonormalize any elements

Concept	In $\mathbb{C}^n$	In a general inner product space
---------	-------------------	----------------------------------

The price of the generalization: verify the inner-product axioms for each specific space.

Instructor Remarks and Study Guidance

- **Schatten norms** unify the three main matrix norms: nuclear ($p = 1$), Frobenius ($p = 2$), operator ($p = \infty$); all are functions of the singular values.
- **Nuclear norm** = convex relaxation of **rank** (the low-rank analog of ℓ_1 sparsity). It powers **matrix completion** / the **Netflix challenge** (low-rank CM factorization won; nuclear-norm minimization is the convex, rank-adaptive alternative).
- **Inner products generalize all linear-algebra geometry** to functions, matrices, and random variables. **Correlation = inner product** (uncorrelated = orthogonal, std-dev = norm) makes Wiener/Kalman filtering a projection problem.
- The **SVD rank-1 components are orthonormal** in the Frobenius inner product; σ_k is the matrix's coordinate along its own natural basis.
- **Gram-Schmidt** in $L^2([-1, 1])$ on monomials yields the **Legendre polynomials**; on random variables it yields the **innovation sequence** (new, unpredictable information = orthogonal component).
- **Projection theorem (exam climax)**: the closest point in a subspace makes the **error orthogonal to the subspace**; this gives the **Gram-matrix normal equations** $G\alpha = b$, which collapse to independent inner products when the basis is **orthonormal**. Least squares (and its weighted/regularized variants) are special cases obtained by choosing the inner product.
- **Exam covers through the projection theorem, inclusive**. Read the supplementary LMMSE, polynomial-approximation, and state-space notes for going further.

Source and Coverage Note

Source: `corrected/lecture23_corrected.md`.

Audit patch addendum: Added transcript-specific details on the informal Netflix matrix setup and held-out ratings; observed-entry/regularization caveats; matrix vectorization as column stacking; the Kronecker-delta and trace-cycling clarification in the SVD orthonormality proof; same-span requirements for Gram-Schmidt on degree- n polynomials; why non-orthogonal monomials require a coupled projection solve; measurement-span/linear-estimator interpretation for innovation sequences; the induced-norm requirement in the projection theorem; and the optional status of post-theorem least-squares variants.

Coverage: Schatten p -norms (definition; nuclear/Frobenius/operator special cases; $p = \infty = \sigma_1 =$ induced 2-2 origin of SVD); nuclear/trace norm as ℓ_1 of singular values and convex relaxation of rank; matrix completion / Netflix challenge (customer $\times$ movie matrix, low-rank CM feature factorization with inner-product scores, fixed-rank Frobenius approach with $r(m+n)$ unknowns and regularization, nuclear-norm convex/rank-adaptive approach, \$1M prize/history); other SVD applications (direction finding / subspace algorithms); abstract inner product spaces (Banach vs Hilbert, axioms, induced norm, utility for projection/least-squares); four examples ($\mathbb{C}^n$ Euclidean and weighted; $L^2[a, b]$ with Fourier transform as inner product and weighted version; matrices/Frobenius inner product; random variables/correlation with std-dev as norm and the entropy aside); SVD rank-1 components orthonormal in Frobenius inner product (full trace proof and norm check, singular value as coordinate); Gram-Schmidt in inner product spaces — Legendre polynomials ($h_1, h_2, v_3 = t^2 - \frac{1}{3}$ steps, why monomials aren't orthogonal, other polynomial families) and the innovation-sequence/Kalman prediction interpretation; the **projection theorem** (statement, orthogonality principle, Pythagoras intuition, Gram-matrix normal equations $G\alpha = b$, orthonormal simplification, least-squares/weighted/regularized connection); exam boundary (through projection theorem inclusive) and recommended further reading; unification table.